Understanding Neural Networks: A Structured Guide to Training and Optimization

Md Shahriar Forhad
https://shahriar88.github.io/



September 13, 2026

Contents

1 Common Neural Network Terms and Intuition **1**

1.1 Why These Terms Matter . . . 1
1.2 Autoencoder . . . 1
1.2.1 Basic Idea . . . 1
1.2.2 Why Autoencoders Are Useful . . . 1
1.2.3 Common Variants . . . 2
1.3 Encoder . . . 2
1.3.1 Important Clarification . . . 2
1.4 Decoder . . . 2
1.5 Hierarchical Representation . . . 3
1.5.1 Hierarchical Encoder . . . 3
1.5.2 Hierarchical Decoder . . . 3
1.5.3 Key Distinction . . . 3
1.5.4 Multi-Stage and Multi-Level Representations . . . 4
1.6 Latent Space . . . 5
1.7 Bottleneck . . . 5
1.8 Feature Map . . . 5
1.9 Downsampling . . . 6
1.10 Upsampling . . . 6
1.11 Relationship Between Hierarchy, Downsampling, and Upsampling . . . 7
1.12 Skip Connection . . . 7
1.12.1 Why Skip Connections Help . . . 7
1.12.2 Skip Connections in U-Net . . . 7
1.13 Residual Connection . . . 8
1.13.1 Why Residual Connections Matter . . . 8
1.14 Difference Between Skip Connection and Residual Connection . . . 8
1.15 Shortcut Connection . . . 9
1.16 Reconstruction Loss . . . 9
1.17 Regularization . . . 9
1.18 Monte Carlo Experiments and Monte Carlo Validation . . . 10
1.18.1 Monte Carlo Experiment . . . 10
1.18.2 Monte Carlo Validation . . . 11
1.18.3 Monte Carlo Validation Versus Deterministic Validation . . . 11
1.18.4 Role of Random Seeds . . . 12
1.18.5 Practical Interpretation . . . 12
1.18.6 Why Monte Carlo Methods Are Useful . . . 12
1.19 Attention . . . 13
1.20 Transformer . . . 13
1.21 Practical Picture of a Typical Image Model . . . 13
1.22 Summary . . . 14

2 Activation Functions, Loss Functions, Optimizers, and Training Mechanics 15
2.1 Activation Functions . . . 15
2.1.1 Sigmoid . . . 15
2.1.2 Tanh . . . 15
2.1.3 ReLU . . . 16
2.1.4 Leaky ReLU . . . 16
2.1.5 Softmax . . . 16
2.1.6 GELU . . . 16
2.2 Visual Interpretation of Activation Functions . . . 17
2.3 Loss Functions . . . 19
2.3.1 Mean Squared Error (MSE) . . . 19
2.3.2 Mean Absolute Error (MAE) . . . 19
2.3.3 Huber Loss . . . 19
2.3.4 Cross Entropy Loss . . . 19
2.3.5 Focal Loss . . . 19
2.3.6 Dice / IoU Loss . . . 19
2.3.7 KL Divergence . . . 19
2.4 Visual Interpretation of Loss Functions . . . 21
2.5 Optimizers . . . 23
2.5.1 Stochastic Gradient Descent (SGD) . . . 23
2.5.2 Momentum . . . 23
2.5.3 Adagrad . . . 23
2.5.4 RMSProp . . . 23
2.5.5 Adam . . . 23
2.5.6 AdamW . . . 23
2.5.7 LAMB . . . 23
2.6 Visual Interpretation of Optimizers . . . 24
2.7 Summary Tables . . . 26
2.8 Neuron and Feature Representation . . . 26
2.9 Visual Interpretation of Training Mechanics . . . 27
2.10 Forward Pass . . . 28
2.11 Backpropagation . . . 28
2.12 Weight and Bias Updates . . . 28

3 General Structure of Forward Pass and Backpropagation 31
3.1 Overview . . . 31
3.2 Forward Pass . . . 31
3.2.1 Basic Structure . . . 31
3.2.2 Computation Flow . . . 31
3.3 Backpropagation . . . 31
3.3.1 Error Propagation . . . 31
3.3.2 Weight Update . . . 32
3.3.3 General Steps . . . 32
3.4 Universality of the Process . . . 32
3.5 Common Variations . . . 32
3.6 Summary . . . 32
3.7 Worked Example I: Single-Neuron Forward and Backpropagation . . . 33
3.8 Worked Example II: Three-Layer Forward and Backpropagation . . . 33
3.8.1 Network Definition (Scalar) . . . 33
3.8.2 Activation Derivatives . . . 34
3.8.3 Numerical Values Used . . . 34

3.9 Matrix-Form Backpropagation, Product Types, and Forward-Pass Memory . . . 37
3.10 Chain Rule Accumulation and Gradient Flow . . . 39
3.11 Hessian–Vector Products and Second-Order Structure . . . 39
3.11.1 Worked Example: Scalar Hessian–Vector Product . . . 40
3.11.2 Hessian–Vector Products via Backpropagation: Concrete Three-Layer Example . . . 40
3.11.3 Role of the Second Backward Pass . . . 46
3.11.4 Direction Vectors in Second-Order Optimization . . . 46
3.11.5 Newton–CG and Hessian-Free Methods . . . 46
3.12 Training vs. Inference in Automatic Differentiation Frameworks . . . 48

4 Jacobians in Backpropagation and Second-Order Analysis 49
4.1 Overview . . . 49
4.2 Jacobian View of the Chain Rule . . . 49
4.3 Why Jacobians Are Not Formed Explicitly . . . 49
4.4 Three-Layer Network: Matrix Form . . . 50
4.5 Numerical Example with Explicit Jacobians (with activation and loss derivatives) 50
4.5.1 Network definition . . . 51
4.5.2 Activation derivatives and loss derivative . . . 51
4.5.3 Forward pass (numerical) . . . 51
4.5.4 Local derivatives (numerical) . . . 52
4.5.5 Explicit Jacobians . . . 52
4.5.6 Backpropagation via Jacobian products (numerical) . . . 52
4.6 Same Numerical Example via Explicit Chain Rule (no Jacobian shortcut) . . . 53
4.6.1 Local derivatives used . . . 53
4.6.2 Forward values (from the numerical example) . . . 53
4.6.3 Backward pass by chain rule (step-by-step) . . . 54
4.6.4 Connection to the Jacobian form . . . 55
4.7 Connection to Second-Order Derivatives . . . 55
4.8 Summary . . . 56
4.9 Worked Example III: Full Forward/Backward Pass and Gradient Descent Update 57
4.9.1 Network definition . . . 57
4.9.2 Derivatives used (local) . . . 57
4.9.3 Numerical values . . . 57
4.9.4 Forward pass (all steps) . . . 57
4.9.5 Backward pass (explicit derivatives and gradients) . . . 57
4.9.6 Gradient summary . . . 59
4.9.7 Gradient descent update (one step) . . . 59
4.10 Jacobian, Delta, and Parameter Gradients in Backpropagation . . . 59
4.10.1 Jacobian . . . 59
4.10.2 Delta . . . 60
4.10.3 Gradients of Weights and Biases . . . 60
4.10.4 Conceptual Hierarchy . . . 60
4.10.5 A Subtle but Important Clarification . . . 61
4.10.6 Mental Model . . . 61
4.10.7 Summary . . . 61
4.10.8 Chronological computation table (forward → backward → update, with Jacobians) . . . 61
4.11 Worked Micro-Example: The First Backprop Signal (dv_7) Comes From the Loss 61
4.11.1 Forward computation (scalar graph) . . . 61
4.11.2 The universal first step in backprop . . . 61

4.11.3 Why sometimes $dv_7 = 1$ . . . 63
4.11.4 Target-matching: making v_7 approach a desired value . . . 63
4.11.5 Different losses give different dv_7 (only the first line changes) . . . 63
4.12 Parameter Shapes and Gradient Shapes . . . 64
4.12.1 Single Neuron Case . . . 64
4.12.2 Layer Case (Multiple Neurons) . . . 64
4.12.3 Summary Table: Parameters and Their Gradients . . . 64
4.12.4 Operational Summary . . . 64
4.12.5 Reference Python Implementation . . . 65
4.13 Linear Algebra Operations in Forward and Backward Propagation . . . 65
4.14 Commutativity of Linear Algebra Operations . . . 65
4.15 Numerical Experiments: Effect of the Initial Backprop Signal dv_7 . . . 68
4.15.1 Direct minimization of v_7 ($dv_7 = 1$) . . . 69
4.15.2 Forcing $v_7 \to 0$ (squared loss, $dv_7 = v_7$) . . . 69
4.15.3 Forcing $v_7 \to 1$ (shifted squared loss, $dv_7 = v_7 - 1$) . . . 70
4.15.4 Interpretation . . . 70

5 The Curse of Dimensionality and Fixed Basis Functions 73
5.1 Polynomial Basis Functions and Parameter Growth . . . 73
5.2 Grid-Based Basis Functions . . . 73
5.3 High-Dimensional Geometry . . . 74
5.3.1 Volume Concentration . . . 74
5.4 Data Manifolds . . . 74
5.4.1 Natural Images . . . 75
5.5 Radial Basis Functions . . . 75
5.6 Motivation for Deep Learning . . . 75

6 Gradients and Optimization Foundations 77
6.1 Gradient and Subgradient . . . 77
6.2 Error Surfaces and Gradient-Based Optimization . . . 79
6.2.1 Error Surfaces . . . 79
6.2.2 Local Change in Error . . . 79
6.2.3 Geometric Meaning of the Gradient . . . 79
6.2.4 Gradient Descent Update . . . 79
6.2.5 Stationary Points . . . 80
6.2.6 Interpretation . . . 80
6.2.7 Local Quadratic Approximation . . . 80
6.2.8 Eigenvalues, Curvature, and Stationary Points . . . 81
6.2.9 Stochastic and Mini-Batch Gradient Descent . . . 82
6.2.10 Convergence Behavior . . . 84
6.2.11 Vanishing and Exploding Gradients . . . 85
6.2.12 Gradient Propagation via the Chain Rule . . . 85
6.2.13 Vanishing Gradients . . . 86
6.2.14 Exploding Gradients . . . 86
6.3 Normalization Methods . . . 87

7 Regularization Principles: Tikhonov, Bayesian Priors, and Modern Deep Networks 89
7.1 Learning Curves and Generalization . . . 89
7.2 Tikhonov Regularization: The Classical Perspective . . . 89
7.2.1 Closed-Form Solution . . . 90
7.3 Statistical Learning Theory View . . . 90

7.4 Functional Analysis View: Ill-Posed Operators . . . 90
7.5 Bayesian Interpretation . . . 91
7.6 Connections to Modern Deep Learning . . . 91
7.6.1 Weight Decay . . . 91
7.6.2 Dropout . . . 91
7.6.3 Early Stopping . . . 92
7.6.4 Normalization Layers . . . 92
7.7 Conceptual Synthesis . . . 92
7.8 Regularization in Deep Networks . . . 92
7.9 Symmetry, Invariance, and Equivariance . . . 93
7.9.1 Symmetry . . . 93
7.9.2 Invariance . . . 93
7.9.3 Equivariance . . . 94
7.9.4 Vision Tasks . . . 94
7.9.5 Summary Table . . . 95
7.9.6 Conceptual Distinction . . . 95
7.10 Enforcing Invariance and Regularization . . . 95
7.10.1 Methods for Enforcing Invariance . . . 95
7.11 Error Function Regularization . . . 96
7.11.1 Interpretation . . . 96
7.11.2 Gradients of Regularized Error Functions . . . 96
7.11.3 Geometric Interpretation . . . 99
7.11.4 Practical Example: Linear Regression . . . 100
7.11.5 Summary . . . 100
7.12 L_p Regularization in PyTorch . . . 100
7.12.1 PyTorch Implementation . . . 100
7.12.2 Gradient Behavior . . . 101
7.13 Inspecting the Autograd Graph . . . 101
7.13.1 Manual Graph Inspection . . . 101
7.13.2 Graph Visualization with torchviz . . . 102
7.13.3 Key Principle . . . 102
7.14 Consistency of Regularizers and Scaling Symmetries . . . 102
7.14.1 Two-Layer Network Formulation . . . 102
7.14.2 Invariance Under Input Linear Transformations . . . 103
7.14.3 Hidden Unit Rescaling Symmetry . . . 104
7.14.4 Why Standard L_2 Regularization Breaks This Symmetry . . . 104
7.14.5 Concrete Numeric Example . . . 104
7.14.6 Layer-Wise Regularization . . . 105
7.14.7 Bayesian Interpretation . . . 105
7.14.8 Summary . . . 105
7.15 Classical Bias–Variance Trade-Off and the Modern Overparameterized Regime . . . 105
7.15.1 Classical Bias–Variance Trade-Off . . . 105
7.15.2 Modern Overparameterized Regime . . . 106
7.15.3 Double Descent Phenomenon . . . 106
7.15.4 Why Does This Happen? . . . 107
7.15.5 Illustrative Example . . . 107
7.15.6 Key Contrast . . . 107
7.15.7 Implications for Deep Architectures . . . 108
7.16 Double Descent and Structural Regularization . . . 108
7.16.1 Statistical Framework . . . 108
7.16.2 Classical Regime . . . 108

7.16.3 Interpolation Threshold . . . 109
7.16.4 Double Descent . . . 109
7.16.5 Implicit Bias and Low-Norm Selection . . . 109
7.17 Structural Regularization . . . 110
7.17.1 Parameter Sharing . . . 110
7.17.2 Residual Connections . . . 110
7.18 Model Averaging, Boosting, and Dropout . . . 111
7.18.1 Model Averaging (Ensembles) . . . 111
7.18.2 Variance Reduction . . . 111
7.18.3 Bagging . . . 112
7.18.4 Boosting . . . 112
7.18.5 Dropout . . . 113
7.18.6 Monte Carlo Dropout . . . 114
7.18.7 Bias–Variance Perspective . . . 116

8 Ensemble Learning, Parameter Sharing, and the Interpolation Threshold 117
8.1 Overview . . . 117
8.2 Model Averaging: Ensembles and Committees . . . 118
8.2.1 Flow of a Parallel Committee . . . 118
8.2.2 Numerical Example: Soft Voting . . . 118
8.3 Bias–Variance Trade-Off . . . 119
8.4 Expected Error of Individual Models . . . 119
8.4.1 Classification Error of an Individual Member . . . 119
8.5 Expected Error of the Committee . . . 120
8.5.1 Regression Committee with Correlated Members . . . 120
8.5.2 Numerical Example: Variance Reduction . . . 120
8.5.3 Majority-Vote Classification Committee . . . 121
8.6 Parameter Sharing . . . 121
8.7 Hard vs. Soft Parameter Sharing . . . 122
8.7.1 Hard Parameter Sharing . . . 122
8.7.2 Soft Parameter Sharing . . . 122
8.8 Interpolation Threshold . . . 123
8.9 Bagging: Bootstrap Aggregation . . . 124
8.9.1 Bootstrap Sampling . . . 124
8.9.2 Bagging Flowchart . . . 127
8.9.3 Example: Bagged Regression Trees . . . 128
8.10 Boosting: Sequential Model Combination . . . 128
8.10.1 Gradient-Boosting View . . . 128
8.10.2 Boosting Flowchart . . . 129
8.10.3 Numerical Example: Residual Correction . . . 129
8.10.4 AdaBoost View for Classification . . . 130
8.11 Comparison of Bagging and Boosting . . . 130
8.11.1 Practical Selection Guide . . . 131
8.12 Unified Perspective . . . 131
Chapter Summary . . . 132

9 Convolutional Neural Networks 133
9.1 Motivation . . . 133
9.2 Images as Input Data . . . 133
9.3 Convolution Operation . . . 133
9.4 Convolution vs. Cross-Correlation in 2D . . . 134
9.4.1 Mathematical Difference . . . 134

9.4.2 Numerical Example . . . 135
9.4.3 Why Deep Learning Still Calls It Convolution . . . 136
9.4.4 Summary . . . 137
9.5 Convolution with Multiple Input and Output Channels . . . 137
9.5.1 Convolution Operation with Multiple Input Channels . . . 138
9.5.2 Convolution Operation with Multiple Output Channels . . . 139
9.5.3 Channel Mixing . . . 139
9.5.4 Changing the Number of Channels . . . 140
9.5.5 Examples of Channel Transformation . . . 140
9.5.6 Understanding the Table . . . 140
9.5.7 Example: How a 1×1 Convolution Changes Channels . . . 142
9.5.8 Summary . . . 143
9.6 Kernel, Feature Map, and Channel . . . 143
9.6.1 Feature Map Example . . . 144
9.6.2 Example with Multiple Input Channels . . . 144
9.6.3 How Kernel Values Are Learned . . . 145
9.7 Stride, Padding, and Output Size . . . 145
9.7.1 Stride . . . 146
9.7.2 Padding . . . 146
9.8 Example: Convolution Output Size . . . 146
9.9 Dilated Convolution . . . 147
9.9.1 Effective Kernel Size . . . 147
9.9.2 2D Dilated Convolution Example . . . 148
9.9.3 Output Size with Dilation . . . 148
9.9.4 Example: Output Size with Dilation . . . 149
9.9.5 Why Dilation Is Useful . . . 149
9.9.6 Summary . . . 150
9.10 Receptive Field . . . 150
9.10.1 Simple Receptive Field Example . . . 150
9.10.2 Receptive Field with Stride . . . 151
9.10.3 Example: Receptive Field with Two Layers . . . 151
9.10.4 Example: Receptive Field with Pooling . . . 152
9.11 Why CNNs Are Hierarchical . . . 152
9.12 Number of Parameters in a Convolution Layer . . . 153
9.12.1 Example . . . 153
9.13 Activation Function . . . 153
9.14 Pooling Layers . . . 153
9.14.1 Max Pooling . . . 153
9.14.2 Average Pooling . . . 153
9.15 Fully Connected Layer in CNN . . . 154
9.16 Basic CNN Architecture . . . 154
9.17 CNN for Image Classification . . . 154
9.18 CNN for Semantic Segmentation . . . 155
9.19 CNN for Object Detection and Instance Segmentation . . . 156
9.19.1 R-CNN . . . 156
9.19.2 Fast R-CNN . . . 157
9.19.3 Faster R-CNN . . . 160
9.19.4 Mask R-CNN . . . 161
9.19.5 Evolution of the R-CNN Family . . . 166
9.20 Why CNNs Work Well for Images . . . 167
9.20.1 Local Connectivity . . . 167

9.20.2 Weight Sharing . . . 167
9.20.3 Translation Equivariance . . . 167
9.21 Comparison with Fully Connected Neural Networks . . . 167
9.22 LeNet: A Classic CNN Example . . . 168
9.22.1 LeNet Layer-by-Layer Summary . . . 168
9.22.2 TikZ Flow Diagram of LeNet . . . 169
9.22.3 Why LeNet is Important . . . 169
9.23 AlexNet: A Deeper CNN for Image Classification . . . 169
9.23.1 AlexNet Layer-by-Layer Summary . . . 170
9.23.2 TikZ Flow Diagram of AlexNet . . . 171
9.23.3 Why AlexNet is Important . . . 171
9.24 VGG16: A Deep CNN with Small Convolution Kernels . . . 172
9.24.1 VGG16 Layer-by-Layer Summary . . . 173
9.24.2 TikZ Flow Diagram of VGG16 . . . 175
9.24.3 Why VGG16 is Important . . . 176
9.25 GoogLeNet: CNN with Inception Modules . . . 176
9.25.1 The Inception Module . . . 176
9.25.2 GoogLeNet Layer-by-Layer Summary . . . 177
9.25.3 TikZ Flow Diagram of an Inception Module . . . 177
9.25.4 TikZ Flow Diagram of GoogLeNet . . . 178
9.25.5 Why GoogLeNet is Important . . . 178
9.26 Summary . . . 179
9.27 Comparison of LeNet, AlexNet, VGG16, and GoogLeNet . . . 179
9.27.1 Architecture-Level Comparison . . . 180
9.27.2 Simple Example-Based Understanding . . . 180
9.27.3 Short Memory Summary . . . 181
9.28 ResNet: Deep CNN with Residual Connections . . . 181
9.28.1 Main Idea of Residual Learning . . . 181
9.28.2 Why Deep Networks Need Residual Connections . . . 182
9.28.3 Parts of a Basic ResNet Block . . . 182
9.28.4 Basic Residual Block . . . 182
9.28.5 Bottleneck Residual Block . . . 183
9.28.6 Identity Shortcut and Projection Shortcut . . . 183
9.28.7 How Bottleneck Blocks Repeat in ResNet . . . 183
9.28.8 TikZ Diagram: Repeating Bottleneck Blocks in ResNet . . . 184
9.28.9 ResNet Layer-by-Layer Summary . . . 185
9.28.10 Common ResNet Variants . . . 186
9.28.11 TikZ Flow Diagram of a Basic Residual Block . . . 187
9.28.12 TikZ Flow Diagram of a Bottleneck Residual Block . . . 188
9.28.13 TikZ Flow Diagram of ResNet . . . 189
9.28.14 Why ResNet is Important . . . 189
9.29 Gabor Filters, Saliency Maps, and Grad-CAM . . . 189
9.29.1 Gabor Filters . . . 190
9.29.2 Saliency Maps . . . 190
9.29.3 Grad-CAM . . . 191
9.29.4 Comparison . . . 192
9.29.5 Short Memory Summary . . . 192
9.30 Upsampling in Convolutional Neural Networks . . . 192
9.30.1 Upsampling . . . 193
9.30.2 Non-Learnable Upsampling . . . 193
9.30.3 Unpooling . . . 193

9.30.4 Learnable Upsampling . . . 194
9.30.5 Transposed Convolution . . . 194
9.30.6 Why It Is Sometimes Called Deconvolution . . . 195
9.30.7 Unpooling versus Transposed Convolution . . . 195
9.30.8 Conceptual Comparison . . . 195

10 Image Segmentation **197**
10.1 Motivation . . . 197
10.2 What is Image Segmentation? . . . 197
10.3 Classification, Detection, and Segmentation . . . 198
10.4 Types of Image Segmentation . . . 198
10.4.1 Semantic Segmentation . . . 198
10.4.2 Instance Segmentation . . . 198
10.4.3 Panoptic Segmentation . . . 199
10.5 Comparison of Segmentation Types . . . 199
10.6 Segmentation as Pixel-Wise Classification . . . 199
10.7 Segmentation Masks . . . 200
10.8 Important Note About Mask Resizing . . . 200
10.9 Encoder-Decoder Structure . . . 201
10.10 TikZ Flow Diagram of Image Segmentation . . . 201
10.11 Fully Convolutional Networks . . . 201
10.12 U-Net . . . 202
10.13 TikZ Flow Diagram of U-Net . . . 202
10.14 Segmentation Loss Functions . . . 202
10.14.1 Pixel-Wise Cross-Entropy Loss . . . 202
10.14.2 Dice Loss . . . 203
10.14.3 Focal Loss . . . 203
10.14.4 IoU Loss . . . 204
10.15 Segmentation Evaluation Metrics . . . 204
10.15.1 Pixel Accuracy . . . 204
10.15.2 Intersection over Union . . . 204
10.15.3 Dice Score . . . 204
10.15.4 Precision, Recall, and F1 Score . . . 205
10.16 Common Segmentation Metrics Summary . . . 205
10.17 Visual Interpretation of Common Segmentation Metrics . . . 207
10.18 Precision–Recall Curve, Average Precision, and Mean Average Precision . . . 209
10.18.1 Precision–Recall Curve . . . 209
10.18.2 Average Precision . . . 210
10.18.3 Mean Average Precision . . . 211
10.18.4 mAP Across Multiple IoU Thresholds . . . 211
10.18.5 mAP at Different IoU Thresholds . . . 212
10.18.6 Relationship Among PR Curve, AP, and mAP . . . 212
10.18.7 Non-Maximum Suppression . . . 213
10.18.8 Conditional Random Field for Segmentation Refinement . . . 214
10.18.9 Summary . . . 218
10.19 Typical Segmentation Training Pipeline . . . 218
10.20 Common Challenges in Image Segmentation . . . 219
10.21 Applications of Image Segmentation . . . 219
10.22 Short Memory Summary . . . 219

11 Vision Transformer (ViT) 221
11.1 Overview . . . 221
11.2 Architectural View of ViT . . . 221
11.3 Patch Embedding and Tokenization . . . 223
11.4 Transformer Depth and Attention Modules . . . 223
11.5 Multi-Head Self-Attention . . . 223
11.6 Computational Scaling . . . 224
11.6.1 Attention Cost . . . 224
11.6.2 MLP Cost . . . 224
11.7 Parameter Impact Summary . . . 224
11.8 Numerical Example . . . 224
11.8.1 Small Example: 16×16 Image . . . 224
11.9 Implications for Detection and Segmentation . . . 226
11.10 Summary . . . 226

12 Swin Transformer 229
12.1 Overview . . . 229
12.2 Architectural View of Swin Transformer . . . 229
12.3 Patch Embedding and Initial Tokenization . . . 231
12.4 Hierarchical Stage Structure . . . 231
12.5 Swin Transformer Blocks . . . 232
12.6 Window-Based Multi-Head Self-Attention . . . 232
12.7 Shifted Window Mechanism . . . 233
12.8 Patch Merging . . . 233
12.9 Computational Scaling . . . 233
12.9.1 Windowed Attention Cost . . . 233
12.9.2 MLP Cost . . . 234
12.10 Parameter Impact Summary . . . 234
12.11 Numerical Example . . . 234
12.11.1 Small Example: 16×16 Image . . . 234
12.12 Implications for Detection and Segmentation . . . 236
12.13 Architectural Comparison with ViT . . . 236
12.14 Summary . . . 238

13 Model Structure Explanation 243
13.1 ResNet-50 . . . 243
13.2 FCN-ResNet50 . . . 249
13.3 DeepLabV3-ResNet50 . . . 253
13.4 Main Difference Between the Models . . . 259
13.5 Chapter Summary . . . 260

14 Fundamental Concepts in Reinforcement Learning 261
14.1 Introduction . . . 261
14.2 Markov Decision Processes . . . 262
14.2.1 State . . . 262
14.2.2 Action . . . 262
14.2.3 Reward . . . 262
14.2.4 Discount Factor . . . 262
14.3 Return and Long-Term Reward . . . 263
14.4 Policy . . . 263
14.4.1 Example . . . 263
14.5 Stochastic Policies . . . 263

14.5.1 Example . . . 264
14.6 Deterministic Policies . . . 264
14.6.1 Example . . . 264
14.7 On-Policy Learning . . . 264
14.7.1 Example . . . 264
14.8 Off-Policy Learning . . . 264
14.8.1 Example . . . 265
14.9 Replay Buffers . . . 265
14.10 Objective Functions in Reinforcement Learning . . . 265
14.10.1 Example . . . 266
14.11 Value Functions . . . 266
14.11.1 State-Value Function . . . 266
14.11.2 Example . . . 266
14.11.3 Action-Value Function . . . 266
14.11.4 Example . . . 266
14.12 Actor-Critic Methods . . . 267
14.12.1 Actor . . . 267
14.12.2 Critic . . . 267
14.12.3 Interaction Between Actor and Critic . . . 267
14.13 Critic Targets . . . 267
14.13.1 Example . . . 268
14.14 Advantage Function . . . 268
14.14.1 Example . . . 268
14.15 Entropy and Exploration . . . 269
14.15.1 Example . . . 269
14.16 Proximal Policy Optimization . . . 269
14.16.1 Motivation . . . 269
14.16.2 Clipped Objective . . . 269
14.16.3 Example . . . 270
14.16.4 Advantages and Limitations . . . 270
14.17 Soft Actor-Critic . . . 270
14.17.1 Motivation . . . 270
14.17.2 Maximum-Entropy Objective . . . 270
14.17.3 Soft Q Target . . . 270
14.17.4 Example . . . 270
14.17.5 Advantages and Limitations . . . 271
14.18 Twin Delayed Deep Deterministic Policy Gradient . . . 271
14.18.1 Motivation . . . 271
14.18.2 Twin Critics . . . 271
14.18.3 Example . . . 271
14.18.4 Delayed Actor Updates . . . 271
14.18.5 Target Policy Smoothing . . . 271
14.18.6 Advantages and Limitations . . . 272
14.19 Optimizers in Reinforcement Learning . . . 272
14.20 Comparison of PPO, SAC, and TD3 . . . 273
14.21 End-to-End Learning Workflow . . . 273
14.22 Summary . . . 274

List of Figures

1.1 Basic autoencoder structure. The encoder maps the input to a compact latent representation, and the decoder reconstructs the desired output from that representation. . . . 1
1.2 Intuition for a hierarchical representation. Different stages encode different levels of detail and abstraction; downsampling is common but is not what defines hierarchy. . . . 3
1.3 Downsampling and upsampling change spatial resolution. An encoder often follows the upper direction toward coarser features, while a decoder often follows the lower direction toward finer resolution. . . . 6
1.4 Conceptual difference between a general skip connection and a residual connection. A skip connection bypasses intermediate computation; a residual connection specifically combines the transformed path with the input, usually by addition. . . 8
1.5 General Monte Carlo idea in machine learning. The same procedure is repeated under controlled randomness, and the collection of outcomes is summarized statistically. . . . 10
1.6 Attention as selective information weighting. The current token or image patch can assign different importance to other tokens or regions before forming a context-aware representation. . . . 13
1.7 A practical encoder-decoder picture combining hierarchical features, downsampling, a bottleneck, upsampling, and skip connections. Individual blocks may also contain residual connections or attention. . . . 14

2.1 Side-by-side comparison of four common activation functions. . . . 17
2.2 A compact conceptual grouping of the activation functions discussed in this chapter. 18
2.3 Side-by-side geometric interpretation of common regression and classification losses. 21
2.4 Conceptual visualizations of focal, Dice, and KL-divergence losses. . . . 22
2.5 Side-by-side conceptual behavior of SGD, Momentum, Adagrad, and Adam. . . . 24
2.6 Conceptual progression from a raw gradient update toward adaptive and large-batch optimization strategies. . . . 25
2.7 Side-by-side visual summary of the mechanics of neural-network training. . . . 27

8.1 Parallel committee architecture. Each member processes the same input, and a combination rule produces the final prediction. . . . 118
8.2 Hard parameter sharing in multi-task learning. The encoder parameters are exactly shared, while the output heads remain task-specific. . . . 122
8.3 Conceptual double-descent curve. Test error may peak near the interpolation threshold and decrease again in the overparameterized regime. . . . 123
8.4 Bootstrap sampling creates a resampled training set and an associated out-of-bag set. . . . 126
8.5 Bagging trains independent members on bootstrap resamples and combines their predictions in parallel. . . . 127

8.6 Boosting is sequential. Each new weak learner is trained using information from the current ensemble error. . . . 129
8.7 Simplified selection guide for bagging, boosting, and general model averaging. . . . 131

9.1 Basic CNN flow diagram in PyTorch tensor order. A CNN applies convolution, activation, and pooling repeatedly to learn hierarchical feature maps. A 1×1 convolution can mix channels and change the number of output feature maps without changing the spatial resolution. . . . 154
9.2 Flow diagram of a basic CNN architecture in PyTorch tensor order. The model takes an input tensor of size $1 \times 28 \times 28$, applies two convolutional layers with 5×5 valid kernels and 2×2 max-pooling, then flattens the resulting feature maps and passes them through fully connected layers for classification. . . . 155
9.3 Basic FCN flow diagram for semantic segmentation in PyTorch tensor order. The encoder extracts hierarchical feature maps using convolution and pooling. A 1×1 convolution maps the learned feature channels to K class-score channels, and upsampling restores the spatial resolution to produce a pixel-wise segmentation map. . . . 156
9.4 Conceptual R-CNN pipeline. Candidate regions are generated first, and each proposed region is independently processed by the CNN before classification and bounding-box refinement. . . . 157
9.5 Conceptual Fast R-CNN architecture. The complete image is processed by the CNN once to obtain a shared feature map. Region proposals are projected onto this feature map and converted to fixed-size representations through RoI pooling before classification and bounding-box regression. . . . 158
9.6 Conceptual Region of Interest (RoI) projected onto a shared CNN feature map. The RoI is specified by its boundary coordinates and can have arbitrary width and height before fixed-size feature extraction. . . . 159
9.7 Conceptual operation of a Region Proposal Network (RPN). The RPN operates on the shared CNN feature map and predicts both objectness scores and bounding-box adjustments, which are used to generate candidate RoIs. . . . 160
9.8 Relationship between the principal Mask R-CNN components and their training objectives. The RPN learns objectness and proposal localization, the detection head learns object classification and final bounding-box refinement, and the mask head learns pixel-level instance segmentation. . . . 166
9.9 Conceptual evolution of the R-CNN family from region-wise CNN processing to shared feature extraction, learnable region proposals, and pixel-level instance mask prediction. . . . 167
9.10 A simple flow diagram of the LeNet-5 architecture. . . . 169
9.11 A simple flow diagram of the AlexNet architecture. . . . 171
9.12 A simple flow diagram of the VGG16 architecture. . . . 175
9.13 A simplified Inception module used in GoogLeNet. Multiple operations are applied in parallel, and their outputs are concatenated along the channel dimension. 177
9.14 A simple flow diagram of the GoogLeNet architecture. . . . 178
9.15 Repeating bottleneck blocks in a ResNet-50 style architecture. The four residual stages repeat bottleneck blocks 3, 4, 6, and 3 times. . . . 184
9.16 A basic ResNet residual block. The input x passes through convolution layers and also skips directly to the addition operation. . . . 187
9.17 A bottleneck ResNet block. The 1×1 convolutions reduce and then expand the channel dimension, while the 3×3 convolution extracts spatial features. . . . 188
9.18 A simple flow diagram of a standard ResNet architecture. . . . 189

10.1 A simple encoder-decoder flow for image segmentation. . . . 201

10.2 A simplified U-Net architecture. Dashed arrows represent skip connections from encoder blocks to decoder blocks. . . . 202
10.3 Conceptual visualizations of pixel accuracy, IoU, mean IoU, and Dice score used in image segmentation. . . . 207
10.4 Conceptual visualizations of precision, recall, F1 score, and Focal Loss. . . . 208
10.5 Conceptual precision–recall curve obtained by varying the prediction confidence threshold. Stronger models maintain high precision over a broad range of recall values. . . . 209
10.6 Conceptual visualization of Average Precision (AP). AP summarizes the precision–recall relationship as the area under the precision–recall curve. A larger area indicates that high precision is maintained over a broader range of recall values. . . 210
10.7 Conceptual visualization of mean Average Precision (mAP). Average Precision is first determined independently for each evaluated class, and the class-specific AP values are subsequently averaged to obtain mAP. The numerical values shown are illustrative. . . . 211
10.8 Conceptual interpretation of mAP evaluated at different IoU thresholds. Lower IoU thresholds permit greater differences between predicted and ground-truth regions, whereas higher IoU thresholds require increasingly accurate spatial localization. . . . 212
10.9 Conceptual calculation of $\text{mAP}_{50:95}$. Performance is evaluated at multiple IoU thresholds from 0.50 to 0.95 in increments of 0.05, and the resulting values are averaged to obtain an overall measure that incorporates both permissive and stringent localization requirements. . . . 212
10.10 Conceptual relationship among confidence thresholds, the precision–recall curve, Average Precision, and mean Average Precision. . . . 213
10.11 Conceptual illustration of Non-Maximum Suppression. Multiple highly overlapping predictions may initially correspond to the same object. NMS retains the highest-confidence prediction and suppresses redundant predictions whose overlap exceeds the specified NMS threshold. . . . 214
10.12 Conceptual use of a Conditional Random Field for semantic segmentation. The CNN produces an initial pixel-wise prediction that supplies unary evidence. The CRF combines this evidence with spatial and appearance relationships between pixels through pairwise potentials to produce a spatially refined segmentation. . 217

11.1 Architectural flow of the Vision Transformer. ViT converts the image into a flat sequence of patch tokens and applies global self-attention across all tokens in every encoder layer. . . . 222
11.2 Small ViT attention example. A 16×16 image with 2×2 patches produces an 8×8 token grid. Since ViT uses global self-attention, the attention matrix has size 64×64. . . . 225
11.3 High-level architecture of the Vision Transformer (ViT). An input image is split into non-overlapping patches, embedded into a token sequence, and processed by a Transformer encoder stack of depth L. Each encoder layer contains exactly one multi-head self-attention (MHSA) module (so the number of attention modules is L), and each MHSA uses h attention heads. . . . 227
11.4 Hydra/tree-style view of ViT without overflow. Tokens pass through an encoder layer repeated $\times L$. Each layer contains exactly one MHSA (so the number of attention modules is L) with h heads and one MLP block. The MHSA splits into h heads and merges back via concatenation to dimension D. . . . 227
11.5 Tree-style view of ViT. The encoder stack has depth L; each layer contains one MHSA (therefore total attention modules $= L$) with h heads and an MLP block. 228

12.1 Architectural flow of the Swin Transformer. Swin keeps image tokens as a two-dimensional feature map, applies attention inside local windows, uses shifted windows for cross-window interaction, and builds a hierarchical feature representation through patch merging. . . . 230
12.2 Small Swin Transformer attention example. A 16×16 image with 2×2 patches produces an 8×8 token grid. With a 2×2 window size, the grid contains 4×4 windows, and attention is computed locally inside each window. . . . 235
12.3 High-level architecture of the Swin Transformer. An input image is partitioned into patches and embedded into a 2D token grid. The token grid is processed by a hierarchical sequence of stages. Within each stage, Swin blocks alternate between regular window attention (W-MSA) and shifted-window attention (SW-MSA). Patch merging layers between stages reduce spatial resolution and increase channel dimension. . . . 238
12.4 Tree-style view of one Swin Transformer stage. Each stage contains L_s Swin blocks with alternating regular window attention (W-MSA) and shifted-window attention (SW-MSA). Patch merging connects one stage to the next by reducing spatial resolution and increasing feature dimension. . . . 239
12.5 Flow of Swin Transformer across stages. The model begins with patch embedding, then processes the token grid through a hierarchy of stages. Each stage applies alternating windowed and shifted-window attention blocks, while patch merging progressively reduces spatial resolution and increases channel dimension. 239
12.6 Overall flow of a Swin Transformer backbone. Here, p denotes patch size and C denotes the base embedding dimension. . . . 240
12.7 Internal structure of Swin stage k, consisting of a stack of Swin blocks. . . . 241
12.8 Internal flow of a Swin Transformer block. . . . 241
12.9 Example Swin Transformer backbone configuration with $embed_dim = 96$, $depths = (2, 2, 6, 2)$, and $num_heads = (3, 6, 12, 24)$. . . . 242

14.1 The reinforcement learning interaction loop. The agent selects an action from the current state, and the environment returns a reward and the next state. . . . 261
14.2 Two independent ways to classify reinforcement learning algorithms: how the policy represents actions and how training data are reused. . . . 265
14.3 Actor–critic architecture. The actor proposes an action, while the critic evaluates the state or state–action pair and supplies a learning signal to improve the actor. 267
14.4 Relationship among the temporal-difference target, the learned action value, the state-value baseline, and the advantage estimate. . . . 268
14.5 Conceptual decision map distinguishing PPO, SAC, and TD3. The diagram is a learning aid rather than a universal algorithm-selection rule. . . . 269
14.6 High-level training pipelines for PPO, SAC, and TD3. PPO uses newly collected on-policy rollouts, whereas SAC and TD3 reuse samples from a replay buffer. . . . 272

List of Tables

2.1 Forward Pass Computations and Data Structures . . . 28
2.2 Comprehensive Backpropagation Steps . . . 29
2.3 Weight and Bias Update Steps . . . 29

3.1 Forward propagation for a single-neuron network . . . 33
3.2 Backpropagation for the output layer (layer 2) . . . 33
3.3 Backpropagation for the hidden layer (layer 1) . . . 34
3.4 Forward propagation for a three-layer network with three different activation functions . . . 34
3.5 Backpropagation through the output layer (layer 3, sigmoid) . . . 35
3.6 Backpropagation through hidden layer 2 (layer 2, tanh) . . . 35
3.7 Backpropagation through hidden layer 1 (layer 1, ReLU) . . . 35
3.8 Quantities stored during forward propagation and their role in backpropagation . . . 38
3.9 Product types appearing in forward and backward propagation . . . 38
3.10 Comparison of PyTorch behavior during training and inference . . . 47

4.1 Forward pass for a three-layer network: ReLU $\rightarrow$ tanh $\rightarrow$ sigmoid . . . 57
4.2 Chronological calculations with explicit Jacobians J_1, J_2, J_3 . . . 62
4.3 Examples of loss functions and the corresponding initial backprop signal $dv_7 = \partial\mathcal{L}/\partial v_7$. . . . 63
4.4 Relationship between parameters and gradient shapes . . . 65
4.5 Distinction between local derivatives, deltas, and gradients in backpropagation . . . 65
4.6 Commutativity properties of common linear algebra operations . . . 66
4.7 Linear algebra operations used in forward and backward propagation . . . 66

5.1 Growth of polynomial coefficients with dimensionality . . . 73
5.2 Exponential growth of grid cells . . . 74
5.3 Natural images versus random noise . . . 75

6.1 Classification of stationary points using first- and second-order derivatives . . . 80
6.2 Comparison of batch, stochastic, and mini-batch gradient descent . . . 83
6.3 Comparison of normalization methods in neural networks . . . 87
6.4 Comparison of Batch Normalization and Layer Normalization . . . 88

8.1 Main ways to combine multiple models. . . . 117
8.2 Interpretation of bias and variance. . . . 119
8.3 Expected-error comparison between an individual model and an averaged committee. . . . 121
8.4 Comparison of hard and soft parameter sharing. . . . 123
8.5 Behavior across model-capacity regimes. . . . 124
8.6 Example bootstrap samples drawn from the same dataset. . . . 125
8.7 Difference between bootstrap sampling and bagging. . . . 126

8.8 Comparison of bagging and boosting. . . . 130
8.9 Summary of the main concepts. . . . 132

9.1 Comparison between 2D cross-correlation and true 2D convolution. . . . 136
9.2 Summary of 2D convolution and 2D cross-correlation. . . . 137
9.3 Examples of convolution layers with multiple input and output channels. . . . 141
9.4 Summary of the architectural evolution from R-CNN to Mask R-CNN. . . . 167
9.5 Short summary of the LeNet-5 architecture. . . . 168
9.6 Short summary of the AlexNet architecture. . . . 170
9.7 Short summary of the VGG16 architecture. . . . 173
9.8 Short summary of the GoogLeNet architecture. . . . 177
9.9 Comparison of classic CNN architectures. . . . 179
9.10 Architecture-level comparison of LeNet, AlexNet, VGG16, and GoogLeNet. . . . 180
9.11 Short summary of a standard ResNet architecture. . . . 185
9.12 Common ResNet variants. . . . 186
9.13 Simple interpretation of Gabor filter parameters. . . . 190
9.14 Comparison of Gabor filters, saliency maps, and Grad-CAM. . . . 192
9.15 Comparison of common spatial upsampling operations in CNN decoders. . . . 195

10.1 Comparison of image classification, object detection, and image segmentation. . . . 198
10.2 Comparison of semantic, instance, and panoptic segmentation. . . . 199
10.3 Example class encoding for a segmentation mask. . . . 200
10.4 Correct resizing methods for images and segmentation masks. . . . 200
10.5 Common evaluation metrics for image segmentation. . . . 205
10.6 Summary of precision–recall-based evaluation metrics. . . . 218
10.7 Example applications of image segmentation. . . . 219

11.1 Impact of Vision Transformer architectural parameters. . . . 224
11.2 Impact of the MLP ratio in Vision Transformers. . . . 226

12.1 Impact of Swin Transformer architectural parameters. . . . 234
12.2 Architectural comparison between ViT and Swin Transformer. . . . 237
12.3 Impact of the MLP ratio in Swin Transformer. . . . 238

14.1 Example policy output. . . . 263
14.2 Example stochastic policy. . . . 264
14.3 Replay buffer samples. . . . 265
14.4 Policy comparison based on average reward. . . . 266
14.5 Example state values. . . . 266
14.6 Example action values. . . . 266
14.7 Example critic evaluation. . . . 267
14.8 Comparison of PPO, SAC, and TD3. . . . 273

Chapter 1

Common Neural Network Terms and Intuition

1.1 Why These Terms Matter

Many terms in machine learning are used frequently but are not always explained clearly. Words such as *autoencoder*, *skip connection*, *residual connection*, *latent space*, and *hierarchical encoder* appear in papers, lectures, and codebases, yet they can feel vague at first. This chapter gives practical meanings for these terms using direct language and simple intuition.

1.2 Autoencoder

An **autoencoder** is a neural network designed to compress an input into a smaller internal representation and then reconstruct the original input from that representation.

It usually has two main parts:

- **Encoder:** maps the input into a compact representation
- **Decoder:** reconstructs the input from that compact representation

The middle compressed representation is often called the **latent representation** or **latent code**.

1.2.1 Basic Idea

If an input image is given to the autoencoder, the encoder tries to extract the most important information and discard unnecessary detail. The decoder then uses this compact information to rebuild the image. Because the model must pass information through a limited internal representation, it is forced to learn useful structure in the data.

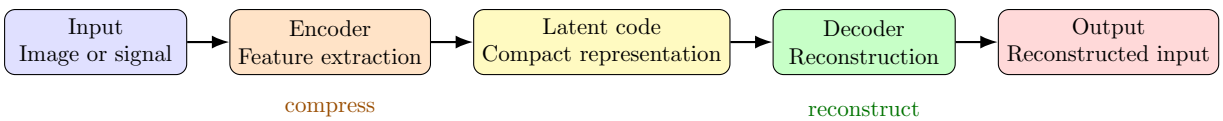


Figure 1.1: Basic autoencoder structure. The encoder maps the input to a compact latent representation, and the decoder reconstructs the desired output from that representation.

1.2.2 Why Autoencoders Are Useful

Autoencoders are commonly used for:

- dimensionality reduction
- denoising
- feature extraction
- anomaly detection
- representation learning

1.2.3 Common Variants

- **Vanilla autoencoder:** reconstructs the original input
- **Denoising autoencoder:** takes a noisy input and learns to reconstruct a clean version
- **Sparse autoencoder:** encourages only a small number of latent units to be active
- **Variational autoencoder (VAE):** learns a probabilistic latent space and is useful for generation

1.3 Encoder

An **encoder** is the part of the network that transforms the raw input into a more informative internal representation.

In image models, the encoder often:

- extracts useful visual features
- may reduce spatial resolution
- often increases semantic abstraction
- learns patterns such as edges, shapes, texture, and object-level structure

The encoder is often viewed as the feature extraction stage of the model.

1.3.1 Important Clarification

An encoder does **not always** have to reduce spatial resolution. Its main role is to convert the input into learned features. Many vision encoders do become hierarchical and use downsampling, but that is a common design choice rather than a strict requirement.

1.4 Decoder

A **decoder** is the part of the network that takes the encoded representation and converts it into the desired output.

Depending on the application, the decoder may produce:

- a reconstructed image
- a segmentation mask
- a denoised signal
- a generated sequence

In image tasks, decoders often increase spatial resolution gradually and recover fine detail.

1.5 Hierarchical Representation

The word **hierarchical** means that the model contains multiple levels or stages of representation. These levels usually differ in spatial resolution, semantic abstraction, or both.

A hierarchical design often looks like this:

$$\text{Fine detail} \rightarrow \text{intermediate structure} \rightarrow \text{coarse semantic representation}$$

So the word *hierarchical* does not by itself mean downsampling or upsampling. Instead, it means that the model processes information across multiple levels.

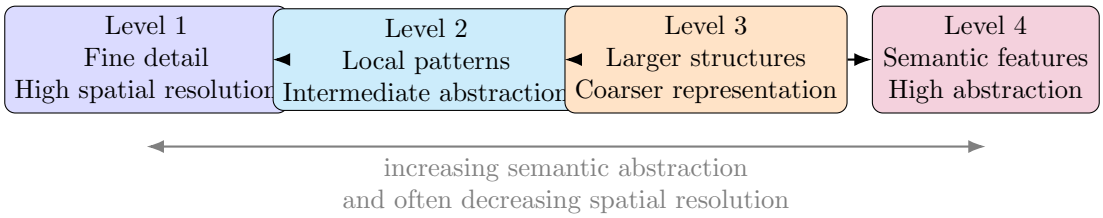


Figure 1.2: Intuition for a hierarchical representation. Different stages encode different levels of detail and abstraction; downsampling is common but is not what defines hierarchy.

1.5.1 Hierarchical Encoder

A **hierarchical encoder** is an encoder that builds features at several levels, often from high resolution to low resolution.

For example:

$$256 \times 256 \rightarrow 128 \times 128 \rightarrow 64 \times 64$$

At the same time, the feature channels often increase, and the features become more abstract.

This type of encoder is common in:

- CNN backbones such as ResNet
- Swin Transformer
- U-Net style encoders

1.5.2 Hierarchical Decoder

A **hierarchical decoder** is a decoder that reconstructs information step by step across multiple levels, often from coarse resolution back to fine resolution.

For example:

$$64 \times 64 \rightarrow 128 \times 128 \rightarrow 256 \times 256$$

This type of decoder is common in segmentation, reconstruction, and super-resolution networks.

1.5.3 Key Distinction

- **Hierarchical** means multi-stage or multi-level
- **Downsampling** and **upsampling** are operations often used to create that hierarchy

Thus, hierarchy describes the structure, while downsampling and upsampling describe the transformation between stages.

1.5.4 Multi-Stage and Multi-Level Representations

The terms **multi-stage** and **multi-level** are closely related, but they do not mean exactly the same thing.

Multi-stage. A model is called **multi-stage** when it is organized into several sequential processing stages. Each stage takes the output of the previous stage, performs further transformation, and passes the result to the next stage. In this sense, the word *stage* refers to the architectural organization of the network.

For example, an image encoder may contain:

- Stage 1: high-resolution feature extraction
- Stage 2: reduced-resolution feature extraction
- Stage 3: deeper semantic feature extraction
- Stage 4: the most compressed and abstract representation

Thus, **multi-stage** emphasizes that the network is built as a sequence of distinct processing blocks or feature extraction steps.

Multi-level. A model is called **multi-level** when it learns or represents information at different levels of detail, scale, or abstraction. In this sense, the word *level* refers to the nature of the learned representation rather than only the arrangement of blocks.

For example, in an image model:

- shallow levels may capture edges, corners, and texture
- intermediate levels may capture shapes, parts, or local patterns
- deeper levels may capture object structure or semantic meaning

Thus, **multi-level** emphasizes that different parts of the network contain different kinds of information.

Relationship Between the Two. In many practical neural network architectures, a model is both **multi-stage** and **multi-level**. This means that the network has several sequential stages, and each stage produces representations at a different level of detail or abstraction.

For example, a hierarchical encoder may process an image as follows:

$$\text{Input} \rightarrow \text{Stage 1} \rightarrow \text{Stage 2} \rightarrow \text{Stage 3} \rightarrow \text{Stage 4}$$

At the same time, the representations may evolve as:

$$\text{fine detail} \rightarrow \text{local structure} \rightarrow \text{coarse structure} \rightarrow \text{semantic abstraction}$$

Therefore:

- **multi-stage** describes how the model is organized
- **multi-level** describes what kind of representations are learned across those stages

Example in Vision Models. In convolutional neural networks, Swin Transformers, and U-Net-style models, the encoder is often both multi-stage and multi-level. Early stages tend to preserve higher spatial resolution and low-level detail, while deeper stages usually operate at lower resolution and encode higher-level semantic information.

Similarly, a decoder may also be multi-stage and multi-level, but in the reverse direction. It may progressively reconstruct spatial resolution while transforming coarse semantic features into finer output detail.

Practical Interpretation. A useful way to remember the distinction is:

- **Multi-stage:** several sequential processing steps
- **Multi-level:** several levels of feature meaning, detail, or abstraction

In many modern vision architectures, these two ideas appear together, which is why the terms are often used side by side.

1.6 Latent Space

The **latent space** is the hidden internal representation learned by the network. It lies between the input and the output.

For an autoencoder:

$$\text{Input} \rightarrow \text{Encoder} \rightarrow \text{Latent Representation} \rightarrow \text{Decoder} \rightarrow \text{Output}$$

The latent representation is important because it captures the essential structure of the data in a more compact form than the original input.

If the model has learned meaningful features, then similar inputs often produce nearby points in latent space.

1.7 Bottleneck

A **bottleneck** is a stage in the network where information is forced into a compressed form.

For example, an image with many pixels may be mapped into a much smaller latent vector. This restriction encourages the model to preserve only the most useful information.

The term *bottleneck* can also appear in residual networks, where it may refer to a block that temporarily reduces the number of channels before expanding them again to reduce computation.

1.8 Feature Map

A **feature map** is the output of a convolutional or spatial processing layer. In image models, a feature map is usually represented as:

$$\text{Channels} \times \text{Height} \times \text{Width}$$

Each channel may respond to a different pattern, such as:

- edges
- corners
- texture

- shape fragments
- object-level structure

Earlier feature maps tend to contain lower-level information, while deeper feature maps often contain more abstract and semantic information.

1.9 Downsampling

Downsampling means reducing the spatial resolution of the data.

For example:

$$256 \times 256 \rightarrow 128 \times 128 \rightarrow 64 \times 64$$

This is useful because it:

- reduces memory usage
- reduces computation
- increases the receptive field
- helps the network capture higher-level structure

Common methods include pooling, strided convolution, and patch merging in transformer-based models.

A disadvantage of downsampling is that fine spatial detail may be lost.

1.10 Upsampling

Upsampling means increasing spatial resolution.

For example:

$$32 \times 32 \rightarrow 64 \times 64 \rightarrow 128 \times 128$$

This is commonly used in:

- image reconstruction
- segmentation
- super-resolution
- decoder architectures

Common upsampling methods include interpolation, transposed convolution, and learned decoder blocks.

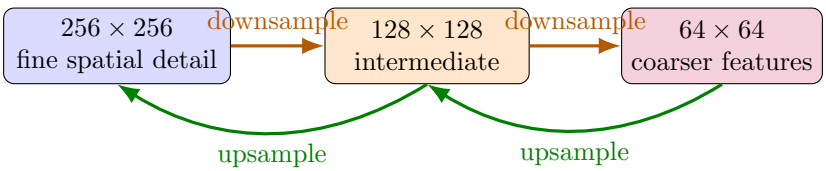


Figure 1.3: Downsampling and upsampling change spatial resolution. An encoder often follows the upper direction toward coarser features, while a decoder often follows the lower direction toward finer resolution.

1.11 Relationship Between Hierarchy, Downsampling, and Upsampling

These terms are closely related but they do not mean the same thing.

- **Hierarchy:** the model has multiple levels or stages
- **Downsampling:** spatial size is reduced
- **Upsampling:** spatial size is increased

A hierarchical encoder usually uses downsampling to move from fine detail to coarse semantics. A hierarchical decoder usually uses upsampling to move from coarse semantics back to fine detail.

Therefore, hierarchy refers to the overall organization of stages, while downsampling and upsampling are the operations that often connect those stages.

1.12 Skip Connection

A **skip connection** means that information from an earlier layer is passed directly to a later layer instead of flowing only through every intermediate layer in sequence.

Without a skip connection, the flow is:

$$\text{Layer } 1 \rightarrow \text{Layer } 2 \rightarrow \text{Layer } 3$$

With a skip connection, the flow may also include:

$$\text{Layer } 1 \rightarrow \text{Layer } 3$$

This helps preserve important information from earlier stages.

1.12.1 Why Skip Connections Help

Skip connections are useful because they can:

- preserve fine detail
- improve gradient flow during training
- reduce information loss
- make deep networks easier to optimize

1.12.2 Skip Connections in U-Net

In U-Net style models, skip connections usually transfer encoder feature maps directly to matching decoder stages. This allows the decoder to combine:

- deep semantic information from later layers
- fine spatial detail from earlier layers

This is especially important in segmentation and reconstruction problems.

1.13 Residual Connection

A **residual connection** is a special type of skip connection in which the input is added directly to the output of a learned transformation.

The basic form is:

$$y = F(x) + x$$

where:

- x is the input
- $F(x)$ is the transformation learned by the block
- y is the final output

Instead of learning the full mapping directly, the block learns the **residual**, which is the correction or change applied to the input.

1.13.1 Why Residual Connections Matter

Residual connections help because they:

- improve gradient propagation
- reduce optimization difficulty in deep networks
- preserve input information
- make it easier to train very deep architectures such as ResNet

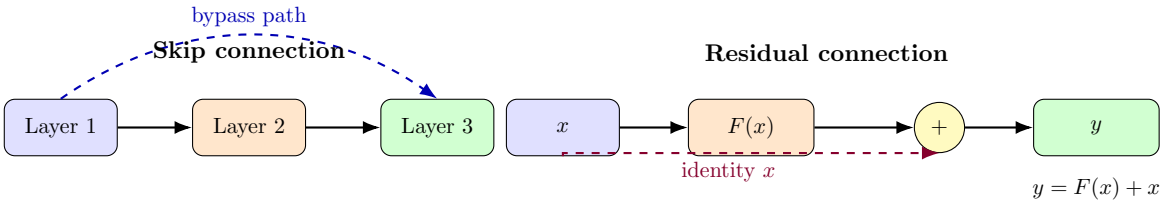


Figure 1.4: Conceptual difference between a general skip connection and a residual connection. A skip connection bypasses intermediate computation; a residual connection specifically combines the transformed path with the input, usually by addition.

1.14 Difference Between Skip Connection and Residual Connection

These terms are related, but they are not exactly the same.

- **Skip connection:** a broad term for passing information from an earlier layer to a later layer
- **Residual connection:** a specific type of skip connection where the earlier input is added to the later output

In practice:

- concatenating encoder and decoder features in U-Net is a skip connection

- computing $F(x) + x$ in ResNet is a residual connection

Thus, every residual connection is a form of skip connection, but not every skip connection is residual.

1.15 Shortcut Connection

The term **shortcut connection** is often used interchangeably with **skip connection**. In many papers, a shortcut simply means that the signal is allowed to bypass one or more layers.

In residual networks, the shortcut path is the branch that carries the original input forward so that it can be added to the transformed output.

1.16 Reconstruction Loss

A **reconstruction loss** measures how close the model output is to the target it is trying to reconstruct.

In image restoration, common reconstruction losses include:

- Mean Squared Error (MSE)
- Mean Absolute Error (L1 loss)
- Charbonnier loss
- Structural Similarity based losses

For autoencoders and denoising models, reconstruction loss guides the model toward outputs that match the desired clean or original target.

1.17 Regularization

Regularization refers to methods that encourage the model to generalize better instead of simply memorizing the training data.

Examples include:

- weight decay
- dropout
- sparsity penalties
- early stopping
- data augmentation

In inverse problems and image restoration, regularization can also refer to adding a penalty term that encourages desirable structure such as smoothness, sparsity, or consistency with learned priors.

1.18 Monte Carlo Experiments and Monte Carlo Validation

The term **Monte Carlo** refers to the use of repeated random sampling in order to estimate a quantity of interest. In machine learning, Monte Carlo methods are often used when the model, the data pipeline, or the evaluation procedure contains some form of randomness. Instead of relying on a single run or a single prediction, the same process is repeated multiple times, and the resulting outputs are summarized using statistics such as the mean, variance, or confidence intervals.

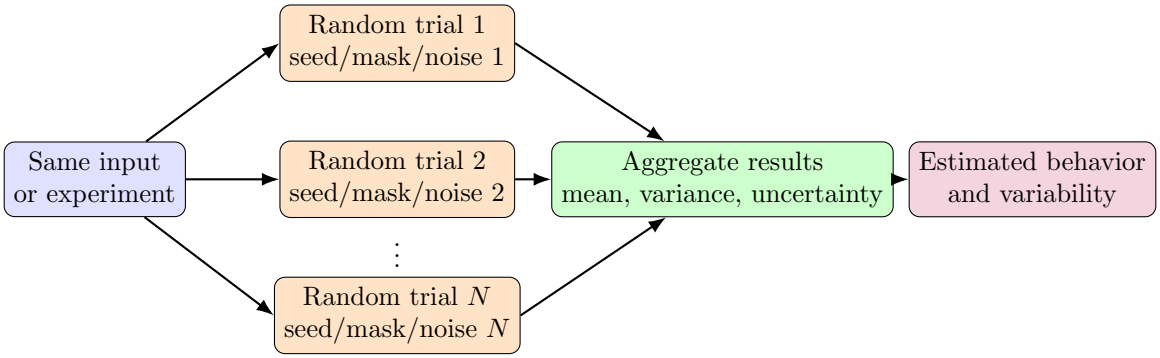


Figure 1.5: General Monte Carlo idea in machine learning. The same procedure is repeated under controlled randomness, and the collection of outcomes is summarized statistically.

1.18.1 Monte Carlo Experiment

A **Monte Carlo experiment** is an experimental procedure in which randomness is intentionally sampled multiple times in order to study average behavior, variability, robustness, or uncertainty.

In machine learning, this usually means that the same model or experimental setup is run multiple times while allowing one or more stochastic elements to vary. Typical sources of randomness include:

- random weight initialization
- random data shuffling
- random train-validation splits
- random augmentation choices
- random dropout masks
- random noise realizations

The main purpose of a Monte Carlo experiment is not simply to produce one final score, but to understand how sensitive the outcome is to randomness. For example, if a model is trained several times with different random seeds, the collection of results can be used to estimate the mean performance and the variability across runs.

Thus, a Monte Carlo experiment provides information about:

- average performance
- stability across repeated runs
- sensitivity to randomness
- uncertainty in the reported result

1.18.2 Monte Carlo Validation

Monte Carlo validation is a validation procedure in which prediction or evaluation is performed repeatedly under stochastic conditions, and the repeated outputs are then aggregated.

The idea is that, instead of using a single deterministic validation pass, the same validation sample or the same validation set may be evaluated multiple times while allowing some random mechanism to remain active. The repeated predictions can then be summarized, for example by computing:

- the mean prediction
- the variance of predictions
- uncertainty measures
- averaged validation metrics

A common example is **Monte Carlo dropout**, in which dropout remains active during validation or inference. The same input is passed through the model multiple times, producing slightly different outputs because different dropout masks are sampled in each pass. The resulting set of predictions can be used to estimate both the expected prediction and the uncertainty of the model.

More generally, Monte Carlo validation may involve repeated stochastic evaluation through:

- dropout at inference time
- stochastic depth or drop path
- random augmentation at test time
- repeated noisy perturbations of the input

1.18.3 Monte Carlo Validation Versus Deterministic Validation

The difference between deterministic validation and Monte Carlo validation is important.

Deterministic validation. In **deterministic validation**, the model is evaluated using a fixed forward process. For the same trained model and the same validation input, the output remains the same each time, assuming no stochastic mechanism is active.

Monte Carlo validation. In **Monte Carlo validation**, the model is evaluated repeatedly under controlled randomness. For the same trained model and the same validation input, the outputs may differ slightly from one pass to another because some stochastic component is intentionally allowed to remain active.

Therefore:

- deterministic validation gives one fixed prediction path
- Monte Carlo validation gives a distribution of predictions

1.18.4 Role of Random Seeds

A **random seed** is not the same as a Monte Carlo method, but it is closely related.

A seed initializes the random number generator so that a stochastic process can be reproduced. In practice, the seed may affect:

- weight initialization
- data order during shuffling
- random augmentation choices
- stochastic masks such as dropout

Thus:

- the **seed** controls the randomness
- the **Monte Carlo procedure** uses repeated sampling from that randomness

If several runs are performed with different seeds, the experiment can measure how performance changes across repeated random trials.

1.18.5 Practical Interpretation

A useful way to remember the distinction is:

- **Monte Carlo experiment:** repeated stochastic runs to estimate average behavior and variability
- **Monte Carlo validation:** repeated stochastic evaluation of validation predictions
- **Deterministic validation:** one fixed evaluation path with no active randomness during inference

1.18.6 Why Monte Carlo Methods Are Useful

Monte Carlo methods are useful because they help answer questions that a single deterministic run cannot answer clearly. For example:

- Is the reported performance stable across random trials?
- Does the model behave consistently?
- How sensitive is the result to initialization or sampling?
- How uncertain is the prediction on a given input?

For this reason, Monte Carlo methods are often used in uncertainty estimation, robustness analysis, repeated-run benchmarking, and stochastic validation studies.

1.19 Attention

Attention is a mechanism that allows the model to focus more strongly on the most relevant parts of the input.

Instead of treating all regions equally, attention learns which parts of the input are more important for the current prediction.

In image models, attention can help relate distant image regions. In language models, attention helps one token use information from other relevant tokens.

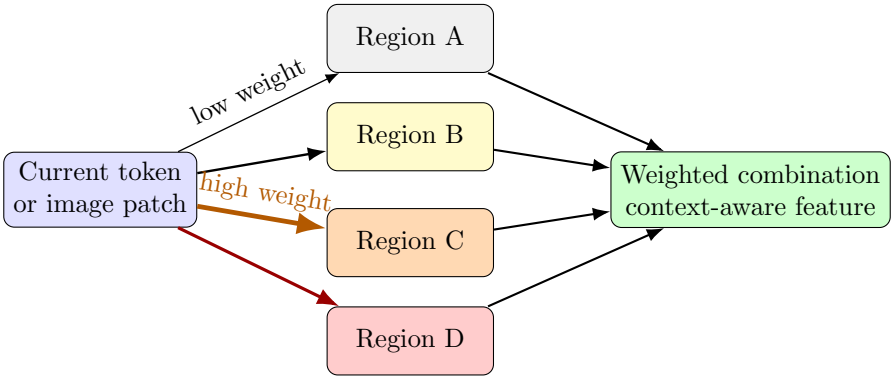


Figure 1.6: Attention as selective information weighting. The current token or image patch can assign different importance to other tokens or regions before forming a context-aware representation.

1.20 Transformer

A **transformer** is a neural network architecture built mainly around attention mechanisms.

Transformers are powerful because they can model long-range interactions effectively.

In computer vision:

- **Vision Transformer (ViT)** treats images as sequences of patches
- **Swin Transformer** processes images using local windows and shifted windows

Transformers are especially useful when global context matters.

1.21 Practical Picture of a Typical Image Model

Many image models combine several of the terms described above. A typical segmentation or restoration model may work as follows:

- the **encoder** extracts increasingly informative features
- **downsampling** may reduce spatial size
- the **hierarchical encoder** builds multiple levels of representation
- the **bottleneck** contains the deepest representation
- the **decoder** upsamples the features
- the **hierarchical decoder** reconstructs detail stage by stage
- **skip connections** transfer useful spatial detail from encoder to decoder

- **residual connections** improve optimization inside blocks
- the final layer produces the output prediction

This overall structure is common in segmentation, denoising, super-resolution, and reconstruction tasks.

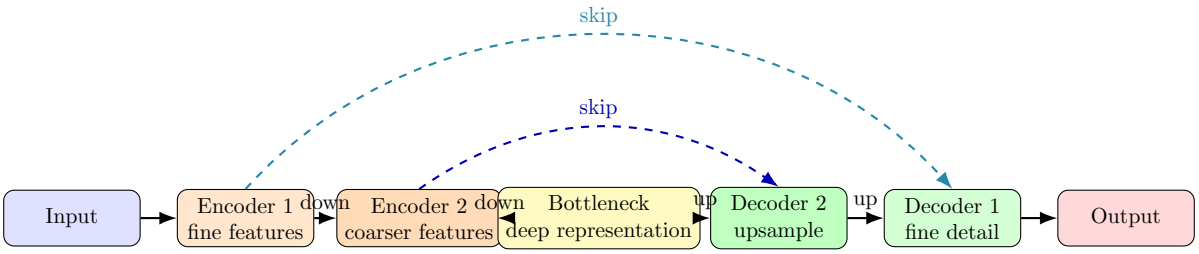


Figure 1.7: A practical encoder-decoder picture combining hierarchical features, downsampling, a bottleneck, upsampling, and skip connections. Individual blocks may also contain residual connections or attention.

1.22 Summary

The following short descriptions can help as a quick reference:

- **Autoencoder:** compresses an input and reconstructs it
- **Encoder:** extracts learned features from the input
- **Decoder:** reconstructs or predicts the final output
- **Hierarchical:** organized into multiple levels or stages
- **Hierarchical encoder:** multi-stage encoder, often using downsampling
- **Hierarchical decoder:** multi-stage decoder, often using upsampling
- **Latent space:** hidden internal representation of the data
- **Bottleneck:** compressed stage that restricts information flow
- **Feature map:** spatial tensor of learned features
- **Downsampling:** reduces spatial resolution
- **Upsampling:** increases spatial resolution
- **Skip connection:** passes early features directly to later layers
- **Residual connection:** adds the input back to the transformed output
- **Regularization:** encourages better generalization
- **Attention:** helps the model focus on important information
- **Transformer:** architecture centered on attention

Chapter 2

Activation Functions, Loss Functions, Optimizers, and Training Mechanics

2.1 Activation Functions

2.1.1 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Applications: Binary classification outputs, logistic regression, simple neural networks. **Advantages:**

- Smooth, differentiable.
- Maps input to $(0, 1)$ range for probability interpretation.

Disadvantages:

- Vanishing gradients for large $|x|$.
- Non-zero-centered outputs slow convergence.

2.1.2 Tanh

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Applications: Hidden layers of RNNs, MLPs. **Advantages:**

- Zero-centered outputs.
- Stronger gradients than Sigmoid.

Disadvantages:

- Still suffers from vanishing gradients.
- Costlier than ReLU.

2.1.3 ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Applications: CNNs, fully connected layers in deep networks. **Advantages:**

- Simple and efficient.
- Sparse activations reduce computation.
- Mitigates vanishing gradient problem.

Disadvantages:

- "Dying ReLU" problem for negative inputs.
- Unbounded positive outputs.

2.1.4 Leaky ReLU

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

Applications: CNNs where ReLU causes dead neurons. **Advantages:** Avoids dying ReLU; small slope for negatives. **Disadvantages:** Requires α tuning.

2.1.5 Softmax

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

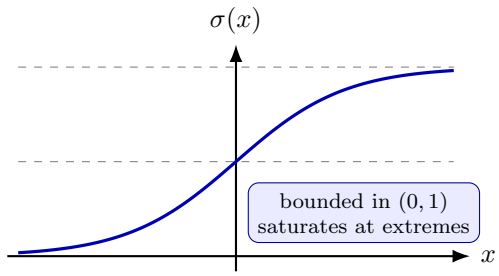
Applications: Multiclass classification output layers. **Advantages:** Converts logits to probabilities. **Disadvantages:** Sensitive to large logits; saturation slows learning.

2.1.6 GELU

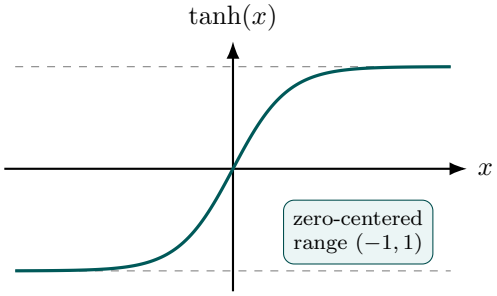
$$\text{GELU}(x) = x\Phi(x)$$

Applications: Transformers, BERT, Vision Transformers. **Advantages:** Smooth and probabilistic activation. **Disadvantages:** More computation-heavy.

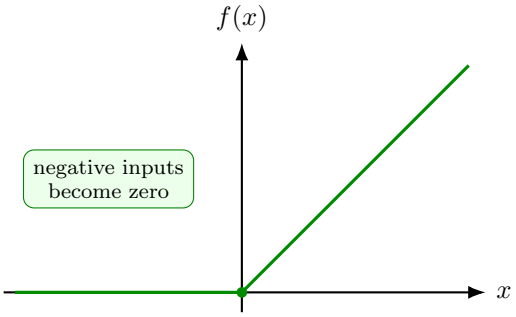
2.2 Visual Interpretation of Activation Functions



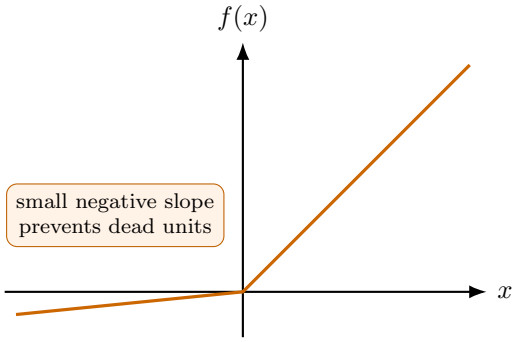
(a) Sigmoid smoothly maps real-valued inputs to the interval $(0, 1)$.



(b) Tanh is zero-centered but still saturates for large positive or negative inputs.

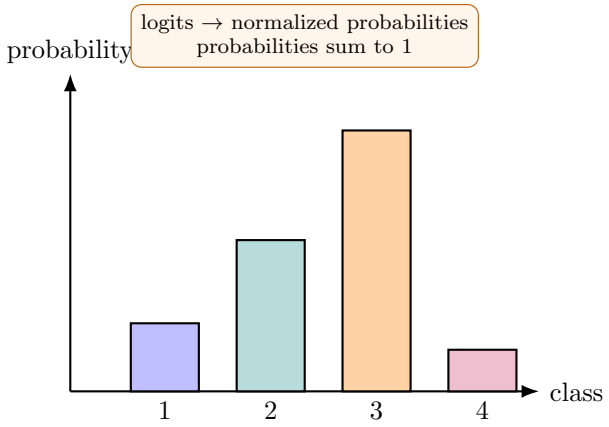


(c) ReLU keeps positive inputs unchanged and clips negative inputs to zero.

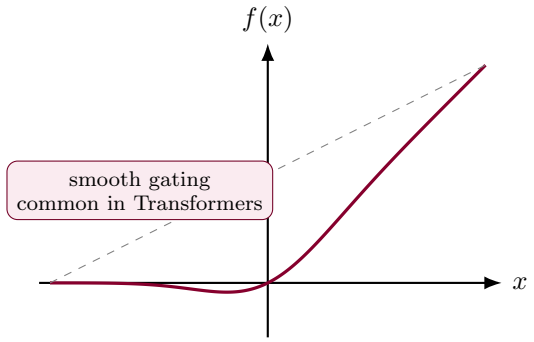


(d) Leaky ReLU retains a small gradient for negative inputs.

Figure 2.1: Side-by-side comparison of four common activation functions.



(a) Softmax converts a vector of logits into a categorical probability distribution.



(b) GELU smoothly gates inputs rather than applying a hard threshold at zero.

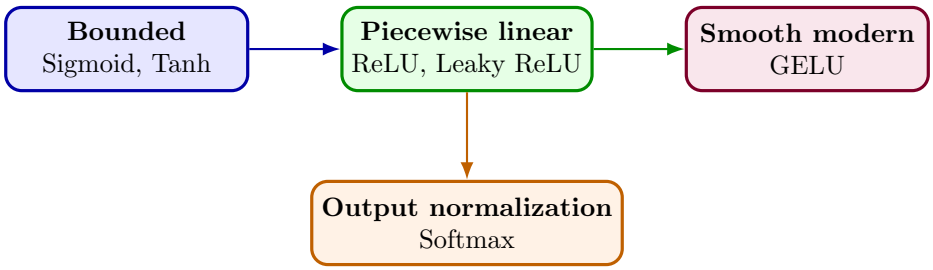


Figure 2.2: A compact conceptual grouping of the activation functions discussed in this chapter.

2.3 Loss Functions

2.3.1 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Applications: Regression problems, autoencoders. **Advantages:** Penalizes large errors heavily; convex. **Disadvantages:** Sensitive to outliers.

2.3.2 Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

Applications: Robust regression. **Advantages:** Robust to outliers. **Disadvantages:** Non-differentiable at 0.

2.3.3 Huber Loss

$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2, & |e| \le \delta \\ \delta(|e| - \frac{1}{2}\delta), & |e| > \delta \end{cases}$$

Applications: Regression with moderate outliers. **Advantages:** Combines smoothness (MSE) and robustness (MAE). **Disadvantages:** Requires tuning δ.

2.3.4 Cross Entropy Loss

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log p_{ik}$$

Applications: Classification (binary, multiclass, multi-label). **Advantages:** Probabilistic; aligns with maximum likelihood. **Disadvantages:** Sensitive to noisy labels; overconfident predictions.

2.3.5 Focal Loss

$$L = -\alpha(1 - p_t)^\gamma \log(p_t)$$

Applications: Object detection (Mask R-CNN, RetinaNet). **Advantages:** Handles class imbalance. **Disadvantages:** Hyperparameters α, γ need tuning.

2.3.6 Dice / IoU Loss

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}, \qquad L = 1 - \text{Dice}$$

Applications: Image segmentation (Mask R-CNN, U-Net). **Advantages:** Works well with imbalanced masks. **Disadvantages:** Non-linear, harder optimization.

2.3.7 KL Divergence

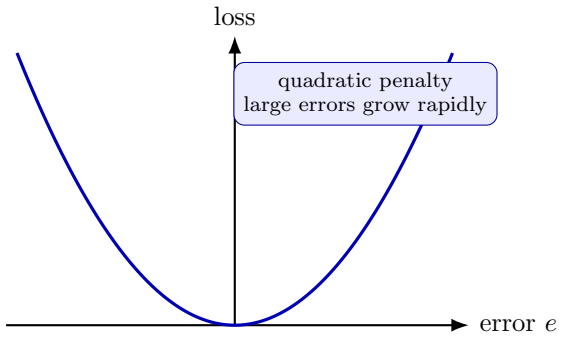
$$D_{KL}(P\|Q) = \sum_i P(i)\log\frac{P(i)}{Q(i)}$$

Applications: Variational Autoencoders, distillation. **Advantages:** Measures information difference. **Disadvantages:** Asymmetric; can be unstable.

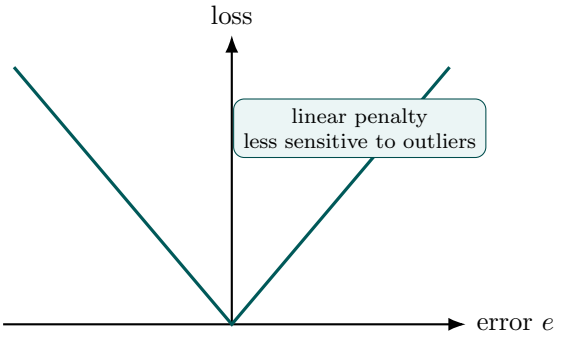
Loss Functions

Loss	**Typical Use**	**Advantages**	**Disadvantages**
MSE ($\frac{1}{N}\sum(y - \hat{y})^2$)	Regression	Convex, smooth, strong penalty on large errors	Sensitive to outliers
MAE ($\frac{1}{N}\sum\|y - \hat{y}\|$)	Robust regression	Robust to outliers	Non-differentiable at 0; slower convergence
Huber / Smooth-L_1	Regression, box regression	Combines MSE and MAE; robust and smooth	Requires tuning of δ
Binary CE	Binary classification	Probabilistic; aligns with MLE	Sensitive to noisy labels
Multiclass CE	Multiclass classification	Standard loss with softmax; probabilistic interpretation	Overconfident predictions are heavily penalized
BCEWithLogits	Multi-label classification	Numerically stable (sigmoid + CE combined)	Ignores label dependencies
Focal Loss	Imbalanced classification/detection	Focuses learning on hard examples	Requires tuning of α and γ
Dice Loss	Image segmentation (masks)	Handles class imbalance; measures overlap directly	Unstable for very small targets
IoU (Jaccard) Loss	Segmentation / bounding boxes	Directly optimizes IoU metric	Non-smooth; slower early convergence
Smooth-L_1 (boxes)	Object detection (R-CNNs)	Robust to outliers; standard for box regression	Scale-sensitive
KL Divergence	VAEs, knowledge distillation	Measures divergence between distributions	Asymmetric; unstable when $Q \approx 0$
Triplet Loss	Metric learning	Learns discriminative embedding spaces	Needs triplet mining; margin tuning
Contrastive Loss	Siamese networks / similarity tasks	Learns pairwise distance relationships	Requires balanced positive/negative pairs
Cosine Embedding	Text/vision embeddings	Scale-invariant and simple	Loses magnitude information
Perceptual Loss	Super-resolution / style transfer	Encourages perceptual similarity using deep features	Computationally heavy; needs pretrained ϕ
Total Variation	Image smoothing / denoising	Removes noise, encourages smoothness	Can over-smooth fine details

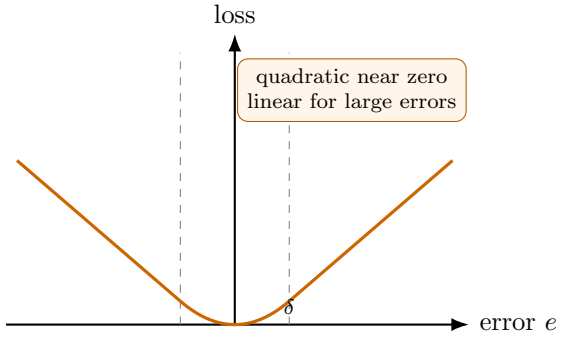
2.4 Visual Interpretation of Loss Functions



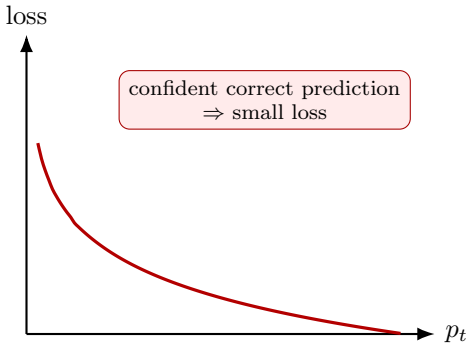
(a) MSE produces a quadratic error surface.



(b) MAE grows linearly with the magnitude of the error.

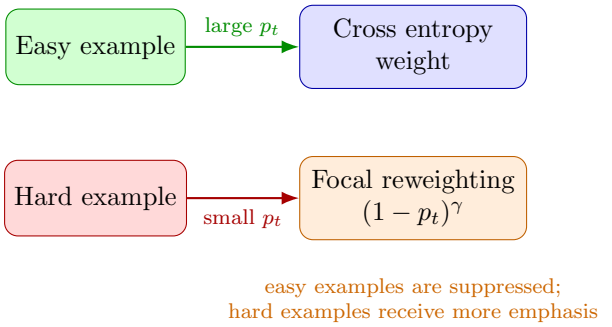


(c) Huber loss combines the local smoothness of MSE with the robustness of MAE.

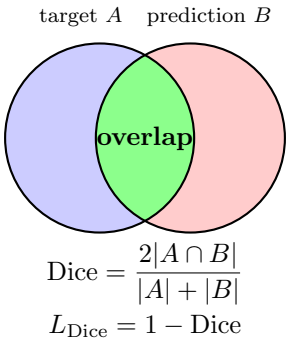


(d) Cross entropy decreases as the probability assigned to the correct class increases.

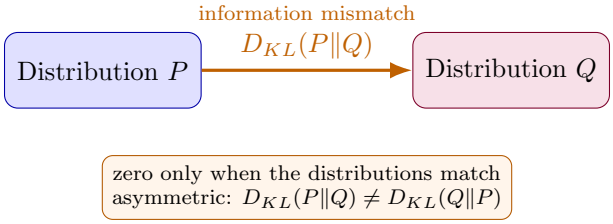
Figure 2.3: Side-by-side geometric interpretation of common regression and classification losses.



(a) Focal loss changes the relative contribution of easy and hard examples.



(b) Dice loss directly rewards overlap between a prediction and the target mask.



(c) KL divergence measures directional mismatch between two probability distributions.

Figure 2.4: Conceptual visualizations of focal, Dice, and KL-divergence losses.

2.5 Optimizers

2.5.1 Stochastic Gradient Descent (SGD)

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta_t)$$

Applications: Classical ML, CNNs. **Advantages:** Simple, effective. **Disadvantages:** Sensitive to learning rate; oscillates.

2.5.2 Momentum

$$v_t = \beta v_{t-1} + (1-\beta)\nabla_\theta J(\theta_t), \quad \theta_{t+1} = \theta_t - \eta v_t$$

Applications: Deep CNNs. **Advantages:** Faster convergence; less noise. **Disadvantages:** Needs tuning of β.

2.5.3 Adagrad

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot \nabla_\theta J(\theta_t)$$

Applications: Sparse features (e.g. NLP embeddings). **Advantages:** Adaptive learning rate. **Disadvantages:** Learning rate decays too fast.

2.5.4 RMSProp

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta) g_t^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Applications: RNNs, time series, non-stationary data. **Advantages:** Stable, adaptive. **Disadvantages:** Sensitive to β.

2.5.5 Adam

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Applications: Deep learning, NLP, Transformers. **Advantages:** Combines momentum + adaptive rate. **Disadvantages:** May generalize poorly.

2.5.6 AdamW

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

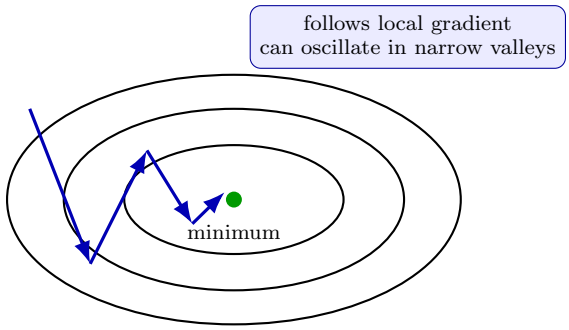
Applications: Transformers, BERT, ViTs. **Advantages:** Decoupled weight decay $\rightarrow$ better generalization. **Disadvantages:** Slightly more computation.

2.5.7 LAMB

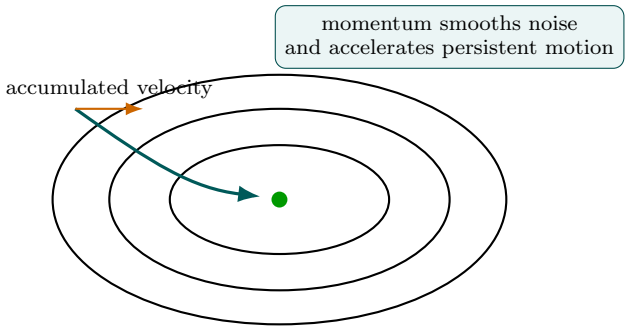
$$r_t = \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \quad \theta_{t+1} = \theta_t - \eta \frac{\|\theta_t\|}{\|r_t\|} r_t$$

Applications: Large-batch Transformer training. **Advantages:** Enables distributed training. **Disadvantages:** Complex; more tuning.

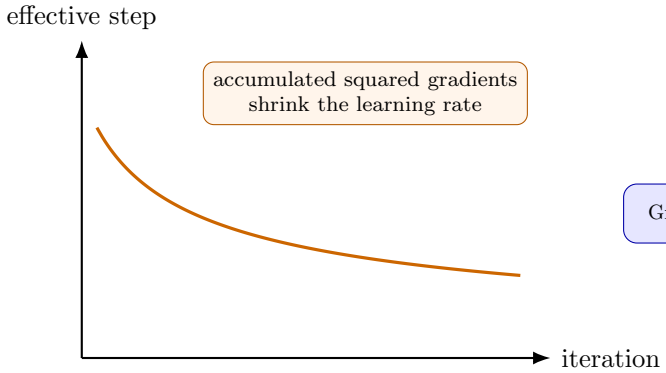
2.6 Visual Interpretation of Optimizers



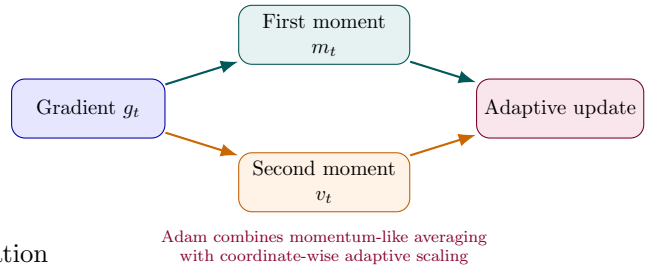
(a) SGD follows the instantaneous gradient and may zig-zag across a curved valley.



(b) Momentum accumulates a velocity term, producing a smoother path toward the minimum.

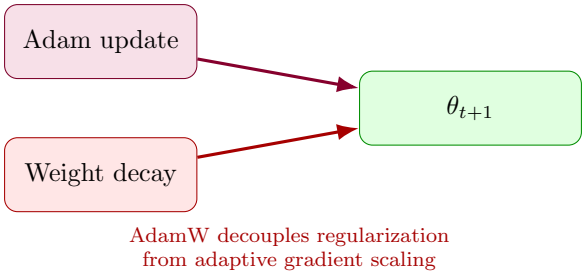


(c) Adagrad progressively decreases the effective step size as gradient history accumulates.

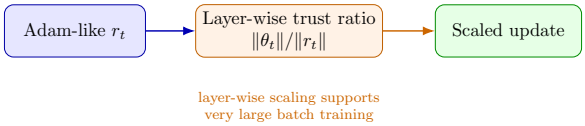


(d) Adam combines estimates of the first and second gradient moments.

Figure 2.5: Side-by-side conceptual behavior of SGD, Momentum, Adagrad, and Adam.



(a) AdamW applies weight decay as a separate contribution to the parameter update.



(b) LAMB rescales an Adam-like direction using the norm of each parameter layer.

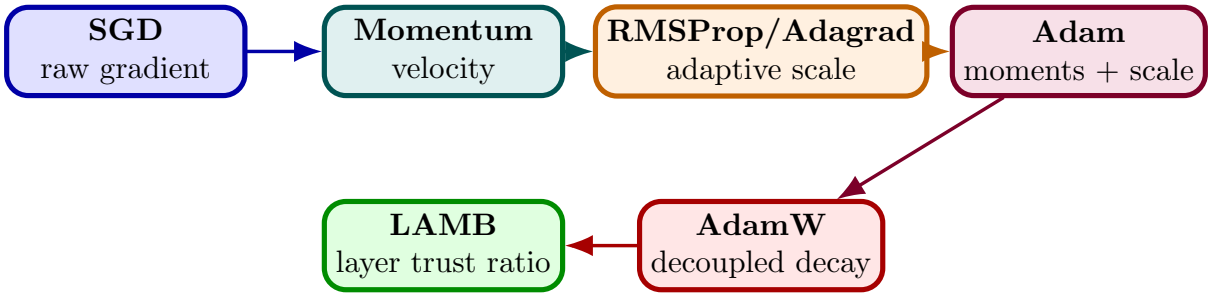


Figure 2.6: Conceptual progression from a raw gradient update toward adaptive and large-batch optimization strategies.

2.7 Summary Tables

Activation Functions

Function	Pros	Cons
Sigmoid	Probabilistic, smooth	Vanishing gradient
Tanh	Zero-centered	Saturation at extremes
ReLU	Sparse, fast	Dead neurons
Leaky ReLU	Fixes dead ReLU	Hyperparam α
Softmax	Probabilities	Saturation
GELU	Smooth	Slower compute

Optimizers

Optimizer	Pros	Cons
SGD	Simple, reliable	Slow, oscillates
Momentum	Smooth convergence	Needs tuning
RMSProp	Stable	Sensitive β
Adam	Fast, adaptive	May overfit
AdamW	Best generalization	Slightly slower

2.8 Neuron and Feature Representation

Mathematical identification of a feature In a hidden layer, each coordinate of the activation vector corresponds to a learned feature. Mathematically,

$$\text{feature}_j \equiv a_j.$$

That is, the activation produced by neuron j represents the j-th feature.

What is a neuron? Strict definition (mathematically correct). A neuron (also called a *unit*) is the computational element that maps its parameters and activation function to an output:

$$(\mathbf{w}_j, b_j, \sigma) \longmapsto a_j.$$

The **neuron** itself is the **function** defined by its weights, bias, and nonlinearity; the scalar quantities z_j and a_j are intermediate values produced by this function.

Neuron and feature (functional form) A neuron (or unit) is a parametric nonlinear function defined by weights, bias, and an activation function. Given an input vector $\mathbf{x}$, it computes

$$z_j = \mathbf{w}_j^\top \mathbf{x} + b_j, \qquad a_j = \sigma(z_j).$$

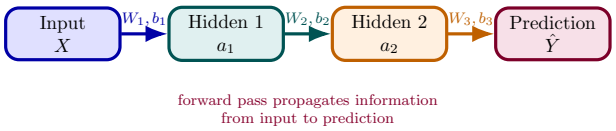
The activation a_j is the neuron output and is interpreted as a learned feature.

Summary

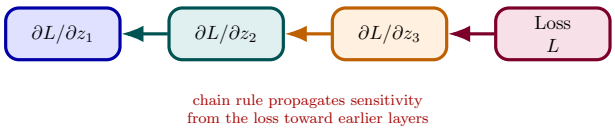
$$\text{neuron} = (\mathbf{w}, b, \sigma), \qquad \text{pre-activation} = z, \qquad \text{activation} = a, \qquad \text{feature} = a.$$

Thus, the output of a neuron is the feature it represents.

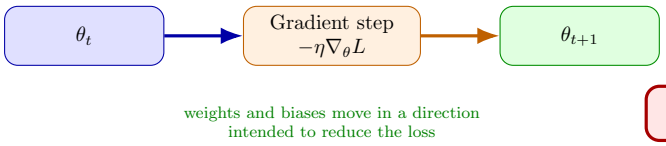
2.9 Visual Interpretation of Training Mechanics



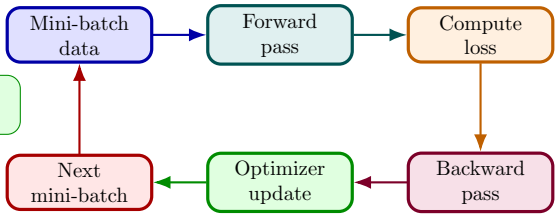
(a) Forward propagation computes layer activations sequentially.



(b) Backpropagation moves gradient information in the reverse direction.



(c) An optimizer converts gradients into parameter updates.



(d) Training repeats the forward, loss, backward, and update cycle over successive mini-batches.

Figure 2.7: Side-by-side visual summary of the mechanics of neural-network training.

2.10 Forward Pass

The forward pass computes activations layer by layer using learned weights and biases. Each layer applies linear transformations followed by nonlinear activation functions (e.g., sigmoid, ReLU, etc.).

Table 2.1: Forward Pass Computations and Data Structures

Step	**Calculation**	**Data Structure**	**Multiplication Type**
Forward Pass			
Input to 1st hidden layer (HI_1)	$HI_1 = W_1 \cdot X + B_1$	HI_1: Matrix, W_1: Matrix, X: Matrix, B_1: Vector	**Dot Product** (between W_1 and X)
Output of 1st hidden layer (HO_1)	$HO_1 = \sigma(HI_1)$	HO_1: Vector	**Element-wise** (Sigmoid applied element-wise)
Input to 2nd hidden layer (HI_2)	$HI_2 = W_2 \cdot HO_1 + B_2$	HI_2: Vector, W_2: Matrix, HO_1: Vector, B_2: Vector	**Dot Product** (between W_2 and HO_1)
Output of 2nd hidden layer (HO_2)	$HO_2 = \sigma(HI_2)$	HO_2: Vector	**Element-wise** (Sigmoid applied element-wise)
Input to output layer (HO_final)	$HO_{final} = W_3 \cdot HO_2 + B_3$	HO_{final}: Vector, W_3: Matrix, HO_2: Vector, B_3: Vector	**Dot Product** (between W_3 and HO_2)
Final output $\hat{Y}$	$\hat{Y} = \sigma(HO_{final})$	$\hat{Y}$: Vector	**Element-wise** (Sigmoid applied element-wise)
Error Calculation	$E = Y - \hat{Y}$	E: Vector, Y: Vector, $\hat{Y}$: Vector	**Element-wise** (Subtraction)

2.11 Backpropagation

$$Error_Layer_N = \sigma_d'(Input_N)\Big(Error_Layer_N + 1(W_N + 1)^T\Big), \qquad Input_N = WX + B.$$

2.12 Weight and Bias Updates

$$W_{new} = W_{old} + lr \cdot Own_Error \cdot Own_Input^T$$

$$Own_Input = X, \quad Own_Input_N = HO_{N-1} = \sigma(HI_{N-1})$$

Table 2.2: Comprehensive Backpropagation Steps

Step	**Calculation**	**Data Structure**	**Multiplication Type**
Backpropagation			
Error at output layer	$err_{HO_{final}} = E \cdot \sigma'(HO_{final})$	$err_{HO_{final}}$: Vector, E: Vector, $\sigma'(HO_{final})$: Vector	**Element-wise** (Multiplication)
Error in 2nd hidden layer (err_HO_2)	$err_{HO_2} = err_{HO_{final}} \cdot W_3^T \cdot \sigma'(HI_2)$	err_{HO_2}: Vector, W_3^T: Matrix (transpose), $\sigma'(HI_2)$: Vector	**Dot Product** (between $err_{HO_{final}}$ and W_3^T) followed by **Element-wise** multiplication with $\sigma'(HI_2)$
Error in 1st hidden layer (err_HO_1)	$err_{HO_1} = err_{HO_2} \cdot W_2^T \cdot \sigma'(HI_1)$	err_{HO_1}: Vector, W_2^T: Matrix (transpose), $\sigma'(HI_1)$: Vector	**Dot Product** (between err_{HO_2} and W_2^T) followed by **Element-wise** multiplication with $\sigma'(HI_1)$
Error Calculation	$E = Y - \hat{Y}$	E: Vector, Y: Vector, $\hat{Y}$: Vector	**Element-wise** (Subtraction)

Table 2.3: Weight and Bias Update Steps

Step	**Calculation**	**Data Structure**	**Multiplication Type**
Weight and Bias Updates			
Update W_1 and B_1	$W_1 = W_1 + lr \cdot X^T \cdot err_{HO_1}$, $B_1 = B_1 + lr \cdot err_{HO_1}$	W_1: Matrix, X^T: Matrix (transpose), err_{HO_1}: Vector	**Dot Product** (between X^T and err_{HO_1})
Update W_2 and B_2	$W_2 = W_2 + lr \cdot HO_1^T \cdot err_{HO_2}$, $B_2 = B_2 + lr \cdot err_{HO_2}$	W_2: Matrix, HO_1^T: Matrix (transpose), err_{HO_2}: Vector	**Dot Product** (between HO_1^T and err_{HO_2})
Update W_3 and B_3	$W_3 = W_3 + lr \cdot HO_2^T \cdot err_{HO_{final}}$, $B_3 = B_3 + lr \cdot err_{HO_{final}}$	W_3: Matrix, HO_2^T: Matrix (transpose), $err_{HO_{final}}$: Vector	**Dot Product** (between HO_2^T and $err_{HO_{final}}$)

Chapter 3

General Structure of Forward Pass and Backpropagation

3.1 Overview

The general structure of the forward pass and backpropagation is the same for all basic neural networks, regardless of how many hidden layers or neurons are used. The following principles hold true for most feedforward neural networks (also known as **multilayer perceptrons (MLPs)**).

3.2 Forward Pass

3.2.1 Basic Structure

- The network consists of an input layer, one or more hidden layers, and an output layer.
- Each layer computes a weighted sum of its inputs, adds a bias, and applies an activation function (such as sigmoid, ReLU, or tanh) to produce its output.

3.2.2 Computation Flow

- Data flows forward through the network from the input layer to the output layer.
- The output of one layer becomes the input to the next layer.

3.3 Backpropagation

3.3.1 Error Propagation

- After computing the final output during the forward pass, the network compares the predicted output with the expected (target) output.
- The error is calculated using a loss function (for example, mean squared error or cross-entropy).
- This error is propagated backward from the output layer to the hidden layers, adjusting the neuron weights to minimize the error.

3.3.2 Weight Update

- The gradients (rate of change of error with respect to weights) are computed using the chain rule of calculus.
- The weights and biases are updated using gradient descent or its variants (e.g., stochastic gradient descent).

3.3.3 General Steps

1. Compute the error at the output layer.
2. Backpropagate the error through each preceding layer.
3. Update the weights and biases to reduce the overall error.

3.4 Universality of the Process

The process of forward pass and backpropagation described above holds for all basic feedforward neural networks, regardless of:

- The number of hidden layers.
- The number of neurons per layer.
- The activation function used.

3.5 Common Variations

While the core steps remain the same, there are variations among different network types and techniques:

- **Activation Functions:** Modern networks often use ReLU or tanh instead of sigmoid in hidden layers.
- **Loss Functions:** Classification problems typically use cross-entropy loss, while regression problems often use mean squared error (MSE).
- **Learning Algorithms:** Standard gradient descent can be replaced by advanced optimizers such as Adam, RMSProp, or Adagrad.

3.6 Summary

- The concept of forward pass and backpropagation is **universal** to most feedforward neural networks.
- Differences arise mainly in the architecture (number of layers or neurons) and the type of activation, loss function, and optimization algorithm used.
- As networks become deeper and more complex, the same principles apply, but across more layers.

Examples

- **Deep Neural Networks (DNNs):** Contain more hidden layers, but the core forward and backward propagation remain identical.
- **Convolutional Neural Networks (CNNs)** and **Recurrent Neural Networks (RNNs):** Both employ forward and backward propagation with specialized modifications.

In essence, for any basic feedforward neural network, the process of forward pass and backpropagation remains fundamentally the same.

3.7 Worked Example I: Single-Neuron Forward and Backpropagation

Table 3.1: Forward propagation for a single-neuron network

Step	Quantity	Expression	Numerical value
1	Input	x	2
2	Weight	w	3
3	Bias	b	1
4	Linear output	$z = wx + b$	7
5	Activation	$\hat{y} = \sigma(z)$	0.999088
6	Target	y	1
7	Loss	$\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$	4.16×10^{-7}

Table 3.2: Backpropagation for the output layer (layer 2)

Step	Gradient	Formula	Value	Value comes from
7	$\frac{\partial \mathcal{L}}{\partial \hat{y}}$	$\hat{y} - y$	=0.119587	Forward step 5 ($\hat{y}$), forward step 6 (y)
8	$\frac{\partial \hat{y}}{\partial z_2}$	$\hat{y}(1 - \hat{y})$	0.105233	Forward step 5 ($\hat{y}$)
9	$\frac{\partial \mathcal{L}}{\partial z_2}$	Step 7 × Step 8	=0.012580	Backprop steps 7 and 8
10	$\frac{\partial z_2}{\partial w_2}$	a_1	0.999088	Forward step 3 (a_1)
11	$\frac{\partial \mathcal{L}}{\partial w_2}$	Step 9 × Step 10	=0.012569	Backprop step 9, forward step 3
12	$\frac{\partial z_2}{\partial b_2}$	1	1	Affine layer definition
13	$\frac{\partial \mathcal{L}}{\partial b_2}$	Step 9 × Step 12	=0.012580	Backprop step 9

Note. Each gradient is computed using the chain rule and depends exclusively on quantities obtained during the forward pass or gradients propagated from subsequent layers. This explicit dependency structure mirrors the computation graph used by automatic differentiation frameworks.

3.8 Worked Example II: Three-Layer Forward and Backpropagation

3.8.1 Network Definition (Scalar)

$$z_1 = w_1 x + b_1, \quad a_1 = \mathrm{ReLU}(z_1),$$

Table 3.3: Backpropagation for the hidden layer (layer 1)

Step	Gradient	Formula	Value	Value comes from
14	$\frac{\partial z_2}{\partial a_1}$	w_2	4	Forward definition of layer 2
15	$\frac{\partial \mathcal{L}}{\partial a_1}$	Step 9 × Step 14	$=0.050319$	Backprop step 9, forward step 4
16	$\frac{\partial a_1}{\partial z_1}$	$a_1(1-a_1)$	0.000911	Forward step 3 (a_1)
17	$\frac{\partial \mathcal{L}}{\partial z_1}$	Step 15 × Step 16	$=4.584\times10^{-5}$	Backprop steps 15 and 16
18	$\frac{\partial z_1}{\partial w_1}$	x	2	Forward step 1 (x)
19	$\frac{\partial \mathcal{L}}{\partial w_1}$	Step 17 × Step 18	$=9.168\times10^{-5}$	Backprop step 17, forward step 1
20	$\frac{\partial z_1}{\partial b_1}$	1	1	Affine layer definition
21	$\frac{\partial \mathcal{L}}{\partial b_1}$	Step 17 × Step 20	$=4.584\times10^{-5}$	Backprop step 17

$$z_2 = w_2 a_1 + b_2, \quad a_2 = \tanh(z_2),$$
$$z_3 = w_3 a_2 + b_3, \quad \hat{y} = \sigma(z_3), \qquad \mathcal{L} = \tfrac{1}{2}(y-\hat{y})^2.$$

3.8.2 Activation Derivatives

$$\frac{d}{dz}\mathrm{ReLU}(z) = \begin{cases} 1, & z > 0 \\ 0, & z \le 0 \end{cases}, \qquad \frac{d}{dz}\tanh(z) = 1 - \tanh^2(z), \qquad \frac{d}{dz}\sigma(z) = \sigma(z)\big(1-\sigma(z)\big).$$

3.8.3 Numerical Values Used

$$x = 2,\ (w_1, b_1) = (1.5, -1),\ (w_2, b_2) = (0.5, 0.1),\ (w_3, b_3) = (2, -0.3),\ y = 1.$$

Table 3.4: Forward propagation for a three-layer network with three different activation functions

Step	Quantity	Expression	Numerical value
1	Input	x	2
2	Layer 1 weight	w_1	1.5
3	Layer 1 bias	b_1	$=1$
4	Pre-activation (L1)	$z_1 = w_1 x + b_1$	2.0
5	Activation (L1, ReLU)	$a_1 = \mathrm{ReLU}(z_1)$	2.0
6	Layer 2 weight	w_2	0.5
7	Layer 2 bias	b_2	0.1
8	Pre-activation (L2)	$z_2 = w_2 a_1 + b_2$	1.1
9	Activation (L2, tanh)	$a_2 = \tanh(z_2)$	0.800499
10	Layer 3 weight	w_3	2
11	Layer 3 bias	b_3	$=0.3$
12	Pre-activation (L3)	$z_3 = w_3 a_2 + b_3$	1.300998
13	Output activation (sigmoid)	$\hat{y} = \sigma(z_3)$	0.786003
14	Target	y	1
15	Loss	$\mathcal{L} = \frac{1}{2}(y-\hat{y})^2$	0.022897

Table 3.5: Backpropagation through the output layer (layer 3, sigmoid)

Step	Gradient	Formula	Value	Value comes from
16	$\frac{\partial \mathcal{L}}{\partial \hat{y}}$	$\hat{y} - y$	=0.213997	Forward step 13 ($\hat{y}$), step 14 (y)
17	$\frac{\partial \hat{y}}{\partial z_3}$	$\hat{y}(1-\hat{y})$	0.168202	Forward step 13 ($\hat{y}$)
18	$\frac{\partial \mathcal{L}}{\partial z_3}$	Step 16 × Step 17	=0.035995	Backprop steps 16 and 17
19	$\frac{\partial z_3}{\partial w_3}$	a_2	0.800499	Forward step 9 (a_2)
20	$\frac{\partial \mathcal{L}}{\partial w_3}$	Step 18 × Step 19	=0.028814	Backprop step 18, forward step 9
21	$\frac{\partial z_3}{\partial b_3}$	1	1	Affine layer definition
22	$\frac{\partial \mathcal{L}}{\partial b_3}$	Step 18 × Step 21	=0.035995	Backprop step 18
23	$\frac{\partial z_3}{\partial a_2}$	w_3	2	Forward step 10 (w_3)
24	$\frac{\partial \mathcal{L}}{\partial a_2}$	Step 18 × Step 23	=0.071990	Backprop step 18, forward step 10

Table 3.6: Backpropagation through hidden layer 2 (layer 2, tanh)

Step	Gradient	Formula	Value	Value comes from
25	$\frac{\partial a_2}{\partial z_2}$	$1 - a_2^2$	0.359201	Forward step 9 (a_2)
26	$\frac{\partial \mathcal{L}}{\partial z_2}$	Step 24 × Step 25	=0.025859	Backprop step 24, backprop step 25
27	$\frac{\partial z_2}{\partial w_2}$	a_1	2.0	Forward step 5 (a_1)
28	$\frac{\partial \mathcal{L}}{\partial w_2}$	Step 26 × Step 27	=0.051718	Backprop step 26, forward step 5
29	$\frac{\partial z_2}{\partial b_2}$	1	1	Affine layer definition
30	$\frac{\partial \mathcal{L}}{\partial b_2}$	Step 26 × Step 29	=0.025859	Backprop step 26
31	$\frac{\partial z_2}{\partial a_1}$	w_2	0.5	Forward step 6 (w_2)
32	$\frac{\partial \mathcal{L}}{\partial a_1}$	Step 26 × Step 31	=0.012929	Backprop step 26, forward step 6

Table 3.7: Backpropagation through hidden layer 1 (layer 1, ReLU)

Step	Gradient	Formula	Value	Value comes from
33	$\frac{\partial a_1}{\partial z_1}$	$\mathbb{1}[z_1 > 0]$	1	Forward step 4 ($z_1 = 2.0 > 0$)
34	$\frac{\partial \mathcal{L}}{\partial z_1}$	Step 32 × Step 33	=0.012929	Backprop step 32, backprop step 33
35	$\frac{\partial z_1}{\partial w_1}$	x	2	Forward step 1 (x)
36	$\frac{\partial \mathcal{L}}{\partial w_1}$	Step 34 × Step 35	=0.025859	Backprop step 34, forward step 1
37	$\frac{\partial z_1}{\partial b_1}$	1	1	Affine layer definition
38	$\frac{\partial \mathcal{L}}{\partial b_1}$	Step 34 × Step 37	=0.012929	Backprop step 34

Note. Each gradient is computed using the chain rule and depends exclusively on quantities obtained during the forward pass or gradients propagated from subsequent layers. The three activation derivatives appear explicitly: sigmoid uses $\hat{y}(1-\hat{y})$, tanh uses $1-a_2^2$, and ReLU uses the gate $\mathbb{1}[z_1 > 0]$.

3.9 Matrix-Form Backpropagation, Product Types, and Forward-Pass Memory

Goal. This section presents forward and backward propagation for a three-layer feedforward neural network using vector and matrix notation. The formulation explicitly identifies the type of each algebraic operation (matrix–vector product, Hadamard product, outer product) and clarifies which intermediate quantities must be stored during the forward pass to enable backpropagation.

Notation and dimensions. Let the input be $\mathbf{x} \in \mathbb{R}^{d_0}$ and define $\mathbf{a}_0 := \mathbf{x}$. For layers $k \in \{1, 2, 3\}$:

$$\mathbf{W}_k \in \mathbb{R}^{d_k \times d_{k-1}}, \qquad \mathbf{b}_k \in \mathbb{R}^{d_k}, \qquad \mathbf{z}_k \in \mathbb{R}^{d_k}, \qquad \mathbf{a}_k \in \mathbb{R}^{d_k}.$$

Each affine transformation uses a **matrix–vector product**:

$$\mathbf{z}_k = \mathbf{W}_k \mathbf{a}_{k-1} + \mathbf{b}_k.$$

Three-layer network with heterogeneous activations.

$$\mathbf{a}_1 = \mathrm{ReLU}(\mathbf{z}_1), \qquad \mathbf{a}_2 = \tanh(\mathbf{z}_2), \qquad \hat{\mathbf{y}} = \mathbf{a}_3 = \sigma(\mathbf{z}_3).$$

For a single training sample, the loss is

$$\mathcal{L} = \tfrac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|^2.$$

Activation derivatives (element-wise).

$$\frac{d}{dz}\mathrm{ReLU}(z) = \begin{cases} 1, & z > 0 \\ 0, & z \le 0 \end{cases}, \qquad \frac{d}{dz}\tanh(z) = 1 - \tanh^2(z), \qquad \frac{d}{dz}\sigma(z) = \sigma(z)\big(1 - \sigma(z)\big).$$

The symbol $\odot$ denotes the *Hadamard (element-wise) product*.

Forward propagation (matrix form).

$$\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{a}_1 = \mathrm{ReLU}(\mathbf{z}_1), \tag{3.1}$$

$$\mathbf{z}_2 = \mathbf{W}_2\mathbf{a}_1 + \mathbf{b}_2, \qquad \mathbf{a}_2 = \tanh(\mathbf{z}_2), \tag{3.2}$$

$$\mathbf{z}_3 = \mathbf{W}_3\mathbf{a}_2 + \mathbf{b}_3, \qquad \hat{\mathbf{y}} = \sigma(\mathbf{z}_3). \tag{3.3}$$

Backpropagation (matrix form). Define error signals ("deltas"):

$$\boldsymbol{\delta}_k := \frac{\partial \mathcal{L}}{\partial \mathbf{z}_k} \in \mathbb{R}^{d_k}.$$

Layer 3 (sigmoid output).

$$\boldsymbol{\delta}_3 = (\hat{\mathbf{y}} - \mathbf{y}) \ \odot \ \big(\hat{\mathbf{y}} \odot (1 - \hat{\mathbf{y}})\big).$$

Layer 2 (tanh).

$$\boldsymbol{\delta}_2 = (\mathbf{W}_3^\top \boldsymbol{\delta}_3) \ \odot \ \big(1 - \mathbf{a}_2^{\odot 2}\big).$$

Layer 1 (ReLU).

$$\boldsymbol{\delta}_1 = (\mathbf{W}_2^\top \boldsymbol{\delta}_2) \ \odot \ \mathbb{1}[\mathbf{z}_1 > 0].$$

Table 3.8: Quantities stored during forward propagation and their role in backpropagation

Stored quantity	**Saved during forward pass**	**Used in backpropagation**
$\mathbf{x} = \mathbf{a}_0$	Input layer	$\boldsymbol{\delta}_1 \mathbf{x}^\top$
$\mathbf{z}_1$	Pre-activation (layer 1)	ReLU mask $\mathbb{1}[\mathbf{z}_1 > 0]$
$\mathbf{a}_1$	Activation (layer 1)	$\boldsymbol{\delta}_2 \mathbf{a}_1^\top$
$\mathbf{z}_2$	Pre-activation (layer 2)	$1 - \mathbf{a}_2^{\odot 2}$
$\mathbf{a}_2$	Activation (layer 2)	$\boldsymbol{\delta}_3 \mathbf{a}_2^\top$
$\mathbf{z}_3$	Pre-activation (output layer)	$\hat{\mathbf{y}} \odot (1 - \hat{\mathbf{y}})$
$\hat{\mathbf{y}}$	Network output	$\hat{\mathbf{y}} - \mathbf{y}$

Parameter gradients. For each layer $k \in \{1, 2, 3\}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_k} = \boldsymbol{\delta}_k \, \mathbf{a}_{k-1}^\top, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_k} = \boldsymbol{\delta}_k.$$

The product $\boldsymbol{\delta}_k \, \mathbf{a}_{k-1}^\top$ is an **outer product**.

Forward-pass memory (saved tensors).

Table 3.9: Product types appearing in forward and backward propagation

Symbol / form	**Name**	**Where it appears**
$\mathbf{Wa}$, $\mathbf{W}^\top \boldsymbol{\delta}$	Matrix–vector product	Forward affine map; backward error propagation
$\odot$	Hadamard (element-wise) product	Activation derivatives and masks
$\boldsymbol{\delta} \, \mathbf{a}^\top$	Outer product	Weight gradients

Summary of product types.

3.10 Chain Rule Accumulation and Gradient Flow

Motivation. In backpropagation, gradients are accumulated whenever a variable influences the loss through multiple computational paths. This section makes the summation in the chain rule explicit and connects it directly to how automatic differentiation frameworks accumulate gradients.

Computation graph example. Consider the scalar computation

$$\begin{aligned} v_3 &= x^2, \\ v_5 &= 2v_3, \\ v_6 &= v_3 + 1, \\ f &= v_5\, v_6. \end{aligned}$$

The intermediate variable v_3 feeds into two downstream nodes.

Forward pass. For $x = 3$:

$$v_3 = 9, \qquad v_5 = 18, \qquad v_6 = 10, \qquad f = 180.$$

Backward pass. The multivariable chain rule gives

$$\frac{\partial f}{\partial v_3} = \frac{\partial f}{\partial v_6}\frac{\partial v_6}{\partial v_3} + \frac{\partial f}{\partial v_5}\frac{\partial v_5}{\partial v_3}.$$

Evaluating:

$$\frac{\partial f}{\partial v_5} = v_6 = 10, \qquad \frac{\partial f}{\partial v_6} = v_5 = 18,$$
$$\frac{\partial v_5}{\partial v_3} = 2, \qquad \frac{\partial v_6}{\partial v_3} = 1.$$

Thus,

$$\frac{\partial f}{\partial v_3} = 18 + 20 = 38.$$

Interpretation. The summation corresponds exactly to gradient accumulation in backpropagation: each outgoing edge contributes a partial gradient, and the total gradient is their sum.

3.11 Hessian–Vector Products and Second-Order Structure

Let $f(\boldsymbol{\theta})$ be a scalar loss with parameters $\boldsymbol{\theta} \in \mathbb{R}^W$. The gradient is $\nabla f(\boldsymbol{\theta}) \in \mathbb{R}^W$ and the Hessian is

$$\mathbf{H} = \nabla^2 f(\boldsymbol{\theta}) \in \mathbb{R}^{W \times W}.$$

Explicit construction of $\mathbf{H}$ is infeasible for modern neural networks. Instead, second-order methods rely on the *Hessian–vector product*

$$\mathbf{Hu} = \nabla^2 f(\boldsymbol{\theta})\, \mathbf{u},$$

for a chosen direction $\mathbf{u} \in \mathbb{R}^W$.

A key identity enables efficient computation:

$$\mathbf{Hu} = \nabla_{\boldsymbol{\theta}} \left(\nabla f(\boldsymbol{\theta})^\top \mathbf{u} \right).$$

This identity allows Hessian–vector products to be computed using standard backpropagation without forming $\mathbf{H}$ explicitly.

3.11.1 Worked Example: Scalar Hessian–Vector Product

Consider

$$f(w_1, w_2) = w_1^2 + 3w_1w_2 + 2w_2^2.$$

The gradient is

$$\nabla f = \begin{pmatrix} 2w_1 + 3w_2 \\ 3w_1 + 4w_2 \end{pmatrix},$$

and the Hessian is

$$\nabla^2 f = \begin{pmatrix} 2 & 3 \\ 3 & 4 \end{pmatrix}.$$

For $\mathbf{u} = (1, -1)^\top$,

$$\mathbf{Hu} = \begin{pmatrix} -1 \\ -1 \end{pmatrix}.$$

Using the nabla identity,

$$\nabla f^\top \mathbf{u} = -(w_1 + w_2), \qquad \nabla_{\mathbf{w}}(-(w_1 + w_2)) = \begin{pmatrix} -1 \\ -1 \end{pmatrix},$$

confirming the result.

3.11.2 Hessian–Vector Products via Backpropagation: Concrete Three-Layer Example

Forward pass. The forward pass computes

$$\hat{y} = w_3\big(w_2(w_1x + b_1) + b_2\big) + b_3,$$

$$\mathcal{L} = \tfrac{1}{2}\big(\hat{y} - y\big)^2.$$

First backward pass (gradient computation). Backpropagation yields the gradient of the loss with respect to all parameters:

$$\nabla f(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial \mathcal{L}}{\partial w_1} \\ \frac{\partial \mathcal{L}}{\partial b_1} \\ \frac{\partial \mathcal{L}}{\partial w_2} \\ \frac{\partial \mathcal{L}}{\partial b_2} \\ \frac{\partial \mathcal{L}}{\partial w_3} \\ \frac{\partial \mathcal{L}}{\partial b_3} \end{pmatrix} = (\hat{y} - y) \begin{pmatrix} w_3w_2x \\ w_3w_2 \\ w_3a_1 \\ w_3 \\ a_2 \\ 1 \end{pmatrix}.$$

This is the standard gradient computed during ordinary backpropagation.

Concrete three-layer network usage. To make the role of the Hessian–vector product explicit in a deep learning setting, consider the three-layer scalar network

$$z_1 = w_1x + b_1, \qquad a_1 = z_1,$$

$$z_2 = w_2a_1 + b_2, \qquad a_2 = z_2,$$

$$z_3 = w_3a_2 + b_3, \qquad \hat{y} = z_3, \qquad \mathcal{L} = \tfrac{1}{2}(\hat{y} - y)^2,$$

with parameter vector

$$\boldsymbol{\theta} = (w_1, b_1, w_2, b_2, w_3, b_3)^\top.$$

First backward pass (gradient). Applying standard backpropagation yields

$$\nabla f(\boldsymbol{\theta}) = (\hat{y} - y) \begin{pmatrix} w_3 w_2 x \\ w_3 w_2 \\ w_3 a_1 \\ w_3 \\ a_2 \\ 1 \end{pmatrix}.$$

This is the gradient used by first-order optimizers such as SGD or Adam.

Scalar construction for curvature probing. Given a direction vector

$$\mathbf{u} = (u_{w_1}, u_{b_1}, u_{w_2}, u_{b_2}, u_{w_3}, u_{b_3})^\top,$$

we form the scalar

$$s(\boldsymbol{\theta}) = \nabla f(\boldsymbol{\theta})^\top \mathbf{u} = (\hat{y} - y)\Big(u_{w_1} w_3 w_2 x + u_{b_1} w_3 w_2 + u_{w_2} w_3 a_1 + u_{b_2} w_3 + u_{w_3} a_2 + u_{b_3}\Big).$$

This scalar measures how the gradient of the loss changes when the parameters are perturbed along the direction $\mathbf{u}$.

Second backward pass (Hessian–vector product). Differentiating $s(\boldsymbol{\theta})$ with respect to the parameters gives

$$\nabla_{\boldsymbol{\theta}} s(\boldsymbol{\theta}) = \nabla^2 f(\boldsymbol{\theta})\, \mathbf{u} = \mathbf{H}\mathbf{u}.$$

Each component of this vector captures the second-order sensitivity of the loss with respect to the corresponding parameter along the chosen direction $\mathbf{u}$.

How this fits into forward and backward propagation. Operationally, the computation can be summarized as:

- **Forward pass:** compute activations $(a_1, a_2, \hat{y})$ and the loss $\mathcal{L}$.
- **First backward pass:** compute the gradient $\nabla f(\boldsymbol{\theta})$ using standard backpropagation.
- **Scalar projection:** form $s(\boldsymbol{\theta}) = \nabla f(\boldsymbol{\theta})^\top \mathbf{u}$.
- **Second backward pass:** backpropagate through $s(\boldsymbol{\theta})$ to obtain $\mathbf{H}\mathbf{u}$.

Interpretation in deep learning terms.

- The first backward pass computes *slopes* of the loss.
- The second backward pass computes how those slopes *change* along a specific direction in parameter space.
- The vector $\mathbf{u}$ selects which direction of curvature is being examined.
- No Hessian matrix is formed; curvature is accessed implicitly through backpropagation.

Practical meaning. In large neural networks, the same mechanism applies when $\boldsymbol{\theta}$ contains millions of parameters. Hessian–vector products computed in this manner are used inside iterative solvers such as conjugate gradient to obtain curvature-aware update directions, analyze sharpness of minima, and enable Hessian-free optimization—all while preserving the standard forward and backward propagation structure used in deep learning frameworks.

Role of the second backward pass: clarification. The purpose of the second backward pass depends on how the Hessian–vector product is used. The same mathematical operation serves two distinct roles in deep learning.

- **Standard first-order training (SGD, Adam).**
  - Training consists of one forward pass and one backward pass per batch.
  - The backward pass computes the gradient $\nabla f(\boldsymbol{\theta})$.
  - Parameter updates depend only on first-order information.
  - No Hessian–vector products are required.
  - In this setting, a second backward pass is unnecessary.
- **Second backward pass used for analysis or diagnostics.**
  - The second backward pass computes $\mathbf{Hu} = \nabla^2 f(\boldsymbol{\theta})\,\mathbf{u}$.
  - The direction $\mathbf{u}$ is chosen externally (e.g., the gradient direction or a random vector).
  - Quantities such as $\mathbf{u}^\top \mathbf{Hu}$ are used to assess curvature, sharpness, or stability.
  - Training updates are not modified.
  - The second pass is used only to monitor or analyze convergence behavior.
- **Second backward pass used for optimization.**
  - In second-order or Hessian-free methods, the second backward pass directly influences parameter updates.
  - Optimization algorithms such as conjugate gradient solve linear systems of the form

    $$\mathbf{Hp} = -\nabla f(\boldsymbol{\theta})$$

    using repeated Hessian–vector products.
  - Each Hessian–vector product requires a second backward pass.
  - The resulting direction $\mathbf{p}$ determines how the model parameters are updated.
  - In this case, the second backward pass is part of the training algorithm itself.
- **Why the second backward pass is not always used.**
  - Hessian–vector products are computationally more expensive than gradients.
  - Stochastic noise in mini-batch training complicates curvature estimation.
  - First-order methods often provide sufficient performance at lower computational cost.
  - As a result, curvature-based methods are applied selectively.

Summary.

- One backward pass computes the slope of the loss.
- A second backward pass computes how that slope changes.
- The second backward pass may be used for analysis only or as an integral part of optimization.
- Whether it affects training depends on how the Hessian–vector product is used.

When Hessian–vector products are used in practice. The use of Hessian–vector products depends on the choice of optimization strategy.

- **Typical deep learning training.**
  - Most deep learning models are trained using first-order optimizers such as SGD, Adam, or RMSProp.
  - These methods require only one forward pass and one backward pass per batch.
  - Curvature information is not explicitly computed.
  - Hessian–vector products are therefore *not* used during standard training.
- **Second-order and curvature-aware optimization.**
  - When second-order information is desired, Hessian–vector products become essential.
  - Methods such as Hessian-free optimization, Newton–CG, and other curvature-aware algorithms rely on repeated evaluations of $\mathbf{Hu}$.
  - The second backward pass is used to compute how gradients change along selected directions.
  - This additional information is used to construct improved update directions for the model parameters.
- **Practical implication.**
  - In most applications, first-order methods are preferred due to their simplicity and computational efficiency.
  - Hessian–vector products are used selectively, either for advanced optimization or for analyzing the geometry of the loss landscape.
  - When a second-order optimizer is employed, the additional backward pass becomes a necessary and integral part of the training procedure.

Key takeaway.

- Standard training does not require Hessian–vector products.
- Hessian–vector products become useful and necessary when second-order optimization methods are employed.

Role and selection of the direction vector in second-order optimization. Second-order optimization methods do not operate on the Hessian matrix $\mathbf{H}$ directly. Instead, they rely on Hessian–vector products of the form $\mathbf{Hu}$, which require the specification of a direction vector $\mathbf{u}$ in parameter space.

- **Why a direction vector is required.**
  - The Hessian $\mathbf{H} \in \mathbb{R}^{W \times W}$ is too large to form or store explicitly in deep learning models.
  - Second-order methods therefore interact with the Hessian only through its action on vectors.
  - The vector $\mathbf{u}$ specifies the direction along which curvature information is extracted.
- **Direction vectors are not arbitrary parameters.**
  - The vector $\mathbf{u}$ is not learned and is not part of the model.

  - It is generated algorithmically by the optimization method.
  - Its purpose is to probe curvature or construct update directions.

- **Typical choices of the direction vector u.**
  - *Gradient direction:*
$$\mathbf{u} = \nabla f(\boldsymbol{\theta}),$$
used to measure curvature along the descent direction and to assess sharpness or stability.
  - *Conjugate gradient search directions:*
$$\mathbf{u} = \mathbf{p}_k,$$
where $\mathbf{p}_k$ is the current search direction generated by the conjugate gradient algorithm when solving
$$\mathbf{H}\mathbf{p} = -\nabla f(\boldsymbol{\theta}).$$
  - *Random probe directions:* used to estimate average curvature or spectral properties of the Hessian.

- **How the direction vector affects optimization.**
  - The quality of the second-order update depends on the choice of $\mathbf{u}$.
  - In Newton–CG and Hessian-free optimization, the direction vectors are chosen adaptively to approximate the Newton step.
  - Poorly chosen directions yield poor curvature estimates and ineffective updates.

- **Connection to the second backward pass.**
  - Each chosen direction $\mathbf{u}$ triggers a Hessian–vector product $\mathbf{H}\mathbf{u}$.
  - Computing $\mathbf{H}\mathbf{u}$ requires a second backward pass through the scalar $\nabla f(\boldsymbol{\theta})^\top \mathbf{u}$.
  - Multiple directions may be evaluated during a single optimization step.

Key takeaway.

- Second-order optimization requires both gradient information and carefully chosen direction vectors.
- Hessian–vector products provide curvature information only along those directions.
- The effectiveness of second-order methods depends critically on how the direction vectors are defined and updated.

Where Newton–CG is used in practice. Newton–Conjugate Gradient (Newton–CG) methods appear in deep learning and scientific computing primarily in settings where curvature information is valuable and the optimization problem is sufficiently structured.

- **Hessian-free neural network training.**
  - Newton–CG is the core optimization engine in Hessian-free training methods.
  - The Newton update direction $\mathbf{p}$ is obtained by approximately solving
$$\mathbf{H}\mathbf{p} = -\nabla f(\boldsymbol{\theta})$$
using conjugate gradient iterations.

  - Each CG iteration requires Hessian–vector products **Hu** rather than the full Hessian.
  - **Example applications:**
    * training deep autoencoders for dimensionality reduction,
    * optimization of recurrent neural networks (RNNs) and LSTMs suffering from vanishing or exploding gradients,
    * training very deep feedforward networks where SGD converges slowly due to ill-conditioned curvature.

- **Large-scale scientific and engineering optimization.**
  - Newton–CG is widely used in physics-based optimization problems involving smooth, differentiable objective functions.
  - **Example applications:**
    * inverse problems in imaging (e.g., tomography and deconvolution),
    * parameter estimation in PDE-constrained optimization,
    * aerodynamic shape optimization and structural mechanics,
    * fluid dynamics and climate modeling simulations.
  - In these problems, curvature information dramatically reduces the number of required optimization iterations.

- **Classical machine learning models.**
  - Newton–CG is commonly used when the loss function is convex or nearly convex.
  - **Example applications:**
    * logistic regression with large feature sets,
    * generalized linear models (GLMs),
    * kernel-based learning methods with smooth regularization.
  - In these settings, Newton–CG often converges in far fewer iterations than gradient descent.

- **Gauss–Newton and Fisher-based methods in deep learning.**
  - Newton–CG is frequently applied to Gauss–Newton or Fisher information matrix approximations of the Hessian.
  - These approximations are positive semi-definite, making CG numerically stable.
  - **Example applications:**
    * training neural networks with squared-error or likelihood- based losses,
    * natural gradient methods and trust-region approaches,
    * curvature-aware fine-tuning of pretrained deep networks.

- **Why Newton–CG is not the default in deep learning.**
  - Each Newton step requires multiple Hessian–vector products, increasing computational cost.
  - Mini-batch stochasticity introduces noise into curvature estimates.
  - First-order methods (SGD, Adam) often achieve competitive performance with significantly lower overhead.
  - As a result, Newton–CG is used selectively rather than as a default optimizer.

Summary.

- Newton–CG is most useful when curvature information significantly improves convergence.
- It is widely used in Hessian-free neural network training and large-scale scientific and engineering optimization.
- In deep learning, it is typically applied with Gauss–Newton or Fisher approximations rather than the exact Hessian.
- Its limited use is due to computational cost and stochastic noise, not lack of effectiveness.

3.11.3 Role of the Second Backward Pass

- **First-order training (SGD, Adam):** one forward pass and one backward pass compute gradients.
- **Analysis and diagnostics:** a second backward pass computes curvature quantities such as $\mathbf{u}^\top \mathbf{H}\mathbf{u}$ without affecting updates.
- **Second-order optimization:** Hessian–vector products directly determine update directions in methods such as Newton–CG and Hessian-free optimization.

The second backward pass is therefore optional and task-dependent.

3.11.4 Direction Vectors in Second-Order Optimization

The vector $\mathbf{u}$ is not a model parameter. It is generated by the optimization algorithm.

Typical choices include:

- the gradient direction $\mathbf{u} = \nabla f(\boldsymbol{\theta})$,
- conjugate gradient search directions,
- random probe directions for curvature estimation.

Each choice extracts curvature information along a specific direction.

3.11.5 Newton–CG and Hessian-Free Methods

Newton–CG methods solve

$$\mathbf{H}\mathbf{p} = -\nabla f(\boldsymbol{\theta})$$

using conjugate gradient iterations. Each iteration requires a Hessian–vector product.

These methods appear in:

- Hessian-free neural network training,
- RNN and LSTM optimization,
- inverse problems and PDE-constrained optimization,
- Gauss–Newton and Fisher-based approximations.

They are used selectively due to computational cost and stochastic noise.

Table 3.10: Comparison of PyTorch behavior during training and inference

Aspect	**Training mode (autograd enabled)**	**Inference mode (autograd disabled)**
Forward pass behavior	Records the full computational graph of tensor operations	Performs forward computation only
Intermediate activations	Stored in memory for backpropagation	Not stored
Gradient tracking	Tracks tensors with `requires_grad=True`	No gradient tracking
Operation metadata	Saved to enable chain rule application during backward pass	Not saved
Backward pass	Required to compute gradients of the loss	Not performed
Gradient storage	Gradients accumulated in `.grad` fields of parameters	No gradient buffers allocated
Higher-order derivatives	Supported when explicitly requested (e.g., Hessian–vector products)	Not available
Memory usage	Higher due to graph and activation storage	Significantly reduced
Runtime performance	Slower due to bookkeeping overhead	Faster and more efficient
Typical use case	Model training and optimization	Evaluation, validation, and deployment
How mode is enabled	Default behavior when gradients are required	Using `torch.no_grad()` and `model.eval()`

3.12 Training vs. Inference in Automatic Differentiation Frameworks

Key takeaway. During training, PyTorch retains intermediate computations and constructs a computational graph to enable automatic differentiation. During inference, this bookkeeping is disabled to reduce memory usage and improve execution speed, while preserving the same forward computation.

Clarification: meaning of the computational graph. In this context, the term *graph* refers to PyTorch's dynamic computational graph constructed during the forward pass when automatic differentiation is enabled. This graph is a directed acyclic graph whose nodes correspond to tensor operations and whose edges represent data dependencies between tensors. Each node stores the local derivative information required to apply the chain rule during backpropagation, along with references to its parent tensors. During the backward pass, PyTorch traverses this graph in reverse to compute gradients of the loss with respect to model parameters.

Chapter 4

Jacobians in Backpropagation and Second-Order Analysis

4.1 Overview

Backpropagation is often introduced algorithmically as a sequence of local gradient computations propagated backward through a network. At a mathematical level, however, backpropagation is an exact application of the multivariable chain rule and can be understood as repeated multiplication by Jacobian transposes.

This chapter presents a unified Jacobian-based interpretation of backpropagation, explains why explicit Jacobian matrices are rarely constructed in practice, and illustrates the concepts through both symbolic and numerical examples. The connection to second-order methods and Hessian–vector products is also clarified.

4.2 Jacobian View of the Chain Rule

Consider a feedforward network expressed as a composition of functions:

$$\mathbf{a}_0 = \mathbf{x}, \qquad \mathbf{a}_k = f_k(\mathbf{a}_{k-1}), \quad k = 1, \dots, L,$$

with scalar loss

$$\mathcal{L} = \ell(\mathbf{a}_L).$$

The Jacobian of layer k with respect to its input is

$$\mathbf{J}_k = \frac{\partial \mathbf{a}_k}{\partial \mathbf{a}_{k-1}}.$$

Applying the multivariable chain rule yields

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_0} = \mathbf{J}_1^\top \mathbf{J}_2^\top \cdots \mathbf{J}_L^\top \frac{\partial \mathcal{L}}{\partial \mathbf{a}_L}.$$

Thus, backpropagation is mathematically equivalent to multiplying the upstream gradient by a sequence of Jacobian transposes.

4.3 Why Jacobians Are Not Formed Explicitly

If layer k maps $\mathbb{R}^{d_{k-1}} \to \mathbb{R}^{d_k}$, then

$$\mathbf{J}_k \in \mathbb{R}^{d_k \times d_{k-1}}.$$

In modern neural networks, d_k and d_{k-1} may be millions, making explicit construction of $\mathbf{J}_k$ infeasible.

Backpropagation requires only the product

$$\mathbf{J}_k^\top \mathbf{v},$$

where $\mathbf{v}$ is the upstream gradient. Automatic differentiation frameworks compute this product implicitly using local derivatives and the chain rule, avoiding explicit Jacobian storage.

This operation is known as a **vector–Jacobian product (VJP)**.

4.4 Three-Layer Network: Matrix Form

Consider a three-layer feedforward network:

$$\mathbf{a}_0 = \mathbf{x},$$

$$\mathbf{z}_1 = \mathbf{W}_1\mathbf{a}_0 + \mathbf{b}_1, \quad \mathbf{a}_1 = \phi(\mathbf{z}_1),$$

$$\mathbf{z}_2 = \mathbf{W}_2\mathbf{a}_1 + \mathbf{b}_2, \quad \mathbf{a}_2 = \psi(\mathbf{z}_2),$$

$$z_3 = \mathbf{W}_3\mathbf{a}_2 + b_3, \quad \hat{y} = \sigma(z_3),$$

with scalar loss

$$\mathcal{L} = \tfrac{1}{2}(\hat{y} - y)^2.$$

The Jacobians of each layer with respect to its input are:

$$\mathbf{J}_1 = \operatorname{diag}(\phi'(\mathbf{z}_1))\,\mathbf{W}_1,$$

$$\mathbf{J}_2 = \operatorname{diag}(\psi'(\mathbf{z}_2))\,\mathbf{W}_2,$$

$$\mathbf{J}_3 = \sigma'(z_3)\,\mathbf{W}_3.$$

Backpropagation computes:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_2} = (\hat{y} - y)\,\mathbf{J}_3, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_2}\mathbf{J}_2, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{a}_0} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_1}\mathbf{J}_1.$$

Equivalently, for any intermediate activation $\mathbf{a}_n$, the chain rule gives

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_n} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}}\,\mathbf{J}_L\mathbf{J}_{L-1}\cdots\mathbf{J}_{n+1},$$

where $\mathbf{J}_k = \partial \mathbf{a}_k / \partial \mathbf{a}_{k-1}$. This expression makes explicit that backpropagation is a sequence of vector–Jacobian products propagating the loss gradient from the output layer back to layer n.

4.5 Numerical Example with Explicit Jacobians (with activation and loss derivatives)

We use a three-layer network with three different nonlinearities: Layer 1 uses ReLU, Layer 2 uses tanh, and the output layer uses a sigmoid. The loss is mean squared error (single sample).

4.5.1 Network definition

Let $d_0 = d_1 = d_2 = 2$ and define

$$\mathbf{a}_0 = \mathbf{x}, \qquad \mathbf{z}_1 = \mathbf{W}_1\mathbf{a}_0 + \mathbf{b}_1, \quad \mathbf{a}_1 = \mathrm{ReLU}(\mathbf{z}_1),$$

$$\mathbf{z}_2 = \mathbf{W}_2\mathbf{a}_1 + \mathbf{b}_2, \quad \mathbf{a}_2 = \tanh(\mathbf{z}_2),$$

$$z_3 = \mathbf{W}_3\mathbf{a}_2 + b_3, \quad \hat{y} = \sigma(z_3), \qquad \mathcal{L} = \tfrac{1}{2}(\hat{y} - y)^2.$$

We choose

$$\mathbf{x} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad \mathbf{W}_1 = \begin{pmatrix} 1 & 2 \\ -0.5 & 1.5 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix},$$

$$\mathbf{W}_2 = \begin{pmatrix} 0.7 & -1.2 \\ 1.1 & 0.3 \end{pmatrix}, \quad \mathbf{b}_2 = \begin{pmatrix} 0.05 \\ -0.1 \end{pmatrix}, \quad \mathbf{W}_3 = \begin{pmatrix} 1.4 & -0.8 \end{pmatrix}, \quad b_3 = 0.2, \quad y = 1.$$

4.5.2 Activation derivatives and loss derivative

The element-wise derivatives used in the Jacobians are:

$$\frac{d}{dz}\mathrm{ReLU}(z) = \begin{cases} 1, & z > 0 \\ 0, & z \le 0 \end{cases}, \qquad \frac{d}{dz}\tanh(z) = 1 - \tanh^2(z), \qquad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

The loss derivative with respect to the prediction is

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y.$$

4.5.3 Forward pass (numerical)

Layer 1 (ReLU).

$$\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1 = \begin{pmatrix} 1 & 2 \\ -0.5 & 1.5 \end{pmatrix}\begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix} = \begin{pmatrix} 4.1 \\ 0.3 \end{pmatrix}.$$

$$\mathbf{a}_1 = \mathrm{ReLU}(\mathbf{z}_1) = \begin{pmatrix} 4.1 \\ 0.3 \end{pmatrix} \quad \text{(both entries are positive).}$$

Layer 2 (tanh).

$$\mathbf{z}_2 = \mathbf{W}_2\mathbf{a}_1 + \mathbf{b}_2 = \begin{pmatrix} 0.7 & -1.2 \\ 1.1 & 0.3 \end{pmatrix}\begin{pmatrix} 4.1 \\ 0.3 \end{pmatrix} + \begin{pmatrix} 0.05 \\ -0.1 \end{pmatrix} = \begin{pmatrix} 2.56 \\ 4.50 \end{pmatrix}.$$

$$\mathbf{a}_2 = \tanh(\mathbf{z}_2) = \begin{pmatrix} \tanh(2.56) \\ \tanh(4.50) \end{pmatrix} = \begin{pmatrix} 0.988119 \\ 0.999753 \end{pmatrix}.$$

Output layer (sigmoid).

$$z_3 = \mathbf{W}_3\mathbf{a}_2 + b_3 = \begin{pmatrix} 1.4 & -0.8 \end{pmatrix}\begin{pmatrix} 0.988119 \\ 0.999753 \end{pmatrix} + 0.2 = 0.783564.$$

$$\hat{y} = \sigma(z_3) = \frac{1}{1 + e^{-z_3}} = 0.686448, \qquad \mathcal{L} = \tfrac{1}{2}(\hat{y} - y)^2 = \tfrac{1}{2}(0.686448 - 1)^2.$$

4.5.4 Local derivatives (numerical)

Loss derivative.

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = 0.686448 - 1 = -0.313552.$$

Sigmoid derivative.

$$\sigma'(z_3) = \hat{y}(1-\hat{y}) = 0.686448(1 - 0.686448) = 0.215271.$$

tanh **derivative (element-wise).**

$$\tanh'(\mathbf{z}_2) = 1 - \mathbf{a}_2^{\odot 2} = \begin{pmatrix} 1 - 0.988119^2 \\ 1 - 0.999753^2 \end{pmatrix} = \begin{pmatrix} 0.023620 \\ 0.000494 \end{pmatrix}.$$

ReLU derivative (element-wise). Since $\mathbf{z}_1 = (4.1, 0.3)^\top$ is strictly positive component-wise,

$$\mathrm{ReLU}'(\mathbf{z}_1) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

4.5.5 Explicit Jacobians

Jacobian of Layer 1: $\mathbf{a}_1 = \mathrm{ReLU}(\mathbf{z}_1)$, $\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$.

$$\mathbf{J}_1 = \frac{\partial \mathbf{a}_1}{\partial \mathbf{x}} = \mathrm{diag}(\mathrm{ReLU}'(\mathbf{z}_1))\,\mathbf{W}_1 = \mathbf{I}\,\mathbf{W}_1 = \mathbf{W}_1.$$

Jacobian of Layer 2: $\mathbf{a}_2 = \tanh(\mathbf{z}_2)$, $\mathbf{z}_2 = \mathbf{W}_2\mathbf{a}_1 + \mathbf{b}_2$.

$$\mathbf{J}_2 = \frac{\partial \mathbf{a}_2}{\partial \mathbf{a}_1} = \mathrm{diag}(\tanh'(\mathbf{z}_2))\,\mathbf{W}_2 = \begin{pmatrix} 0.023620 & 0 \\ 0 & 0.000494 \end{pmatrix}\begin{pmatrix} 0.7 & -1.2 \\ 1.1 & 0.3 \end{pmatrix} = \begin{pmatrix} 0.016535 & -0.028345 \\ 0.000543 & 0.000148 \end{pmatrix}.$$

Jacobian of Output: $\hat{y} = \sigma(z_3)$, $z_3 = \mathbf{W}_3\mathbf{a}_2 + b_3$.

$$\mathbf{J}_3 = \frac{\partial \hat{y}}{\partial \mathbf{a}_2} = \sigma'(z_3)\,\mathbf{W}_3 = 0.215271\begin{pmatrix} 1.4 & -0.8 \end{pmatrix} = \begin{pmatrix} 0.301332 & -0.172190 \end{pmatrix}.$$

4.5.6 Backpropagation via Jacobian products (numerical)

Backpropagation is a sequence of vector–Jacobian products:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_2} = \frac{\partial \mathcal{L}}{\partial \hat{y}}\,\frac{\partial \hat{y}}{\partial \mathbf{a}_2} = (\hat{y} - y)\,\mathbf{J}_3.$$

Numerically,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_2} = (-0.313552)\begin{pmatrix} 0.301332 & -0.172190 \end{pmatrix} = \begin{pmatrix} -0.094506 & 0.053988 \end{pmatrix}.$$

Next,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_2}\,\mathbf{J}_2,$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} = \begin{pmatrix} -0.094506 & 0.053988 \end{pmatrix}\begin{pmatrix} 0.016535 & -0.028345 \\ 0.000543 & 0.000148 \end{pmatrix} = \begin{pmatrix} -0.001435 & 0.002267 \end{pmatrix}.$$

Finally,

$$\nabla_{\mathbf{x}}\mathcal{L} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_1}\,\mathbf{J}_1 = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_1}\,\mathbf{W}_1,$$

$$\nabla_{\mathbf{x}}\mathcal{L} = \begin{pmatrix} -0.001435 & 0.002267 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ -0.5 & 1.5 \end{pmatrix} = \begin{pmatrix} -0.002876 & 0.000531 \end{pmatrix}.$$

Equivalently, written as a column vector:

$$\nabla_{\mathbf{x}}\mathcal{L} = \begin{pmatrix} -0.002876 \\ 0.000531 \end{pmatrix}.$$

4.6 Same Numerical Example via Explicit Chain Rule (no Jacobian shortcut)

This section computes the same gradient as Section **??** (Jacobian products), but now by writing the chain rule explicitly using the loss derivative and the activation derivatives.

4.6.1 Local derivatives used

Recall

$$\mathcal{L} = \tfrac{1}{2}(\hat{y} - y)^2, \qquad \hat{y} = \sigma(z_3), \qquad z_3 = \mathbf{W}_3\mathbf{a}_2 + b_3,$$

$$\mathbf{a}_2 = \tanh(\mathbf{z}_2), \qquad \mathbf{z}_2 = \mathbf{W}_2\mathbf{a}_1 + \mathbf{b}_2, \qquad \mathbf{a}_1 = \mathrm{ReLU}(\mathbf{z}_1), \qquad \mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1.$$

The scalar loss derivative is

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y.$$

The activation derivatives are

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \qquad \tanh'(z) = 1 - \tanh^2(z), \qquad \mathrm{ReLU}'(z) = \mathbb{1}[z > 0].$$

4.6.2 Forward values (from the numerical example)

Using the same numerical values as in the Jacobian section:

$$\mathbf{z}_1 = \begin{pmatrix} 4.1 \\ 0.3 \end{pmatrix}, \quad \mathbf{a}_1 = \begin{pmatrix} 4.1 \\ 0.3 \end{pmatrix}, \quad \mathbf{z}_2 = \begin{pmatrix} 2.56 \\ 4.50 \end{pmatrix}, \quad \mathbf{a}_2 = \begin{pmatrix} 0.98811896 \\ 0.99975321 \end{pmatrix},$$

$$z_3 = 0.78356397, \quad \hat{y} = 0.68644772, \quad y = 1.$$

Therefore

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = -0.31355228, \qquad \sigma'(z_3) = \hat{y}(1 - \hat{y}) = 0.21523725.$$

For the tanh layer (element-wise):

$$\tanh'(\mathbf{z}_2) = \begin{pmatrix} 1 - a_{2,1}^2 \\ 1 - a_{2,2}^2 \end{pmatrix} = \begin{pmatrix} 0.02362093 \\ 0.00049352 \end{pmatrix}.$$

For ReLU:

$$\mathrm{ReLU}'(\mathbf{z}_1) = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad (\text{since } z_{1,1} = 4.1 > 0,\ z_{1,2} = 0.3 > 0).$$

4.6.3 Backward pass by chain rule (step-by-step)

Step 1: from loss to output pre-activation. Because $\hat{y} = \sigma(z_3)$,

$$\frac{\partial \mathcal{L}}{\partial z_3} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3} = (\hat{y} - y)\, \sigma'(z_3).$$

Numerically:

$$\frac{\partial \mathcal{L}}{\partial z_3} = (-0.31355228)(0.21523725) = -0.06748670.$$

Step 2: from z_3 to $\mathbf{a}_2$. Since $z_3 = \mathbf{W}_3 \mathbf{a}_2 + b_3$ with $\mathbf{W}_3 = (1.4,\ -0.8)$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_2} = \frac{\partial \mathcal{L}}{\partial z_3} \frac{\partial z_3}{\partial \mathbf{a}_2} = \frac{\partial \mathcal{L}}{\partial z_3} \mathbf{W}_3.$$

Numerically:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_2} = (-0.06748670) \begin{pmatrix} 1.4 & -0.8 \end{pmatrix} = \begin{pmatrix} -0.09448138 & 0.05398936 \end{pmatrix}.$$

Step 3: from $\mathbf{a}_2$ to $\mathbf{z}_2$ (tanh gate). Because $\mathbf{a}_2 = \tanh(\mathbf{z}_2)$ element-wise,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_2} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_2} \operatorname{diag}\big(\tanh'(\mathbf{z}_2)\big).$$

Numerically:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_2} = \begin{pmatrix} -0.09448138 & 0.05398936 \end{pmatrix} \begin{pmatrix} 0.02362093 & 0 \\ 0 & 0.00049352 \end{pmatrix} = \begin{pmatrix} -0.00223199 & 0.00002664 \end{pmatrix}.$$

Step 4: from $\mathbf{z}_2$ to $\mathbf{a}_1$ (linear map). Since $\mathbf{z}_2 = \mathbf{W}_2 \mathbf{a}_1 + \mathbf{b}_2$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_2} \frac{\partial \mathbf{z}_2}{\partial \mathbf{a}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_2} \mathbf{W}_2,$$

with

$$\mathbf{W}_2 = \begin{pmatrix} 0.7 & -1.2 \\ 1.1 & 0.3 \end{pmatrix}.$$

Numerically:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} = \begin{pmatrix} -0.00223199 & 0.00002664 \end{pmatrix} \begin{pmatrix} 0.7 & -1.2 \\ 1.1 & 0.3 \end{pmatrix} = \begin{pmatrix} -0.00153294 & 0.00268614 \end{pmatrix}.$$

Step 5: from $\mathbf{a}_1$ to $\mathbf{z}_1$ (ReLU gate). Because $\mathbf{a}_1 = \mathrm{ReLU}(\mathbf{z}_1)$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} \operatorname{diag}\big(\mathrm{ReLU}'(\mathbf{z}_1)\big).$$

Here $\mathrm{ReLU}'(\mathbf{z}_1) = (1, 1)^\top$, so the diagonal is $\mathbf{I}$ and

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} = \begin{pmatrix} -0.00153294 & 0.00268614 \end{pmatrix}.$$

Step 6: from $\mathbf{z}_1$ to input $\mathbf{x}$. Since $\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$,

$$\nabla_{\mathbf{x}}\mathcal{L} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_1}\,\mathbf{W}_1, \quad \mathbf{W}_1 = \begin{pmatrix} 1 & 2 \\ -0.5 & 1.5 \end{pmatrix}.$$

Numerically:

$$\nabla_{\mathbf{x}}\mathcal{L} = \begin{pmatrix} -0.00153294 & 0.00268614 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ -0.5 & 1.5 \end{pmatrix} = \begin{pmatrix} -0.00287601 & 0.00096332 \end{pmatrix}.$$

Equivalently as a column vector:

$$\nabla_{\mathbf{x}}\mathcal{L} = \begin{pmatrix} -0.00287601 \\ 0.00096332 \end{pmatrix}.$$

4.6.4 Connection to the Jacobian form

Grouping the same steps into layer Jacobians gives exactly:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_2} = (\hat{y} - y)\,\mathbf{J}_3, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_2}\mathbf{J}_2, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{a}_0} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_1}\mathbf{J}_1,$$

with

$$\mathbf{J}_1 = \operatorname{diag}(\mathrm{ReLU}'(\mathbf{z}_1))\,\mathbf{W}_1, \qquad \mathbf{J}_2 = \operatorname{diag}(\tanh'(\mathbf{z}_2))\,\mathbf{W}_2, \qquad \mathbf{J}_3 = \sigma'(z_3)\,\mathbf{W}_3.$$

Interpretation.

- The loss derivative $\partial\mathcal{L}/\partial\hat{y}$ produces the initial upstream signal.
- The output Jacobian $\mathbf{J}_3$ injects the sigmoid derivative and the linear weights $\mathbf{W}_3$.
- The hidden Jacobian $\mathbf{J}_2$ injects the tanh derivative via a diagonal gate.
- The input Jacobian $\mathbf{J}_1$ injects the ReLU derivative via another diagonal gate.
- At no point is any full Jacobian of the entire network constructed; only local Jacobians are used in products.

Note. Minor differences at the last digit can occur depending on rounding of $\tanh(\cdot)$ and $\sigma(\cdot)$.

4.7 Connection to Second-Order Derivatives

The Jacobian framework naturally extends to second-order analysis. Given the gradient $\nabla f(\boldsymbol{\theta})$, the Hessian–vector product is defined as

$$\mathbf{H}\mathbf{u} = \nabla_{\boldsymbol{\theta}}\big(\nabla f(\boldsymbol{\theta})^\top \mathbf{u}\big).$$

This computation requires:

- one forward pass,
- one backward pass to compute $\nabla f(\boldsymbol{\theta})$,
- formation of the scalar $\nabla f(\boldsymbol{\theta})^\top \mathbf{u}$,
- a second backward pass.

No Hessian matrix is ever formed.

4.8 Summary

- Backpropagation is mathematically equivalent to multiplying by Jacobian transposes.
- Automatic differentiation computes vector–Jacobian products implicitly.
- Explicit Jacobian matrices are avoided due to prohibitive size.
- The same machinery enables efficient computation of Hessian–vector products.
- First-order and second-order optimization methods differ only in how many backward passes they require.

4.9 Worked Example III: Full Forward/Backward Pass and Gradient Descent Update

We consider a three-layer scalar network with three different activation functions: Layer 1 uses ReLU, Layer 2 uses tanh, and the output layer uses a sigmoid. The loss is mean squared error (single sample):

$$\mathcal{L} = \tfrac{1}{2}\big(\hat{y} - y\big)^2.$$

4.9.1 Network definition

$$\begin{aligned} z_1 &= w_1 x + b_1, \qquad a_1 = \mathrm{ReLU}(z_1), \\ z_2 &= w_2 a_1 + b_2, \qquad a_2 = \tanh(z_2), \\ z_3 &= w_3 a_2 + b_3, \qquad \hat{y} = \sigma(z_3), \qquad \mathcal{L} = \tfrac{1}{2}(\hat{y} - y)^2. \end{aligned}$$

4.9.2 Derivatives used (local)

$$\frac{d}{dz}\mathrm{ReLU}(z) = \begin{cases} 1, & z > 0 \\ 0, & z \le 0 \end{cases}, \qquad \frac{d}{dz}\tanh(z) = 1 - \tanh^2(z), \qquad \sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big).$$

The loss derivative is

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y.$$

4.9.3 Numerical values

We choose:

$$\begin{gathered} x = 2, \quad y = 1, \\ (w_1, b_1) = (1.5, -1), \qquad (w_2, b_2) = (0.5, 0.1), \qquad (w_3, b_3) = (2, -0.3). \end{gathered}$$

4.9.4 Forward pass (all steps)

Table 4.1: Forward pass for a three-layer network: ReLU → tanh → sigmoid

Step	Quantity	Expression	Value
1	Input	x	2
2	Layer 1 pre-activation	$z_1 = w_1 x + b_1$	$1.5(2) + (-1) = 2.0$
3	Layer 1 activation (ReLU)	$a_1 = \mathrm{ReLU}(z_1)$	2.0
4	Layer 2 pre-activation	$z_2 = w_2 a_1 + b_2$	$0.5(2.0) + 0.1 = 1.1$
5	Layer 2 activation (tanh)	$a_2 = \tanh(z_2)$	0.800499
6	Layer 3 pre-activation	$z_3 = w_3 a_2 + b_3$	$2(0.800499) - 0.3 = 1.300998$
7	Output activation (sigmoid)	$\hat{y} = \sigma(z_3)$	0.786003
8	Target	y	1
9	Loss (MSE, single sample)	$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$	0.022897

4.9.5 Backward pass (explicit derivatives and gradients)

Step A: loss to output.

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = 0.786003 - 1 = -0.213997.$$

Step B: sigmoid derivative.

$$\frac{\partial \hat{y}}{\partial z_3} = \hat{y}(1-\hat{y}) = 0.786003(1-0.786003) = 0.168202.$$

Step C: delta at layer 3.

$$\delta_3 := \frac{\partial \mathcal{L}}{\partial z_3} = \frac{\partial \mathcal{L}}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3} = (-0.213997)(0.168202) = -0.035995.$$

Gradients for w_3, b_3. Since $z_3 = w_3 a_2 + b_3$:

$$\frac{\partial \mathcal{L}}{\partial w_3} = \delta_3 \frac{\partial z_3}{\partial w_3} = \delta_3 a_2 = (-0.035995)(0.800499) = -0.028814,$$

$$\frac{\partial \mathcal{L}}{\partial b_3} = \delta_3 \frac{\partial z_3}{\partial b_3} = \delta_3 = -0.035995.$$

Step D: propagate to a_2.

$$\frac{\partial \mathcal{L}}{\partial a_2} = \delta_3 \frac{\partial z_3}{\partial a_2} = \delta_3 w_3 = (-0.035995)(2) = -0.071990.$$

Step E: tanh derivative.

$$\frac{\partial a_2}{\partial z_2} = 1 - a_2^2 = 1 - (0.800499)^2 = 0.359201.$$

Step F: delta at layer 2.

$$\delta_2 := \frac{\partial \mathcal{L}}{\partial z_2} = \frac{\partial \mathcal{L}}{\partial a_2}\frac{\partial a_2}{\partial z_2} = (-0.071990)(0.359201) = -0.025859.$$

Gradients for w_2, b_2. Since $z_2 = w_2 a_1 + b_2$:

$$\frac{\partial \mathcal{L}}{\partial w_2} = \delta_2 \frac{\partial z_2}{\partial w_2} = \delta_2 a_1 = (-0.025859)(2.0) = -0.051718,$$

$$\frac{\partial \mathcal{L}}{\partial b_2} = \delta_2 = -0.025859.$$

Step G: propagate to a_1.

$$\frac{\partial \mathcal{L}}{\partial a_1} = \delta_2 \frac{\partial z_2}{\partial a_1} = \delta_2 w_2 = (-0.025859)(0.5) = -0.012929.$$

Step H: ReLU derivative. Here $z_1 = 2.0 > 0$, so $\mathrm{ReLU}'(z_1) = 1$.

Step I: delta at layer 1.

$$\delta_1 := \frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial a_1}\mathrm{ReLU}'(z_1) = (-0.012929)(1) = -0.012929.$$

Gradients for w_1, b_1. Since $z_1 = w_1 x + b_1$:

$$\frac{\partial \mathcal{L}}{\partial w_1} = \delta_1 \frac{\partial z_1}{\partial w_1} = \delta_1 x = (-0.012929)(2) = -0.025859,$$

$$\frac{\partial \mathcal{L}}{\partial b_1} = \delta_1 = -0.012929.$$

4.9.6 Gradient summary

Collecting all gradients:

$$\frac{\partial \mathcal{L}}{\partial w_1} = -0.025859, \quad \frac{\partial \mathcal{L}}{\partial b_1} = -0.012929,$$
$$\frac{\partial \mathcal{L}}{\partial w_2} = -0.051718, \quad \frac{\partial \mathcal{L}}{\partial b_2} = -0.025859,$$
$$\frac{\partial \mathcal{L}}{\partial w_3} = -0.028814, \quad \frac{\partial \mathcal{L}}{\partial b_3} = -0.035995.$$

4.9.7 Gradient descent update (one step)

Let $\eta > 0$ be the learning rate. Gradient descent updates each parameter by

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}.$$

Therefore,

$$w_1^+ = w_1 - \eta \frac{\partial \mathcal{L}}{\partial w_1} = 1.5 - \eta(-0.025859) = 1.5 + 0.025859\,\eta, \quad b_1^+ = b_1 - \eta \frac{\partial \mathcal{L}}{\partial b_1} = -1 - \eta(-0.012929) = -1 + 0.01292$$
$$w_2^+ = 0.5 - \eta(-0.051718) = 0.5 + 0.051718\,\eta, \quad b_2^+ = 0.1 - \eta(-0.025859) = 0.1 + 0.025859\,\eta,$$
$$w_3^+ = 2 - \eta(-0.028814) = 2 + 0.028814\,\eta, \quad b_3^+ = -0.3 - \eta(-0.035995) = -0.3 + 0.035995\,\eta.$$

Numerical update example. For $\eta = 0.1$:

$$w_1^+ = 1.502586, \quad b_1^+ = -0.998707, \quad w_2^+ = 0.505172, \quad b_2^+ = 0.102586, \quad w_3^+ = 2.002881, \quad b_3^+ = -0.296401.$$

Interpretation. All gradients above are negative, so each parameter increases slightly under the update rule $\theta \leftarrow \theta - \eta\,\partial\mathcal{L}/\partial\theta$. This is consistent with reducing the error $\hat{y} - y < 0$ for this sample.

4.10 Jacobian, Delta, and Parameter Gradients in Backpropagation

Yes — the summary is **conceptually correct**. We restate it concisely below, tightening the language and highlighting one subtle but important point to establish a clear and robust mental model.

4.10.1 Jacobian

Jacobian denotes the derivative of a layer's output with respect to its input. For a layer defined as

$$a_\ell = f_\ell(a_{\ell-1}),$$

the Jacobian is

$$J_\ell = \frac{\partial a_\ell}{\partial a_{\ell-1}}.$$

The Jacobian:

- describes the **local transformation** performed by the layer,
- does **not** depend on the loss function,
- is a property of the layer itself (weights and activation).

In practice, full Jacobian matrices are never formed explicitly; instead, backpropagation relies on efficient **vector–Jacobian products**.

4.10.2 Delta

Delta is the gradient of the final loss with respect to the pre-activation (latent variable) of a layer. Formally,

$$\delta_\ell = \frac{\partial \mathcal{L}}{\partial z_\ell}.$$

Delta:

- **depends on the final loss**,
- is the quantity that propagates backward through the network,
- lives in the same space as the latent variable z_ℓ.

Thus, describing delta as the "gradient of the final loss with respect to the latent or hidden representation" is exactly correct.

4.10.3 Gradients of Weights and Biases

Gradients of the parameters are always gradients of the **final loss**. Specifically,

$$\frac{\partial \mathcal{L}}{\partial W_\ell}, \quad \frac{\partial \mathcal{L}}{\partial b_\ell}.$$

These gradients are computed directly from delta:

$$\frac{\partial \mathcal{L}}{\partial W_\ell} = \delta_\ell \, a_{\ell-1}^\top, \qquad \frac{\partial \mathcal{L}}{\partial b_\ell} = \delta_\ell.$$

Weights and biases never possess local gradients independent of the loss.

4.10.4 Conceptual Hierarchy

Object	Interpretation
Jacobian J_ℓ	Local sensitivity of layer output to its input
Delta δ_ℓ	Gradient of the final loss w.r.t. layer pre-activation
$\nabla_{W_\ell}\mathcal{L}$	Gradient of the final loss w.r.t. weights
$\nabla_{b_\ell}\mathcal{L}$	Gradient of the final loss w.r.t. bias

$$z_\ell = W_\ell a_{\ell-1} + b_\ell \qquad \text{(pre-activation)}$$

$$a_\ell = \phi_\ell(z_\ell) \qquad \text{(post-activation)}$$

$$J_\ell = \frac{\partial a_\ell}{\partial a_{\ell-1}} \qquad \text{(layer Jacobian)}$$

$$\delta_L = \frac{\partial \mathcal{L}}{\partial z_L} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_L} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \, \phi_L'(z_L) \qquad \text{(delta; output-layer pre-activation gradient, chain rule)}$$

$$\delta_\ell = \frac{\partial \mathcal{L}}{\partial z_\ell} = \left(W_{\ell+1}^\top \delta_{\ell+1}\right) \odot \phi_\ell'(z_\ell), \quad \ell < L \qquad \text{(delta; hidden-layer pre-activation gradient)}$$

$$\nabla_{W_\ell}\mathcal{L} = \frac{\partial \mathcal{L}}{\partial W_\ell} = \delta_\ell \, a_{\ell-1}^\top \qquad \text{(weight gradient)}$$

$$\nabla_{b_\ell}\mathcal{L} = \frac{\partial \mathcal{L}}{\partial b_\ell} = \delta_\ell \qquad \text{(bias gradient)}$$

4.10.5 A Subtle but Important Clarification

Referring to the Jacobian as the "gradient of a layer's output with respect to its input" is correct, provided one remembers that:

- the Jacobian is generally **matrix-valued**,
- a gradient is a special case corresponding to scalar output.

Thus, gradients are a subset of Jacobians, and in deep networks Jacobians are almost always handled implicitly.

4.10.6 Mental Model

> **Jacobians describe local layer physics; deltas carry global error; parameter gradients convert error into learning.**

4.10.7 Summary

- Jacobians represent local layer derivatives,
- deltas represent gradients of the final loss with respect to latent variables,
- weight and bias gradients are always gradients of the final loss,
- backpropagation consists of chaining Jacobians with deltas.

This structure applies uniformly across multilayer perceptrons, convolutional networks, transformers, and automatic differentiation systems.

4.10.8 Chronological computation table (forward → backward → update, with Jacobians)

4.11 Worked Micro-Example: The First Backprop Signal (dv_7) Comes From the Loss

This section emphasizes a simple but universal point: *the backward pass must start from the derivative of the chosen loss with respect to the final node.* In other words, the **first upstream signal** is determined entirely by the loss function.

4.11.1 Forward computation (scalar graph)

Consider the scalar computation

$$v_1 = x_1, \qquad v_2 = x_2, \qquad v_3 = \exp(v_2), \qquad v_4 = v_1 v_3, \qquad v_5 = v_1 v_2, \qquad v_6 = \cos(v_5), \qquad v_7 = v_4 + v_6.$$

Thus the final output is

$$v_7(x_1, x_2) = x_1 e^{x_2} + \cos(x_1 x_2).$$

4.11.2 The universal first step in backprop

Backpropagation begins at the output node by differentiating the loss $\mathcal{L}$ with respect to v_7:

$$\boxed{dv_7 \;=\; \frac{\partial \mathcal{L}}{\partial v_7}}$$

This quantity dv_7 is the **initial upstream gradient** that seeds the entire backward pass. Once dv_7 is known, all other local derivatives propagate backward through the same chain rule mechanics.

Table 4.2: Chronological calculations with explicit Jacobians J_1, J_2, J_3

Step	Quantity	Formula	Value	Jacobian role
Forward pass (explicitly computed)				
1	Input	x	2	—
2	Pre-act (L1)	$z_1 = w_1 x + b_1$	2.0	—
3	Act (L1, ReLU)	$a_1 = \mathrm{ReLU}(z_1)$	2.0	—
4	Pre-act (L2)	$z_2 = w_2 a_1 + b_2$	1.1	—
5	Act (L2, tanh)	$a_2 = \tanh(z_2)$	0.800499	—
6	Pre-act (L3)	$z_3 = w_3 a_2 + b_3$	1.300998	—
7	Output	$\hat{y} = \sigma(z_3)$	0.786003	—
8	Loss	$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$	0.022897	—
Local derivatives (explicitly computed)				
9	Loss deriv.	$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y$	-0.213997	scalar loss derivative
10	Sigmoid deriv.	$\sigma'(z_3) = \hat{y}(1 - \hat{y})$	0.168202	local nonlinearity
11	tanh deriv.	$\tanh'(z_2) = 1 - a_2^2$	0.359201	local nonlinearity
12	ReLU deriv.	$\mathrm{ReLU}'(z_1) = \mathbb{1}[z_1 > 0]$	1	local nonlinearity
Backward pass (Jacobian–vector products)				
13	Delta (L3)	$\delta_3 = \frac{\partial \mathcal{L}}{\partial z_3} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \sigma'(z_3)$	-0.035995	chain rule
14	J_3	$J_3 = \frac{\partial \hat{y}}{\partial a_2} = \sigma'(z_3) w_3$	0.336404	conceptual Jacobian
15	Grad w_3	$\frac{\partial \mathcal{L}}{\partial w_3} = \delta_3 a_2$	-0.028814	via J_3
16	Grad b_3	$\frac{\partial \mathcal{L}}{\partial b_3} = \delta_3$	-0.035995	via J_3
17	Prop to a_2	$\frac{\partial \mathcal{L}}{\partial a_2} = \frac{\partial \mathcal{L}}{\partial \hat{y}} J_3$	-0.071990	vector–Jacobian product
18	Delta (L2)	$\delta_2 = \frac{\partial \mathcal{L}}{\partial z_2} = \frac{\partial \mathcal{L}}{\partial a_2} \tanh'(z_2)$	-0.025859	chain rule
19	J_2	$J_2 = \frac{\partial a_2}{\partial a_1} = (1 - a_2^2) w_2$	0.179600	conceptual Jacobian
20	Grad w_2	$\frac{\partial \mathcal{L}}{\partial w_2} = \delta_2 a_1$	-0.051718	via J_2
21	Grad b_2	$\frac{\partial \mathcal{L}}{\partial b_2} = \delta_2$	-0.025859	via J_2
22	Prop to a_1	$\frac{\partial \mathcal{L}}{\partial a_1} = \frac{\partial \mathcal{L}}{\partial a_2} J_2$	-0.012929	vector–Jacobian product
23	Delta (L1)	$\delta_1 = \frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial a_1} \mathrm{ReLU}'(z_1)$	-0.012929	chain rule
24	J_1	$J_1 = \frac{\partial a_1}{\partial x} = \mathrm{ReLU}'(z_1) w_1$	1.5	conceptual Jacobian
25	Grad w_1	$\frac{\partial \mathcal{L}}{\partial w_1} = \delta_1 x$	-0.025859	via J_1
26	Grad b_1	$\frac{\partial \mathcal{L}}{\partial b_1} = \delta_1$	-0.012929	via J_1
Gradient descent update (explicitly applied)				
27	Update rule	$\theta^+ = \theta - \eta\, \partial \mathcal{L} / \partial \theta$	—	applied to all parameters
28	w_1^+	$1.5 - \eta(-0.025859)$	1.502586 ($\eta = 0.1$)	—
29	b_1^+	$-1 - \eta(-0.012929)$	-0.998707 ($\eta = 0.1$)	—
30	w_2^+	$0.5 - \eta(-0.051718)$	0.505172 ($\eta = 0.1$)	—
31	b_2^+	$0.1 - \eta(-0.025859)$	0.102586 ($\eta = 0.1$)	—
32	w_3^+	$2 - \eta(-0.028814)$	2.002881 ($\eta = 0.1$)	—
33	b_3^+	$-0.3 - \eta(-0.035995)$	-0.296401 ($\eta = 0.1$)	—

Color convention: Blue quantities are explicitly computed during forward or backward propagation. Red quantities denote conceptual Jacobians shown for explanation only and are never formed explicitly.

4.11.3 Why sometimes $dv_7 = 1$

If we choose the objective to be the output itself,

$$\mathcal{L}(x_1, x_2) = v_7(x_1, x_2),$$

then

$$dv_7 = \frac{\partial \mathcal{L}}{\partial v_7} = \frac{\partial v_7}{\partial v_7} = 1.$$

This is the common "minimize the output" setup, and it corresponds to writing in code:

```
dv7 = 1.0.
```

4.11.4 Target-matching: making v_7 approach a desired value

Often we do not want to minimize v_7 directly. Instead, we want v_7 to match a target value t. A standard choice is the squared error loss:

$$\mathcal{L}(x_1, x_2) = \frac{1}{2}\big(v_7(x_1, x_2) - t\big)^2.$$

Differentiating with respect to v_7 gives

$$dv_7 = \frac{\partial \mathcal{L}}{\partial v_7} = \frac{\partial}{\partial v_7}\left[\frac{1}{2}(v_7 - t)^2\right] = v_7 - t.$$

So the only change needed in the backward initialization is:

```
dv7 = v7 - t.
```

Two important special cases are:

$$t = 0 \;\Rightarrow\; dv_7 = v_7, \qquad t = 1 \;\Rightarrow\; dv_7 = v_7 - 1.$$

4.11.5 Different losses give different dv_7 (only the first line changes)

The computational graph (the equations defining $v_1, \ldots, v_7$) does not change. What changes is *only* the loss derivative at the output node. Table 4.3 lists common choices.

Table 4.3: Examples of loss functions and the corresponding initial backprop signal $dv_7 = \partial \mathcal{L}/\partial v_7$.

Goal	**Loss $\mathcal{L}(v_7)$**	$dv_7 = \partial \mathcal{L}/\partial v_7$
Minimize v_7	$\mathcal{L} = v_7$	$dv_7 = 1$
Match target t	$\mathcal{L} = \frac{1}{2}(v_7 - t)^2$	$dv_7 = v_7 - t$
Robust (L1) match	$\mathcal{L} = \lvert v_7 - t \rvert$	$dv_7 = \operatorname{sign}(v_7 - t)$
Soft (log-squared) match	$\mathcal{L} = \log\big(1 + (v_7 - t)^2\big)$	$dv_7 = \dfrac{2(v_7 - t)}{1 + (v_7 - t)^2}$

Interpretation.

- The backward pass always starts by computing $dv_7 = \partial \mathcal{L}/\partial v_7$.
- For $\mathcal{L} = v_7$, the seed gradient is constant: $dv_7 = 1$.
- For target matching $\mathcal{L} = \frac{1}{2}(v_7 - t)^2$, the seed gradient becomes the error: $dv_7 = v_7 - t$.
- After choosing dv_7, the remaining backward pass uses the same chain rule steps through $v_6, v_5, v_4, v_3, v_2, v_1$.

4.12 Parameter Shapes and Gradient Shapes

A frequent source of confusion in backpropagation is why a weight vector sometimes produces a vector gradient, while in other cases the gradient appears as a matrix. The resolution lies entirely in the dimensionality of the parameter itself.

The fundamental rule is:

The gradient of a parameter always has the same shape as the parameter.

This section clarifies this principle by contrasting single-neuron and multi-neuron (layer-wise) parameterizations.

4.12.1 Single Neuron Case

Consider a single neuron with input $a \in \mathbb{R}^n$ and parameters $w \in \mathbb{R}^n$, $b \in \mathbb{R}$:

$$z = w^\top a + b.$$

The loss $\mathcal{L}$ depends on z, and the gradient with respect to the weights is

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z}\frac{\partial z}{\partial w} = \delta\, a, \qquad \delta := \frac{\partial \mathcal{L}}{\partial z}.$$

Thus, the gradient $\partial \mathcal{L}/\partial w$ is a vector with the same shape as w.

4.12.2 Layer Case (Multiple Neurons)

Now consider a fully connected layer with m neurons:

$$z = Wa + b,$$

where

$$W \in \mathbb{R}^{m\times n}, \qquad a \in \mathbb{R}^n, \qquad b \in \mathbb{R}^m, \qquad z \in \mathbb{R}^m.$$

Each row of W corresponds to the weight vector of one neuron. The loss gradient with respect to W is obtained by stacking the gradients of all neurons:

$$\frac{\partial \mathcal{L}}{\partial W} = \begin{pmatrix} \delta_1 a^\top \\ \delta_2 a^\top \\ \vdots \\ \delta_m a^\top \end{pmatrix} = \delta\, a^\top, \qquad \delta := \frac{\partial \mathcal{L}}{\partial z} \in \mathbb{R}^m.$$

This expression is an outer product between the delta vector and the input activation, producing a matrix with the same shape as W.

4.12.3 Summary Table: Parameters and Their Gradients

4.12.4 Operational Summary

For a linear layer $z = Wa + b$, the core backpropagation operations are:

$$\boxed{\begin{aligned} z &= Wa + b, \\ \delta &= \frac{\partial \mathcal{L}}{\partial z}, \\ \nabla_W \mathcal{L} &= \delta\, a^\top, \\ \nabla_b \mathcal{L} &= \delta, \\ \frac{\partial \mathcal{L}}{\partial a} &= W^\top \delta. \end{aligned}}$$

Table 4.4: Relationship between parameters and gradient shapes

Scenario	**Parameter**	**Shape**	**Gradient**	**Gradient Shape**
Single neuron	w	$(n,)$	$\nabla_w \mathcal{L}$	$(n,)$
Single neuron	b	scalar	$\partial \mathcal{L}/\partial b$	scalar
Layer (m neurons)	W	(m, n)	$\nabla_W \mathcal{L}$	(m, n)
Layer (m neurons)	b	$(m,)$	$\nabla_b \mathcal{L}$	$(m,)$
Activation	a	$(n,)$	$\partial \mathcal{L}/\partial a$	$(n,)$
Pre-activation	z	$(m,)$	$\delta = \partial \mathcal{L}/\partial z$	$(m,)$

The outer product $\delta\, a^\top$ appears because each weight connects one input component to one output component, requiring a distinct gradient for every pair.

Key takeaway. A gradient is never larger or smaller than the object it differentiates. Its shape always mirrors the shape of the parameter itself.

4.12.5 Reference Python Implementation

Table 4.5: Distinction between local derivatives, deltas, and gradients in backpropagation

Quantity	**Mathematical form**	**Name**	**Role**
Layer output sensitivity	$\delta_z = \dfrac{\partial \mathcal{L}}{\partial z}$	**Delta**	Backward-propagated error signal
Local layer derivative	$\dfrac{\partial z}{\partial W}$	Local derivative / local Jacobian	Layer-specific sensitivity (loss-independent)
Parameter gradient	$\dfrac{\partial \mathcal{L}}{\partial W}$	**Gradient**	Quantity used for parameter updates
Linear layer relation	$\dfrac{\partial \mathcal{L}}{\partial W} = \dfrac{\partial \mathcal{L}}{\partial z}\dfrac{\partial z}{\partial W}$	Chain rule	Connects delta to gradient

4.13 Linear Algebra Operations in Forward and Backward Propagation

4.14 Commutativity of Linear Algebra Operations

Not all linear algebra operations used in forward and backward propagation are commutative. This section clarifies which operations commute, which do not, and how this impacts backpropagation formulas.

Key identities. For vectors $a, b \in \mathbb{R}^n$,

$$a \mathbin{@} b = a^\top b = b^\top a = b \mathbin{@} a,$$

so the dot product is commutative.

Table 4.6: Commutativity properties of common linear algebra operations

Operation	**Inputs**	**Expression**	**Commutative?**	**Notes**
Dot product	vector, vector	$a \mathbin{@} b$	Yes	$a \mathbin{@} b = b \mathbin{@} a$
Outer product	vector, vector	$\text{outer}(a, b)$	No	$\text{outer}(b, a) = \text{outer}(a, b)^\top$
Matrix–vector	matrix, vector	$W \mathbin{@} a$	No	Order defines the linear map
Matrix–matrix	matrix, matrix	$A \mathbin{@} B$	No	In general $AB \neq BA$

In contrast, the outer product satisfies

$$\text{outer}(a, b) = ab^\top, \qquad \text{outer}(b, a) = ba^\top,$$

and therefore

$$\text{outer}(b, a) = \text{outer}(a, b)^\top,$$

which shows that outer products are *not* commutative.

Implication for backpropagation.

- Forward propagation uses matrix–vector products ($W@a$).
- Backward signal propagation uses transpose maps ($W^\top @ dz$).
- Weight gradients use outer products ($dW = dz\, a^\top$).

The lack of commutativity explains why the ordering of operands in backpropagation formulas is essential and cannot be rearranged arbitrarily.

Summary.

Dot products commute. Outer products transpose. Matrix multiplication does not commute.

Table 4.7: Linear algebra operations used in forward and backward propagation

Context	**Inputs**	**Operation**	**Result**	**Code**
Forward pass	matrix, vector	Linear map	vector	`z = W @ a`
Backward pass (input gradient)	matrix, vector	Transpose linear map	vector	`da = W.T @ dz`
Backward pass (weight gradient)	vector, vector	Outer product	matrix	`dW = np.outer(dz, a)`
Dot product	vector, vector	Inner product (summation)	scalar	`u @ v`
Outer product	vector, vector	No summation	matrix	`np.outer(u, v)`
Matrix product	matrix, matrix	Matrix multiplication	matrix	`A @ B`

The `@` operator propagates signals through summation, whereas the outer product constructs parameter gradients without summation.

For completeness and reproducibility, we list the core Python functions used in all numerical experiments. These functions implement the forward computation, the backward pass via the chain rule, and a single gradient descent update. Only the initialization of the upstream gradient dv_7 differs across loss functions.

Objective function. The scalar output of the computational graph is

$$v_7(x_1, x_2) = x_1 e^{x_2} + \cos(x_1 x_2).$$

```
import math

def f(x1, x2):
    return x1 * math.exp(x2) + math.cos(x1 * x2)
```

Direct minimization of v_7 ($dv_7 = 1$). This function corresponds to the loss $\mathcal{L} = v_7$.

```
def compute_deltas(x1, x2, lr=1e-2, prnt=False, prnt_delta=False):
    # ----- forward pass -----
    v1 = x1
    v2 = x2
    v3 = math.exp(v2)
    v4 = v1 * v3
    v5 = v1 * v2
    v6 = math.cos(v5)
    v7 = v4 + v6
    f_old = v7

    # ----- backward pass -----
    dv7 = 1.0

    dv4 = dv7
    dv6 = dv7
    dv5 = dv6 * (-math.sin(v5))

    dv1 = dv4 * v3 + dv5 * v2
    dv2 = dv5 * v1 + (dv4 * v1) * math.exp(v2)

    # ----- gradient descent update -----
    x1_new = x1 - lr * dv1
    x2_new = x2 - lr * dv2

    if prnt_delta:
        f_new = f(x1_new, x2_new)
        print(f"delta_f = {f_new - f_old}")
        print(f"f_new={f_new}, f_old={f_old}")
        print("-" * 25)

    return x1_new, x2_new
```

Highlighted line. The blue assignment `dv7 = 1.0` encodes the loss $\mathcal{L} = v_7$. Changing the loss function modifies only this line via $dv_7 = \partial\mathcal{L}/\partial v_7$.

Targeted optimization ($v_7 \to t$). This function minimizes the squared loss

$$\mathcal{L} = \tfrac{1}{2}(v_7 - t)^2, \qquad dv_7 = v_7 - t.$$

```
def compute_deltas_make_v7_zero(
    x1, x2, v_target=0, lr=1e-3,
    prnt=False, prnt_delta=False
):
    # ----- forward pass -----
```

```
    v1 = x1
    v2 = x2
    v3 = math.exp(v2)
    v4 = v1 * v3
    v5 = v1 * v2
    v6 = math.cos(v5)
    v7 = v4 + v6

    # ----- loss and initial gradient -----
    dv7 = v7 - v_target
    L_old = 0.5 * (v7 - v_target)**2

    # ----- backward pass -----
    dv4 = dv7
    dv6 = dv7
    dv5 = dv6 * (-math.sin(v5))

    dv1 = dv4 * v3 + dv5 * v2
    dv2 = dv5 * v1 + (dv4 * v1) * math.exp(v2)

    # ----- gradient descent update -----
    x1_new = x1 - lr * dv1
    x2_new = x2 - lr * dv2

    if prnt_delta:
        v7_new = f(x1_new, x2_new)
        L_new = 0.5 * (v7_new - v_target)**2
        print(f"v7_old={v7}, v7_new={v7_new}")
        print(f"L_old={L_old}, L_new={L_new}, delta_L={L_new - L_old}")
        print("-" * 25)

    return x1_new, x2_new
```

Highlighted line. The blue assignment `dv7 = v7 - v_target` corresponds to the squared loss $\mathcal{L} = \frac{1}{2}(v_7 - v_{\text{target}})^2$. The target value enters the optimization exclusively through this initial backpropagation signal.

Remark. The forward graph and all chain-rule computations are identical in both functions. The optimization objective enters the algorithm exclusively through the choice of

$$dv_7 = \frac{\partial \mathcal{L}}{\partial v_7}.$$

4.15 Numerical Experiments: Effect of the Initial Backprop Signal dv_7

This section presents concrete numerical experiments using the scalar computational graph

$$v_7(x_1, x_2) = x_1 e^{x_2} + \cos(x_1 x_2),$$

and illustrates how *changing only the initial backprop signal*

$$dv_7 = \frac{\partial \mathcal{L}}{\partial v_7}$$

alters the qualitative behavior of gradient descent.

All experiments were executed using the Python implementation shown below :contentReference[oaicite:0]index=0.

4.15.1 Direct minimization of v_7 ($dv_7 = 1$)

We first minimize the output directly by choosing

$$\mathcal{L} = v_7, \qquad dv_7 = 1.$$

Code.

```
x1, x2 = 0.6, 0.6

for i in range(10):
    x1, x2 = compute_deltas(
        x1, x2,
        lr=1e-2,
        prnt=False,
        prnt_delta=True
    )

print(f"x1={x1}, x2={x2}")
```

Observed behavior.

- The function value v_7 decreases monotonically.
- The parameters drift toward a region that reduces $x_1 e^{x_2}$.
- No target value is enforced; the algorithm only seeks smaller output values.

A representative outcome after a few steps is

$$x_1 \approx 0.441, \qquad x_2 \approx 0.523,$$

with v_7 steadily decreasing.

4.15.2 Forcing $v_7 \to 0$ (squared loss, $dv_7 = v_7$)

We now change only the loss:

$$\mathcal{L} = \tfrac{1}{2} v_7^2, \qquad dv_7 = v_7.$$

Code.

```
x1, x2 = 0.5, 0.5

for i in range(5000):
    x1, x2 = compute_deltas_make_v7_zero(
        x1, x2,
        v_target=0,
        lr=1e-3,
        prnt=False,
        prnt_delta=False
    )

print(f"x1={x1}, x2={x2}")
```

Observed behavior.

- The loss $\frac{1}{2}v_7^2$ decreases toward zero.
- The parameters converge to a finite solution.
- The gradient magnitude vanishes as $v_7 \to 0$.

  Typical convergence:

$$x_1 \approx -0.555, \qquad x_2 \approx 0.543, \qquad v_7 \approx 0.$$

4.15.3 Forcing $v_7 \to 1$ (shifted squared loss, $dv_7 = v_7 - 1$)

Next, we enforce a nonzero target by choosing

$$\mathcal{L} = \tfrac{1}{2}(v_7 - 1)^2, \qquad dv_7 = v_7 - 1.$$

Code.

```
x1, x2 = 1.5, 1.5

for i in range(5000):
    x1, x2 = compute_deltas_make_v7_zero(
        x1, x2,
        v_target=1,
        lr=1e-3,
        prnt=False,
        prnt_delta=False
    )

print(f"x1={x1}, x2={x2}")
print(f(x1, x2))
```

Observed behavior.

- The parameters adjust until $v_7 \approx 1$.
- Multiple parameter configurations can satisfy the target.
- Large parameter magnitudes may appear if the solution manifold is poorly conditioned.

  One observed outcome:

$$x_1 \approx -42.54, \qquad x_2 \approx -82.86, \qquad v_7 \approx 0.99999.$$

4.15.4 Interpretation

These experiments demonstrate that:

- The forward computational graph is unchanged.
- The backward mechanics are unchanged.
- *Only the initial gradient*

$$dv_7 = \frac{\partial \mathcal{L}}{\partial v_7}$$

  determines the optimization objective.

Thus:

$$\begin{aligned} \mathcal{L} = v_7 &\;\Rightarrow\; dv_7 = 1, \\ \mathcal{L} = \tfrac{1}{2}v_7^2 &\;\Rightarrow\; dv_7 = v_7, \\ \mathcal{L} = \tfrac{1}{2}(v_7 - 1)^2 &\;\Rightarrow\; dv_7 = v_7 - 1. \end{aligned}$$

Key takeaway.

Backpropagation does not encode the goal. The loss function does. The goal enters the algorithm *only* through dv_7.

Chapter 5

The Curse of Dimensionality and Fixed Basis Functions

5.1 Polynomial Basis Functions and Parameter Growth

Consider a polynomial model of order M in D input dimensions. The number of coefficients required grows on the order of

$$\mathcal{O}(D^M). \tag{5.1}$$

This growth is combinatorial and quickly becomes infeasible as the input dimension increases.

Example

Dimension D	Order M	Approx. Parameters
5	3	$\sim 10^2$
50	3	$\sim 10^5$
100	3	$\sim 10^6$
1024 (image)	3	$\sim 10^9$

Table 5.1: Growth of polynomial coefficients with dimensionality

Even for moderate values of D, the number of parameters exceeds what can be reliably estimated from data. This phenomenon is known as the *curse of dimensionality*.

5.2 Grid-Based Basis Functions

An alternative approach is to partition the input space into a regular grid. If each dimension is divided into K intervals, the total number of grid cells is

$$\mathcal{O}(K^D). \tag{5.2}$$

Each grid cell can be interpreted as defining a separate basis function.

Example

Consequences

- Data become sparse, with most grid cells empty.

Dimensions D	Bins per Dimension K	Grid Cells
2	10	10^2
5	10	10^5
10	10	10^{10}

Table 5.2: Exponential growth of grid cells

- Predictions rely on very few samples.
- Distance-based intuition becomes unreliable.
- Coverage of the input space requires exponentially many basis functions.

5.3 High-Dimensional Geometry

Geometric intuition developed in low-dimensional spaces fails in high dimensions.

5.3.1 Volume Concentration

The volume of a D-dimensional hypersphere of radius r is given by

$$V_D(r) = K_D r^D, \tag{5.3}$$

where K_D depends only on the dimensionality D.

Consider a unit hypersphere ($r = 1$). The fraction of volume lying between radii $1 - \varepsilon$ and 1 is

$$1 - (1 - \varepsilon)^D. \tag{5.4}$$

As D increases, this fraction rapidly approaches one, indicating that most of the volume concentrates near the boundary.

Implications

- Most points lie near the boundary of the space.
- Distances between points become increasingly similar.
- Euclidean distance loses discriminative power.

5.4 Data Manifolds

Although the ambient dimensionality of data may be large, real-world data typically lie on low-dimensional manifolds embedded within the high-dimensional space.

Examples

- Handwritten digits vary in stroke shape and orientation.
- Natural images are constrained by object geometry and lighting.
- Speech signals are governed by vocal tract dynamics.

5.4.1 Natural Images

A 64×64 grayscale image corresponds to a point in a 12,288-dimensional space. However, natural images occupy only a small fraction of this space due to strong correlations between neighboring pixels.

Image Type	Structure	Pixel Correlation
Natural images	Structured	High
Random noise	Unstructured	None

Table 5.3: Natural images versus random noise

5.5 Radial Basis Functions

Radial basis functions (RBFs) associate a basis function with each training sample:

$$\phi_n(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_n\|^2}{s^2}\right), \tag{5.5}$$

where s controls the width of the basis function.

Advantages

- Data-dependent basis functions
- Local similarity modeling

Limitations

- Poor scalability with dataset size
- Sensitivity to distance concentration
- High computational cost
- Careful tuning of s required

5.6 Motivation for Deep Learning

The primary limitations of fixed basis function methods are:

- Basis functions are chosen independently of the data.
- The number of required functions grows exponentially with dimension.
- High-dimensional geometry undermines low-dimensional intuition.

Deep neural networks address these challenges by learning:

- Data-dependent representations
- Hierarchical and compositional features
- Scalable models for high-dimensional inputs

Chapter Summary

Method	Scaling with D	High-D Feasible
Polynomial bases	$\mathcal{O}(D^M)$	No
Grid-based methods	$\mathcal{O}(K^D)$	No
RBF networks	$\mathcal{O}(N)$	Limited
Deep networks	Data-adaptive	Yes

Chapter 6

Gradients and Optimization Foundations

This chapter introduces the mathematical foundations underlying gradient-based learning in neural networks. We begin with differential operators and subgradients, then develop a geometric view of optimization through error surfaces, curvature, stochastic methods, and convergence behavior.

6.1 Gradient and Subgradient

Function

$$f : \mathbb{R}^n \to \mathbb{R}$$

Nabla (del) operator

$$\nabla \equiv \left(\frac{\partial}{\partial x_1}, \frac{\partial}{\partial x_2}, \ldots, \frac{\partial}{\partial x_n} \right)$$

Gradient

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{bmatrix} \in \mathbb{R}^n$$

Subgradient A vector $g \in \mathbb{R}^n$ is a subgradient of f at x_0 if

$$f(x) \geq f(x_0) + g^\top (x - x_0), \quad \forall x \in \mathbb{R}^n$$

Meaning of g

$g \in \mathbb{R}^n$ is a fixed slope vector defining a supporting affine function

$$x \mapsto f(x_0) + g^\top (x - x_0)$$

g is constant with respect to x and is not a function of x

Subdifferential

$$\partial f(x_0) = \left\{ g \in \mathbb{R}^n : f(x) \geq f(x_0) + g^\top (x - x_0),\ \forall x \right\}$$

Smooth case If f is differentiable at x_0,

$$\partial f(x_0) = \{\nabla f(x_0)\}$$

Non-smooth case If f is not differentiable at x_0,

$$|\partial f(x_0)| > 1$$

One-dimensional case

$$\nabla f(x) = \frac{df}{dx}(x)$$

Optimization update

$$g_k \in \partial f(x_k)$$

$$x_{k+1} = x_k - \eta g_k$$

Key distinction

$$\nabla = \text{operator}, \quad \nabla f = \text{gradient vector (unique if exists)}, \quad \partial f = \text{subgradient set}$$

One-line interpretation

$$\nabla f(x_0) = g \text{ (smooth case)}, \qquad g \in \partial f(x_0) \text{ (non-smooth case)}$$

Example 1 (Quadratic, smooth)

$$f(x) = \tfrac{1}{2}\|x\|_2^2$$

$$\nabla f(x) = x, \qquad \partial f(x) = \{x\}$$

Example 2 (Absolute value, non-smooth)

$$f(x) = |x|$$

$$\partial f(x) = \begin{cases} \{1\}, & x > 0 \\ [-1, 1], & x = 0 \\ \{-1\}, & x < 0 \end{cases}$$

Example 3 (ReLU)

$$f(x) = \max(0, x)$$

$$\partial f(x) = \begin{cases} \{1\}, & x > 0 \\ [0, 1], & x = 0 \\ \{0\}, & x < 0 \end{cases}$$

Example 4 (Linear function)

$$f(x) = a^\top x + b$$

$$\nabla f(x) = a, \qquad \partial f(x) = \{a\}$$

6.2 Error Surfaces and Gradient-Based Optimization

6.2.1 Error Surfaces

Let

$$\mathbf{w} \in \mathbb{R}^W$$

denote the vector of all learnable parameters (weights and biases) of a neural network, and let

$$E(\mathbf{w}) : \mathbb{R}^W \to \mathbb{R}$$

denote the associated error (loss) function.

Geometrically, $E(\mathbf{w})$ defines a scalar-valued surface over the W-dimensional parameter space. Training a neural network can therefore be interpreted as navigating this surface in search of parameter values that yield small error.

6.2.2 Local Change in Error

Consider a small perturbation of the parameters

$$\mathbf{w} \to \mathbf{w} + \delta\mathbf{w}.$$

Using a first-order Taylor approximation, the change in the error function is

$$E(\mathbf{w} + \delta\mathbf{w}) \;\approx\; E(\mathbf{w}) + \nabla E(\mathbf{w})^\top \delta\mathbf{w}.$$

Thus, the incremental change in error is approximately

$$\delta E \;\approx\; \delta\mathbf{w}^\top \nabla E(\mathbf{w}),$$

where

$$\nabla E(\mathbf{w}) = \begin{bmatrix} \frac{\partial E}{\partial w_1} \\ \vdots \\ \frac{\partial E}{\partial w_W} \end{bmatrix}$$

is the gradient of the error function with respect to the parameters.

6.2.3 Geometric Meaning of the Gradient

The gradient vector $\nabla E(\mathbf{w})$ points in the direction of the steepest local increase of the error function. Formally,

$$\max_{\|\delta\mathbf{w}\|=1} \delta\mathbf{w}^\top \nabla E(\mathbf{w}) = \|\nabla E(\mathbf{w})\|.$$

Consequently, moving in the direction

$$-\nabla E(\mathbf{w})$$

produces the greatest local decrease in the error.

6.2.4 Gradient Descent Update

A gradient-based optimization step is therefore given by

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \eta \nabla E(\mathbf{w}_k),$$

where $\eta > 0$ is the learning rate.

Substituting into the first-order approximation yields

$$\delta E \approx -\eta \|\nabla E(\mathbf{w}_k)\|^2 \leq 0,$$

indicating a local decrease in the error.

6.2.5 Stationary Points

A necessary condition for a stationary point of the error surface is

$$\nabla E(\mathbf{w}) = \mathbf{0}.$$

At such points, no first-order change in the parameters produces a change in the error. Stationary points may correspond to **local minima, local maxima, or saddle points**.

Second-order characterization Let $\nabla^2 E(\mathbf{w})$ denote the Hessian matrix of second derivatives.

Table 6.1: Classification of stationary points using first- and second-order derivatives

Condition	Second-order behavior	Point type
$\nabla E(\mathbf{w}) = \mathbf{0}$	$\nabla^2 E(\mathbf{w}) \succ 0$	Local minimum
$\nabla E(\mathbf{w}) = \mathbf{0}$	$\nabla^2 E(\mathbf{w}) \prec 0$	Local maximum
$\nabla E(\mathbf{w}) = \mathbf{0}$	Indefinite $\nabla^2 E(\mathbf{w})$	Saddle point
$\nabla E(\mathbf{w}) = \mathbf{0}$	$\nabla^2 E(\mathbf{w}) = 0$ or singular	Higher-order analysis required

In high-dimensional neural network models, saddle points are particularly common.

6.2.6 Interpretation

Training a neural network consists of iteratively computing the gradient of the error surface and updating the parameters in a direction that locally reduces the error. Backpropagation provides an efficient mechanism for evaluating $\nabla E(\mathbf{w})$, enabling gradient-based optimization in high-dimensional parameter spaces.

6.2.7 Local Quadratic Approximation

Insight into the local geometry of the error surface can be obtained via a second-order Taylor expansion of the error function $E(\mathbf{w})$ about a reference point $\tilde{\mathbf{w}}$:

$$E(\mathbf{w}) \;\approx\; E(\tilde{\mathbf{w}}) + (\mathbf{w} - \tilde{\mathbf{w}})^\top \mathbf{b} + \frac{1}{2}(\mathbf{w} - \tilde{\mathbf{w}})^\top \mathbf{H}(\mathbf{w} - \tilde{\mathbf{w}}).$$

Here,

$$\mathbf{b} = \nabla E(\mathbf{w})\big|_{\mathbf{w}=\tilde{\mathbf{w}}}$$

is the gradient of the error function evaluated at $\tilde{\mathbf{w}}$, and

$$\mathbf{H} = \nabla^2 E(\mathbf{w})\big|_{\mathbf{w}=\tilde{\mathbf{w}}}$$

is the Hessian matrix of second-order partial derivatives.

Interpretation of the terms

- $E(\tilde{\mathbf{w}})$: error value at the reference point,
- $(\mathbf{w} - \tilde{\mathbf{w}})^\top \mathbf{b}$: first-order (linear) variation describing local slope,
- $\frac{1}{2}(\mathbf{w} - \tilde{\mathbf{w}})^\top \mathbf{H}(\mathbf{w} - \tilde{\mathbf{w}})$: second-order variation describing local curvature.

Stationary point If $\mathbf{w}^*$ is a stationary point of the error surface, then

$$\nabla E(\mathbf{w}^*) = \mathbf{0}.$$

In this case, the linear term vanishes and the approximation reduces to

$$E(\mathbf{w}) \approx E(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^\top \mathbf{H}(\mathbf{w} - \mathbf{w}^*).$$

Second-order characterization The local behavior of the error surface near $\mathbf{w}^*$ is determined by the Hessian $\mathbf{H}$:

- $\mathbf{H} \succ 0$: local minimum,
- $\mathbf{H} \prec 0$: local maximum,
- $\mathbf{H}$ indefinite: saddle point.

Geometric view Near a stationary point, the error surface behaves locally as a quadratic form. The eigenvalues of the Hessian determine the curvature along different directions in parameter space, explaining the prevalence of saddle points in high-dimensional neural networks.

6.2.8 Eigenvalues, Curvature, and Stationary Points

Let $\mathbf{H} = \nabla^2 E(\mathbf{w}^*)$ denote the Hessian matrix of the error function evaluated at a stationary point $\mathbf{w}^*$. Let $\{\lambda_i, \mathbf{u}_i\}$ denote the eigenvalue–eigenvector pairs of $\mathbf{H}$, satisfying

$$\mathbf{H}\mathbf{u}_i = \lambda_i \mathbf{u}_i.$$

Since the Hessian is symmetric, its eigenvectors form a complete orthonormal basis:

$$\mathbf{u}_i^\top \mathbf{u}_j = \delta_{ij},$$

where δ_{ij} is the Kronecker delta defined by

$$\delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Thus, $\|\mathbf{u}_i\| = 1$ and $\mathbf{u}_i \perp \mathbf{u}_j$ for $i \neq j$.

Expansion in the Eigenbasis

Any displacement from the stationary point can be expressed in the eigenbasis of the Hessian as

$$\mathbf{w} - \mathbf{w}^* = \sum_i \alpha_i \mathbf{u}_i.$$

Substituting this expansion into the local quadratic approximation,

$$E(\mathbf{w}) \approx E(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^\top \mathbf{H}(\mathbf{w} - \mathbf{w}^*),$$

yields

$$E(\mathbf{w}) = E(\mathbf{w}^*) + \frac{1}{2}\sum_i \lambda_i \alpha_i^2.$$

Each eigen-direction contributes independently to the change in error.

Classification of Stationary Points

From the quadratic form

$$E(\mathbf{w}) - E(\mathbf{w}^*) = \frac{1}{2} \sum_i \lambda_i \alpha_i^2,$$

the nature of the stationary point follows directly:

- If $\lambda_i > 0$ for all i, the point is a local minimum.
- If $\lambda_i < 0$ for all i, the point is a local maximum.
- If the eigenvalues have mixed signs, the point is a saddle point.

Positive Definiteness

The Hessian $\mathbf{H}$ is positive definite if

$$\mathbf{v}^\top \mathbf{H} \mathbf{v} > 0 \quad \forall \mathbf{v} \neq \mathbf{0},$$

which is equivalent to

$$\lambda_i > 0 \quad \forall i.$$

Positive definiteness guarantees that $\mathbf{w}^*$ is a strict local minimum.

Geometric Interpretation

Contours of constant error near $\mathbf{w}^*$ satisfy

$$\sum_i \lambda_i \alpha_i^2 = \text{const.}$$

These contours form ellipses (or ellipsoids in higher dimensions) whose principal axes are aligned with the eigenvectors $\mathbf{u}_i$. The lengths of the axes are proportional to $\lambda_i^{-1/2}$, indicating steep directions for large eigenvalues and flat directions for small eigenvalues.

This local geometry explains the behavior of gradient-based optimization methods near stationary points.

6.2.9 Stochastic and Mini-Batch Gradient Descent

Assume the error function decomposes over individual data points:

$$E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}),$$

where $E_n(\mathbf{w})$ denotes the loss associated with the n-th sample.

The full gradient is

$$\nabla E(\mathbf{w}) = \sum_{n=1}^{N} \nabla E_n(\mathbf{w}),$$

which becomes computationally expensive for large N.

Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent updates the parameters using a single randomly selected data point:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla E_n(\mathbf{w}^{(t)}),$$

where n is sampled uniformly from $\{1, \ldots, N\}$.

The stochastic gradient satisfies

$$\mathbb{E}[\nabla E_n(\mathbf{w})] = \nabla E(\mathbf{w}),$$

so the update direction is an unbiased but noisy estimate of the true gradient.

The noise introduced by SGD can help escape shallow local minima and saddle points.

Mini-Batch Gradient Descent

Mini-batch gradient descent uses a subset $B \subset \{1, \ldots, N\}$ of size $|B|$:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \sum_{n \in B} \nabla E_n(\mathbf{w}^{(t)}).$$

This approach balances computational efficiency and variance reduction by averaging gradients over multiple samples.

Gradient Noise Scaling

Assuming the per-sample gradient has standard deviation σ, the standard deviation of the mini-batch gradient estimator scales as

$$\frac{\sigma}{\sqrt{|B|}}.$$

Thus, increasing the batch size yields diminishing returns: increasing $|B|$ by a factor of 100 reduces the gradient noise by only a factor of 10.

Shuffling and Randomness

Mini-batches should be formed by random sampling. Datasets are typically shuffled between epochs to avoid correlations arising from ordered data and to ensure that each mini-batch is a representative sample of the full dataset.

Terminology

Even when mini-batches are used, the optimization method is commonly referred to as stochastic gradient descent due to the inherent randomness in gradient estimation.

Table 6.2: Comparison of batch, stochastic, and mini-batch gradient descent

Method	Batch size	Gradient noise	Computation per step
Batch Gradient Descent	N	None	High
Stochastic Gradient Descent	1	High	Low
Mini-Batch Gradient Descent	$\|B\| \ll N$	Moderate	Moderate

Practical considerations Mini-batch sizes are often chosen to maximize hardware efficiency, particularly on GPUs. Powers of two such as $|B| \in \{32, 64, 128, 256\}$ are commonly used in practice.

Notation The superscript (t) denotes the iteration index of the optimization algorithm. For example, $\mathbf{w}^{(t)}$ represents the parameter vector after t gradient descent updates.

6.2.10 Convergence Behavior

Near a local minimum $\mathbf{w}^*$, the error surface can be approximated by a quadratic function. Let $\mathbf{H}$ denote the Hessian matrix at $\mathbf{w}^*$, with eigenpairs $(\lambda_i, \mathbf{u}_i)$.

Any displacement from the minimum can be written as

$$\mathbf{w}^{(t)} - \mathbf{w}^* = \sum_i \alpha_i^{(t)} \mathbf{u}_i.$$

For this quadratic approximation, the gradient is

$$\nabla E(\mathbf{w}^{(t)}) = \mathbf{H}(\mathbf{w}^{(t)} - \mathbf{w}^*).$$

Applying the gradient descent update

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla E(\mathbf{w}^{(t)}),$$

and projecting onto the eigenvector direction $\mathbf{u}_i$, yields the scalar recursion

$$\alpha_i^{(t+1)} = (1 - \eta\lambda_i)\, \alpha_i^{(t)}.$$

This equation shows that gradient descent converges *linearly* and *independently* along each eigenvector direction of the Hessian.

Convergence condition For convergence along direction i, the magnitude of the update factor must be less than one:

$$|1 - \eta\lambda_i| < 1.$$

This implies the learning rate must satisfy

$$0 < \eta < \frac{2}{\lambda_{\max}},$$

where $\lambda_{\max} = \max_i \lambda_i$.

Thus, the largest curvature direction determines the maximum stable learning rate.

Example 1: One-dimensional quadratic Consider

$$E(w) = \tfrac{1}{2}\lambda w^2, \quad \lambda > 0.$$

Gradient descent gives

$$w^{(t+1)} = (1 - \eta\lambda) w^{(t)}.$$

Convergence occurs if

$$|1 - \eta\lambda| < 1,$$

which yields $0 < \eta < \frac{2}{\lambda}$. If η is too large, the iterates oscillate or diverge.

Example 2: Narrow valley (two dimensions) Let

$$E(w_1, w_2) = \tfrac{1}{2}(\lambda_1 w_1^2 + \lambda_2 w_2^2), \quad \lambda_1 \gg \lambda_2 > 0.$$

The updates become

$$w_1^{(t+1)} = (1 - \eta\lambda_1) w_1^{(t)}, \qquad w_2^{(t+1)} = (1 - \eta\lambda_2) w_2^{(t)}.$$

To ensure stability in the steep direction w_1, the learning rate must satisfy

$$\eta < \frac{2}{\lambda_1}.$$

However, since $\lambda_2 \ll \lambda_1$,

$$1 - \eta\lambda_2 \approx 1,$$

and convergence along w_2 is extremely slow.

This mismatch causes the iterates to bounce across the steep direction while making slow progress along the flat direction, producing the characteristic *zig-zag* trajectory.

Interpretation

- Each eigenvalue λ_i controls the convergence rate along direction $\mathbf{u}_i$.
- Large eigenvalues enforce small learning rates.
- Small eigenvalues cause slow convergence.
- Ill-conditioned problems ($\lambda_{\max} \gg \lambda_{\min}$) lead to zig-zag behavior.

Summary Gradient descent converges linearly near a local minimum, with the slowest convergence governed by the smallest eigenvalues of the Hessian and the maximum allowable learning rate governed by the largest eigenvalue.

6.2.11 Vanishing and Exploding Gradients

Training deep neural networks using gradient-based optimization relies on the efficient propagation of error signals from the output layer back to the early layers of the network. As network depth increases, however, gradients may become exponentially small or exponentially large.

These phenomena are known as *vanishing gradients* and *exploding gradients*, respectively.

6.2.12 Gradient Propagation via the Chain Rule

Consider a feedforward neural network with K layers. Let

$$\mathbf{z}^{(k)} = \mathbf{W}^{(k)}\mathbf{a}^{(k-1)} + \mathbf{b}^{(k)}, \qquad \mathbf{a}^{(k)} = \sigma(\mathbf{z}^{(k)}),$$

where $\mathbf{a}^{(k)}$ denotes the activation vector at layer k, and E denotes the loss function.

The gradient of the loss with respect to parameters in early layers is obtained by repeated application of the chain rule. For example, the gradient with respect to the activations of the first layer satisfies

$$\frac{\partial E}{\partial \mathbf{a}^{(1)}} = \left(\prod_{k=2}^{K} \frac{\partial \mathbf{a}^{(k)}}{\partial \mathbf{a}^{(k-1)}} \right) \frac{\partial E}{\partial \mathbf{a}^{(K)}}.$$

Layer-wise Jacobian The Jacobian matrix of layer k is given by

$$\mathbf{J}^{(k)} = \frac{\partial \mathbf{a}^{(k)}}{\partial \mathbf{a}^{(k-1)}} = \operatorname{diag}\Big(\sigma'(\mathbf{z}^{(k)})\Big)\, \mathbf{W}^{(k)}.$$

Thus, backpropagation is governed by the product of Jacobians across layers:

$$\frac{\partial E}{\partial \mathbf{a}^{(1)}} = \left(\prod_{k=2}^{K} \mathbf{J}^{(k)}\right) \frac{\partial E}{\partial \mathbf{a}^{(K)}}.$$

6.2.13 Vanishing Gradients

If the spectral norms (or singular values) of the Jacobian matrices satisfy

$$\|\mathbf{J}^{(k)}\| < 1 \quad \text{for most layers,}$$

then repeated multiplication yields

$$\left\| \prod_{k=2}^{K} \mathbf{J}^{(k)} \right\| \xrightarrow[K\to\infty]{} 0.$$

Consequently, gradients shrink exponentially with network depth:

$$\left\| \frac{\partial E}{\partial \mathbf{a}^{(1)}} \right\| \ll \left\| \frac{\partial E}{\partial \mathbf{a}^{(K)}} \right\|.$$

Implications

- Early layers learn extremely slowly.
- Gradient updates become numerically insignificant.
- Feature extraction in early layers is impaired.

Example: Sigmoid activation For the sigmoid function,

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)(1-\sigma(z)) \le \tfrac{1}{4}.$$

Thus, each Jacobian contains factors strictly less than one. Repeated multiplication across layers guarantees exponential decay of gradients, explaining why deep sigmoid networks suffer from vanishing gradients.

6.2.14 Exploding Gradients

Conversely, if the Jacobian norms satisfy

$$\|\mathbf{J}^{(k)}\| > 1 \quad \text{for many layers,}$$

then

$$\left\| \prod_{k=2}^{K} \mathbf{J}^{(k)} \right\| \xrightarrow[K\to\infty]{} \infty.$$

This leads to exploding gradients, causing unstable parameter updates and numerical overflow.

Example: Large weight matrices If weight matrices are initialized with large spectral norms, then even moderate activation derivatives can cause Jacobian norms greater than one, leading to exponential growth of gradients during backpropagation.

Summary

- Backpropagation is governed by products of Jacobians of the form

$$\operatorname{diag}(\sigma'(\mathbf{z}^{(k)}))\mathbf{W}^{(k)}.$$

- Jacobian norms less than one cause vanishing gradients.
- Jacobian norms greater than one cause exploding gradients.
- Activation functions, weight initialization, and network depth jointly determine gradient stability.

6.3 Normalization Methods

Normalization rescales variables in a neural network to improve numerical stability and optimization efficiency. Table 6.3 compares common normalization techniques based on where and how normalization is applied.

Table 6.3: Comparison of normalization methods in neural networks

Method	**Normalization equation**	**Axis of normalization**	**Typical use / example**
Input (data) normalization	$\tilde{\mathbf{x}} = \frac{\mathbf{x}-\boldsymbol{\mu}_{\text{data}}}{\boldsymbol{\sigma}_{\text{data}}}$	Across dataset samples (per feature)	Image pixel normalization or feature standardization before training
Batch normalization	$\hat{\mathbf{z}}^{(k)} = \frac{\mathbf{z}^{(k)}-\boldsymbol{\mu}_B}{\sqrt{\boldsymbol{\sigma}_B^2+\epsilon}}$, $\mathbf{a}^{(k)} = \gamma\hat{\mathbf{z}}^{(k)} + \beta$	Across mini-batch samples (per feature)	Deep CNNs and feedforward networks with large batch sizes
Layer normalization	$\hat{\mathbf{z}}^{(k)} = \frac{\mathbf{z}^{(k)}-\mu_L}{\sqrt{\sigma_L^2+\epsilon}}$, $\mathbf{a}^{(k)} = \gamma\hat{\mathbf{z}}^{(k)} + \beta$	Across features within a single sample	Transformers and recurrent networks with variable batch sizes

Explanation Input normalization standardizes raw data and remains fixed throughout training. Batch normalization normalizes intermediate pre-activations using mini-batch statistics, stabilizing gradient propagation but introducing dependence on batch size. Layer normalization operates independently on each sample, making it suitable for sequence models and architectures with small or varying batch sizes.

Example

- Input normalization scales image pixels to zero mean and unit variance before training a convolutional network.

- Batch normalization enables stable training of very deep CNNs by controlling the distribution of hidden-layer activations.
- Layer normalization is used in Transformer models to stabilize training across time steps and sequence lengths.

Table 6.4: Comparison of Batch Normalization and Layer Normalization

Aspect	**Batch Normalization (BN)**	**Layer Normalization (LN)**
What is normalized?	Pre-activations $\mathbf{z}^{(k)}$ for each feature (neuron) separately	Pre-activations $\mathbf{z}^{(k)}$ across all features within one sample
Mean and variance computed over	Mini-batch dimension (across samples)	Feature dimension (within a single sample)
Mean	$\mu_{B,j} = \frac{1}{B}\sum_{i=1}^{B} z_{i,j}$	$\mu_{L,i} = \frac{1}{d}\sum_{j=1}^{d} z_{i,j}$
Variance	$\sigma^2_{B,j} = \frac{1}{B}\sum_{i=1}^{B}(z_{i,j} - \mu_{B,j})^2$	$\sigma^2_{L,i} = \frac{1}{d}\sum_{j=1}^{d}(z_{i,j} - \mu_{L,i})^2$
Normalization step	$\hat{z}_{i,j} = \frac{z_{i,j} - \mu_{B,j}}{\sqrt{\sigma^2_{B,j} + \epsilon}}$	$\hat{z}_{i,j} = \frac{z_{i,j} - \mu_{L,i}}{\sqrt{\sigma^2_{L,i} + \epsilon}}$
Dependence on batch size	Strong dependence; unstable for small batches	Independent of batch size
Training vs inference	Uses batch statistics during training and running averages during inference	Same computation during training and inference
Typical architectures	Convolutional and feedforward networks with large batches	Transformers and recurrent networks
Key intuition	"Normalize across samples for each neuron"	"Normalize across neurons for each sample"

Chapter 7

Regularization Principles: Tikhonov, Bayesian Priors, and Modern Deep Networks

7.1 Learning Curves and Generalization

Learning curves show how training and validation error change as a function of training iteration, epoch, or model complexity. They are useful for diagnosing underfitting, overfitting, and the effect of regularization.

In a typical training process, the training error decreases as optimization proceeds. The validation error often decreases at first, but may increase later when the model begins to fit noise or sample-specific patterns in the training data. This behavior indicates overfitting.

Therefore, learning curves provide a practical way to monitor generalization. The model checkpoint with the lowest validation error is usually preferred, because it represents the best trade-off between fitting the training data and maintaining performance on unseen data.

7.2 Tikhonov Regularization: The Classical Perspective

Tikhonov regularization arises as a principled solution to ill-posed inverse problems. Consider a linear operator

$$A : \mathbb{R}^n \to \mathbb{R}^m$$

and the linear system

$$Ax = b,$$

where $b \in \mathbb{R}^m$ is observed data and $x \in \mathbb{R}^n$ is unknown.

In practice, one or more of the following occur:

- A is ill-conditioned (small singular values),
- the system is underdetermined ($m < n$),
- the data b contains noise.

Direct inversion is unstable. Small perturbations in b may produce arbitrarily large changes in x.

Tikhonov proposed solving instead the penalized problem

$$\min_{x \in \mathbb{R}^n} \left\{ \|Ax - b\|_2^2 + \lambda \|x\|_2^2 \right\}, \qquad \lambda > 0.$$

The quadratic penalty enforces stability by controlling the norm of the solution.

7.2.1 Closed-Form Solution

The minimizer satisfies the normal equation

$$(A^\top A + \lambda I)x_\lambda = A^\top b,$$

hence

$$x_\lambda = (A^\top A + \lambda I)^{-1} A^\top b.$$

Since $A^\top A + \lambda I$ is strictly positive definite for $\lambda > 0$, the solution exists uniquely and depends continuously on b.

Thus,

$$\text{Tikhonov regularization} = \text{L2 regularization} = \text{Ridge regression}.$$

They are mathematically identical.

7.3 Statistical Learning Theory View

Consider empirical risk minimization over a hypothesis space $\mathcal{H}$:

$$\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i).$$

With finite data, a model may interpolate the training set while generalizing poorly.

Generalization bounds typically take the form

$$\mathcal{E}_{\text{test}}(f) \leq \mathcal{E}_{\text{train}}(f) + \mathcal{C}(\mathcal{H}),$$

where $\mathcal{C}(\mathcal{H})$ is a complexity term depending on

- VC dimension,
- Rademacher complexity,
- norm constraints.

Imposing a constraint such as

$$\|w\|_2 \leq C$$

shrinks the effective capacity of the hypothesis class.

Therefore, L2 regularization reduces statistical complexity in addition to stabilizing numerical inversion.

7.4 Functional Analysis View: Ill-Posed Operators

Let

$$A : \mathcal{X} \to \mathcal{Y}$$

be a bounded linear operator between Hilbert spaces.

A problem is well-posed if:

1. A solution exists,
2. The solution is unique,
3. The solution depends continuously on the data.

If A has small singular values σ_i, then inversion involves division by σ_i, and instability arises as $\sigma_i \to 0$.

Using the singular value decomposition

$$A = U\Sigma V^\top,$$

Tikhonov replaces the inverse

$$A^{-1}$$

with the regularized inverse

$$(A^\top A + \lambda I)^{-1} A^\top.$$

In spectral form,

$$\frac{\sigma_i}{\sigma_i^2 + \lambda}$$

damps directions corresponding to small σ_i.

This mechanism is known as spectral filtering.

In deep learning, weight decay performs the same stabilization in parameter space.

7.5 Bayesian Interpretation

Assume a linear Gaussian model:

$$b = Ax + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2 I).$$

The likelihood is

$$p(b \mid x) \propto \exp\left(-\frac{1}{2\sigma^2}\|Ax - b\|_2^2\right).$$

Assume a Gaussian prior:

$$x \sim \mathcal{N}(0, \tau^2 I).$$

The posterior maximizer (MAP estimator) solves

$$\min_x \left\{\|Ax - b\|_2^2 + \lambda\|x\|_2^2\right\}, \qquad \lambda = \frac{\sigma^2}{\tau^2}.$$

Thus,

$$\text{L2 regularization} = \text{Gaussian prior}.$$

7.6 Connections to Modern Deep Learning

7.6.1 Weight Decay

Neural network training solves

$$\min_\theta \mathcal{L}(\theta) + \lambda\|\theta\|_2^2.$$

This is precisely Tikhonov regularization applied in high-dimensional parameter space.

7.6.2 Dropout

Dropout introduces multiplicative Bernoulli noise,

$$h \mapsto m \odot h, \quad m_i \sim \text{Bernoulli}(p),$$

which approximates Bayesian model averaging and induces implicit regularization.

7.6.3 Early Stopping

Early stopping is an implicit regularization method that limits the effective complexity of a model by stopping optimization before full convergence.

In practice, the validation loss is monitored during training. Training is stopped when the validation loss no longer improves for a specified number of epochs, often called the patience parameter. The model checkpoint with the lowest validation loss is then used for testing or deployment.

For least squares, gradient descent yields

$$x^{(k)} = \sum_i \left(1 - (1 - \eta\sigma_i^2)^k\right) \frac{u_i^\top b}{\sigma_i} v_i.$$

Stopping early suppresses small singular directions, which are often associated with noise amplification. Therefore, early stopping acts as implicit spectral regularization.

For quadratic loss functions, early stopping can produce a solution similar to L_2 regularization. The learning rate and stopping time together control the effective regularization strength. If training continues too long, the model may move closer to the unregularized solution and begin to overfit.

7.6.4 Normalization Layers

Batch normalization and layer normalization do not directly penalize norms but modify the geometry of the optimization landscape, often leading to flatter minima and improved generalization.

7.7 Conceptual Synthesis

Regularization mechanisms all serve to:

1. Restrict the effective hypothesis space,
2. Stabilize inverse problems,
3. Introduce inductive bias,
4. Improve generalization.

Without such bias, learning from finite data is fundamentally ill-posed, consistent with the No Free Lunch theorem.

7.8 Regularization in Deep Networks

Training a deep model solves a nonlinear inverse problem:

$$f_\theta(x_i) \approx y_i.$$

Infinitely many parameters θ interpolate the data.

Regularization selects among them by enforcing structure:

- Small parameter norm,
- Smoothness,
- Stability,

- Simplicity.

In this sense, Tikhonov regularization extends naturally to modern deep learning as a high-dimensional stabilization principle.

7.9 Symmetry, Invariance, and Equivariance

In many machine learning applications, predictions should behave consistently under certain transformations of the input. These transformations may represent symmetries of the underlying task. Understanding how a model responds to such transformations leads to the notions of invariance and equivariance.

Let the input be an image I, and let T denote a transformation acting on the input (e.g., translation, rotation, scaling). Let S denote a neural network or function mapping inputs to outputs.

7.9.1 Symmetry

A transformation T represents a symmetry of a task if applying T to the input does not change the essential property being predicted.

Formally, a task exhibits symmetry under transformation T if

$$\text{Task}(T(I)) = \text{Task}(I).$$

Example 1: Object presence In image classification, the question

"Is there a cat in the image?"

is invariant to translations of the cat inside the image. Thus translation is a symmetry of the task.

Example 2: Physical laws In physics-informed learning, many systems are translation invariant:

$$f(x + c) = f(x),$$

for constant shift c. This reflects translational symmetry of the governing equations.

Transformations such as translation, rotation, or scaling that leave relevant properties unchanged are examples of symmetries.

7.9.2 Invariance

A function f is invariant to a transformation T if

$$f(T(I)) = f(I).$$

Invariance means the output remains unchanged when the input is transformed.

Example 1: Image classification Suppose $f(I)$ predicts a class label. If the image is shifted:

$$f(T_{\text{translate}}(I)) = f(I).$$

Similarly, many classification models aim for rotation invariance:

$$f(T_{\text{rotate}}(I)) = f(I).$$

Example 2: Norm function Consider the Euclidean norm:

$$f(\mathbf{x}) = \|\mathbf{x}\|_2.$$

This function is invariant to orthogonal rotations R:

$$f(R\mathbf{x}) = \|R\mathbf{x}\|_2 = \|\mathbf{x}\|_2.$$

Thus the norm is rotation invariant.

Example 3: Set pooling If a model aggregates features using summation,

$$f(\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}) = \sum_i \mathbf{x}_i,$$

the function is invariant to permutation of inputs. This is crucial in graph neural networks.

7.9.3 Equivariance

A function S is equivariant to transformation T if transforming the input results in a predictable transformation of the output:

$$S(T(I)) = \tilde{T}(S(I)),$$

where $\tilde{T}$ is the corresponding transformation acting on the output space.

Equivariance means the output changes in the same structured way as the input.

Example 1: Image segmentation Suppose $S(I)$ produces a pixel-wise segmentation mask. If the input image is translated by 10 pixels to the right, the segmentation mask should also translate by 10 pixels:

$$S(T_{\text{translate}}(I)) = T_{\text{translate}}(S(I)).$$

Example 2: Convolution Let $\mathcal{C}$ denote convolution with kernel K. Then

$$\mathcal{C}(T_{\text{translate}}(I)) = T_{\text{translate}}(\mathcal{C}(I)).$$

Thus convolution is translation equivariant.

Example 3: Linear maps Let

$$S(\mathbf{x}) = A\mathbf{x},$$

where A is a matrix. If $T(\mathbf{x}) = R\mathbf{x}$ is a rotation and A commutes with R, then

$$S(R\mathbf{x}) = AR\mathbf{x} = RA\mathbf{x} = RS(\mathbf{x}),$$

which shows equivariance.

7.9.4 Vision Tasks

Different tasks require different transformation behavior:

- **Classification:** translation invariant
- **Segmentation:** translation equivariant
- **Object detection:** partially equivariant (bounding boxes shift)

- **Pose estimation:** rotation equivariant

For classification, translation equivariant feature maps are typically followed by pooling operations, producing approximate translation invariance:

$$\text{Pooling}(\mathcal{C}(T(I))) = \text{Pooling}(\mathcal{C}(I)).$$

7.9.5 Summary Table

Property	Mathematical Form	Example
Symmetry	$\text{Task}(T(I)) = \text{Task}(I)$	Object presence under translation
Invariance	$f(T(I)) = f(I)$	Classification, norm function
Equivariance	$S(T(I)) = \tilde{T}(S(I))$	Segmentation, convolution

7.9.6 Conceptual Distinction

- **Symmetry** is a property of the task.
- **Invariance** is a property of the model output remaining unchanged.
- **Equivariance** is a property of the model output transforming predictably.

Invariance is appropriate for global prediction tasks such as classification, while equivariance is essential for structured output tasks such as segmentation, detection, and localization.

7.10 Enforcing Invariance and Regularization

7.10.1 Methods for Enforcing Invariance

Invariance can be encouraged through several complementary strategies:

1. **Pre-processing:** Construct invariant features.
2. **Regularized error functions:** Penalize sensitivity to transformations.
3. **Data augmentation:** Expand training data using transformed samples.
4. **Network architecture:** Encode invariance structurally.

Example 1: Pre-processing In image classification, instead of using raw pixel coordinates, one may use

$$f(I) = \text{histogram of gradients}(I),$$

which is more robust to small translations and lighting variations.

Example 2: Data augmentation If T is a translation operator, training on

$$\{I, T(I), T^2(I), \dots\}$$

encourages the model to satisfy approximately

$$f(T(I)) \approx f(I).$$

Example 3: Architectural enforcement Convolution satisfies

$$\mathcal{C}(T(I)) = T(\mathcal{C}(I)),$$

which enforces translation equivariance structurally.

Pooling then yields approximate invariance.

7.11 Error Function Regularization

A common approach to regularization is to modify the error function by adding a penalty discouraging large parameter values:

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{2}\mathbf{w}^\top \mathbf{w}.$$

Here:

- $\mathbf{w}$ is the parameter vector,
- $E(\mathbf{w})$ is the original loss,
- $\lambda > 0$ controls regularization strength.

This is known as **L2 regularization** or **weight decay**.

7.11.1 Interpretation

The added term is

$$\frac{\lambda}{2}\|\mathbf{w}\|_2^2.$$

It penalizes large parameter magnitudes and encourages solutions near the origin.

Effect of λ

- $\lambda = 0$: no regularization.
- Large λ: strong shrinkage toward zero.

7.11.2 Gradients of Regularized Error Functions

Regularization modifies the objective function:

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \lambda\,\Omega(\mathbf{w}),$$

where $\Omega(\mathbf{w})$ is a penalty term.

L2 Regularization (Weight Decay)

Penalty:

$$\Omega(\mathbf{w}) = \frac{1}{2}\|\mathbf{w}\|_2^2 = \frac{1}{2}\mathbf{w}^\top \mathbf{w}.$$

Regularized objective:

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{2}\mathbf{w}^\top \mathbf{w}.$$

Gradient:

$$\nabla \tilde{E}(\mathbf{w}) = \nabla E(\mathbf{w}) + \lambda \mathbf{w}.$$

Gradient descent update:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \left(\nabla E(\mathbf{w}_t) + \lambda \mathbf{w}_t \right).$$

Rewriting,

$$\mathbf{w}_{t+1} = (1 - \eta\lambda)\mathbf{w}_t - \eta \nabla E(\mathbf{w}_t).$$

Thus, even if $\nabla E = 0$,

$$\mathbf{w}_{t+1} = (1 - \eta\lambda)\mathbf{w}_t,$$

showing exponential shrinkage toward zero.
This is why L2 regularization is called **weight decay**.

L1 Regularization

Penalty:

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_i |w_i|.$$

Regularized objective:

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \lambda \|\mathbf{w}\|_1.$$

The L1 norm is not differentiable at $w_i = 0$. Its subgradient is:

$$\frac{\partial}{\partial w_i} |w_i| = \begin{cases} 1, & w_i > 0 \\ -1, & w_i < 0 \\ \in [-1, 1], & w_i = 0 \end{cases}$$

Thus, the subgradient of the regularized objective is:

$$\nabla \tilde{E}(\mathbf{w}) = \nabla E(\mathbf{w}) + \lambda \, \text{sign}(\mathbf{w}),$$

where

$$\text{sign}(w_i) = \begin{cases} 1, & w_i > 0 \\ -1, & w_i < 0 \\ \in [-1, 1], & w_i = 0 \end{cases}$$

Update rule:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \left(\nabla E(\mathbf{w}_t) + \lambda \, \text{sign}(\mathbf{w}_t) \right).$$

Key property Unlike L2, L1 does not scale shrinkage with magnitude. Instead, it applies a constant pull toward zero.

This encourages **sparsity**, meaning many weights become exactly zero.

General L_p Regularization

Penalty:

$$\Omega(\mathbf{w}) = \frac{1}{p}\|\mathbf{w}\|_p^p = \frac{1}{p}\sum_i |w_i|^p.$$

Regularized objective:

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{p}\sum_i |w_i|^p.$$

For $p > 1$, the gradient is:

$$\frac{\partial}{\partial w_i}\left(\frac{1}{p}|w_i|^p\right) = |w_i|^{p-2}w_i.$$

Thus,

$$\nabla\tilde{E}(\mathbf{w}) = \nabla E(\mathbf{w}) + \lambda\,|\mathbf{w}|^{p-2} \odot \mathbf{w},$$

where $\odot$ denotes elementwise multiplication.
Special cases:

$$p = 2 \Rightarrow \lambda\mathbf{w}$$

$$p = 1 \Rightarrow \lambda\,\mathrm{sign}(\mathbf{w})$$

Behavior comparison

- $p = 2$: smooth shrinkage, no sparsity.
- $p = 1$: promotes sparsity.
- $p > 2$: strongly penalizes large weights.
- $p < 1$: non-convex, strongly sparse but harder to optimize.

Summary

$$\text{L2:} \quad +\lambda\mathbf{w}$$

$$\text{L1:} \quad +\lambda\,\mathrm{sign}(\mathbf{w})$$

$$\text{Lp:} \quad +\lambda\,|\mathbf{w}|^{p-2} \odot \mathbf{w}$$

L2 yields smooth weight decay, L1 yields sparse solutions, general L_p interpolates between these behaviors.

Constrained Optimization Interpretation

The penalized objective

$$\min_{\mathbf{w}} \left\{ E(\mathbf{w}) + \frac{\lambda}{p} \sum_i |w_i|^p \right\}$$

can also be interpreted as a constrained optimization problem:

$$\min_{\mathbf{w}} E(\mathbf{w}) \quad \text{subject to} \quad \sum_i |w_i|^p \leq \eta,$$

for an appropriate constraint radius η.

This interpretation shows that regularization restricts the feasible region of the parameter vector. For $p = 2$, the constraint region is smooth and circular in two dimensions. For $p = 1$, the constraint region has sharp corners, which encourages sparse solutions where some coefficients become exactly zero. For $p < 1$, the constraint becomes non-convex and can promote stronger sparsity, but the optimization problem becomes harder.

7.11.3 Geometric Interpretation

Suppose the original loss near its minimum is quadratic:

$$E(\mathbf{w}) = \frac{1}{2}\mathbf{w}^\top H \mathbf{w}.$$

Contours of $E(\mathbf{w})$ are ellipses.
The regularization term has contours:

$$\|\mathbf{w}\|_2^2 = \text{constant},$$

which are circles.
The minimum of

$$E(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

occurs where elliptical contours intersect circular contours.

Example: Two-dimensional case Let

$$E(w_1, w_2) = \frac{1}{2}(4w_1^2 + w_2^2).$$

Unregularized minimum: $(0, 0)$.
Suppose instead the minimum without regularization were at $(2, 1)$.
Adding regularization yields

$$\tilde{E}(w_1, w_2) = \frac{1}{2}(4w_1^2 + w_2^2) + \frac{\lambda}{2}(w_1^2 + w_2^2).$$

The effective curvature becomes

$$(4 + \lambda) \text{ in direction } w_1, \quad (1 + \lambda) \text{ in direction } w_2.$$

The solution shifts toward the origin.

Directional effect Regularization shrinks more strongly in directions where:

$$\text{data curvature is small.}$$

This suppresses parameters that contribute weakly to reducing training error.

7.11.4 Practical Example: Linear Regression

Consider linear regression:

$$E(\mathbf{w}) = \frac{1}{2N}\|X\mathbf{w} - \mathbf{y}\|^2.$$

The L2-regularized solution satisfies:

$$(X^\top X + \lambda I)\mathbf{w} = X^\top \mathbf{y}.$$

Thus,

$$\mathbf{w} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}.$$

Adding λI stabilizes inversion and reduces variance.
This is precisely Tikhonov regularization.

7.11.5 Summary

- Regularization modifies the objective:

$$E(\mathbf{w}) \to E(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|^2.$$

- The gradient becomes:

$$\nabla E + \lambda \mathbf{w}.$$

- Geometrically, solutions move toward the origin.
- Regularization reduces overfitting by suppressing weakly supported parameter directions.

7.12 L_p Regularization in PyTorch

Consider the regularized objective

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \frac{\lambda}{p}\sum_{\ell} \|W_\ell\|_p^p, \qquad p > 1.$$

Here only weight matrices W_ℓ are regularized (biases excluded).

7.12.1 PyTorch Implementation

```
p = 3.0
lam = 1e-4

def lp_weight_penalty(model, p: float):
    reg = 0.0
    for name, param in model.named_parameters():
        if not param.requires_grad:
            continue
        if name.endswith("weight"):  # weights only
            reg = reg + param.abs().pow(p).sum()
    return reg

for x, y in train_loader:
    optimizer.zero_grad(set_to_none=True)
```

```
15
16      yhat = model(x)
17      data_loss = criterion(yhat, y)
18
19      reg = lp_weight_penalty(model, p)
20      loss = data_loss + (lam / p) * reg
21
22      loss.backward()
23      optimizer.step()
```

7.12.2 Gradient Behavior

Because the regularizer depends only on weights,

$$\tilde{\mathcal{L}} = \mathcal{L} + R(W),$$

we obtain

$$\frac{\partial \tilde{\mathcal{L}}}{\partial a} = \frac{\partial \mathcal{L}}{\partial a}, \qquad \frac{\partial \tilde{\mathcal{L}}}{\partial z} = \frac{\partial \mathcal{L}}{\partial z}.$$

Thus, backpropagation deltas are unchanged.

Only parameter gradients are modified:

$$\frac{\partial \tilde{\mathcal{L}}}{\partial W} = \frac{\partial \mathcal{L}}{\partial W} + \lambda |W|^{p-2} \odot W.$$

For special cases:

$$p = 2 \Rightarrow +\lambda W, \qquad p = 3 \Rightarrow +\lambda |W| W.$$

Bias gradients remain unchanged when biases are not regularized.

7.13 Inspecting the Autograd Graph

PyTorch builds a dynamic computation graph. The loss

$$\texttt{loss} = \texttt{data_loss} + \frac{\lambda}{p}\texttt{reg}$$

creates two separate computational branches:

$$\tilde{\mathcal{L}} = \mathcal{L}_{\text{data}} + R(W).$$

7.13.1 Manual Graph Inspection

Each tensor stores a `grad_fn` describing how it was created:

```
loss = data_loss + (lam / p) * reg
print(loss.grad_fn)
```

One can traverse backward:

```
fn = loss.grad_fn
while fn is not None:
    print(fn)
    next_fns = fn.next_functions
    if not next_fns:
        break
    fn = next_fns[0][0]
```

Typical nodes include:

```
AddBackward0
MseLossBackward0
SigmoidBackward0
PowBackward0
AbsBackward0
```

The `AddBackward0` node merges:

- the data loss branch,
- the regularization branch.

7.13.2 Graph Visualization with torchviz

```
# install once
pip install torchviz
```

```
from torchviz import make_dot

loss = data_loss + (lam / p) * reg
graph = make_dot(loss, params=dict(model.named_parameters()))
graph.render("autograd_graph", format="png")
```

This produces a visualization where:

- one branch flows through activations to the data loss,
- one branch flows directly from weights to the regularizer,
- both merge at the final loss node.

7.13.3 Key Principle

Autograd does not label gradients by source. It applies the chain rule over the computation graph. If a term does not depend on a tensor, its derivative with respect to that tensor is structurally zero:

$$\frac{\partial R(W)}{\partial a} = 0, \qquad \frac{\partial R(W)}{\partial z} = 0.$$

Therefore, regularization affects only the gradients of parameters appearing in the regularizer.

7.14 Consistency of Regularizers and Scaling Symmetries

A desirable property of regularization is *consistency*: if two parameter sets represent the same function, the regularizer should assign them the same penalty.

Neural networks often admit multiple equivalent parameterizations due to internal scaling symmetries. Standard weight decay does not always respect these symmetries.

7.14.1 Two-Layer Network Formulation

Consider a two-layer neural network.

First layer

$$z_j = \sum_i w_{ji} x_i + w_{j0}, \qquad a_j = \sigma(z_j),$$

where:

- $W_1 = \{w_{ji}\}$ are first-layer weights,
- w_{j0} are first-layer biases,
- $\sigma(\cdot)$ is a nonlinear activation function.

Second layer (linear output)

$$y_k = \sum_j v_{kj} a_j + v_{k0},$$

where:

- $W_2 = \{v_{kj}\}$ are second-layer weights,
- v_{k0} are output biases.

We now analyze invariances of this model.

7.14.2 Invariance Under Input Linear Transformations

Suppose inputs undergo a linear transformation:

$$x_i \to \tilde{x}_i = a x_i + b.$$

The first-layer pre-activation becomes:

$$\tilde{z}_j = \sum_i w_{ji} \tilde{x}_i + w_{j0} = \sum_i w_{ji}(a x_i + b) + w_{j0}.$$

Expanding,

$$\tilde{z}_j = a \sum_i w_{ji} x_i + b \sum_i w_{ji} + w_{j0}.$$

To preserve the original mapping, define transformed parameters:

$$\tilde{w}_{ji} = \frac{1}{a} w_{ji}, \qquad \tilde{w}_{j0} = w_{j0} - \frac{b}{a} \sum_i w_{ji}.$$

Substituting,

$$\tilde{z}_j = \sum_i \tilde{w}_{ji} \tilde{x}_i + \tilde{w}_{j0} = z_j.$$

Therefore,

$$\tilde{a}_j = \sigma(\tilde{z}_j) = \sigma(z_j) = a_j.$$

The overall function remains unchanged.

7.14.3 Hidden Unit Rescaling Symmetry

A more fundamental symmetry arises from rescaling hidden activations.

Suppose we rescale hidden units:

$$a_j \to \tilde{a}_j = ca_j.$$

To preserve the output,

$$y_k = \sum_j v_{kj} a_j,$$

we compensate by transforming second-layer weights:

$$v_{kj} \to \tilde{v}_{kj} = \frac{1}{c} v_{kj}.$$

Then:

$$\sum_j \tilde{v}_{kj} \tilde{a}_j = \sum_j \frac{1}{c} v_{kj} (ca_j) = \sum_j v_{kj} a_j.$$

Thus,

$$(W_1, W_2) \quad \text{and} \quad (cW_1, \tfrac{1}{c} W_2)$$

represent identical functions.

This symmetry exists even without changing the inputs.

7.14.4 Why Standard L_2 Regularization Breaks This Symmetry

Standard weight decay is

$$\Omega(W) = \frac{\lambda}{2} \left(\sum_{w \in W_1} w^2 + \sum_{v \in W_2} v^2 \right).$$

Under hidden scaling:

$$W_1 \to cW_1, \qquad W_2 \to \frac{1}{c} W_2,$$

the penalty becomes:

$$\Omega = \frac{\lambda}{2} \left(c^2 \sum_{W_1} w^2 + \frac{1}{c^2} \sum_{W_2} v^2 \right).$$

The penalty changes with c, even though the network function is unchanged.

Therefore:

> Standard weight decay penalizes parameterization, not function complexity.

7.14.5 Concrete Numeric Example

Let

$$W_1 = 2, \qquad W_2 = 3.$$

Then

$$\Omega = \frac{\lambda}{2} (2^2 + 3^2) = \frac{13\lambda}{2}.$$

Rescale hidden units with $c = 2$:

$$W_1 \to 4, \qquad W_2 \to \frac{3}{2}.$$

New penalty:

$$\frac{\lambda}{2}(16 + 2.25) = \frac{\lambda}{2}(18.25).$$

Different penalty. Same function.
This illustrates the inconsistency.

7.14.6 Layer-Wise Regularization

One improvement is to use layer-specific penalties:

$$\Omega(W) = \frac{\lambda_1}{2} \sum_{w \in W_1} w^2 + \frac{\lambda_2}{2} \sum_{v \in W_2} v^2.$$

Under hidden rescaling by c, invariance can be preserved if

$$\lambda_1 \to c^{-2}\lambda_1, \qquad \lambda_2 \to c^2\lambda_2.$$

This aligns regularization with network symmetries.

7.14.7 Bayesian Interpretation

Quadratic regularization corresponds to Gaussian priors:

$$p(W) \propto \exp\left(-\frac{\alpha_1}{2} \sum_{W_1} w^2 - \frac{\alpha_2}{2} \sum_{W_2} v^2 \right).$$

Equivalently,

$$W_1 \sim \mathcal{N}(0, \alpha_1^{-1}), \qquad W_2 \sim \mathcal{N}(0, \alpha_2^{-1}).$$

Larger precision parameters α imply stronger shrinkage.

7.14.8 Summary

Neural networks possess internal scaling symmetries:

$$(W_1, W_2) \sim (cW_1, \tfrac{1}{c}W_2).$$

Standard weight decay breaks this symmetry. Invariant regularization seeks to penalize functions rather than arbitrary parameter scalings.

7.15 Classical Bias–Variance Trade-Off and the Modern Overparameterized Regime

7.15.1 Classical Bias–Variance Trade-Off

Consider the standard supervised learning model

$$y = f^\star(x) + \varepsilon, \qquad \mathbb{E}[\varepsilon] = 0, \qquad \mathrm{Var}(\varepsilon) = \sigma^2,$$

where $f^\star$ is the true function and ε represents irreducible noise.

For a learned predictor $\hat{f}(x)$, the expected squared test error decomposes as

$$\mathbb{E}\big[(\hat{f}(x) - y)^2\big] = \underbrace{\big(\mathbb{E}[\hat{f}(x)] - f^\star(x)\big)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\big[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\big]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible error}}.$$

Bias. Bias measures systematic error:

$$\text{Bias}(x) = \mathbb{E}[\hat{f}(x)] - f^\star(x).$$

High bias corresponds to underfitting. For example, fitting a linear model to quadratic data results in both high training and test errors.

Variance. Variance measures sensitivity to fluctuations in the training data:

$$\text{Var}(x) = \mathbb{E}\left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\right].$$

High variance corresponds to overfitting. For example, a high-degree polynomial may interpolate noisy data, achieving near-zero training error but large test error.

Classical complexity curve. As model complexity increases:

- Bias decreases,
- Variance increases.

This yields the classical U-shaped test error curve:

- Low complexity: underfitting (high bias),
- Intermediate complexity: optimal trade-off,
- High complexity: overfitting (high variance).

In classical statistics, increasing complexity beyond the interpolation threshold was expected to degrade generalization.

7.15.2 Modern Overparameterized Regime

Modern deep neural networks often operate in a regime where

$$\text{Number of parameters} \gg \text{Number of training samples}.$$

Such models can interpolate the training data:

$$\text{Training error} = 0.$$

According to the classical view, this should imply extreme variance and poor generalization. However, empirical evidence shows that in many cases:

$$\text{Test error decreases even beyond interpolation}.$$

This phenomenon leads to what is known as *double descent*.

7.15.3 Double Descent Phenomenon

As model size increases, test error often behaves as follows:

1. In the classical regime:
   - Test error decreases,
   - Then increases near the interpolation threshold.
2. In the overparameterized regime:
   - Test error decreases again.

Thus, the curve transitions from the classical U-shape to a *double descent* curve:

$$\text{U-shape} \longrightarrow \text{Peak at interpolation} \longrightarrow \text{Second descent}.$$

7.15.4 Why Does This Happen?

Several mechanisms help explain this behavior.

Implicit regularization of SGD. Stochastic gradient descent tends to prefer:

- Low-norm solutions,
- Flat minima,
- Structured representations.

Even in large parameter spaces, optimization dynamics restrict the effective solution complexity.

Parameterization versus function complexity. Increasing parameter count does not necessarily increase effective function complexity. Many parameter configurations represent the same function.

Flat minima. Overparameterized models often admit wide, flat valleys in the loss landscape. Solutions in flat regions tend to generalize better than sharp minima.

7.15.5 Illustrative Example

Suppose:

- Dataset size: 100 samples,
- Model A: 50 parameters,
- Model B: 10,000 parameters.

Both models may interpolate the data. However:

- Model A has limited representation flexibility,
- Model B has many equivalent interpolating solutions,
- SGD often converges to the minimum-norm interpolating solution.

Minimum-norm interpolators frequently generalize well, linking this behavior to:

- Ridge regression,
- Kernel methods,
- Neural tangent kernel (NTK) theory.

7.15.6 Key Contrast

Classical view.

$$\text{Complexity} \uparrow \Rightarrow \ \text{Bias} \downarrow \Rightarrow \ \text{Variance} \uparrow \Rightarrow \ \text{Test error} \uparrow .$$

Modern deep learning view.

$$\text{Complexity} \uparrow \Rightarrow \ \text{Bias} \downarrow,$$

$$\text{Variance initially} \uparrow, \quad \text{then implicit regularization dominates},$$

$$\text{Test error} \downarrow \ \text{again}.$$

7.15.7 Implications for Deep Architectures

Convolutional neural networks and transformers:

- Are highly overparameterized,
- Are trained with SGD,
- Possess strong architectural inductive biases.

Despite perfectly fitting training data, they often generalize well. Thus, generalization in deep learning depends not only on parameter count but also on:

- Optimization dynamics,
- Architectural structure,
- Data geometry,
- Implicit biases.

Conclusion. The classical bias–variance framework assumes a fixed hypothesis class and explicit regularization. Modern deep learning operates in highly overparameterized regimes with strong implicit regularization. A complete theory of generalization must therefore extend beyond the classical bias–variance trade-off.

7.16 Double Descent and Structural Regularization

7.16.1 Statistical Framework

Assume the data model

$$Y = f^\star(X) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid X] = 0, \qquad \mathrm{Var}(\varepsilon \mid X) = \sigma^2.$$

Let $\hat{f}_{\mathcal{D}}$ be the predictor obtained from a random dataset $\mathcal{D}$.
For squared loss,

$$\mathbb{E}_{\mathcal{D}}\Big[(\hat{f}(x) - Y)^2\Big] = \underbrace{\big(\mathbb{E}_{\mathcal{D}}\hat{f}(x) - f^\star(x)\big)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}\Big[(\hat{f}(x) - \mathbb{E}_{\mathcal{D}}\hat{f}(x))^2\Big]}_{\text{Var}} + \sigma^2.$$

Bias measures approximation error of the hypothesis class. Variance measures sensitivity to sampling fluctuations.

7.16.2 Classical Regime

Let $\mathcal{H}_d$ be a hypothesis class of complexity d.

$$d \uparrow \implies \text{Bias} \downarrow, \quad \text{Var} \uparrow .$$

If d is small,

$$f^\star \notin \mathcal{H}_d \;\Rightarrow\; \inf_{f \in \mathcal{H}_d} \|f - f^\star\| > 0,$$

so bias dominates.

This yields the classical U-shaped test error curve.

7.16.3 Interpolation Threshold

The interpolation threshold is the point where a model first becomes capable of fitting the training data exactly. At this point, the training error can approach zero, but the learned solution may become highly sensitive to noise, initialization, and small changes in the training set.

Let n be the sample size. Interpolation occurs when

$$\dim(\mathcal{H}_d) \approx n,$$

so there exists $\hat{f}$ satisfying

$$\hat{f}(x_i) = y_i \quad \forall i.$$

Near this threshold, variance can dominate generalization behavior. The model has just enough capacity to fit the data, but not enough redundancy to select among many stable interpolating solutions.

In linear regression with

$$X = U\Sigma V^\top,$$

the least-squares solution contains terms of the form

$$\frac{1}{\sigma_i}.$$

If $\sigma_i \to 0$, small perturbations in the data can be amplified, causing the variance to increase sharply. This produces the test-error spike observed near the interpolation threshold.

Beyond the interpolation threshold, modern overparameterized models may contain many interpolating solutions. Gradient-based optimization often selects solutions with lower effective complexity, such as lower-norm or smoother solutions, which can reduce test error again.

7.16.4 Double Descent

In modern models,

$$d \gg n.$$

There are infinitely many interpolating solutions.

Empirically,

$$\text{Test error decreases again as } d \to \infty.$$

Thus test error behaves as

$$\text{Decrease} \;\to\; \text{Spike} \;\to\; \text{Second Decrease}.$$

7.16.5 Implicit Bias and Low-Norm Selection

Let $\theta \in \mathbb{R}^p$.

Among interpolating parameters

$$\{\theta : f_\theta(x_i) = y_i\},$$

gradient-based optimization often selects

$$\theta^\star = \arg \min_{\theta \text{ interpolates}} \|\theta\|_2.$$

For linear models,

$$\theta^\star = \arg \min_{X\theta = y} \|\theta\|_2.$$

Norm control implies

$$|f_\theta(x)| \le \|\theta\|_2 \|x\|_2,$$

so smaller norm reduces effective variance.

Thus overparameterization creates redundancy, and optimization selects stable solutions.

7.17 Structural Regularization

Generalization is controlled not only by parameter count but by structural constraints.

7.17.1 Parameter Sharing

Let $\theta \in \mathbb{R}^p$.

Hard Sharing. Impose equality constraints

$$\theta_i = \theta_j \quad (i,j) \in \mathcal{G}.$$

The admissible set becomes

$$\Theta_{\text{hard}} = \{\theta \in \mathbb{R}^p : \theta_i = \theta_j \text{ for } (i,j) \in \mathcal{G}\},$$

so

$$\dim(\Theta_{\text{hard}}) < p.$$

Example (Convolution):

$$(f_K(x))(u) = \sum_v K(v)x(u-v).$$

Parameter count depends on kernel size, not input size.

Soft Sharing. Encourage clustering via

$$w_i \sim \sum_{j=1}^{K} \pi_j \mathcal{N}(\mu_j, \sigma_j^2).$$

Induced penalty:

$$\Omega(w) = -\sum_i \log\left(\sum_{j=1}^{K} \pi_j \mathcal{N}(w_i \mid \mu_j, \sigma_j^2)\right).$$

Hard sharing restricts feasible parameters. Soft sharing reshapes parameter distribution.

7.17.2 Residual Connections

Standard block:

$$x_{k+1} = F_k(x_k).$$

Residual block:

$$x_{k+1} = x_k + F_k(x_k).$$

Jacobian:

$$Dx_{k+1} = I + DF_k(x_k).$$

Eigenvalues shift:

$$\lambda(Dx_{k+1}) = 1 + \lambda(DF_k).$$

Thus gradient propagation involves

$$\prod_k (I + DF_k),$$

rather than

$$\prod_k DF_k.$$

Identity terms prevent vanishing gradients and stabilize optimization.

Synthesis

Modern generalization depends on

$$\text{Optimization bias} + \text{Architectural constraints} + \text{Data geometry}.$$

Overparameterization alone does not determine behavior.

Double descent reflects instability near interpolation, while structural mechanisms (norm bias, sharing, residual paths) restore stability in highly redundant regimes.

7.18 Model Averaging, Boosting, and Dropout

7.18.1 Model Averaging (Ensembles)

Let

$$f_m(x; w_m, b_m), \quad m = 1, \dots, M,$$

be trained predictors.

The ensemble predictor is

$$f_{\text{ens}}(x) = \frac{1}{M} \sum_{m=1}^{M} f_m(x; w_m, b_m).$$

For probabilistic classification,

$$p_m(y \mid x) = \text{softmax}(z_m(x)), \quad z_m = W_m a_m + b_m.$$

The ensemble probability is

$$p_{\text{ens}}(y \mid x) = \frac{1}{M} \sum_{m=1}^{M} p_m(y \mid x).$$

7.18.2 Variance Reduction

Assume regression model

$$f_m(x) = f^\star(x) + \varepsilon_m(x), \qquad \mathbb{E}[\varepsilon_m(x)] = 0.$$

Then

$$f_{\text{ens}}(x) = f^\star(x) + \frac{1}{M} \sum_{m=1}^{M} \varepsilon_m(x).$$

If

$$\mathbb{E}[\varepsilon_m(x)\varepsilon_\ell(x)] = 0 \quad (m \neq \ell),$$

then

$$\text{Var}(f_{\text{ens}}(x)) = \frac{1}{M} \text{Var}(f_m(x)).$$

More generally,

$$\text{Var}\left(\frac{1}{M} \sum_{m=1}^{M} f_m(x) \right) = \frac{1}{M^2} \sum_{m,\ell} \text{Cov}(f_m(x), f_\ell(x)).$$

7.18.3 Bagging

Given dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N},$$

construct bootstrap samples

$$\mathcal{D}^{(m)} \sim \text{sampling with replacement.}$$

Train

$$f_m(x; w_m, b_m) \quad \text{on } \mathcal{D}^{(m)}.$$

Final predictor:

$$f_{\text{bag}}(x) = \frac{1}{M} \sum_{m=1}^{M} f_m(x).$$

7.18.4 Boosting

Initialize

$$f_0(x) = 0.$$

At iteration t, compute residuals

$$r_t(x_i) = y_i - f_{t-1}(x_i).$$

Fit a weak learner g_t to $\{(x_i, r_t(x_i))\}$ and update

$$f_t(x) = f_{t-1}(x) + \alpha_t g_t(x),$$

where $\alpha_t > 0$ is a step size.

Thus boosting performs sequential residual correction.

Concrete Example (Regression). Assume training points

$$(x_1, y_1) = (1, 2), \qquad (x_2, y_2) = (2, 4).$$

Step 0.

$$f_0(x) = 0.$$

Residuals:

$$r_1(1) = 2, \qquad r_1(2) = 4.$$

Fit a weak linear model

$$g_1(x) = 2x.$$

Choose $\alpha_1 = 1$, so

$$f_1(x) = 2x.$$

Step 1. Residuals:

$$r_2(1) = 2 - 2 = 0, \qquad r_2(2) = 4 - 4 = 0.$$

Training error is now zero.

Noisy Example. Suppose instead

$$(x_1, y_1) = (1, 2), \qquad (x_2, y_2) = (2, 5).$$

Step 0:

$$f_0(x) = 0.$$

Residuals:

$$r_1(1) = 2, \qquad r_1(2) = 5.$$

Fit weak learner

$$g_1(x) = 2.5x.$$

Then

$$f_1(x) = 2.5x.$$

Residuals:

$$r_2(1) = 2 - 2.5 = -0.5, \qquad r_2(2) = 5 - 5 = 0.$$

Fit second weak learner g_2 to these residuals and update

$$f_2(x) = f_1(x) + \alpha_2 g_2(x).$$

Each iteration reduces bias by correcting remaining error.

Interpretation. Boosting constructs

$$f_T(x) = \sum_{t=1}^{T} \alpha_t g_t(x),$$

a weighted sum of weak learners.

Bias decreases as T increases, while variance is controlled by step sizes α_t and weak learner complexity.

7.18.5 Dropout

Consider a layer

$$z = Wx + b, \qquad a = \sigma(z).$$

Introduce a random mask

$$R \in \{0, 1\}^d, \qquad R_i \sim \text{Bernoulli}(p),$$

independently for each unit.

The dropout activation is

$$\tilde{a} = R \odot a,$$

where $\odot$ denotes elementwise multiplication.

This is called inverted dropout. The scaling factor $1/p$ is applied during training so that no additional scaling is required at test time.

Training minimizes the expected loss

$$\mathbb{E}_R\big[\mathcal{L}(f(x; w, b, R), y)\big].$$

If the layer has d hidden units, the number of possible subnetworks is

$$2^d,$$

since each unit may be either active or removed.

Thus dropout approximates averaging over exponentially many subnetworks.

Concrete Example (Two Hidden Units). Let

$$x \in \mathbb{R}^2, \qquad W = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad b = 0, \qquad \sigma(t) = t \quad \text{(identity)}.$$

Then

$$z = Wx = x, \qquad a = x.$$

Let $d = 2$ and $p = 1/2$. Possible masks:

$$R \in \{(0,0), (1,0), (0,1), (1,1)\}.$$

Corresponding subnetworks produce:

$$R = (1,1) \Rightarrow \tilde{a} = (x_1, x_2),$$

$$R = (1,0) \Rightarrow \tilde{a} = (x_1, 0),$$

$$R = (0,1) \Rightarrow \tilde{a} = (0, x_2),$$

$$R = (0,0) \Rightarrow \tilde{a} = (0,0).$$

Thus the model randomly selects among $2^2 = 4$ subnetworks.

Expectation Effect. Since

$$\mathbb{E}[R_i] = p,$$

we obtain

$$\mathbb{E}[\tilde{a}_i] = \mathbb{E}[R_i a_i] = p a_i.$$

To preserve scale at test time, one commonly rescales:

$$\tilde{a} = \frac{1}{p} R \odot a.$$

Then

$$\mathbb{E}[\tilde{a}_i] = a_i.$$

Interpretation. Dropout injects multiplicative noise:

$$\tilde{a}_i = a_i + (R_i - 1) a_i.$$

This reduces co-adaptation of units and acts as implicit model averaging:

$$f(x) \approx \frac{1}{2^d} \sum_{\text{subnetworks}} f_{\text{sub}}(x).$$

Variance across masks induces regularization and improves generalization.

7.18.6 Monte Carlo Dropout

Keep dropout active at inference time and perform multiple stochastic forward passes.

For $t = 1, \ldots, T$,

$$\hat{y}^{(t)}(x) = f(x; w, b, R^{(t)}),$$

where each $R^{(t)}$ is an independent dropout mask.

Regression. Predictive mean:

$$\mu(x) = \frac{1}{T}\sum_{t=1}^{T} \hat{y}^{(t)}(x).$$

Predictive variance:

$$\sigma^2(x) = \frac{1}{T}\sum_{t=1}^{T} \left(\hat{y}^{(t)}(x) - \mu(x)\right)^2.$$

Thus uncertainty is estimated via stochastic forward passes.

Concrete Example (Regression). Assume $T = 4$ stochastic passes produce

$$\hat{y}^{(1)} = 2.1, \quad \hat{y}^{(2)} = 1.9, \quad \hat{y}^{(3)} = 2.0, \quad \hat{y}^{(4)} = 2.4.$$

Then

$$\mu = \frac{2.1 + 1.9 + 2.0 + 2.4}{4} = 2.1,$$

$$\sigma^2 = \frac{1}{4}\left[(2.1-2.1)^2 + (1.9-2.1)^2 + (2.0-2.1)^2 + (2.4-2.1)^2\right] = 0.035.$$

The model predicts 2.1 with nonzero epistemic uncertainty.

Classification. For softmax outputs,

$$p^{(t)}(y \mid x) = \text{softmax}(z^{(t)}).$$

Averaged predictive distribution:

$$\bar{p}(y \mid x) = \frac{1}{T}\sum_{t=1}^{T} p^{(t)}(y \mid x).$$

Predictive entropy:

$$H(x) = -\sum_{k} \bar{p}_k(x) \log \bar{p}_k(x).$$

Concrete Example (Binary Classification). Suppose $T = 3$ passes yield

$$p^{(1)}(y = 1 \mid x) = 0.9, \quad p^{(2)}(y = 1 \mid x) = 0.6, \quad p^{(3)}(y = 1 \mid x) = 0.8.$$

Then

$$\bar{p}(y = 1 \mid x) = \frac{0.9 + 0.6 + 0.8}{3} = 0.7667.$$

Predictive entropy:

$$H(x) = -0.7667 \log(0.7667) - 0.2333 \log(0.2333).$$

Higher entropy indicates greater model uncertainty.

Interpretation. Monte Carlo dropout approximates Bayesian model averaging:

$$p(y \mid x) \approx \int p(y \mid x, w) q(w)\, dw,$$

where dropout induces an approximate posterior $q(w)$.
Thus predictive uncertainty arises from weight uncertainty induced by stochastic masks.

7.18.7 Bias–Variance Perspective

For squared loss,

$$\mathbb{E}[(f(x) - y)^2] = \text{Bias}^2 + \text{Var} + \sigma^2.$$

- Model averaging and dropout reduce variance.
- Boosting reduces bias.

Thus averaging diverse predictors improves generalization by reducing variance while maintaining low bias.

Chapter 8

Ensemble Learning, Parameter Sharing, and the Interpolation Threshold

8.1 Overview

A single learned model can be sensitive to the particular training sample, initialization, optimization path, or feature representation used during training. Ensemble methods reduce this sensitivity by combining several predictors. The individual predictors are often called *base learners*, *members*, or *committee members*.

Let

$$f_m(x), \qquad m = 1, \ldots, M,$$

denote M trained models. Their predictions can be combined by averaging, voting, or a learned weighted combination. The central principle is

$$\text{accurate members} + \text{diverse errors} \quad \Longrightarrow \quad \text{strong ensemble.}$$

Accuracy alone is insufficient. If every member makes the same error, averaging provides little benefit.

Table 8.1: Main ways to combine multiple models.

Combination	Mathematical form	Typical use
Mean prediction	$\hat{y}(x) = \frac{1}{M} \sum_{m=1}^{M} f_m(x)$	Regression
Weighted mean	$\hat{y}(x) = \sum_{m=1}^{M} \alpha_m f_m(x)$, $\alpha_m \geq 0$, $\sum_m \alpha_m = 1$	Regression or calibrated model fusion
Hard voting	$\hat{y}(x) = \text{mode}\{\hat{y}_1(x), \ldots, \hat{y}_M(x)\}$	Classification
Soft voting	$\hat{y}(x) = \arg\max_k \frac{1}{M} \sum_{m=1}^{M} p_m(y = k \mid x)$	Probabilistic classification
Stacking	$\hat{y}(x) = g(f_1(x), \ldots, f_M(x))$	Learned model combination

8.2 Model Averaging: Ensembles and Committees

For regression, the simplest committee prediction is the arithmetic mean

$$\bar{f}(x) = \frac{1}{M} \sum_{m=1}^{M} f_m(x).$$

A weighted committee uses

$$\bar{f}_\alpha(x) = \sum_{m=1}^{M} \alpha_m f_m(x), \qquad \alpha_m \geq 0, \qquad \sum_{m=1}^{M} \alpha_m = 1.$$

Equal weights are robust when the models have similar quality. Learned weights can be useful when validation performance differs substantially across members.

For a K-class classifier, model m produces a probability vector

$$\boldsymbol{p}_m(x) = \Big[p_m(y = 1 \mid x) \quad \cdots \quad p_m(y = K \mid x) \Big]^T.$$

Soft voting averages the probability vectors:

$$\bar{\boldsymbol{p}}(x) = \frac{1}{M} \sum_{m=1}^{M} \boldsymbol{p}_m(x), \qquad \hat{y}_{\text{soft}}(x) = \arg\max_k \bar{p}_k(x).$$

Hard voting first converts each probability vector to a class label and then selects the majority class:

$$\hat{y}_{\text{hard}}(x) = \text{mode} \left\{ \arg\max_k p_1(y = k \mid x), \ldots, \arg\max_k p_M(y = k \mid x) \right\}.$$

Soft voting retains confidence information and is usually preferable when the member probabilities are reasonably calibrated.

8.2.1 Flow of a Parallel Committee

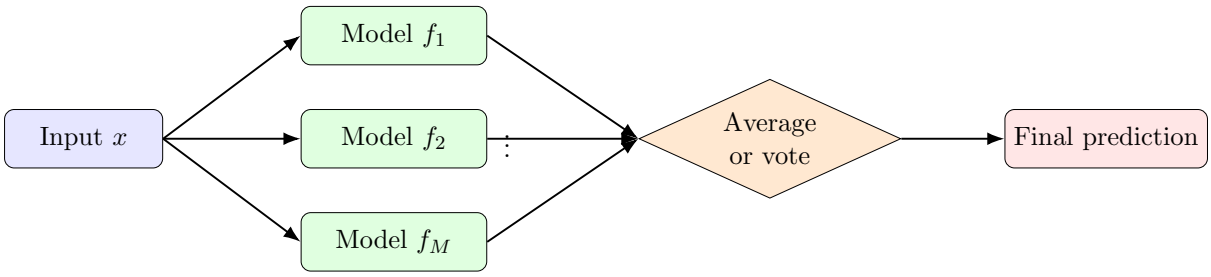


Figure 8.1: Parallel committee architecture. Each member processes the same input, and a combination rule produces the final prediction.

8.2.2 Numerical Example: Soft Voting

Suppose three classifiers produce the following probabilities for two classes:

$$\boldsymbol{p}_1 = (0.80, 0.20), \qquad \boldsymbol{p}_2 = (0.55, 0.45), \qquad \boldsymbol{p}_3 = (0.30, 0.70).$$

The average probability is

$$\bar{\boldsymbol{p}} = \frac{1}{3} \left[(0.80, 0.20) + (0.55, 0.45) + (0.30, 0.70) \right] = (0.55, 0.45).$$

Therefore, the committee predicts class 1. Hard voting gives the same result because two of the three models individually predict class 1.

8.3 Bias–Variance Trade-Off

Assume the regression model

$$Y = f^\star(X) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid X] = 0, \qquad \mathrm{Var}(\varepsilon \mid X) = \sigma^2.$$

A training set $\mathcal{D}$ produces a predictor $\hat{f}_{\mathcal{D}}$. At a fixed input x, the expected squared prediction error is

$$\mathbb{E}_{\mathcal{D},\varepsilon}\left[\left(Y - \hat{f}_{\mathcal{D}}(x)\right)^2\right] = \underbrace{\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)] - f^\star(x)\right)^2}_{\text{Bias}^2} \tag{8.1}$$

$$+ \underbrace{\mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}_{\mathcal{D}}(x) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]\right)^2\right]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}. \tag{8.2}$$

Bias measures systematic approximation error. Variance measures sensitivity to the sampled training set. Increasing model flexibility usually decreases bias but can increase variance. Ensemble averaging primarily targets variance, whereas boosting often targets bias by sequentially correcting errors.

Table 8.2: Interpretation of bias and variance.

Condition	Typical behavior	Common remedy
High bias	Training and validation errors are both high; the model underfits	Increase model capacity, improve features, reduce excessive regularization, or use boosting
High variance	Training error is low but validation error is substantially higher	Add data, regularize, simplify the model, or use bagging/model averaging
High irreducible noise	Even a suitable model cannot perfectly predict the target	Improve measurement quality, collect more informative variables, or model uncertainty explicitly

8.4 Expected Error of Individual Models

Let an individual regression model be

$$f_m(x) = f^\star(x) + b(x) + \eta_m(x),$$

where $b(x)$ is the common bias and $\eta_m(x)$ is a zero-mean random deviation satisfying

$$\mathbb{E}[\eta_m(x)] = 0, \qquad \mathrm{Var}(\eta_m(x)) = v(x).$$

For a noisy target $Y = f^\star(x) + \varepsilon$, the expected squared error of one model is

$$\mathbb{E}\left[(Y - f_m(x))^2\right] = b^2(x) + v(x) + \sigma^2.$$

Thus, an individual model carries its squared bias, its full prediction variance, and the irreducible noise.

8.4.1 Classification Error of an Individual Member

For binary classification, let

$$P(\text{member } m \text{ is wrong}) = p.$$

The expected 0–1 loss of one member is simply

$$\mathbb{E}[L_m] = p.$$

An individual member is better than random guessing when $p < 0.5$ for a balanced binary problem.

8.5 Expected Error of the Committee

8.5.1 Regression Committee with Correlated Members

Consider the mean committee

$$\bar{f}(x) = \frac{1}{M} \sum_{m=1}^{M} f_m(x).$$

Assume every member has prediction variance $v(x)$ and every pair of distinct members has correlation $\rho(x)$. Therefore,

$$\mathrm{Cov}(f_m(x), f_\ell(x)) = \rho(x)v(x), \qquad m \neq \ell.$$

The committee variance is

$$\mathrm{Var}(\bar{f}(x)) = \frac{1}{M^2} \left[Mv(x) + M(M-1)\rho(x)v(x) \right] \tag{8.3}$$

$$= v(x) \left[\rho(x) + \frac{1-\rho(x)}{M} \right]. \tag{8.4}$$

Hence, the committee expected error is

$$\boxed{\mathbb{E}\left[(Y - \bar{f}(x))^2 \right] = b^2(x) + v(x) \left[\rho(x) + \frac{1-\rho(x)}{M} \right] + \sigma^2}$$

when all members have the same bias $b(x)$.

This equation reveals three important cases:

- If $\rho = 0$, then the variance becomes v/M.
- If $\rho = 1$, then the variance remains v, so averaging gives no variance reduction.
- If $0 < \rho < 1$, the variance decreases with M but approaches the nonzero limit ρv.

8.5.2 Numerical Example: Variance Reduction

Suppose

$$b^2 = 0.04, \qquad v = 0.25, \qquad \sigma^2 = 0.01, \qquad M = 5, \qquad \rho = 0.20.$$

The expected error of one model is

$$0.04 + 0.25 + 0.01 = 0.30.$$

The committee variance is

$$0.25 \left[0.20 + \frac{0.80}{5} \right] = 0.25(0.36) = 0.09.$$

Therefore, the committee expected error is

$$0.04 + 0.09 + 0.01 = 0.14.$$

The improvement comes from variance reduction. The common bias and irreducible noise are unchanged.

8.5.3 Majority-Vote Classification Committee

Assume an odd number M of independent binary classifiers, each with error probability $p < 0.5$. The committee is wrong when more than half of the members are wrong. Therefore,

$$P_{\text{committee}} = \sum_{k=(M+1)/2}^{M} \binom{M}{k} p^k (1-p)^{M-k}.$$

For $M = 3$,

$$P_{\text{committee}} = \binom{3}{2} p^2(1-p) + p^3 = 3p^2 - 2p^3.$$

If each model has error $p = 0.30$, then

$$P_{\text{committee}} = 3(0.30)^2 - 2(0.30)^3 = 0.216.$$

The committee error is 21.6%, compared with 30% for an individual member. This improvement depends strongly on the independence assumption. Positively correlated classification errors reduce the gain.

Table 8.3: Expected-error comparison between an individual model and an averaged committee.

Quantity	Individual model	Mean committee of M exchangeable models
Bias squared	b^2	b^2 if all members share the same bias
Variance	v	$v\left[\rho + \frac{1-\rho}{M}\right]$
Irreducible noise	σ^2	σ^2
Expected squared error	$b^2 + v + \sigma^2$	$b^2 + v\left[\rho + \frac{1-\rho}{M}\right] + \sigma^2$
Main requirement for improvement	Low individual error	Low member error and low error correlation

8.6 Parameter Sharing

Parameter sharing uses the same parameter or parameter block in multiple parts of a model. It reduces the number of independent degrees of freedom and introduces a structural assumption about the task.

A fully connected mapping from an input with n_{in} units to an output with n_{out} units requires

$$n_{\text{in}} n_{\text{out}} + n_{\text{out}}$$

parameters. In contrast, a two-dimensional convolution with kernel size $k_h \times k_w$, C_{in} input channels, and C_{out} output channels requires

$$k_h k_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}$$

parameters, independent of the spatial image size. The same kernel is reused at every spatial location:

$$y_c(u,v) = \sum_{d=1}^{C_{\text{in}}} \sum_{i=1}^{k_h} \sum_{j=1}^{k_w} K_{c,d,i,j}\, x_d(u+i, v+j) + b_c.$$

This sharing encodes translation-equivariant local feature detection.

Other examples include:

- recurrent neural networks, where the same transition parameters are reused across time steps;
- Siamese networks, where two or more branches use the same encoder;
- transformer encoders applied repeatedly to different tokens, where projection matrices are shared across token positions;
- multi-task models, where early layers are shared across related tasks.

8.7 Hard vs. Soft Parameter Sharing

8.7.1 Hard Parameter Sharing

Hard sharing imposes exact equality. For two tasks A and B, a shared representation may be written as

$$h_A(x) = h(x; \theta_s), \qquad h_B(x) = h(x; \theta_s),$$

with task-specific heads

$$\hat{y}_A = g_A(h_A(x); \theta_A), \qquad \hat{y}_B = g_B(h_B(x); \theta_B).$$

The same parameter vector θ_s is used by both tasks. The joint objective is

$$\mathcal{L}_{exthard} = \mathcal{L}_A(\theta_s, \theta_A) + \lambda_B \mathcal{L}_B(\theta_s, \theta_B).$$

Hard sharing strongly reduces the effective parameter count and can improve generalization when tasks are closely related.

8.7.2 Soft Parameter Sharing

Soft sharing gives each task its own parameters but penalizes disagreement:

$$\mathcal{L}_{extsoft} = \mathcal{L}_A(\theta_A) + \mathcal{L}_B(\theta_B) + \lambda \|\theta_A - \theta_B\|_2^2.$$

The models may specialize while remaining close. A small λ allows substantial task-specific adaptation, whereas a large λ approaches hard sharing.

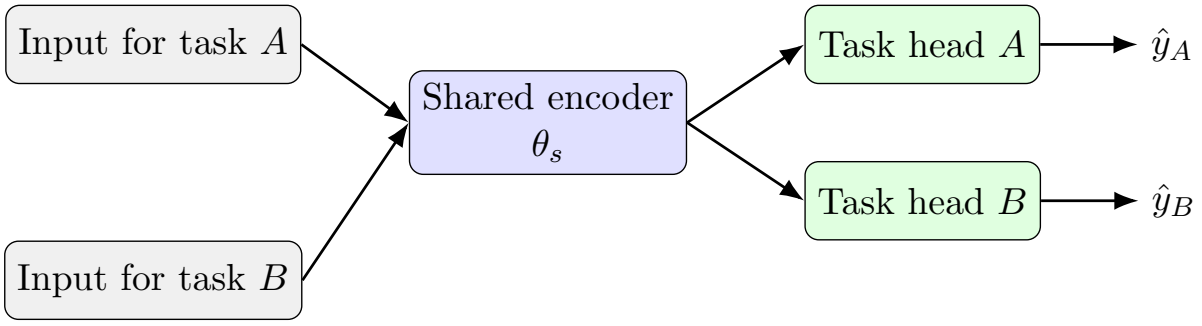


Figure 8.2: Hard parameter sharing in multi-task learning. The encoder parameters are exactly shared, while the output heads remain task-specific.

Table 8.4: Comparison of hard and soft parameter sharing.

Property	Hard sharing	Soft sharing
Constraint	Exact equality of shared parameters	Similarity encouraged by a penalty or prior
Typical form	Common encoder with task-specific heads	Separate encoders with $\lambda\|\theta_A - \theta_B\|^2$
Parameter count	Lower	Higher
Flexibility	Lower	Higher
Regularization strength	Strong	Tunable through λ
Best suited for	Closely related tasks or repeated structure	Related but nonidentical tasks
Main risk	Negative transfer if tasks conflict	Weak knowledge transfer or excessive model size

8.8 Interpolation Threshold

A model interpolates the training data when it achieves zero training error:

$$\hat{f}(x_i) = y_i, \qquad i = 1, \ldots, N.$$

The *interpolation threshold* is the effective model capacity at which exact fitting first becomes possible. In a linear regression problem with N samples and p adjustable parameters, this transition often occurs near

$$p \approx N,$$

although the exact location depends on rank, regularization, noise, optimization, and model structure.

Near the threshold, the fitted solution can become highly sensitive to small changes in the data. In linear regression,

$$\hat{\boldsymbol{w}} = (X^T X)^{-1} X^T \boldsymbol{y}$$

when the inverse exists. If $X^T X$ is nearly singular, small perturbations in $\boldsymbol{y}$ can cause large changes in $\hat{\boldsymbol{w}}$. This instability produces high variance and may create a peak in test error.

Beyond the interpolation threshold, an overparameterized system has many exact-fitting solutions. Gradient-based optimization and architectural structure may select a stable, low-complexity solution, producing the second descent in the double-descent curve.

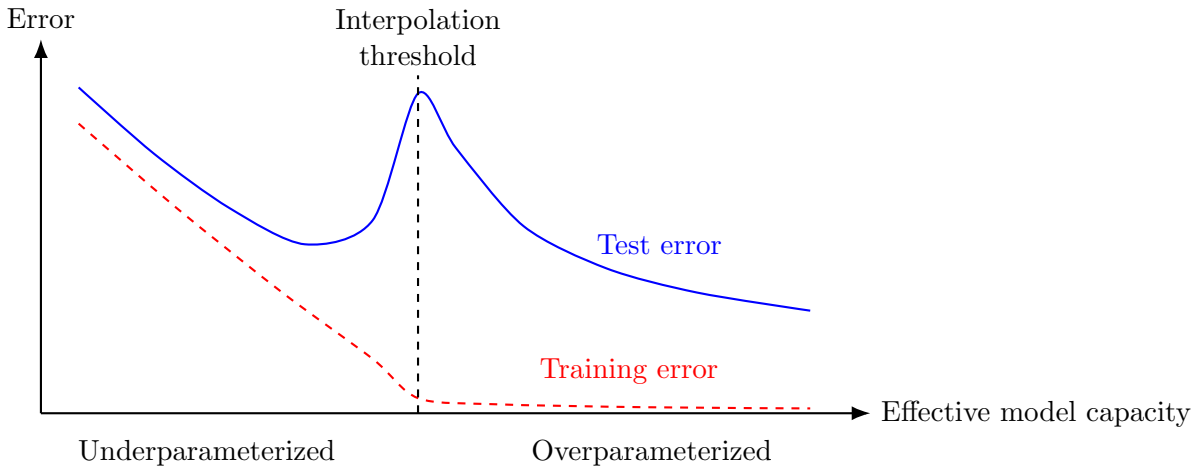


Figure 8.3: Conceptual double-descent curve. Test error may peak near the interpolation threshold and decrease again in the overparameterized regime.

Table 8.5: Behavior across model-capacity regimes.

Regime	Training behavior	Generalization behavior
Underparameterized	Cannot exactly fit the training data	Bias is often dominant
Near interpolation	Training error approaches zero	Variance and numerical instability may peak
Overparameterized	Many exact-fitting solutions exist	Implicit regularization may select stable interpolating solutions

8.9 Bagging: Bootstrap Aggregation

8.9.1 Bootstrap Sampling

Bootstrap sampling is a statistical resampling procedure used to construct new datasets from an observed dataset. Let the original training set be

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}.$$

A bootstrap sample, denoted by $\mathcal{D}^*$, is obtained by drawing N observations from $\mathcal{D}$ *with replacement*. Sampling with replacement means that after an observation is selected, it remains available for subsequent selections. Consequently, the same observation may appear several times in a bootstrap sample, while other observations may not appear at all.

A bootstrap dataset can therefore be written as

$$\mathcal{D}^* = \{(x_{i_1}, y_{i_1}), (x_{i_2}, y_{i_2}), \ldots, (x_{i_N}, y_{i_N})\},$$

where each index i_j is sampled independently from

$$\{1, 2, \ldots, N\}.$$

The size of $\mathcal{D}^*$ is the same as the size of the original dataset, but the composition of the two datasets is generally different.

Example of Bootstrap Sampling

Consider the dataset

$$\mathcal{D} = \{A, B, C, D, E\}.$$

A bootstrap sample of size five may be

$$\mathcal{D}_1^* = \{A, C, C, D, E\},$$

while another bootstrap sample may be

$$\mathcal{D}_2^* = \{B, B, C, E, E\}.$$

In the first sample, observation C appears twice and observation B is absent. In the second sample, observations B and E are repeated, while observations A and D are absent.

Table 8.6: Example bootstrap samples drawn from the same dataset.

Dataset	Sampled observations	Main observation
Original dataset	$A,\ B,\ C,\ D,\ E$	Each observation appears exactly once
Bootstrap sample 1	$A,\ C,\ C,\ D,\ E$	C is repeated and B is omitted
Bootstrap sample 2	$B,\ B,\ C,\ E,\ E$	B and E are repeated; A and D are omitted

Probability That an Observation Is Omitted

For a dataset containing N observations, the probability of selecting a particular observation in one draw is

$$\frac{1}{N}.$$

Therefore, the probability that the observation is not selected in one draw is

$$1 - \frac{1}{N}.$$

Because a bootstrap sample contains N independent draws, the probability that the observation is never selected is

$$P(\text{observation omitted}) = \left(1 - \frac{1}{N}\right)^N.$$

As N becomes large,

$$\left(1 - \frac{1}{N}\right)^N \longrightarrow e^{-1} \approx 0.368.$$

Thus, approximately 36.8% of the original observations are omitted from a typical bootstrap sample. Equivalently, the expected proportion of observations selected at least once is

$$1 - e^{-1} \approx 0.632.$$

Therefore, a bootstrap sample of size N contains approximately 63.2% unique observations from the original dataset. The remaining sample positions are occupied by repeated observations.

Out-of-Bag Observations

The observations that are not selected in a particular bootstrap sample are called *out-of-bag* observations. For the mth bootstrap sample, the corresponding out-of-bag set may be written as

$$\mathcal{D}_{\text{OOB}}^{(m)} = \mathcal{D} \setminus \mathcal{D}^{(m)}.$$

A model trained on $\mathcal{D}^{(m)}$ can be evaluated using $\mathcal{D}_{\text{OOB}}^{(m)}$ because these observations were not used to train that model. Out-of-bag evaluation provides an internal estimate of prediction error without requiring a separate validation set.

For an observation (x_i, y_i), let

$$\mathcal{M}_i = \left\{ m : (x_i, y_i) \notin \mathcal{D}^{(m)} \right\}$$

denote the set of models for which observation i is out of bag. The out-of-bag prediction for regression can be computed as

$$\hat{y}_i^{\text{OOB}} = \frac{1}{|\mathcal{M}_i|} \sum_{m \in \mathcal{M}_i} f_m(x_i).$$

For classification, the corresponding prediction may be obtained using majority voting:

$$\hat{y}_i^{\text{OOB}} = \text{mode}\left\{ f_m(x_i) : m \in \mathcal{M}_i \right\}.$$

The overall out-of-bag error can then be estimated by comparing the out-of-bag predictions with the corresponding target values.

Bootstrap Sampling Flowchart

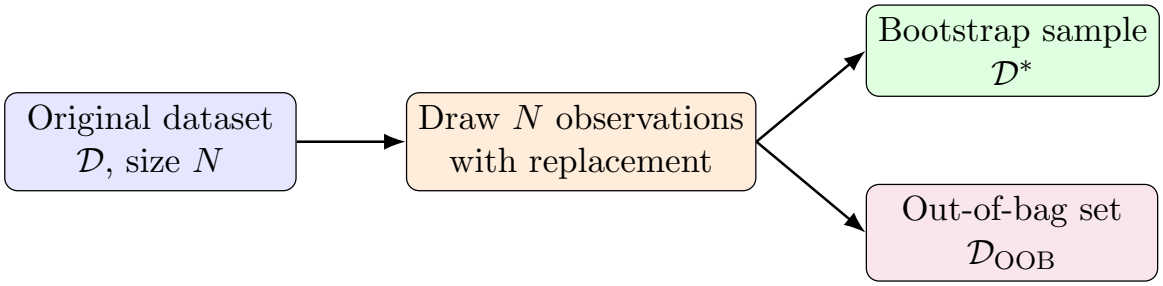


Figure 8.4: Bootstrap sampling creates a resampled training set and an associated out-of-bag set.

Bootstrap Sampling and Ensemble Learning

Bootstrap sampling and bagging are related but distinct concepts. Bootstrap sampling is a general-purpose statistical resampling method, whereas bagging is an ensemble-learning method that uses bootstrap samples to train multiple models.

Bootstrap = resampling observations with replacement

Bagging = training multiple models on bootstrap samples and combining them

The distinction is summarized in Table 8.7.

Table 8.7: Difference between bootstrap sampling and bagging.

Property	Bootstrap sampling	Bagging
Primary purpose	Estimate sampling variability, uncertainty, or model stability	Improve predictive stability and reduce variance
Basic operation	Generate datasets by sampling with replacement	Train multiple models on different bootstrap datasets
Model combination	Not required	Predictions are combined by averaging or voting
Typical output	Bootstrap statistics, standard errors, confidence intervals, or resampled datasets	A single ensemble prediction
Use of out-of-bag data	May be used for evaluation	Commonly used to estimate ensemble generalization error
Examples	Bootstrap confidence intervals and uncertainty estimation	Bagged decision trees and random forests

Why Bootstrap Sampling Helps Bagging

Many learning algorithms, particularly decision trees, are sensitive to small changes in the training data. Training the same algorithm on different bootstrap samples produces models with different structures and prediction errors. This variation creates diversity among the ensemble members.

Suppose that M bootstrap datasets are generated:

$$\mathcal{D}^{(1)}, \mathcal{D}^{(2)}, \ldots, \mathcal{D}^{(M)}.$$

A separate model is trained on each dataset:

$$f_1, f_2, \ldots, f_M.$$

Although the individual models may have relatively high variance, their errors are unlikely to be identical. Averaging their predictions can cancel part of the model-specific variation. Bootstrap sampling therefore provides the diversity required for bagging to reduce variance effectively.

> Bootstrap sampling creates different training datasets. Bagging converts the resulting model diversity into a more stable combined prediction.

Bagging constructs multiple training sets through bootstrap sampling. From the original data

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N},$$

each bootstrap dataset $\mathcal{D}^{(m)}$ is formed by drawing N observations independently with replacement. Model f_m is trained on $\mathcal{D}^{(m)}$, and the final prediction is obtained by averaging or voting:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{M} \sum_{m=1}^{M} f_m(x)$$

for regression, or

$$\hat{y}_{\text{bag}}(x) = \text{mode}\{f_1(x), \ldots, f_M(x)\}$$

for classification.

The probability that a particular training observation is not selected in one bootstrap sample is

$$\left(1 - \frac{1}{N}\right)^{N} \approx e^{-1} \approx 0.368.$$

Therefore, a bootstrap sample contains approximately 63.2% unique training observations, with the remaining positions occupied by repeated observations. The unused observations form the *out-of-bag* set for that member and can be used for internal validation.

8.9.2 Bagging Flowchart

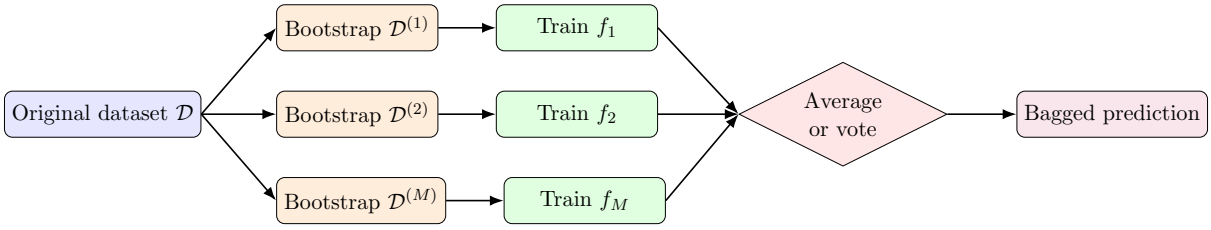


Figure 8.5: Bagging trains independent members on bootstrap resamples and combines their predictions in parallel.

8.9.3 Example: Bagged Regression Trees

Suppose five regression trees predict

$$4.8, \quad 5.4, \quad 5.1, \quad 4.9, \quad 5.3.$$

The bagged prediction is

$$\hat{y}_{\text{bag}} = \frac{4.8 + 5.4 + 5.1 + 4.9 + 5.3}{5} = 5.1.$$

Individual trees are unstable because small data changes can alter their split structure. Averaging many trees reduces this instability. Random forests extend bagging by also randomizing the candidate features considered at each split, which further reduces correlation among trees.

8.10 Boosting: Sequential Model Combination

Boosting trains learners sequentially. Each new learner focuses on deficiencies of the current ensemble. The final model is an additive expansion

$$F_T(x) = F_0(x) + \sum_{t=1}^{T} \alpha_t h_t(x),$$

where h_t is a weak learner and α_t controls its contribution.

8.10.1 Gradient-Boosting View

For a differentiable loss $\ell(y, F(x))$, initialize F_0 with a simple constant predictor. At stage t, compute pseudo-residuals

$$r_{it} = - \left. \frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{t-1}}.$$

Fit h_t to the pairs (x_i, r_{it}), then update

$$F_t(x) = F_{t-1}(x) + \nu \alpha_t h_t(x),$$

where $0 < \nu \leq 1$ is the learning rate or shrinkage parameter.

For squared error,

$$\ell(y, F) = \frac{1}{2}(y - F)^2,$$

the pseudo-residual is the ordinary residual:

$$r_{it} = y_i - F_{t-1}(x_i).$$

Thus, each new learner predicts the errors left by the current ensemble.

8.10.2 Boosting Flowchart

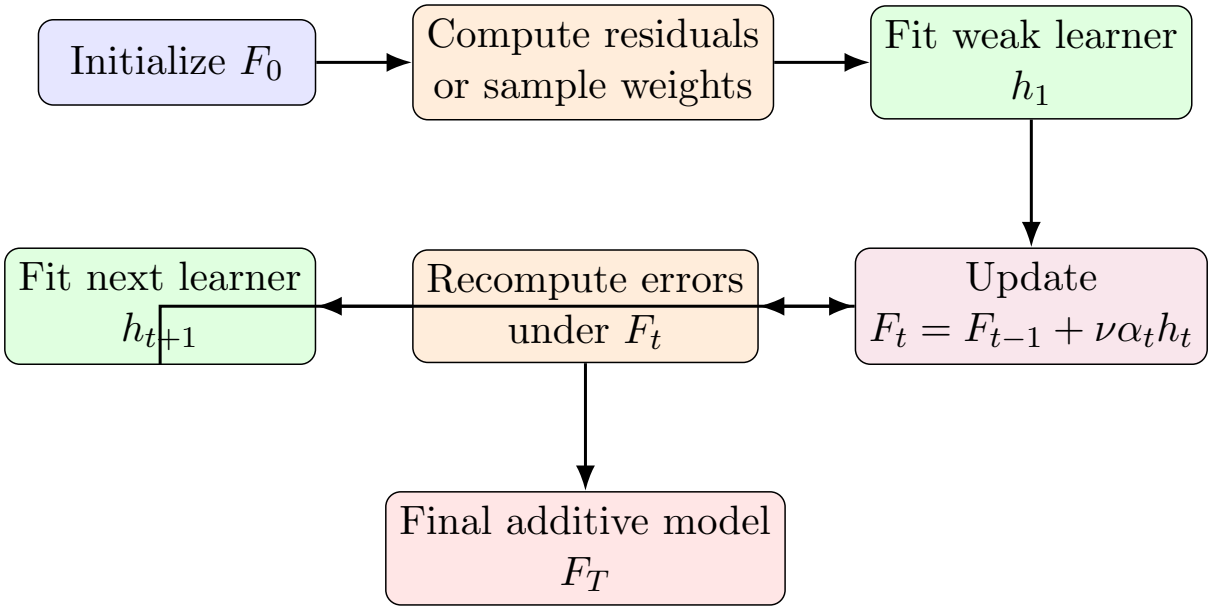


Figure 8.6: Boosting is sequential. Each new weak learner is trained using information from the current ensemble error.

8.10.3 Numerical Example: Residual Correction

Consider three observations:

$$(x_1, y_1) = (1, 2), \qquad (x_2, y_2) = (2, 3), \qquad (x_3, y_3) = (3, 5).$$

Initialize with the target mean:

$$F_0(x) = \bar{y} = \frac{2+3+5}{3} = \frac{10}{3}.$$

The first residuals are

$$r_{i1} = y_i - F_0(x_i),$$

so

$$r_{11} = -\frac{4}{3}, \qquad r_{21} = -\frac{1}{3}, \qquad r_{31} = \frac{5}{3}.$$

Suppose a weak learner predicts

$$h_1(x) = \begin{cases} -\frac{5}{6}, & x < 2.5, \\ \frac{5}{3}, & x \geq 2.5. \end{cases}$$

With learning rate $\nu = 0.5$ and $\alpha_1 = 1$,

$$F_1(x) = F_0(x) + 0.5 h_1(x).$$

The predictions become

$$F_1(1) = F_1(2) = \frac{35}{12} \approx 2.917, \qquad F_1(3) = \frac{25}{6} \approx 4.167.$$

The new residuals are smaller in aggregate than the initial residuals. Additional learners continue correcting the remaining structure.

8.10.4 AdaBoost View for Classification

AdaBoost maintains observation weights $w_i^{(t)}$. After fitting classifier h_t, its weighted error is

$$\epsilon_t = \frac{\sum_{i=1}^{N} w_i^{(t)} \mathbb{I}[h_t(x_i) \neq y_i]}{\sum_{i=1}^{N} w_i^{(t)}}.$$

The learner weight is

$$\alpha_t = \frac{1}{2} \log\left(\frac{1-\epsilon_t}{\epsilon_t}\right),$$

and misclassified observations receive increased weight. The final binary classifier is

$$H(x) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right).$$

8.11 Comparison of Bagging and Boosting

Table 8.8: Comparison of bagging and boosting.

Property	Bagging	Boosting
Training organization	Members are trained independently and can be parallelized	Members are trained sequentially
Data treatment	Bootstrap samples with replacement	Reweighted observations or pseudo-residual targets
Primary objective	Reduce variance	Reduce bias, while regularization controls variance
Member dependence	Low; models are conditionally independent given sampled data	High; each stage depends on previous stages
Typical base learner	Deep or unpruned trees can be effective	Shallow trees or weak learners are common
Combination rule	Usually equal averaging or majority vote	Weighted additive combination
Sensitivity to noise	Usually comparatively robust	Can focus excessively on noisy or mislabeled observations
Parallel computation	Natural	Limited across boosting stages
Representative method	Random forest	AdaBoost, gradient boosting, XGBoost
Stopping control	Number of members mainly controls variance stabilization	Number of stages, learning rate, and learner depth control fit

8.11.1 Practical Selection Guide

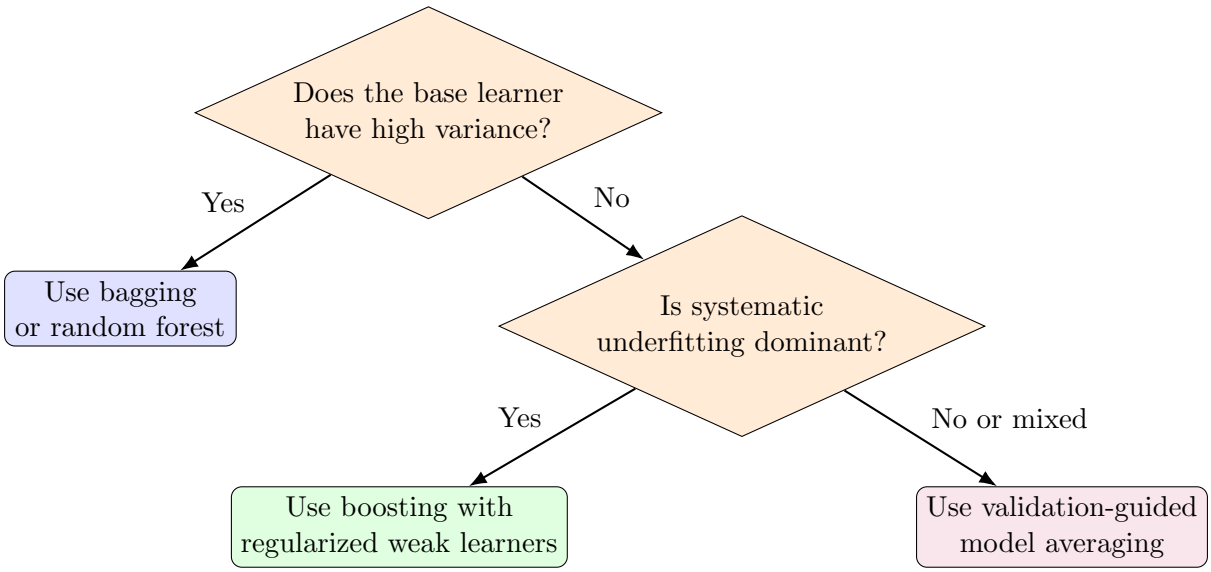


Figure 8.7: Simplified selection guide for bagging, boosting, and general model averaging.

8.12 Unified Perspective

The methods in this chapter control generalization through different mechanisms:

- **Model averaging** combines predictions and reduces variance when member errors are not perfectly correlated.
- **Bagging** creates diversity through bootstrap resampling and is particularly effective for unstable learners.
- **Boosting** constructs an additive model sequentially and reduces systematic residual error.
- **Hard parameter sharing** reduces the feasible parameter space through exact equality constraints.
- **Soft parameter sharing** encourages related parameters to remain close while allowing specialization.
- **Interpolation analysis** explains why model behavior can become unstable near exact fitting and why overparameterized models may generalize again beyond that threshold.

A successful ensemble therefore requires more than a large number of models. Its members should be individually useful, their errors should be sufficiently diverse, and the combination rule should match the prediction task.

Chapter Summary

Table 8.9: Summary of the main concepts.

Concept	Main mechanism	Main effect
Model averaging	Average or vote across predictors	Reduces variance when errors are diverse
Committee error	Depends on member error and error correlation	More members help only when dependence is controlled
Bias–variance trade-off	Decomposes expected squared error	Separates underfitting, instability, and noise
Hard sharing	Reuses exactly the same parameters	Strong structural regularization
Soft sharing	Penalizes disagreement between parameter sets	Tunable information transfer
Interpolation threshold	Marks the onset of exact training fit	Often associated with peak instability
Bagging	Parallel bootstrap training	Primarily reduces variance
Boosting	Sequential residual or weight correction	Primarily reduces bias

Chapter 9

Convolutional Neural Networks

9.1 Motivation

Convolutional Neural Networks, commonly called CNNs, are neural network architectures designed mainly for grid-structured data such as images. Unlike fully connected neural networks, CNNs use local connections, shared weights, and spatially organized feature extraction. These properties make CNNs especially useful for computer vision tasks such as image classification, object detection, semantic segmentation, instance segmentation, denoising, and defect detection.

In a standard fully connected neural network, every input pixel may be connected to every neuron in the next layer. For an image, this quickly creates a very large number of parameters. CNNs reduce this complexity by applying small learnable filters across the image. Each filter detects a local pattern, such as an edge, corner, texture, or shape.

The main idea of a CNN is that important visual patterns can appear at different spatial locations. Therefore, instead of learning a separate detector for every image location, CNNs reuse the same filter across the entire image.

9.2 Images as Input Data

A grayscale image can be represented as a two-dimensional matrix,

$$X \in \mathbb{R}^{H \times W},$$

where H is the image height and W is the image width. A color image usually has three channels: red, green, and blue. Therefore, an RGB image can be written as

$$X \in \mathbb{R}^{H \times W \times C},$$

where $C = 3$ for RGB images.

For a batch of images, the input tensor is often written as

$$X \in \mathbb{R}^{B \times C \times H \times W},$$

where B is the batch size, C is the number of channels, H is the height, and W is the width. This is the common PyTorch format.

9.3 Convolution Operation

The core operation in a CNN is convolution. A convolution layer applies a learnable filter, also called a kernel, to local regions of the input image.

For a two-dimensional input X and a kernel K, the convolution output at location (i, j) can be written as

$$Y(i,j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} K(m,n)X(i+m, j+n),$$

where k_h and k_w are the height and width of the kernel.

In deep learning, this operation is often implemented as cross-correlation rather than strict mathematical convolution because the kernel is not flipped. However, it is still commonly called convolution.

9.4 Convolution vs. Cross-Correlation in 2D

In classical signal processing, convolution and cross-correlation are different operations. The main difference is whether the kernel is flipped before applying it to the input.

In deep learning libraries such as PyTorch and TensorFlow, the operation commonly called "convolution" is usually cross-correlation. The kernel is not flipped. However, the name convolution is still used because the kernel values are learned during training, so the distinction usually does not affect model learning.

9.4.1 Mathematical Difference

Let the input image be

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix},$$

and let the kernel be

$$K = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

For cross-correlation, the kernel is applied directly:

$$Y(i,j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} K(m,n)X(i+m, j+n).$$

For true convolution, the kernel is flipped both vertically and horizontally before it is applied:

$$K_{\text{flipped}} = \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}.$$

Then the convolution output is

$$Y(i,j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} K_{\text{flipped}}(m,n)X(i+m, j+n).$$

9.4.2 Numerical Example

The following table compares cross-correlation and true convolution for the same input and kernel.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad K = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

Table 9.1: Comparison between 2D cross-correlation and true 2D convolution.

Item	**Cross-Correlation**	**True Convolution**
Kernel used	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	$\begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}$
Kernel flipping	No flipping. The kernel is applied as it is.	The kernel is flipped vertically and horizontally before applying it.
Top-left output value	$(1)(1) + (2)(2) + (3)(4) + (4)(5)$ $= 1 + 4 + 12 + 20 = 37$	$(4)(1) + (3)(2) + (2)(4) + (1)(5)$ $= 4 + 6 + 8 + 5 = 23$
Top-right output value	$(1)(2) + (2)(3) + (3)(5) + (4)(6)$ $= 2 + 6 + 15 + 24 = 47$	$(4)(2) + (3)(3) + (2)(5) + (1)(6)$ $= 8 + 9 + 10 + 6 = 33$
Bottom-left output value	$(1)(4) + (2)(5) + (3)(7) + (4)(8)$ $= 4 + 10 + 21 + 32 = 67$	$(4)(4) + (3)(5) + (2)(7) + (1)(8)$ $= 16 + 15 + 14 + 8 = 53$
Bottom-right output value	$(1)(5) + (2)(6) + (3)(8) + (4)(9)$ $= 5 + 12 + 24 + 36 = 77$	$(4)(5) + (3)(6) + (2)(8) + (1)(9)$ $= 20 + 18 + 16 + 9 = 63$
Final output	$Y_{\text{corr}} = \begin{bmatrix} 37 & 47 \\ 67 & 77 \end{bmatrix}$	$Y_{\text{conv}} = \begin{bmatrix} 23 & 33 \\ 53 & 63 \end{bmatrix}$

9.4.3 Why Deep Learning Still Calls It Convolution

Although deep learning libraries usually compute cross-correlation, the operation is still called convolution in CNNs.

The reason is that the kernel values are learned from data. If the model needs a flipped version of a pattern, it can learn that flipped kernel directly during training. Therefore, for learned filters, the practical difference between convolution and cross-correlation is usually not

important.

For example, suppose a model learns the kernel

$$K = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

Using cross-correlation with K gives the same type of learnable operation as using true convolution with the flipped kernel

$$K_{\text{flipped}} = \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}.$$

Since the kernel weights are trainable, the network can learn whichever orientation is useful for reducing the loss.

9.4.4 Summary

Table 9.2: Summary of 2D convolution and 2D cross-correlation.

Property	**Cross-Correlation**	**True Convolution**
Kernel orientation	Kernel is used directly.	Kernel is flipped before use.
Operation	Measures similarity between the kernel and local input region.	Applies a flipped kernel to the local input region.
Formula	$Y(i,j) = \sum_m \sum_n K(m,n) \times X(i+m, j+n)$	$Y(i,j) = \sum_m \sum_n K(k_h - 1 - m, k_w - 1 - n) \times X(i+m, j+n)$
Used in CNN libraries	Yes. This is what many libraries implement.	Usually not directly used in CNN layers.
Common name in deep learning	Still called convolution.	Mathematical convolution.
Practical effect in CNNs	No major issue because kernels are learned.	Equivalent behavior can be learned by adjusting kernel weights.

9.5 Convolution with Multiple Input and Output Channels

In the simplest case, a convolution operation can be explained using one input image and one kernel. However, real CNN layers usually work with multiple input channels and multiple output channels.

For example, an RGB image has three input channels:

$$X \in \mathbb{R}^{3 \times H \times W}.$$

A convolution layer may transform this input into 64 output channels:

$$Y \in \mathbb{R}^{64 \times H_{\text{out}} \times W_{\text{out}}}.$$

This section explains how convolution works when the input has multiple channels, when the output has multiple channels, and how a CNN changes the number of channels.

9.5.1 Convolution Operation with Multiple Input Channels

Suppose the input tensor has C_{in} channels:

$$X \in \mathbb{R}^{C_{\text{in}} \times H \times W}.$$

To produce one output feature map, the convolution layer uses one filter with the same depth as the input:

$$K \in \mathbb{R}^{C_{\text{in}} \times K_h \times K_w}.$$

This means one filter is not just a two-dimensional matrix. It is a three-dimensional tensor. It has one small $K_h \times K_w$ kernel for each input channel.

The output at spatial location (i, j) is computed by summing over all input channels and all spatial kernel locations:

$$Y(i,j) = \sum_{c=1}^{C_{\text{in}}} \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} K(c,m,n) X(c, i+m, j+n).$$

Therefore, one filter combines information from all input channels and produces one output feature map.

Example: RGB Input and One Output Feature Map

Suppose the input is an RGB image:

$$X \in \mathbb{R}^{3 \times 32 \times 32}.$$

Let the convolution layer use one 3×3 filter. Since the input has 3 channels, the full filter shape is

$$K \in \mathbb{R}^{3 \times 3 \times 3}.$$

This filter contains:

$$3 \times 3 \times 3 = 27$$

learnable weights, not counting bias.

The filter has three parts:

$$K = [K_R,\ K_G,\ K_B],$$

where

$$K_R \in \mathbb{R}^{3 \times 3}, \qquad K_G \in \mathbb{R}^{3 \times 3}, \qquad K_B \in \mathbb{R}^{3 \times 3}.$$

The filter is applied to the red, green, and blue channels separately. Then the three results are added together to produce one output feature map.

If stride $S = 1$ and padding $P = 1$, then the spatial size remains 32×32. Therefore, one filter produces

$$Y \in \mathbb{R}^{1 \times 32 \times 32}.$$

9.5.2 Convolution Operation with Multiple Output Channels

A convolution layer usually produces more than one output feature map. To produce multiple output channels, the layer uses multiple filters.

If the layer has C_{out} filters, then it produces C_{out} output feature maps.

The full convolution weight tensor has shape

$$K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w}.$$

Each output channel has its own filter:

$$K_o \in \mathbb{R}^{C_{\text{in}} \times K_h \times K_w}, \qquad o = 1, 2, \ldots, C_{\text{out}}.$$

The o-th output feature map is computed as

$$Y_o(i,j) = \sum_{c=1}^{C_{\text{in}}} \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} K(o,c,m,n) X(c, i+m, j+n).$$

Therefore, each output channel is produced by one separate filter, and each filter looks at all input channels.

Example: RGB Input and 64 Output Channels

Suppose the input is

$$X \in \mathbb{R}^{3 \times 32 \times 32}.$$

Now suppose the convolution layer uses

$$C_{\text{out}} = 64$$

filters, each with kernel size 3×3.

The weight tensor has shape

$$K \in \mathbb{R}^{64 \times 3 \times 3 \times 3}.$$

Each of the 64 filters looks at all 3 input channels and produces one output feature map.

If stride $S = 1$ and padding $P = 1$, then the output is

$$Y \in \mathbb{R}^{64 \times 32 \times 32}.$$

So the layer changes the number of channels from 3 to 64.

9.5.3 Channel Mixing

Channel mixing means that each output channel is computed using all input channels.

For example, in an RGB image, one output feature map is not computed from only the red channel, or only the green channel, or only the blue channel. Instead, the convolution filter combines information from all three channels.

For one output channel, the computation can be written as

$$Y_o = K_{o,1} * X_1 + K_{o,2} * X_2 + \cdots + K_{o,C_{\text{in}}} * X_{C_{\text{in}}},$$

where X_c is the c-th input channel, $K_{o,c}$ is the kernel slice connecting input channel c to output channel o, and $*$ denotes the convolution operation used in CNNs.

This is why convolution can learn relationships across channels. For example, a filter may learn to respond to a pattern involving red and green contrast, or a combination of color and texture.

In deeper CNN layers, the channels no longer represent RGB values. They represent learned feature maps. Channel mixing then means that the next layer combines lower-level learned features into more complex features.

9.5.4 Changing the Number of Channels

A convolution layer changes the number of channels by changing the number of filters.

The rule is simple:

$$\text{number of output channels} = \text{number of filters}.$$

The number of input channels is determined by the previous layer. The number of output channels is chosen by the model designer.

For example:

$$3 \to 64$$

means the layer takes an RGB image with 3 input channels and produces 64 output feature maps.

Similarly,

$$64 \to 128$$

means the layer takes 64 input feature maps and produces 128 output feature maps.

The convolution weight tensor for a layer with C_{in} input channels and C_{out} output channels is

$$C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w.$$

The number of learnable weights is

$$C_{\text{out}} C_{\text{in}} K_h K_w.$$

If bias is used, the number of parameters is

$$C_{\text{out}} C_{\text{in}} K_h K_w + C_{\text{out}}.$$

9.5.5 Examples of Channel Transformation

9.5.6 Understanding the Table

The first example takes a grayscale image with one input channel and produces 16 output feature maps. Since there are 16 filters, the output has 16 channels.

The second example takes an RGB image with 3 channels and produces 64 output feature maps. Each of the 64 filters has depth 3, because each filter must look across all RGB channels.

The third example takes 64 learned feature maps from a previous layer and produces 128 new feature maps. This is common in CNNs, where deeper layers usually increase the number of channels.

The fourth example increases the number of channels from 128 to 256, while also reducing the spatial size from 32×32 to 16×16 using stride $S = 2$.

The fifth example uses a 1×1 convolution. A 1×1 convolution does not mix neighboring spatial pixels, but it does mix information across channels. It is often used to reduce or increase the number of channels efficiently.

Table 9.3: Examples of convolution layers with multiple input and output channels.

Input Tensor	**Kernel Size**	**Number of Filters**	**Weight Tensor Shape**	**Output Tensor**
$1 \times 28 \times 28$	3×3	16	$16 \times 1 \times 3 \times 3$	$16 \times 28 \times 28$ with $S = 1, P = 1$
$3 \times 32 \times 32$	3×3	64	$64 \times 3 \times 3 \times 3$	$64 \times 32 \times 32$ with $S = 1, P = 1$
$64 \times 32 \times 32$	3×3	128	$128 \times 64 \times 3 \times 3$	$128 \times 32 \times 32$ with $S = 1, P = 1$
$128 \times 32 \times 32$	3×3	256	$256 \times 128 \times 3 \times 3$	$256 \times 16 \times 16$ with $S = 2, P = 1$
$256 \times 16 \times 16$	1×1	64	$64 \times 256 \times 1 \times 1$	$64 \times 16 \times 16$ with $S = 1, P = 0$

9.5.7 Example: How a 1×1 Convolution Changes Channels

A 1×1 convolution is useful for changing the number of channels without changing the height and width of the feature map.

Unlike a 3×3 convolution, a 1×1 convolution does not combine neighboring spatial pixels. Instead, it combines channel values at the same spatial location.

Suppose the input is

$$X \in \mathbb{R}^{C_{\text{in}} \times H \times W}.$$

At one spatial location (i, j), the input vector is

$$X(:, i, j) = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{C_{\text{in}}} \end{bmatrix}.$$

For one output channel, a 1×1 convolution computes

$$Y_o(i, j) = w_{o,1}x_1 + w_{o,2}x_2 + \cdots + w_{o,C_{\text{in}}}x_{C_{\text{in}}} + b_o.$$

Therefore, a 1×1 convolution performs a learned weighted combination of the input channels at each pixel location. It is not simply an average of the channels.

It becomes an average only in the special case where

$$w_{o,1} = w_{o,2} = \cdots = w_{o,C_{\text{in}}} = \frac{1}{C_{\text{in}}}, \qquad b_o = 0.$$

In normal CNN training, these weights are learned from data, so each output channel can learn a different combination of the input channels.

For example, for an RGB input at one pixel location,

$$X(:, i, j) = \begin{bmatrix} R \\ G \\ B \end{bmatrix},$$

one 1×1 filter may learn

$$Y_1(i, j) = 0.2R + 0.5G - 0.1B + b_1.$$

Another 1×1 filter may learn

$$Y_2(i, j) = -0.3R + 0.1G + 0.8B + b_2.$$

Thus, different 1×1 filters produce different output channels by learning different channel combinations.

For a full feature tensor example, suppose

$$X \in \mathbb{R}^{256 \times 16 \times 16}.$$

A 1×1 convolution with 64 filters has weight tensor

$$K \in \mathbb{R}^{64 \times 256 \times 1 \times 1}.$$

The output is

$$Y \in \mathbb{R}^{64\times16\times16}.$$

So the layer changes the channel dimension from 256 to 64, while keeping the spatial size fixed:

$$256 \times 16 \times 16 \to 64 \times 16 \times 16.$$

The key idea is

$$1 \times 1 \text{ convolution mixes channels, not neighboring pixels.}$$

9.5.8 Summary

For a convolution layer, the input has shape

$$X \in \mathbb{R}^{C_{\text{in}}\times H\times W}.$$

The convolution weight tensor has shape

$$K \in \mathbb{R}^{C_{\text{out}}\times C_{\text{in}}\times K_h\times K_w}.$$

The output has shape

$$Y \in \mathbb{R}^{C_{\text{out}}\times H_{\text{out}}\times W_{\text{out}}}.$$

The number of output channels is controlled by the number of filters:

$$C_{\text{out}} = \text{number of filters.}$$

Each output channel is produced by one filter, and each filter combines information from all input channels.

9.6 Kernel, Feature Map, and Channel

A convolutional layer uses learnable kernels to extract local patterns from an input image or feature tensor. A kernel is a small matrix of trainable weights. During training, the values inside the kernel are updated by backpropagation.

For a grayscale input image,

$$X \in \mathbb{R}^{H\times W},$$

a single kernel

$$K \in \mathbb{R}^{k_h\times k_w}$$

slides over the image and produces one output feature map.

A feature map is the result of applying a kernel to all valid local regions of the input. It shows where a particular learned pattern appears in the input.

For example, one kernel may respond strongly to vertical edges, another kernel may respond to horizontal edges, and another may respond to texture-like patterns.

If a convolution layer has C_{out} kernels, then it produces C_{out} feature maps. Therefore, the number of output channels is equal to the number of kernels.

For an input tensor

$$X \in \mathbb{R}^{C_{\text{in}}\times H\times W},$$

a convolution layer with C_{out} kernels produces

$$Y \in \mathbb{R}^{C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}}.$$

Each output channel is one feature map.

9.6.1 Feature Map Example

Suppose the input is a grayscale image of size

$$5 \times 5,$$

and we apply one kernel of size

$$3 \times 3$$

with stride $S = 1$ and no padding $P = 0$. The output height and width are

$$H_{\text{out}} = \left\lfloor \frac{5 + 2(0) - 3}{1} \right\rfloor + 1 = 3,$$

$$W_{\text{out}} = \left\lfloor \frac{5 + 2(0) - 3}{1} \right\rfloor + 1 = 3.$$

Therefore, one 3×3 kernel produces one feature map of size

$$3 \times 3.$$

If the layer uses 8 different kernels, then the output has 8 feature maps:

$$Y \in \mathbb{R}^{8 \times 3 \times 3}.$$

This means the spatial size is 3×3, and the channel dimension is 8.

9.6.2 Example with Multiple Input Channels

For an RGB image, the input has three channels:

$$X \in \mathbb{R}^{3 \times H \times W}.$$

In this case, each convolutional kernel must also have depth 3. Therefore, one kernel has shape

$$K \in \mathbb{R}^{3 \times k_h \times k_w}.$$

This one kernel looks across all input channels and produces one output feature map.
For example, suppose

$$X \in \mathbb{R}^{3 \times 32 \times 32},$$

and the convolution layer uses

$$C_{\text{out}} = 64$$

kernels, each with size

$$3 \times 3.$$

Each kernel has shape

$$3 \times 3 \times 3.$$

The first 3 corresponds to the RGB input channels, and the last two dimensions correspond to the spatial kernel size.

If stride $S = 1$ and padding $P = 1$, then the output spatial size remains

$$32 \times 32.$$

Therefore, the output tensor is

$$Y \in \mathbb{R}^{64 \times 32 \times 32}.$$

This means the convolution layer converts an image with 3 channels into a feature tensor with 64 channels.

9.6.3 How Kernel Values Are Learned

The values inside the kernels are not manually selected in modern CNNs. They are learned from data.

At the beginning of training, the kernel weights are usually initialized randomly or with a standard initialization method. During forward propagation, the kernels produce feature maps. The network output is compared with the target label using a loss function.

During backpropagation, the gradient of the loss with respect to each kernel weight is computed:

$$\frac{\partial \mathcal{L}}{\partial K}.$$

Then the kernel weights are updated using an optimizer such as gradient descent:

$$K^{(t+1)} = K^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial K^{(t)}},$$

where η is the learning rate.

Therefore, the CNN automatically learns kernel values that reduce the loss. Early layers often learn low-level patterns such as edges and textures, while deeper layers learn more abstract patterns.

9.7 Stride, Padding, and Output Size

The spatial size of the output feature map depends on the input size, kernel size, stride, and padding.

For a two-dimensional convolution, the output height and width are

$$H_{\text{out}} = \left\lfloor \frac{H + 2P_h - K_h}{S_h} \right\rfloor + 1,$$

$$W_{\text{out}} = \left\lfloor \frac{W + 2P_w - K_w}{S_w} \right\rfloor + 1,$$

where

$$H, W$$

are the input height and width,

$$K_h, K_w$$

are the kernel height and width,

$$P_h, P_w$$

are the padding sizes, and

$$S_h, S_w$$

are the stride values.

9.7.1 Stride

Stride controls how far the kernel moves at each step. A stride of 1 moves the kernel one pixel at a time. A stride of 2 skips every other position and reduces the spatial size of the output.

9.7.2 Padding

Padding adds extra pixels around the input image boundary. It is often used to preserve spatial size after convolution.

For example, if

$$K = 3, \qquad S = 1, \qquad P = 1,$$

then the output height and width remain the same as the input height and width.

9.8 Example: Convolution Output Size

Suppose the input image has size

$$32 \times 32,$$

the kernel size is

$$3 \times 3,$$

the stride is

$$S = 1,$$

and the padding is

$$P = 1.$$

Then

$$H_{\text{out}} = \left\lfloor \frac{32 + 2(1) - 3}{1} \right\rfloor + 1 = 32.$$

Similarly,

$$W_{\text{out}} = 32.$$

Therefore, the output feature map size is

$$32 \times 32.$$

If the convolution layer has 64 filters, then the output tensor has shape

$$64 \times 32 \times 32.$$

9.9 Dilated Convolution

Dilated convolution, also called atrous convolution, is a convolution operation that inserts gaps between kernel elements. It allows the network to increase the receptive field without increasing the number of kernel parameters.

In a standard convolution, neighboring kernel elements are applied to neighboring pixels. In a dilated convolution, the kernel elements are spaced apart by a dilation rate.

For a one-dimensional illustration, a standard 3-element kernel uses adjacent positions:

$$[x_{i-1}, x_i, x_{i+1}].$$

With dilation rate $d = 2$, the same kernel uses spaced positions:

$$[x_{i-2}, x_i, x_{i+2}].$$

Therefore, dilation increases the area covered by the kernel while keeping the number of learnable weights unchanged.

9.9.1 Effective Kernel Size

For a kernel of size K and dilation rate d, the effective kernel size is

$$K_{\text{eff}} = K + (K-1)(d-1).$$

For example, if

$$K = 3, \qquad d = 1,$$

then

$$K_{\text{eff}} = 3 + (3-1)(1-1) = 3.$$

So a 3×3 kernel with dilation 1 behaves like a normal 3×3 convolution.

If

$$K = 3, \qquad d = 2,$$

then

$$K_{\text{eff}} = 3 + (3-1)(2-1) = 5.$$

So a 3×3 kernel with dilation 2 covers a 5×5 region, but it still has only $3 \times 3 = 9$ learnable weights.

If

$$K = 3, \qquad d = 3,$$

then

$$K_{\text{eff}} = 3 + (3-1)(3-1) = 7.$$

So a 3×3 kernel with dilation 3 covers a 7×7 region, again using only 9 learnable weights.

9.9.2 2D Dilated Convolution Example

Consider a 3×3 kernel.

With dilation $d = 1$, the kernel samples adjacent pixels:

$$\begin{bmatrix} \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet \end{bmatrix}$$

The effective kernel size is

$$3 \times 3.$$

With dilation $d = 2$, one gap is inserted between neighboring kernel elements:

$$\begin{bmatrix} \bullet & 0 & \bullet & 0 & \bullet \\ 0 & 0 & 0 & 0 & 0 \\ \bullet & 0 & \bullet & 0 & \bullet \\ 0 & 0 & 0 & 0 & 0 \\ \bullet & 0 & \bullet & 0 & \bullet \end{bmatrix}$$

$$\begin{bmatrix} \bullet & \cdot & \bullet & \cdot & \bullet \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \bullet & \cdot & \bullet & \cdot & \bullet \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \bullet & \cdot & \bullet & \cdot & \bullet \end{bmatrix}$$

The effective kernel size becomes

$$5 \times 5.$$

However, only the $\bullet$ positions contain learnable kernel weights. The empty positions represent skipped input locations. They are not learnable weights and are not additional kernel parameters.

With dilation $d = 3$, the same 3×3 kernel covers a

$$7 \times 7$$

region of the input.

9.9.3 Output Size with Dilation

When dilation is used, the output size formula becomes

$$H_{\text{out}} = \left\lfloor \frac{H + 2P_h - d_h(K_h - 1) - 1}{S_h} \right\rfloor + 1,$$

$$W_{\text{out}} = \left\lfloor \frac{W + 2P_w - d_w(K_w - 1) - 1}{S_w} \right\rfloor + 1.$$

Here,

$$d_h, d_w$$

are the dilation rates along the height and width directions.
When

$$d_h = d_w = 1,$$

the formula reduces to the standard convolution output size formula.

9.9.4 Example: Output Size with Dilation

Suppose the input size is

$$32 \times 32,$$

the kernel size is

$$3 \times 3,$$

the stride is

$$S = 1,$$

the padding is

$$P = 2,$$

and the dilation rate is

$$d = 2.$$

The output height is

$$H_{\text{out}} = \left\lfloor \frac{32 + 2(2) - 2(3-1) - 1}{1} \right\rfloor + 1.$$

Therefore,

$$H_{\text{out}} = \lfloor 32 + 4 - 4 - 1 \rfloor + 1 = 32.$$

Similarly,

$$W_{\text{out}} = 32.$$

So with $K = 3$, $d = 2$, $S = 1$, and $P = 2$, the output spatial size remains

$$32 \times 32.$$

9.9.5 Why Dilation Is Useful

Dilation is useful because it increases the receptive field without requiring more parameters or aggressive downsampling.

This is especially important in dense prediction tasks such as semantic segmentation. In segmentation, the network needs both local detail and broader context. If the feature maps are downsampled too much, spatial information may be lost. Dilated convolution helps the network capture larger context while preserving feature map resolution.

For example, DeepLab-style segmentation models use dilated convolution to enlarge the receptive field while maintaining dense spatial predictions.

9.9.6 Summary

A standard convolution with a 3×3 kernel and dilation 1 sees a local 3×3 region.

A dilated convolution with a 3×3 kernel and dilation 2 sees an effective 5×5 region.

A dilated convolution with a 3×3 kernel and dilation 3 sees an effective 7×7 region.

Therefore, dilation increases the receptive field without increasing the number of learnable kernel weights.

9.10 Receptive Field

The receptive field of a neuron in a CNN is the region of the original input image that influences that neuron.

In a fully connected layer, each output neuron may depend on all input values. In a convolution layer, each output value depends only on a local region of the input. This local region is called the receptive field.

For example, if a convolution layer uses a 3×3 kernel, then each output value depends on a 3×3 region of the input. Therefore, the receptive field of one output value in the first convolution layer is

$$3 \times 3.$$

As we go deeper into the CNN, the receptive field becomes larger. This is one reason CNNs are called hierarchical models. Early layers see small local patterns, while deeper layers combine these patterns over larger regions of the image.

9.10.1 Simple Receptive Field Example

Suppose an input image passes through two convolution layers. Both layers use

$$3 \times 3$$

kernels, stride $S = 1$, and no pooling.

The first convolution layer has receptive field

$$3 \times 3.$$

Each output value in the second convolution layer depends on a 3×3 region of the first feature map. However, each value in the first feature map already depends on a 3×3 region of the original input.

Therefore, the receptive field of the second convolution layer becomes

$$5 \times 5.$$

So with two 3×3 convolution layers, one output value in the second layer sees a 5×5 region of the original input.

With three 3×3 convolution layers, the receptive field becomes

$$7 \times 7.$$

In general, for L convolution layers with kernel size 3×3, stride 1, and no dilation, the receptive field size is

$$R_L = 1 + 2L.$$

For example,

$$L = 1 \Rightarrow R_1 = 3,$$

$$L = 2 \Rightarrow R_2 = 5,$$

$$L = 3 \Rightarrow R_3 = 7.$$

9.10.2 Receptive Field with Stride

Stride increases how quickly the receptive field grows because each output location jumps over input positions.

For a stack of convolution layers, the receptive field can be computed recursively using

$$R_l = R_{l-1} + (K_l - 1)J_{l-1},$$

where R_l is the receptive field at layer l, K_l is the kernel size at layer l, and J_{l-1} is the effective jump between adjacent features from the previous layer.

The jump is updated as

$$J_l = J_{l-1}S_l,$$

where S_l is the stride at layer l.

The initialization is

$$R_0 = 1, \qquad J_0 = 1.$$

Here, $R_0 = 1$ means one input pixel initially sees only itself, and $J_0 = 1$ means adjacent input pixels are one pixel apart.

9.10.3 Example: Receptive Field with Two Layers

Consider the following CNN:

$$\text{Input} \rightarrow \text{Conv}_{3\times3,S=1} \rightarrow \text{Conv}_{3\times3,S=2}.$$

For the first convolution layer,

$$R_1 = R_0 + (K_1 - 1)J_0 = 1 + (3 - 1)(1) = 3.$$

The jump after the first layer is

$$J_1 = J_0S_1 = 1(1) = 1.$$

For the second convolution layer,

$$R_2 = R_1 + (K_2 - 1)J_1 = 3 + (3 - 1)(1) = 5.$$

The jump after the second layer is

$$J_2 = J_1S_2 = 1(2) = 2.$$

Therefore, each output value in the second layer sees a

$$5 \times 5$$

region of the original input, and adjacent output values correspond to positions that are 2 pixels apart in the input.

9.10.4 Example: Receptive Field with Pooling

Pooling also increases the effective receptive field. Suppose we have

$$\text{Input} \to \text{Conv}_{3\times3,S=1} \to \text{MaxPool}_{2\times2,S=2} \to \text{Conv}_{3\times3,S=1}.$$

We initialize

$$R_0 = 1, \qquad J_0 = 1.$$

For the first convolution layer,

$$R_1 = 1 + (3-1)(1) = 3,$$

$$J_1 = 1(1) = 1.$$

For the max pooling layer,

$$R_2 = R_1 + (2-1)J_1 = 3 + (1)(1) = 4,$$

$$J_2 = J_1(2) = 2.$$

For the second convolution layer,

$$R_3 = R_2 + (3-1)J_2 = 4 + (2)(2) = 8.$$

Thus, each output value after the second convolution layer sees an

$$8 \times 8$$

region of the original input.

This shows why deeper CNNs can learn larger and more abstract structures. The first layers detect local patterns, while deeper layers combine information from larger regions.

9.11 Why CNNs Are Hierarchical

CNNs are called hierarchical because each layer builds features from the features learned by previous layers.

The first layers usually learn low-level features such as edges, corners, and simple textures. Middle layers combine these features into more meaningful patterns such as curves, repeated textures, and object parts. Deeper layers combine these object parts into high-level semantic representations.

This hierarchy can be summarized as

$$\text{pixels} \to \text{edges} \to \text{textures} \to \text{parts} \to \text{objects}.$$

The hierarchy is not only because the spatial size may decrease and the number of channels may increase. It is mainly because deeper layers represent increasingly abstract features using larger receptive fields.

For example, in a classification CNN, early layers may detect edges, middle layers may detect shapes or object parts, and deeper layers may detect class-specific structures.

In segmentation models, this hierarchy is also important. Encoder layers learn increasingly abstract features, while decoder layers recover spatial resolution to produce pixel-level predictions.

9.12 Number of Parameters in a Convolution Layer

The number of learnable parameters in a convolution layer is

$$\text{Parameters} = C_{\text{out}} \left(C_{\text{in}} K_h K_w + 1 \right),$$

where the $+1$ corresponds to the bias term for each output channel.

If bias is not used, then the number of parameters is

$$\text{Parameters} = C_{\text{out}} C_{\text{in}} K_h K_w.$$

9.12.1 Example

Suppose a convolution layer has

$$C_{\text{in}} = 3, \qquad C_{\text{out}} = 64, \qquad K_h = K_w = 3.$$

With bias, the number of parameters is

$$64(3 \times 3 \times 3 + 1) = 64(28) = 1792.$$

Without bias, the number of parameters is

$$64(3 \times 3 \times 3) = 1728.$$

9.13 Activation Function

After convolution, an activation function is usually applied. The most common activation function in CNNs is the Rectified Linear Unit, or ReLU:

$$\text{ReLU}(x) = \max(0, x).$$

ReLU introduces nonlinearity into the network. Without nonlinear activation functions, a deep network would behave like a single linear transformation.

9.14 Pooling Layers

Pooling layers reduce the spatial size of feature maps. This helps reduce computation and makes the model less sensitive to small spatial shifts.

The two most common pooling operations are max pooling and average pooling.

9.14.1 Max Pooling

Max pooling takes the maximum value from a local region:

$$Y(i, j) = \max_{(m,n) \in \Omega} X(i + m, j + n),$$

where Ω is the local pooling window.

9.14.2 Average Pooling

Average pooling takes the average value from a local region:

$$Y(i, j) = \frac{1}{|\Omega|} \sum_{(m,n) \in \Omega} X(i + m, j + n).$$

9.15 Fully Connected Layer in CNN

After several convolution and pooling layers, the extracted feature maps may be flattened and passed into a fully connected layer.

For classification, the final fully connected layer usually outputs a vector of class scores:

$$z \in \mathbb{R}^K,$$

where K is the number of classes.

The class probabilities can then be computed using the softmax function:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

9.16 Basic CNN Architecture

A simple CNN architecture can be written as

Input → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Flatten → Fully Connected

In practice, modern CNNs may include additional components such as batch normalization, dropout, residual connections, skip connections, and multi-scale feature extraction.

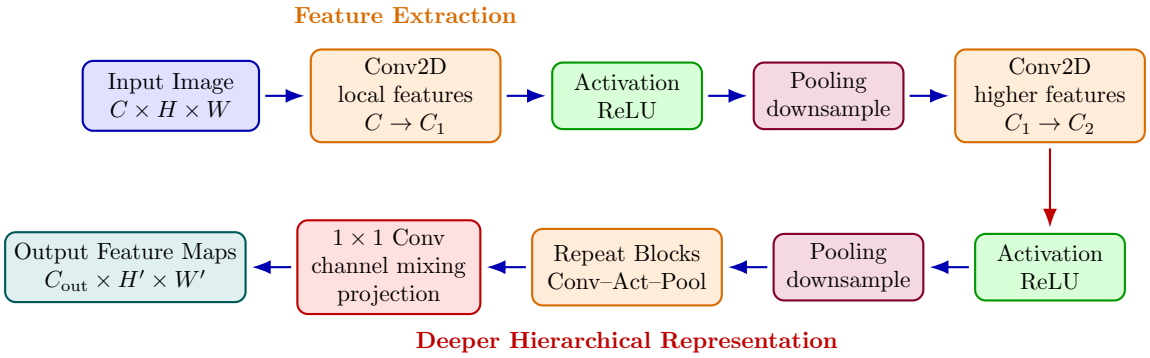


Figure 9.1: Basic CNN flow diagram in PyTorch tensor order. A CNN applies convolution, activation, and pooling repeatedly to learn hierarchical feature maps. A 1×1 convolution can mix channels and change the number of output feature maps without changing the spatial resolution.

9.17 CNN for Image Classification

For image classification, the CNN receives an image as input and predicts one class label. The network learns to extract useful visual features and map them to class probabilities.

For example, if the task is to classify an image into one of K classes, the output layer produces

$$\hat{y} = \text{softmax}(z),$$

where

$$\hat{y} \in \mathbb{R}^K.$$

The predicted class is

$$\hat{k} = \arg\max_k \hat{y}_k.$$

Figure 9.2 illustrates a basic CNN architecture for image classification. The network first extracts spatial features using convolution and max-pooling layers, then converts the final feature maps into a vector by flattening, and finally performs classification using fully connected layers.

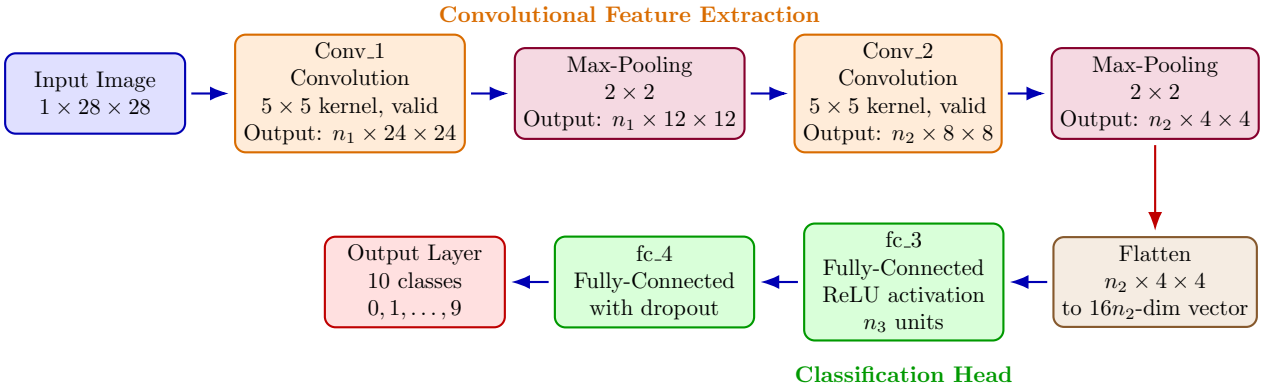


Figure 9.2: Flow diagram of a basic CNN architecture in PyTorch tensor order. The model takes an input tensor of size $1 \times 28 \times 28$, applies two convolutional layers with 5×5 valid kernels and 2×2 max-pooling, then flattens the resulting feature maps and passes them through fully connected layers for classification.

9.18 CNN for Semantic Segmentation

In semantic segmentation, the goal is to assign a class label to every pixel in the image. Instead of producing one label for the whole image, the network produces a dense prediction map.

If the input image has size

$$H \times W,$$

and there are K semantic classes, then the output is often written as

$$Y \in \mathbb{R}^{K \times H \times W}.$$

Each pixel has a class probability vector across the K classes.

CNN-based semantic segmentation models include architectures such as FCN, U-Net, SegNet, DeepLab, and related encoder-decoder networks.

Figure 9.3 shows the basic structure of a fully convolutional network for semantic segmentation. Unlike a classification CNN, an FCN does not flatten the feature maps into a vector. Instead, it keeps the spatial structure, maps the feature channels to class-score maps using a 1×1 convolution, and upsamples the result to obtain a pixel-wise prediction.

In the FCN output,

$$Y \in \mathbb{R}^{H \times W \times K},$$

where K is the number of semantic classes. Each pixel location has K scores, and the final predicted class at each pixel is usually selected by

$$\hat{y}(i, j) = \arg\max_k Y(i, j, k).$$

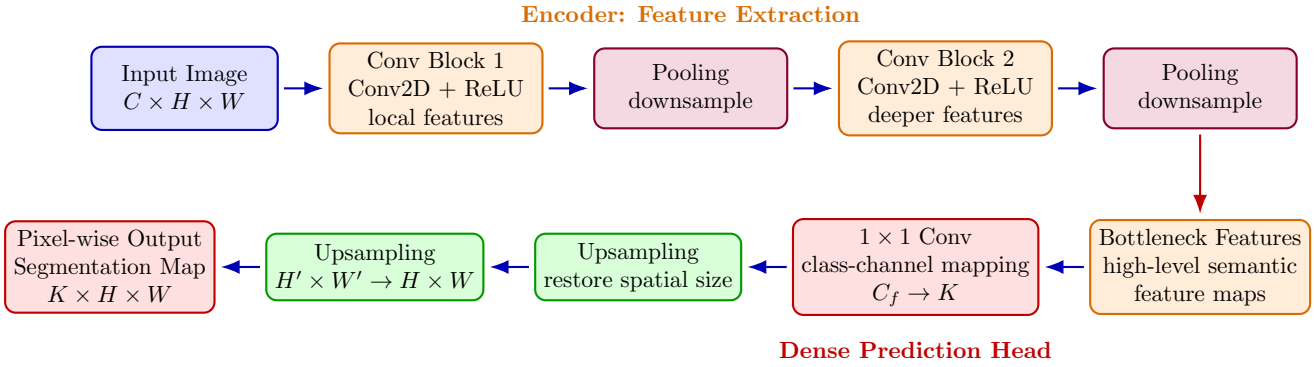


Figure 9.3: Basic FCN flow diagram for semantic segmentation in PyTorch tensor order. The encoder extracts hierarchical feature maps using convolution and pooling. A 1×1 convolution maps the learned feature channels to K class-score channels, and upsampling restores the spatial resolution to produce a pixel-wise segmentation map.

Thus, FCN converts image-level convolutional feature extraction into dense pixel-level classification.

9.19 CNN for Object Detection and Instance Segmentation

CNNs are also widely used for object detection and instance segmentation. In image classification, a model predicts a single class label for an entire image. Object detection extends this task by identifying both *what* objects are present and *where* they are located, typically using class labels and bounding boxes. Instance segmentation extends object detection further by predicting a pixel-level mask for each detected object, allowing different instances of the same semantic class to be distinguished.

A major family of CNN-based object detectors is the Region-based Convolutional Neural Network (R-CNN) family. Its development can be summarized as

$$\text{R-CNN} \to \text{Fast R-CNN} \to \text{Faster R-CNN} \to \text{Mask R-CNN}.$$

Each successive architecture addresses an important limitation of the previous model, particularly redundant convolutional computation, external region proposal generation, and the absence of pixel-level instance masks.

9.19.1 R-CNN

Region-based Convolutional Neural Network (R-CNN) combines region proposals with CNN-based feature extraction for object detection. Instead of evaluating every possible image location, R-CNN first generates a limited number of candidate regions that are likely to contain objects. An external region proposal method, such as Selective Search, is commonly used to generate these candidate regions.

For an input image, each proposed region is cropped or warped to a fixed spatial size and passed independently through a CNN. For a candidate region R_i, the extracted feature representation can be written as

$$\mathbf{f}_i = \text{CNN}(R_i), \tag{9.1}$$

where $\mathbf{f}_i$ denotes the CNN feature vector for the i-th region proposal. The extracted features are then used for object classification, while a separate bounding-box regression stage refines the predicted object location.

The basic R-CNN pipeline can therefore be summarized as

Input Image → Region Proposals → Crop/Warp → CNN Features → Classification → Bounding-Box Regressi

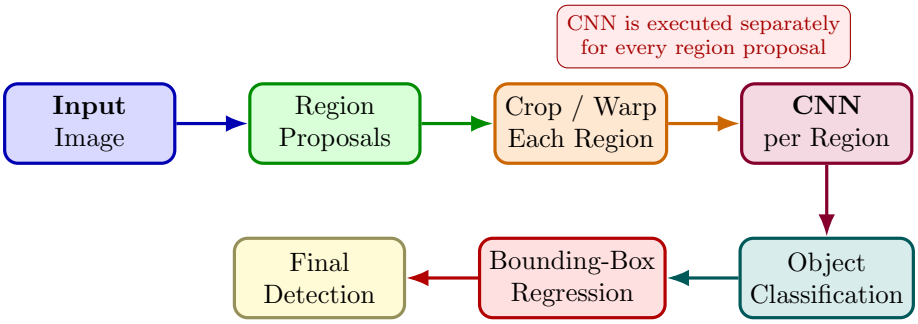


Figure 9.4: Conceptual R-CNN pipeline. Candidate regions are generated first, and each proposed region is independently processed by the CNN before classification and bounding-box refinement.

The main limitation of R-CNN is computational redundancy. If an image contains N candidate regions, the CNN is evaluated approximately N times:

$$\{R_1, R_2, \ldots, R_N\} \rightarrow \{\text{CNN}(R_1), \text{CNN}(R_2), \ldots, \text{CNN}(R_N)\}.$$

Because many proposed regions overlap, the same image content may be processed repeatedly. R-CNN also relies on a multi-stage training pipeline in which CNN fine-tuning, region classification, and bounding-box regression are handled as separate components. These limitations motivated Fast R-CNN.

9.19.2 Fast R-CNN

Fast R-CNN improves R-CNN by sharing convolutional computation across all region proposals. Instead of passing every candidate region independently through the CNN, the complete input image is processed once to generate a shared convolutional feature map:

$$\mathbf{F} = \text{CNN}(I), \tag{9.2}$$

where I denotes the input image and $\mathbf{F}$ is the resulting feature map.

Region proposals are still generated using an external method such as Selective Search. However, each proposed region is now projected onto the shared feature map rather than independently processed from the original image. Region of Interest (RoI) pooling converts each variable-size proposal into a fixed-size feature representation:

$$\mathbf{z}_i = \text{RoIPool}(\mathbf{F}, R_i), \tag{9.3}$$

where $\mathbf{z}_i$ represents the fixed-size feature representation for proposal R_i.

The RoI features are passed through fully connected layers and then divided into two prediction branches: an object classification branch and a bounding-box regression branch. Thus, the Fast R-CNN pipeline can be represented as

$$\text{Image} \rightarrow \text{Shared CNN Feature Map} \rightarrow \text{RoI Pooling} \rightarrow \begin{cases} \text{Classification}, \\ \text{Bounding-Box Regression}. \end{cases}$$

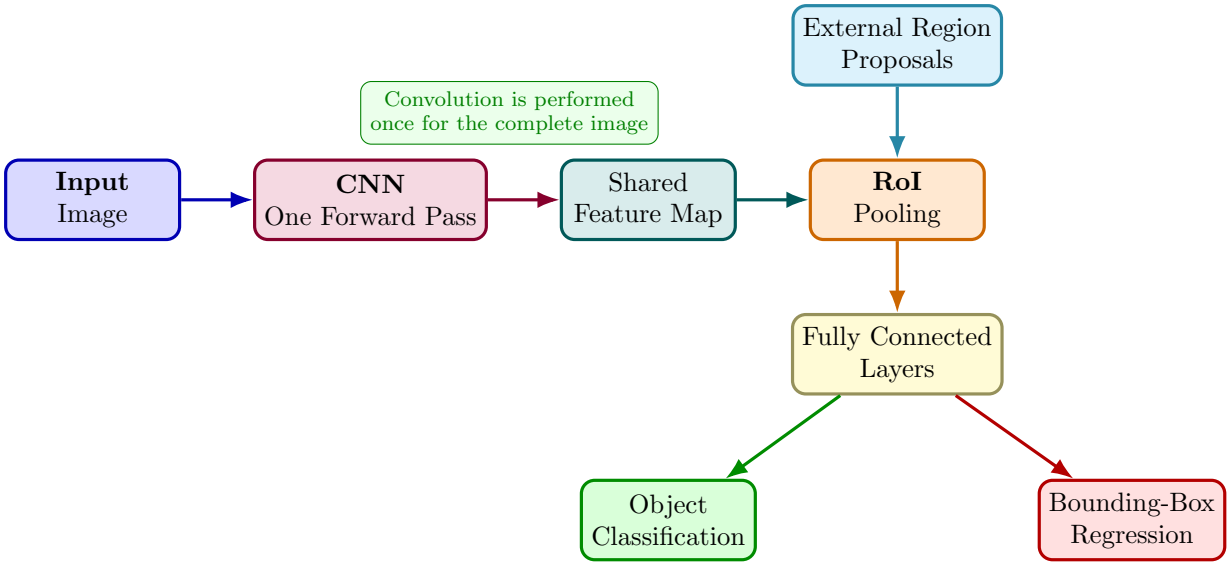


Figure 9.5: Conceptual Fast R-CNN architecture. The complete image is processed by the CNN once to obtain a shared feature map. Region proposals are projected onto this feature map and converted to fixed-size representations through RoI pooling before classification and bounding-box regression.

The major advantage of Fast R-CNN is the reuse of convolutional features. R-CNN requires approximately N CNN forward passes for N proposals, whereas Fast R-CNN requires only one CNN forward pass for the complete image, followed by comparatively inexpensive RoI-specific operations.

Fast R-CNN also jointly optimizes classification and localization using a multi-task objective that can be written in general form as

$$L = L_{\text{cls}} + \lambda L_{\text{loc}}, \tag{9.4}$$

where L_{cls} represents the classification loss, L_{loc} represents the localization loss, and λ controls their relative contribution.

Although Fast R-CNN substantially reduces redundant CNN computation, it still depends on an external region proposal algorithm. Region proposal generation therefore remains separate from the neural network and can become a computational bottleneck. This limitation led to Faster R-CNN.

RoI Pooling and Quantization

Region of Interest (RoI) pooling is used in Fast R-CNN to convert variable-size region proposals into fixed-size feature representations. This is necessary because the subsequent fully connected layers require inputs with a consistent spatial dimension.

Suppose an RoI projected onto the convolutional feature map is represented by

$$R = (x_1, y_1, x_2, y_2), \tag{9.5}$$

where (x_1, y_1) and (x_2, y_2) denote the upper-left and lower-right coordinates of the region, respectively.

Quantization Step. In conventional RoI pooling, the continuous or fractional RoI coordinates are quantized to discrete feature-map coordinates. Conceptually,

$$\hat{x}_1 = Q(x_1), \qquad \hat{y}_1 = Q(y_1), \qquad \hat{x}_2 = Q(x_2), \qquad \hat{y}_2 = Q(y_2), \tag{9.6}$$

where $Q(\cdot)$ denotes the coordinate quantization operation.

Quantization forces the RoI boundaries onto integer-valued locations on the feature map. Although this simplifies pooling, the operation can slightly shift the original region boundaries and therefore introduce spatial misalignment.

Bin Division. After quantization, the RoI is divided into a fixed number of spatial bins. If the desired pooled output has dimensions

$$H_{\text{out}} \times W_{\text{out}},$$

then the quantized RoI width and height are

$$W_{\text{RoI}} = \hat{x}_2 - \hat{x}_1, \tag{9.7}$$

and

$$H_{\text{RoI}} = \hat{y}_2 - \hat{y}_1. \tag{9.8}$$

The approximate width and height of each pooling bin are then

$$W_{\text{bin}} = \frac{W_{\text{RoI}}}{W_{\text{out}}}, \tag{9.9}$$

and

$$H_{\text{bin}} = \frac{H_{\text{RoI}}}{H_{\text{out}}}. \tag{9.10}$$

Each bin covers a portion of the RoI, and max pooling is applied to the feature values falling within that bin. The final output therefore has the fixed size

$$H_{\text{out}} \times W_{\text{out}},$$

regardless of the original RoI dimensions.

The major limitation of RoI pooling is that quantization is applied both to the RoI boundaries and, depending on the implementation, to the boundaries of the individual bins. These rounding operations can produce a mismatch between the original object location and the extracted feature representation. This loss of spatial precision is particularly undesirable for instance segmentation, where accurate pixel-level alignment is required.

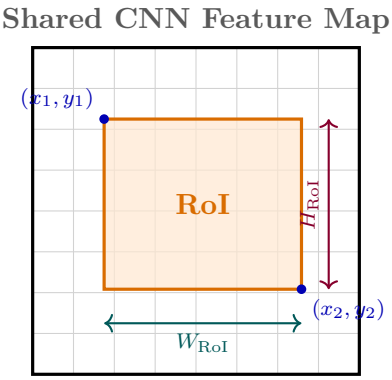


Figure 9.6: Conceptual Region of Interest (RoI) projected onto a shared CNN feature map. The RoI is specified by its boundary coordinates and can have arbitrary width and height before fixed-size feature extraction.

9.19.3 Faster R-CNN

Faster R-CNN replaces the external region proposal stage used in R-CNN and Fast R-CNN with a learnable Region Proposal Network (RPN). The RPN operates directly on the shared convolutional feature map and predicts candidate object regions together with objectness scores. Because the RPN and the detection network share convolutional features, region proposal generation becomes part of the neural network pipeline.

The overall structure can be summarized as

$$\text{Input Image} \rightarrow \text{CNN Feature Map} \rightarrow \text{RPN} \rightarrow \text{RoIs}$$

$$\rightarrow \text{RoI Feature Extraction} \rightarrow \begin{cases} \text{Classification}, \\ \text{Bounding-Box Regression}. \end{cases}$$

The key transition from Fast R-CNN to Faster R-CNN is therefore the replacement of externally generated proposals with proposals learned directly from the shared convolutional representation.

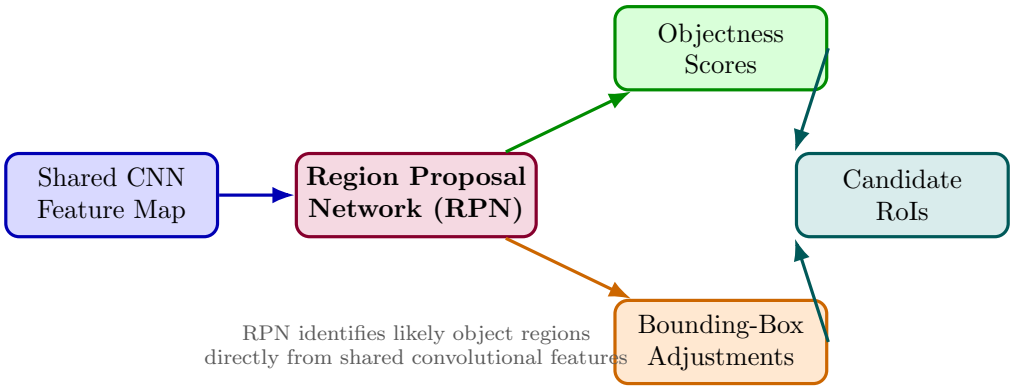


Figure 9.7: Conceptual operation of a Region Proposal Network (RPN). The RPN operates on the shared CNN feature map and predicts both objectness scores and bounding-box adjustments, which are used to generate candidate RoIs.

The overall structure can be summarized as

$$\text{Input Image} \rightarrow \text{CNN Feature Map} \rightarrow \text{RPN}$$

$$\rightarrow \text{RoIs} \rightarrow \text{RoI Feature Extraction} \rightarrow \begin{cases} \text{Classification}, \\ \text{Bounding-Box Regression}. \end{cases}$$

Region Proposal Network

The Region Proposal Network is the main architectural contribution that distinguishes Faster R-CNN from Fast R-CNN. Instead of relying on an external region proposal algorithm, the RPN learns to generate candidate object regions directly from the shared convolutional feature map.

At each spatial location of the feature map, the RPN evaluates a set of reference boxes, commonly referred to as anchors. These anchors represent different candidate scales and aspect ratios. For each anchor, the RPN predicts two quantities:

1. an **objectness score**, indicating whether the anchor is likely to contain an object or background;

2. a set of **bounding-box regression offsets**, which refine the anchor coordinates so that the proposal better matches the underlying object.

Thus, the RPN can be represented conceptually as

$$\text{Shared Feature Map} \rightarrow \text{Anchors} \rightarrow \begin{cases} \text{Objectness Classification,} \\ \text{Bounding-Box Regression.} \end{cases}$$

The objectness branch does not determine the final semantic class of the object. Its purpose is only to determine whether a candidate region is likely to contain an object. Final object classification is performed later by the second-stage detection head.

The bounding-box regression branch predicts adjustments to the anchor coordinates. If an anchor is represented by its center coordinates, width, and height,

$$A = (x_a, y_a, w_a, h_a),$$

the RPN predicts offsets

$$t = (t_x, t_y, t_w, t_h),$$

which are used to obtain a refined proposal.

After objectness prediction and bounding-box refinement, candidate proposals are ranked according to their objectness scores. Low-confidence proposals are discarded, and highly overlapping proposals are reduced using Non-Maximum Suppression (NMS). The remaining proposals form the Regions of Interest (RoIs) that are passed to the second-stage detection network.

The RPN pipeline can therefore be summarized as

$$\begin{aligned} &\text{Feature Map} \rightarrow \text{Anchor Generation} \rightarrow \text{Objectness} + \text{Box Regression} \\ &\qquad \rightarrow \text{Proposal Refinement} \rightarrow \text{NMS} \rightarrow \text{Candidate RoIs.} \end{aligned}$$

RPN Loss

The RPN is trained using a multi-task objective that combines objectness classification and bounding-box regression:

$$\mathcal{L}_{\text{RPN}} = \mathcal{L}_{\text{RPN,cls}} + \lambda \mathcal{L}_{\text{RPN,box}}, \tag{9.11}$$

where $\mathcal{L}_{\text{RPN,cls}}$ measures whether anchors are correctly classified as object or background, and $\mathcal{L}_{\text{RPN,box}}$ measures the accuracy of the predicted bounding-box adjustments for positive anchors.

9.19.4 Mask R-CNN

Mask R-CNN extends Faster R-CNN from object detection to instance segmentation. In addition to the classification and bounding-box regression branches, Mask R-CNN introduces a parallel mask prediction branch that estimates a binary segmentation mask for each detected instance.

A central architectural modification is RoIAlign, which avoids the coordinate quantization used in conventional RoI pooling and preserves more precise spatial correspondence between the original image and the extracted RoI features. This improved alignment is important for pixel-level mask prediction.

The prediction head of Mask R-CNN therefore produces three outputs for each detected instance:

1. an object class label,
2. a refined bounding box, and
3. a pixel-level instance mask.

Conceptually,

$$\text{Faster R-CNN} + \text{RoIAlign} + \text{Mask Branch} \rightarrow \text{Mask R-CNN}.$$

This architecture allows different objects belonging to the same semantic class to be separated as individual instances, which distinguishes instance segmentation from conventional semantic segmentation.

RoIAlign

RoIAlign was introduced in Mask R-CNN to eliminate the coordinate quantization used in RoI pooling. Instead of rounding RoI coordinates to integer feature-map locations, RoIAlign preserves their continuous coordinates and evaluates feature values at fractional sampling locations using bilinear interpolation.

The procedure can be described in several steps.

Compute RoI Width and Height. For an RoI with continuous coordinates

$$R = (x_1, y_1, x_2, y_2),$$

the RoI width and height are computed directly as

$$W_{\text{RoI}} = x_2 - x_1, \tag{9.12}$$

and

$$H_{\text{RoI}} = y_2 - y_1. \tag{9.13}$$

Unlike conventional RoI pooling, these values are not rounded to integer coordinates.

Compute Bin Size. If the required RoIAlign output has dimensions

$$H_{\text{out}} \times W_{\text{out}},$$

the continuous RoI is divided into bins of size

$$W_{\text{bin}} = \frac{W_{\text{RoI}}}{W_{\text{out}}}, \tag{9.14}$$

and

$$H_{\text{bin}} = \frac{H_{\text{RoI}}}{H_{\text{out}}}. \tag{9.15}$$

Because the RoI coordinates remain continuous, the resulting bin boundaries can also lie at fractional feature-map locations.

Compute Sampling Points (Bin Centers). A simple conceptual implementation evaluates the feature map at the center of each bin. For a bin with row index i and column index j, the center can be written as

$$x_{ij} = x_1 + \left(j + \frac{1}{2}\right) W_{\text{bin}}, \tag{9.16}$$

and

$$y_{ij} = y_1 + \left(i + \frac{1}{2}\right) H_{\text{bin}}, \tag{9.17}$$

where

$$i = 0, \ldots, H_{\text{out}} - 1$$

and

$$j = 0, \ldots, W_{\text{out}} - 1.$$

These sampling locations are generally fractional and therefore do not necessarily coincide with discrete feature-map pixels.

In practical RoIAlign implementations, more than one sampling point may be used inside each bin. The resulting sampled values are then combined, commonly by averaging, to obtain the final feature value for that bin.

Bilinear Interpolation. Because a sampling point may occur at a non-integer location (x, y), its feature value is estimated from the four neighboring feature-map values.

Let the neighboring feature-map coordinates be

$$(x_0, y_0), \quad (x_1, y_0), \quad (x_0, y_1), \quad (x_1, y_1).$$

If the corresponding feature values are

$$F_{00}, \quad F_{10}, \quad F_{01}, \quad F_{11},$$

then the interpolated value can be written as

$$\begin{aligned} F(x, y) = (1 - \alpha)(1 - \beta) F_{00} + \alpha(1 - \beta) F_{10} \\ + (1 - \alpha)\beta F_{01} + \alpha\beta F_{11}, \end{aligned} \tag{9.18}$$

where

$$\alpha = x - x_0, \qquad \beta = y - y_0. \tag{9.19}$$

Thus, nearby feature values contribute according to their spatial distance from the sampling point.

Remaining Bin Centers. The same procedure is repeated for the remaining bins. For every bin, RoIAlign determines the corresponding sampling locations, performs bilinear interpolation at those locations, and aggregates the sampled feature values.

For an output grid containing

$$H_{\text{out}} \times W_{\text{out}}$$

bins, the process is repeated until all bin locations have been evaluated. The resulting fixed-size feature map preserves substantially better spatial alignment with the original RoI than conventional RoI pooling.

RoI Pooling versus RoIAlign. The fundamental distinction between RoI pooling and RoIAlign is therefore the treatment of spatial coordinates:

RoI Pooling :	quantize coordinates → divide into bins → pool,
RoIAlign :	preserve fractional coordinates → sample → bilinear interpolation.

By avoiding coordinate quantization, RoIAlign preserves a more accurate correspondence between the original image, the convolutional feature map, and the extracted RoI features. This improvement is particularly important for the mask prediction branch of Mask R-CNN because even small spatial misalignments can degrade pixel-level segmentation boundaries.

Loss Components of Mask R-CNN

Mask R-CNN is trained using a multi-task objective that combines the losses associated with region proposal generation, object classification, bounding-box localization, and instance-mask prediction. Since Mask R-CNN is built upon Faster R-CNN, losses are generated by both the Region Proposal Network (RPN) and the second-stage detection and mask heads.

The overall loss can be expressed as

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{RPN,cls}} + \mathcal{L}_{\text{RPN,box}} + \mathcal{L}_{\text{ROI,cls}} + \mathcal{L}_{\text{ROI,box}} + \mathcal{L}_{\text{mask}}. \tag{9.20}$$

RPN Classification Loss. The RPN classification loss, $\mathcal{L}_{\text{RPN,cls}}$, trains the Region Proposal Network to determine whether an anchor corresponds to an object or background. This term is therefore commonly referred to as the RPN objectness loss.

RPN Bounding-Box Regression Loss. The RPN bounding-box regression loss, $\mathcal{L}_{\text{RPN,box}}$, trains the network to refine positive anchors so that the resulting region proposals more closely match the corresponding ground-truth objects.

RoI Classification Loss. After candidate regions are generated by the RPN, the second-stage detection head predicts the object class associated with each Region of Interest (RoI). Let

$$p = (p_0, p_1, \ldots, p_K)$$

represent the predicted probability distribution over the $K + 1$ possible classes, including background, and let u denote the ground-truth class label. The classification loss for an RoI can be written as

$$\mathcal{L}_{\text{ROI,cls}}(p, u) = -\log(p_u), \tag{9.21}$$

where p_u is the predicted probability assigned to the correct class u.

Equation (9.21) penalizes the model when it assigns a low probability to the correct class. When p_u approaches one, the classification loss approaches zero. Conversely, assigning a low probability to the correct class results in a larger loss.

RoI Bounding-Box Regression Loss. In addition to predicting the object class, the detection head refines the location and dimensions of each positive RoI. The predicted bounding-box adjustments can be represented as

$$t = (t_x, t_y, t_w, t_h)\,, \tag{9.22}$$

where t_x and t_y describe spatial displacement and t_w and t_h describe adjustments to the width and height of the bounding box. Let

$$v = (v_x, v_y, v_w, v_h)$$

denote the corresponding ground-truth regression targets.

The bounding-box regression loss can be expressed as

$$\mathcal{L}_{\text{ROI,box}}(t, v) = \sum_{i \in \{x,y,w,h\}} \text{SmoothL1}\left(t_i - v_i\right). \tag{9.23}$$

The Smooth L1 function is defined as

$$\text{SmoothL1}(x) = \begin{cases} \frac{1}{2}x^2, & |x| < 1, \\ |x| - \frac{1}{2}, & |x| \geq 1. \end{cases} \tag{9.24}$$

For small localization errors, Smooth L1 behaves quadratically, which provides smooth optimization near the target. For larger errors, it behaves linearly, making it less sensitive to large regression errors than a purely squared-error loss. This behavior supports stable bounding-box refinement.

Mask Loss. The distinguishing component of Mask R-CNN is the mask prediction branch. For each positive RoI, the mask head predicts a binary segmentation mask for the corresponding object instance.

Let $y_i \in \{0, 1\}$ denote the ground-truth binary mask value at pixel i, and let $\hat{y}_i$ denote the corresponding predicted foreground probability. The mask loss can be expressed using binary cross-entropy as

$$\mathcal{L}_{\text{mask}} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right], \tag{9.25}$$

where N denotes the number of pixels included in the mask.

The mask loss is evaluated for the mask corresponding to the ground-truth object class. This allows the mask branch to learn pixel-level object shape independently of the class prediction performed by the classification branch.

Mask Loss.

Focal Loss. For highly imbalanced segmentation problems, the large number of easy background pixels can dominate a standard binary cross-entropy objective. Focal Loss modifies cross-entropy by reducing the contribution of well-classified examples and placing greater emphasis on difficult or misclassified examples.

For a binary target, define

$$p_t = \begin{cases} p_i, & y_i = 1, \\ 1 - p_i, & y_i = 0, \end{cases} \tag{9.26}$$

where p_i is the predicted foreground probability and $y_i \in \{0, 1\}$ is the ground-truth label.

The Focal Loss can then be written as

$$\mathcal{L}_{\text{Focal}} = -\frac{1}{N} \sum_{i=1}^{N} \alpha_t (1 - p_t)^{\gamma} \log(p_t), \tag{9.27}$$

where $\gamma \geq 0$ is the focusing parameter and α_t is an optional class-balancing factor. When p_t is close to one, the factor $(1-p_t)^\gamma$ becomes small, reducing the contribution of an easy, correctly classified example. Difficult examples with smaller p_t retain a larger contribution to the total loss.

When $\gamma = 0$, the focusing term becomes one and Focal Loss reduces to weighted cross-entropy.

Interpretation of the Combined Loss.

Interpretation of the Combined Loss.

RPN :	objectness + proposal localization,
Detection Head :	class prediction + final box localization,
Mask Head :	instance-mask prediction.

The individual loss terms correspond to three major functional components of Mask R-CNN:

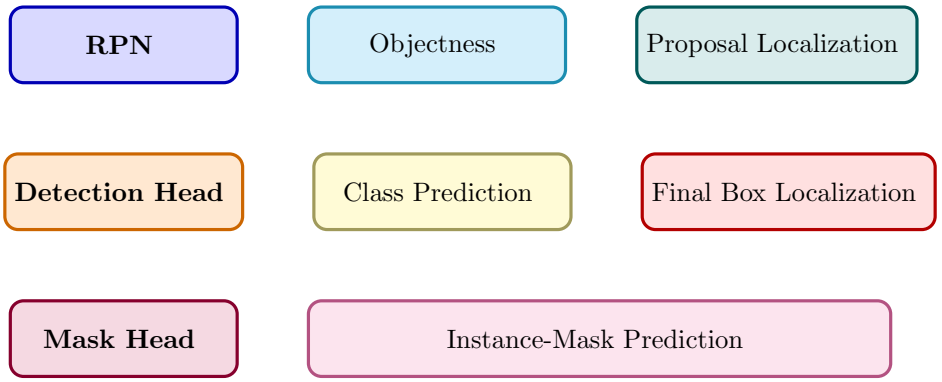


Figure 9.8: Relationship between the principal Mask R-CNN components and their training objectives. The RPN learns objectness and proposal localization, the detection head learns object classification and final bounding-box refinement, and the mask head learns pixel-level instance segmentation.

9.19.5 Evolution of the R-CNN Family

The evolution of the R-CNN family reflects a sequence of architectural improvements that progressively reduce computational redundancy and extend the prediction capability from bounding-box detection to pixel-level instance segmentation.

Table 9.4: Summary of the architectural evolution from R-CNN to Mask R-CNN.

Model	Region Proposal	Feature Processing	Main Contribution
R-CNN	External proposal method	CNN applied independently to each proposed region	Introduced CNN-based region-level object detection.
Fast R-CNN	External proposal method	One shared CNN feature map with RoI pooling	Eliminated repeated CNN computation for individual proposals.
Faster R-CNN	Learnable Region Proposal Network	Shared feature map used by both RPN and detection head	Integrated region proposal generation into the neural network.
Mask R-CNN	Learnable Region Proposal Network	Shared features with RoIAlign and parallel prediction branches	Added pixel-level mask prediction for instance segmentation.

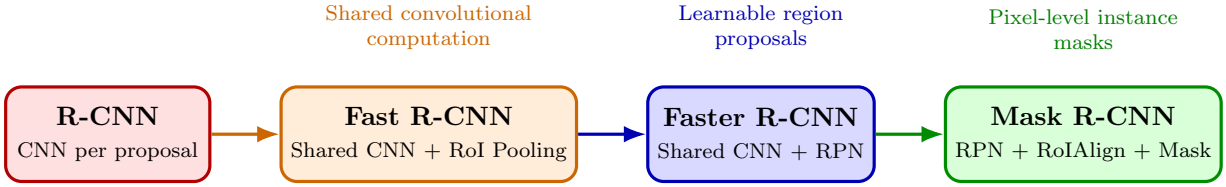


Figure 9.9: Conceptual evolution of the R-CNN family from region-wise CNN processing to shared feature extraction, learnable region proposals, and pixel-level instance mask prediction.

9.20 Why CNNs Work Well for Images

CNNs work well for images because they use several important assumptions about visual data.

9.20.1 Local Connectivity

Nearby pixels are usually more related than distant pixels. CNNs exploit this by applying filters to local image regions.

9.20.2 Weight Sharing

The same kernel is applied across the whole image. This greatly reduces the number of parameters and allows the network to detect the same pattern at different locations.

9.20.3 Translation Equivariance

A convolution operation is translation equivariant. This means that if the input shifts, the feature map shifts in a corresponding way.

This property is useful because objects and patterns may appear in different locations within an image.

9.21 Comparison with Fully Connected Neural Networks

A fully connected neural network treats each pixel as an independent input feature. This ignores the spatial structure of the image and leads to a large number of parameters.

A CNN preserves spatial structure by using local filters. It also reuses the same weights across different spatial locations.

Therefore, CNNs are usually more efficient and more effective for image-related tasks.

9.22 LeNet: A Classic CNN Example

LeNet, especially LeNet-5, is one of the earliest and most influential convolutional neural networks. It was introduced by Yann LeCun and was designed mainly for handwritten digit recognition. LeNet is important because it clearly shows the basic CNN idea: convolution layers extract features, pooling layers reduce spatial size, and fully connected layers perform classification.

The original LeNet-5 takes a grayscale image of size 32×32 as input. As the image passes through the network, the spatial dimensions decrease while the learned feature representation becomes more abstract. Early layers learn simple patterns such as edges and corners, while deeper layers learn more meaningful shapes and digit-specific structures.

A simplified view of LeNet is:

$$\text{Input} \to \text{Conv} \to \text{Pool} \to \text{Conv} \to \text{Pool} \to \text{Conv} \to \text{FC} \to \text{Output}.$$

In the original LeNet, the activation was typically `tanh`, and the pooling operation was average pooling (also called subsampling). In modern teaching, LeNet is often used as a simple example of how CNNs work.

9.22.1 LeNet Layer-by-Layer Summary

Table 9.5: Short summary of the LeNet-5 architecture.

Layer	**Operation**	**Output Size**	**Main Purpose**
Input	Grayscale image	$1 \times 32 \times 32$	Original input image.
C1	Convolution, 5×5, 6 filters	$6 \times 28 \times 28$	Learns simple local features such as edges and corners.
S2	Average pooling, 2×2, stride 2	$6 \times 14 \times 14$	Reduces spatial size and keeps important information.
C3	Convolution, 5×5, 16 filters	$16 \times 10 \times 10$	Learns more complex patterns from earlier features.
S4	Average pooling, 2×2, stride 2	$16 \times 5 \times 5$	Further reduces spatial size and computation.
C5	Convolution, 5×5, 120 filters	$120 \times 1 \times 1$	Combines the extracted features into a compact high-level representation.
F6	Fully connected, 84 units	84	Learns a compact representation for classification.
Output	Fully connected, 10 units	10	Produces class scores for digits 0 to 9.

9.22.2 TikZ Flow Diagram of LeNet

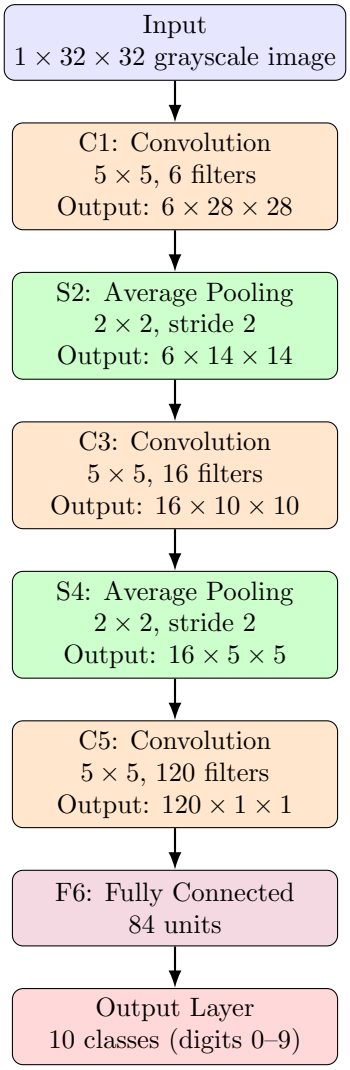


Figure 9.10: A simple flow diagram of the LeNet-5 architecture.

9.22.3 Why LeNet is Important

LeNet is important because it demonstrates the core design of many CNNs in a simple and understandable form. Even though modern CNNs are much deeper and more powerful, the same main idea remains: use convolution layers to learn spatial features, use pooling to reduce dimension, and use final dense layers to perform classification.

9.23 AlexNet: A Deeper CNN for Image Classification

AlexNet is one of the most important CNN architectures in deep learning. It was introduced by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, and it became widely known after winning the ImageNet Large Scale Visual Recognition Challenge in 2012. AlexNet showed that deep convolutional neural networks could significantly outperform traditional computer vision methods when trained on large datasets with GPU acceleration.

Compared with LeNet, AlexNet is much deeper and larger. It was designed for color images and large-scale classification tasks. The network uses multiple convolution layers to learn increasingly complex visual features, followed by fully connected layers for classification.

Some important features of AlexNet are:

- **ReLU activation**, which helps faster training than sigmoid or `tanh`,
- **max pooling**, which reduces spatial dimensions,
- **dropout**, which helps reduce overfitting in fully connected layers,
- and a **deep layered structure**, which allows learning rich hierarchical features.

In a simple view, AlexNet follows this flow:

Input → Conv → Pool → Conv → Pool → Conv → Conv → Conv → Pool → FC → FC → Output.

9.23.1 AlexNet Layer-by-Layer Summary

Table 9.6: Short summary of the AlexNet architecture.

Layer	**Operation**	**Output Size**	**Main Purpose**
Input	RGB image	$3 \times 227 \times 227$	Original input image.
Conv1	Convolution, 11×11, stride 4, 96 filters + ReLU	$96 \times 55 \times 55$	Learns low-level features such as edges and color patterns.
Pool1	Max pooling, 3×3, stride 2	$96 \times 27 \times 27$	Reduces spatial size and keeps strong responses.
Conv2	Convolution, 5×5, 256 filters + ReLU	$256 \times 27 \times 27$	Learns more complex textures and shape patterns.
Pool2	Max pooling, 3×3, stride 2	$256 \times 13 \times 13$	Further reduces feature map size.
Conv3	Convolution, 3×3, 384 filters + ReLU	$384 \times 13 \times 13$	Learns richer intermediate visual features.
Conv4	Convolution, 3×3, 384 filters + ReLU	$384 \times 13 \times 13$	Refines and combines intermediate features.
Conv5	Convolution, 3×3, 256 filters + ReLU	$256 \times 13 \times 13$	Learns higher-level object-related features.
Pool3	Max pooling, 3×3, stride 2	$256 \times 6 \times 6$	Produces a compact spatial representation.
FC1	Fully connected, 4096 units + ReLU + Dropout	4096	Learns a high-level global representation.
FC2	Fully connected, 4096 units + ReLU + Dropout	4096	Further refines features for classification.
Output	Fully connected, 1000 units	1000	Produces class scores for ImageNet classes.

9.23.2 TikZ Flow Diagram of AlexNet

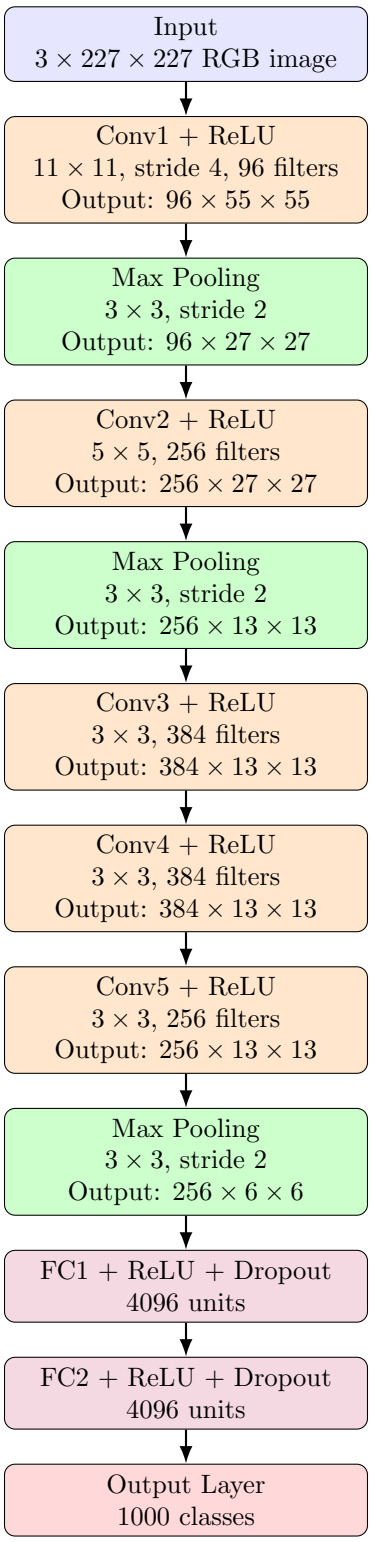


Figure 9.11: A simple flow diagram of the AlexNet architecture.

9.23.3 Why AlexNet is Important

AlexNet is important because it demonstrated that deep CNNs can achieve powerful performance on large image classification problems. It helped establish modern deep learning in

computer vision. Compared with LeNet, AlexNet is deeper, uses more filters, works on color images, and is designed for much larger datasets. It also popularized practical ideas such as ReLU and dropout, which became standard in many later neural network architectures.

9.24 VGG16: A Deep CNN with Small Convolution Kernels

VGG16 is a deep convolutional neural network introduced by the Visual Geometry Group at the University of Oxford. It became widely known for showing that strong image classification performance can be achieved using a simple and uniform CNN design. The main idea of VGG16 is to use many small 3×3 convolution kernels instead of using larger kernels.

Compared with AlexNet, VGG16 is deeper and more systematic. AlexNet uses different kernel sizes such as 11×11, 5×5, and 3×3. In contrast, VGG16 mostly uses repeated 3×3 convolutions. This makes the architecture easier to understand and easier to extend.

The name VGG16 means that the network has 16 learnable layers: 13 convolution layers and 3 fully connected layers. The convolution layers extract hierarchical image features, while the fully connected layers perform classification.

Some important features of VGG16 are:

- **Small** 3×3 **convolution kernels**, which allow deep feature extraction with fewer parameters than large kernels,
- **repeated convolution blocks**, which make the architecture simple and organized,
- **max pooling**, which reduces spatial size after each block,
- **deep hierarchical feature learning**, where early layers learn edges and textures, and deeper layers learn object-level patterns,
- and **fully connected layers**, which perform final classification.

In a simple view, VGG16 follows this flow:

$$\begin{aligned}\text{Input} &\to \text{Conv Block 1} \to \text{Pool} \to \text{Conv Block 2} \to \text{Pool} \to \text{Conv Block 3} \\ &\to \text{Pool} \to \text{Conv Block 4} \to \text{Pool} \to \text{Conv Block 5} \to \text{Pool} \\ &\to \text{FC} \to \text{FC} \to \text{Output}.\end{aligned}$$

In a simple view, VGG16 follows this flow:

$$\begin{aligned}\text{Input} &\to \text{Conv Block 1} \to \text{Pool} \to \text{Conv Block 2} \to \text{Pool} \\ &\to \text{Conv Block 3} \to \text{Pool} \to \text{Conv Block 4} \to \text{Pool} \\ &\to \text{Conv Block 5} \to \text{Pool} \to \text{FC}_1 \to \text{FC}_2 \to \text{FC}_3 \to \text{Output}.\end{aligned}$$

9.24.1 VGG16 Layer-by-Layer Summary

Table 9.7: Short summary of the VGG16 architecture.

Layer	**Operation**	**Output Size**	**Main Purpose**
Input	RGB image	$3 \times 224 \times 224$	Original input image.
Block 1	Two 3×3 convolutions, 64 filters + ReLU	$64 \times 224 \times 224$	Learns low-level features such as edges, corners, and simple textures.
Pool 1	Max pooling, 2×2, stride 2	$64 \times 112 \times 112$	Reduces spatial size by half.
Block 2	Two 3×3 convolutions, 128 filters + ReLU	$128 \times 112 \times 112$	Learns richer texture and local shape patterns.
Pool 2	Max pooling, 2×2, stride 2	$128 \times 56 \times 56$	Further reduces spatial size.
Block 3	Three 3×3 convolutions, 256 filters + ReLU	$256 \times 56 \times 56$	Learns intermediate-level visual features.
Pool 3	Max pooling, 2×2, stride 2	$256 \times 28 \times 28$	Reduces spatial size and computation.
Block 4	Three 3×3 convolutions, 512 filters + ReLU	$512 \times 28 \times 28$	Learns high-level object parts and complex patterns.
Pool 4	Max pooling, 2×2, stride 2	$512 \times 14 \times 14$	Produces smaller high-level feature maps.
Block 5	Three 3×3 convolutions, 512 filters + ReLU	$512 \times 14 \times 14$	Learns deeper object-level features.
Pool 5	Max pooling, 2×2, stride 2	$512 \times 7 \times 7$	Produces a compact spatial representation.
FC1	Fully connected, 4096 units + ReLU + Dropout	4096	Learns a high-level global representation.
FC2	Fully connected, 4096 units + ReLU + Dropout	4096	Further refines features for classification.
Output	Fully connected, 1000 units	1000	Produces class scores for ImageNet classes.

9.24.2 TikZ Flow Diagram of VGG16

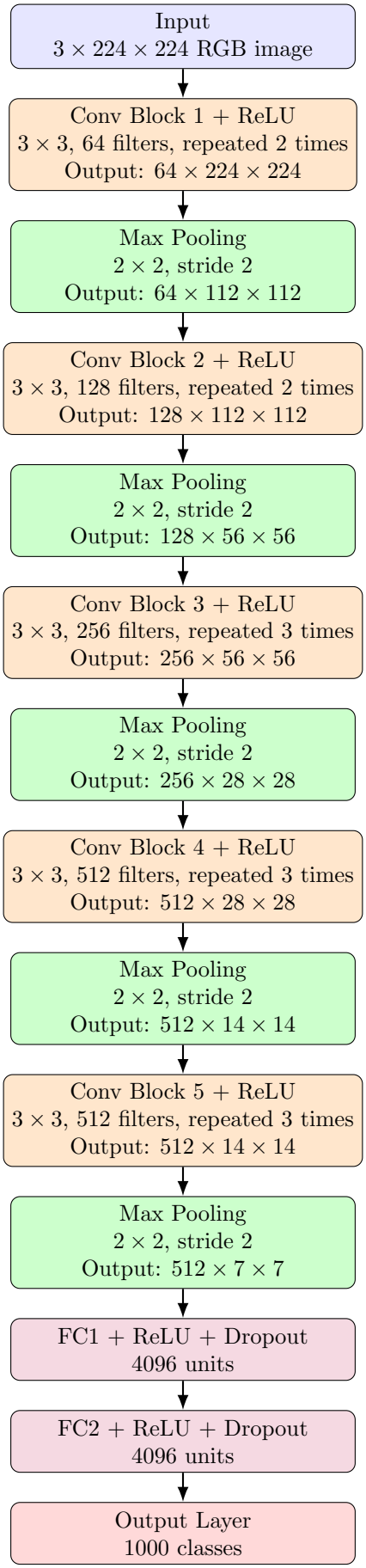


Figure 9.12: A simple flow diagram of the VGG16 architecture.

VGG16 can be remembered as a deep and simple CNN built from repeated 3×3 convolution blocks: *small kernels → repeated blocks → deeper features → classification*. Its main lesson is that depth and architectural simplicity can work very well together.

9.24.3 Why VGG16 is Important

VGG16 is important because it showed that CNNs can be made deeper using a very simple design principle: repeat small 3×3 convolution kernels many times. This makes the architecture easy to understand, while still allowing the network to learn powerful hierarchical features.

Compared with AlexNet, VGG16 uses smaller convolution kernels, more convolution layers, and a more uniform block-based design. Its simplicity made it widely used as a backbone for image classification, transfer learning, object detection, and segmentation models.

9.25 GoogLeNet: CNN with Inception Modules

GoogLeNet is an important convolutional neural network architecture introduced by researchers at Google. It won the ImageNet Large Scale Visual Recognition Challenge in 2014. GoogLeNet is best known for introducing the **Inception module**, which allows the network to learn features at multiple spatial scales within the same layer.

Compared with VGG16, GoogLeNet is deeper but more parameter-efficient. VGG16 uses a simple repeated block design with many 3×3 convolutions and large fully connected layers. GoogLeNet, instead, uses Inception modules to combine 1×1, 3×3, and 5×5 convolutions in parallel. This allows the model to capture both small local patterns and larger spatial structures.

The original GoogLeNet is also called Inception-v1. It has 22 learnable layers, but it uses far fewer parameters than VGG16 because it avoids large fully connected layers near the end. Instead, it uses global average pooling before the final classifier.

Some important features of GoogLeNet are:

- **Inception modules**, which apply multiple operations in parallel,
- 1×1 **convolutions**, which reduce channel dimension and computational cost,
- **multi-scale feature extraction**, which captures both small and large visual patterns,
- **global average pooling**, which replaces large fully connected layers,
- and **parameter efficiency**, which makes the network deep without becoming extremely large.

In a simple view, GoogLeNet follows this flow:

Input → Initial Conv/Pool Layers → Inception Modules → Global Average Pooling → Output.

9.25.1 The Inception Module

The main building block of GoogLeNet is the Inception module. Instead of choosing only one filter size, the Inception module applies different operations to the same input in parallel. The outputs are then concatenated along the channel dimension.

A simplified Inception module can be written as:

$$\text{Input} \rightarrow \begin{cases} 1 \times 1 \text{ convolution}, \\ 3 \times 3 \text{ convolution}, \\ 5 \times 5 \text{ convolution}, \\ 3 \times 3 \text{ max pooling}. \end{cases} \rightarrow \text{concatenate}.$$

The 1×1 convolution is especially important because it can reduce the number of channels before applying more expensive 3×3 and 5×5 convolutions. This keeps the network computationally efficient.

9.25.2 GoogLeNet Layer-by-Layer Summary

Table 9.8: Short summary of the GoogLeNet architecture.

Layer	**Operation**	**Output Size**	**Main Purpose**
Input	RGB image	$3 \times 224 \times 224$	Original input image.
Initial layers	Convolution and max pooling layers	Smaller feature maps	Extracts low-level features and reduces spatial size.
Inception 3a–3b	Two Inception modules	Medium-level feature maps	Learns multi-scale local features using parallel filters.
Max Pooling	Pooling after early Inception modules	Reduced spatial size	Reduces computation and increases receptive field.
Inception 4a–4e	Five Inception modules	Deeper feature maps	Learns richer object parts and complex visual structures.
Max Pooling	Pooling after middle Inception modules	Reduced spatial size	Further reduces spatial dimension.
Inception 5a–5b	Two Inception modules	High-level feature maps	Learns object-level semantic features.
Global Average Pooling	Average over spatial locations	Feature vector	Converts each feature map into one value, reducing parameters.
Dropout	Randomly drops activations during training	Feature vector	Reduces overfitting.
Output	Fully connected layer, 1000 units	1000	Produces class scores for ImageNet classes.

9.25.3 TikZ Flow Diagram of an Inception Module

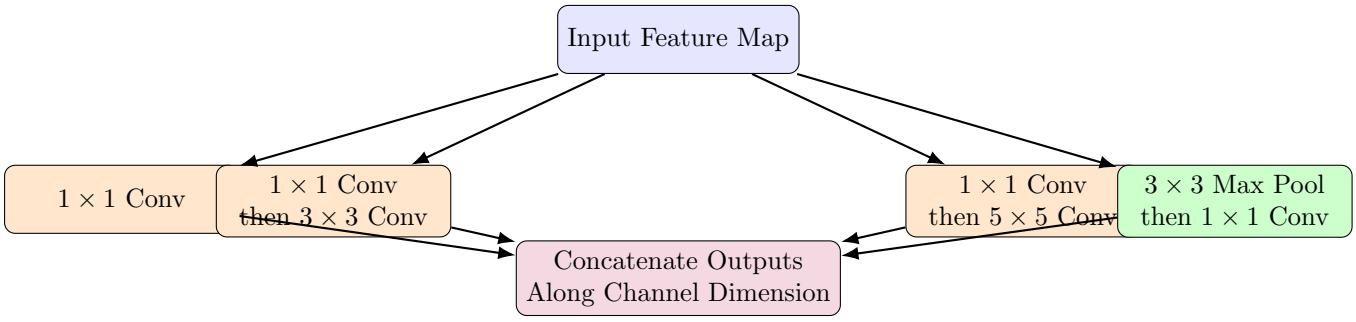


Figure 9.13: A simplified Inception module used in GoogLeNet. Multiple operations are applied in parallel, and their outputs are concatenated along the channel dimension.

9.25.4 TikZ Flow Diagram of GoogLeNet

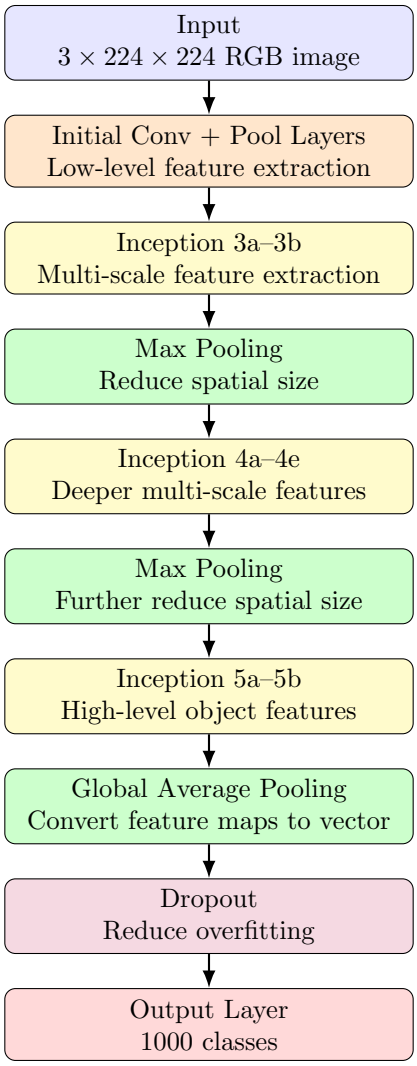


Figure 9.14: A simple flow diagram of the GoogLeNet architecture.

GoogLeNet can be remembered as a CNN built from Inception modules: *parallel filters* → *multi-scale features* → *concatenate channels* → *efficient classification*. Its main lesson is that a network can become deeper and smarter without simply adding more parameters.

9.25.5 Why GoogLeNet is Important

GoogLeNet is important because it introduced the Inception module, which made CNNs deeper and more efficient. Instead of increasing depth and width in a simple way, GoogLeNet used parallel convolution paths to learn features at different scales. This allowed the network to capture small details and larger object patterns at the same time.

Compared with VGG16, GoogLeNet uses fewer parameters because it replaces large fully connected layers with global average pooling and uses 1×1 convolutions to reduce channel dimensions. This makes GoogLeNet a strong example of efficient CNN design.

9.26 Summary

Convolutional Neural Networks are a central architecture in deep learning for computer vision. Their main components are convolution layers, activation functions, pooling layers, and fully connected layers. The convolution operation allows the network to learn local spatial features, while weight sharing reduces the number of parameters.

CNNs form the foundation of many modern vision models, including image classifiers, object detectors, semantic segmentation models, and instance segmentation models. Although Transformer-based vision models are now widely used, CNNs remain important because of their efficiency, inductive bias, and strong performance on image-based tasks.

9.27 Comparison of LeNet, AlexNet, VGG16, and GoogLeNet

LeNet, AlexNet, VGG16, and GoogLeNet represent important stages in the development of convolutional neural networks. LeNet introduced the basic CNN pipeline, AlexNet showed the power of deep CNNs on large-scale image classification, VGG16 demonstrated the strength of repeated small convolution kernels, and GoogLeNet introduced efficient multi-scale feature extraction through Inception modules.

Table 9.9: Comparison of classic CNN architectures.

Model	**Main Idea**	**Input Example**	**Typical Task**	**Important Lesson**
LeNet	Basic CNN pipeline with convolution, pooling, and fully connected layers.	Grayscale digit image, such as MNIST.	Handwritten digit recognition.	CNNs can learn local image features using shared convolution filters.
AlexNet	Deeper CNN with ReLU, max pooling, dropout, and large fully connected layers.	RGB natural image, such as a dog, car, or airplane image.	Large-scale image classification.	Deep CNNs trained with GPUs can outperform traditional computer vision methods.
VGG16	Very deep CNN built from repeated 3×3 convolution blocks.	RGB image resized to 224×224.	Image classification and transfer learning.	A simple repeated-block design can learn strong hierarchical features.
GoogLeNet	CNN with Inception modules for parallel multi-scale feature extraction.	RGB image resized to 224×224.	Image classification with efficient deep architecture.	A network can be deep and accurate without using too many parameters.

9.27.1 Architecture-Level Comparison

Table 9.10: Architecture-level comparison of LeNet, AlexNet, VGG16, and GoogLeNet.

Feature	**LeNet**	**AlexNet**	**VGG16**	**GoogLeNet**
Input type	Grayscale image	RGB image	RGB image	RGB image
Typical input size	$1 \times 32 \times 32$	$3 \times 227 \times 227$	$3 \times 224 \times 224$	$3 \times 224 \times 224$
Main dataset example	MNIST-style digit images	ImageNet	ImageNet	ImageNet
Main building block	Conv + pooling	Conv + max pooling	Repeated 3×3 conv blocks	Inception module
Depth	Shallow	Deeper than LeNet	Very deep	Very deep but efficient
Convolution kernel style	Mostly 5×5	Large early kernels, such as 11×11, then smaller kernels	Mostly 3×3	Parallel 1×1, 3×3, and 5×5 paths
Pooling type	Average pooling	Max pooling	Max pooling	Max pooling and global average pooling
Activation	Usually `tanh`	ReLU	ReLU	ReLU
Fully connected layers	Small FC layers	Large FC layers	Large FC layers	Mostly replaced by global average pooling
Parameter efficiency	Small model	Large model	Very large model	More efficient than VGG16
Best remembered as	First classic CNN	CNN breakthrough for ImageNet	Deep CNN with repeated small kernels	CNN with Inception modules

9.27.2 Simple Example-Based Understanding

Suppose the input is an image of a car.

- **LeNet-style thinking:** The network learns simple local patterns, such as edges and curves, and then uses fully connected layers to decide the class. This is suitable for simple images such as handwritten digits.
- **AlexNet-style thinking:** The network learns low-level patterns in early layers, then textures, object parts, and finally object-level features. It is suitable for large natural image classification.
- **VGG16-style thinking:** The network repeatedly applies small 3×3 convolutions. Each block makes the representation deeper and more abstract. Early blocks may detect edges, middle blocks may detect wheels and windows, and later blocks may detect car-level structures.

- **GoogLeNet-style thinking:** The network looks at the same feature map using different filter sizes in parallel. Small filters can detect small details, while larger filters can capture wider spatial patterns. The outputs are combined to form a stronger multi-scale representation.

9.27.3 Short Memory Summary

A simple way to remember these CNN architectures is:

$$\text{LeNet} \rightarrow \text{basic CNN idea}$$

$$\text{AlexNet} \rightarrow \text{deeper CNN for large RGB image classification}$$

$$\text{VGG16} \rightarrow \text{many repeated } 3 \times 3 \text{ convolution blocks}$$

$$\text{GoogLeNet} \rightarrow \text{Inception modules with parallel multi-scale filters.}$$

In short, LeNet introduced the CNN structure, AlexNet scaled it to large image classification, VGG16 made it deeper using a clean repeated-block design, and GoogLeNet made it more efficient using Inception modules.

9.28 ResNet: Deep CNN with Residual Connections

ResNet, short for Residual Network, is one of the most important CNN architectures in deep learning. It was introduced to solve a major problem in very deep neural networks: as networks become deeper, they can become harder to optimize. This problem is not only caused by overfitting. Even training accuracy can become worse when many layers are stacked directly.

The main idea of ResNet is to use a **skip connection**, also called a **shortcut connection**. Instead of forcing a stack of layers to learn a complete mapping from input to output, ResNet allows the layers to learn a residual correction.

If the desired mapping is $H(x)$, ResNet rewrites it as

$$H(x) = F(x) + x,$$

where x is the input and $F(x)$ is the residual function learned by the convolution layers.

This means the network does not need to learn the entire transformation directly. It only needs to learn the difference between the input and the desired output. This simple idea makes very deep CNNs much easier to train.

9.28.1 Main Idea of Residual Learning

In a traditional CNN block, the input passes through several layers:

$$x \rightarrow \text{Conv} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{Output}.$$

In a ResNet block, the input also skips around these layers and is added back to the output:

$$x \rightarrow F(x) \rightarrow F(x) + x.$$

Here, $F(x)$ is the residual branch, and x is the shortcut branch.

A residual block can be remembered as:

$$\text{Output} = \text{learned residual} + \text{original input}.$$

This helps preserve useful information and improves gradient flow during backpropagation.

9.28.2 Why Deep Networks Need Residual Connections

Before ResNet, researchers observed that simply adding more layers did not always improve performance. A deeper network should, in principle, be able to perform at least as well as a shallower network. However, in practice, very deep plain CNNs often became difficult to train.

This issue is called the **degradation problem**. It means that after a certain depth, training error may increase even though the network has more layers.

Residual connections help because the network can easily learn an identity mapping. If extra layers are not useful, the residual branch can learn values close to zero:

$$F(x) \approx 0.$$

Then,

$$F(x) + x \approx x.$$

So the block can simply pass information forward without damaging the representation. This makes it safer to build very deep networks.

9.28.3 Parts of a Basic ResNet Block

A basic ResNet block has two main paths:

- **Residual branch:** This branch contains convolution, batch normalization, and activation layers. It learns the residual function $F(x)$.
- **Shortcut branch:** This branch carries the input x directly to a later layer. If the input and output dimensions match, the shortcut can be an identity connection.

The two branches are added together using element-wise addition:

$$y = F(x) + x.$$

Then a ReLU activation is usually applied:

$$\text{Output} = \text{ReLU}(F(x) + x).$$

9.28.4 Basic Residual Block

The basic residual block is commonly used in smaller ResNet models such as ResNet-18 and ResNet-34. It usually contains two 3×3 convolution layers.

A simplified basic block is:

$$x \to 3 \times 3 \text{ Conv} \to \text{BatchNorm} \to \text{ReLU} \to 3 \times 3 \text{ Conv} \to \text{BatchNorm} \to +x \to \text{ReLU}.$$

The first convolution learns local features, and the second convolution refines them. The shortcut connection adds the original input back to the learned residual features.

9.28.5 Bottleneck Residual Block

Deeper ResNet models such as ResNet-50, ResNet-101, and ResNet-152 often use a bottleneck residual block. The bottleneck block uses three convolution layers:

$$1 \times 1 \to 3 \times 3 \to 1 \times 1.$$

The first 1×1 convolution reduces the number of channels. The 3×3 convolution performs spatial feature extraction. The final 1×1 convolution expands the number of channels again.

A simplified bottleneck block is:

$$x \to 1 \times 1 \text{ Conv} \to 3 \times 3 \text{ Conv} \to 1 \times 1 \text{ Conv} \to +x.$$

The purpose of the bottleneck design is to reduce computation while still allowing the network to be very deep.

9.28.6 Identity Shortcut and Projection Shortcut

There are two common shortcut types in ResNet.

1. Identity shortcut

If the input and output have the same shape, the shortcut simply passes the input forward:

$$y = F(x) + x.$$

This is the simplest and most common residual connection.

2. Projection shortcut

If the input and output shapes are different, they cannot be added directly. This happens when the spatial size changes or the number of channels changes. In that case, ResNet uses a 1×1 convolution in the shortcut branch:

$$y = F(x) + W_s x,$$

where W_s represents the 1×1 projection convolution.

The projection shortcut changes the shape of x so that it matches the shape of $F(x)$.

9.28.7 How Bottleneck Blocks Repeat in ResNet

In deeper ResNet models such as ResNet-50, ResNet-101, and ResNet-152, the network is not built from only one bottleneck block. Instead, many bottleneck blocks are repeated inside residual stages.

Each bottleneck block follows the same internal pattern:

$$1 \times 1 \text{ Conv} \to 3 \times 3 \text{ Conv} \to 1 \times 1 \text{ Conv} \to \text{Add shortcut}.$$

A ResNet stage repeats this bottleneck block several times. The first block of a new stage may reduce the spatial size and increase the number of channels. The following blocks usually keep the same feature-map size and channel dimension.

For example, ResNet-50 uses bottleneck blocks in four main stages:

$$3,\ 4,\ 6,\ 3$$

which means the four residual stages contain 3, 4, 6, and 3 bottleneck blocks, respectively.

9.28.8 TikZ Diagram: Repeating Bottleneck Blocks in ResNet

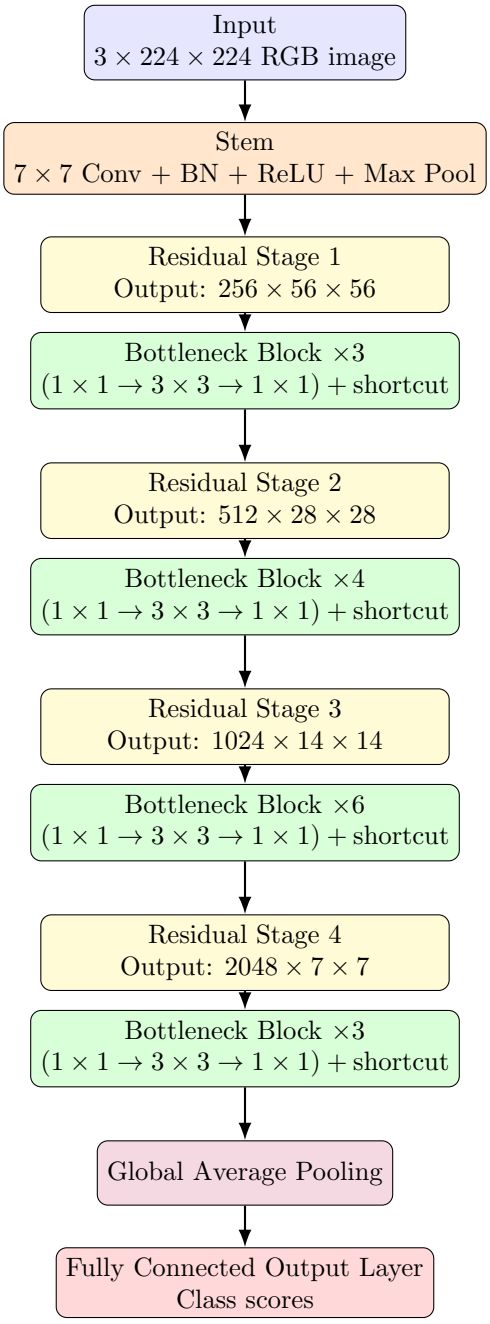


Figure 9.15: Repeating bottleneck blocks in a ResNet-50 style architecture. The four residual stages repeat bottleneck blocks 3, 4, 6, and 3 times.

9.28.9 ResNet Layer-by-Layer Summary

Table 9.11: Short summary of a standard ResNet architecture.

Part	**Operation**	**Output Example**	**Main Purpose**
Input	RGB image	$3 \times 224 \times 224$	Original input image.
Initial layer	7×7 convolution, stride 2, batch normalization, ReLU	$64 \times 112 \times 112$	Extracts low-level features and reduces spatial size.
Max pooling	3×3 max pooling, stride 2	$64 \times 56 \times 56$	Further reduces spatial size.
Residual stage 1	Residual blocks with 64 channels	$64 \times 56 \times 56$	Learns early residual features.
Residual stage 2	Residual blocks with 128 channels, downsampling at stage start	$128 \times 28 \times 28$	Learns richer features with smaller spatial size.
Residual stage 3	Residual blocks with 256 channels, downsampling at stage start	$256 \times 14 \times 14$	Learns high-level object-part features.
Residual stage 4	Residual blocks with 512 channels, downsampling at stage start	$512 \times 7 \times 7$	Learns deep object-level features.
Global average pooling	Average over spatial dimensions	512 or larger feature vector	Converts feature maps into a compact vector.
Output	Fully connected layer	Number of classes	Produces class scores for classification.

9.28.10 Common ResNet Variants

Table 9.12: Common ResNet variants.

Model	Depth	Block Type	Main Use
ResNet-18	18 layers	Basic residual block	Lightweight image classification and feature extraction.
ResNet-34	34 layers	Basic residual block	Deeper than ResNet-18 but still relatively efficient.
ResNet-50	50 layers	Bottleneck residual block	Very common backbone for classification, detection, and segmentation.
ResNet-101	101 layers	Bottleneck residual block	Stronger but more computationally expensive feature extractor.
ResNet-152	152 layers	Bottleneck residual block	Very deep CNN used for high-capacity feature learning.

9.28.11 TikZ Flow Diagram of a Basic Residual Block

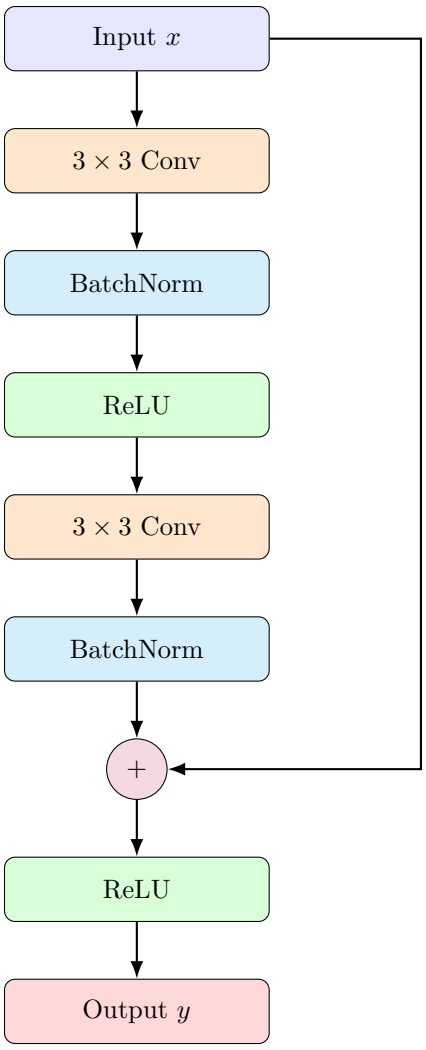


Figure 9.16: A basic ResNet residual block. The input x passes through convolution layers and also skips directly to the addition operation.

9.28.12 TikZ Flow Diagram of a Bottleneck Residual Block

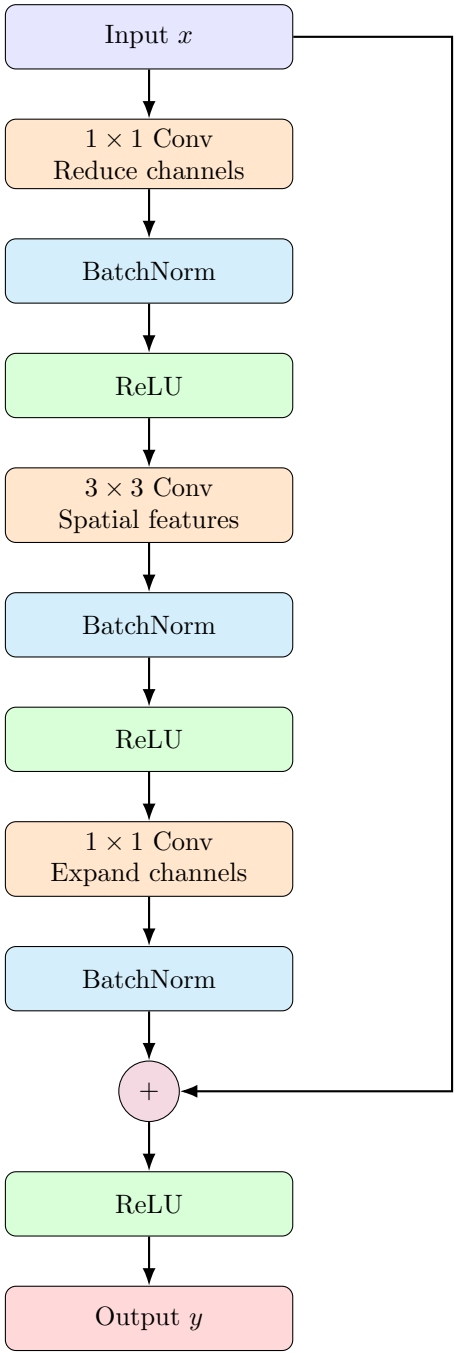


Figure 9.17: A bottleneck ResNet block. The 1×1 convolutions reduce and then expand the channel dimension, while the 3×3 convolution extracts spatial features.

9.28.13 TikZ Flow Diagram of ResNet

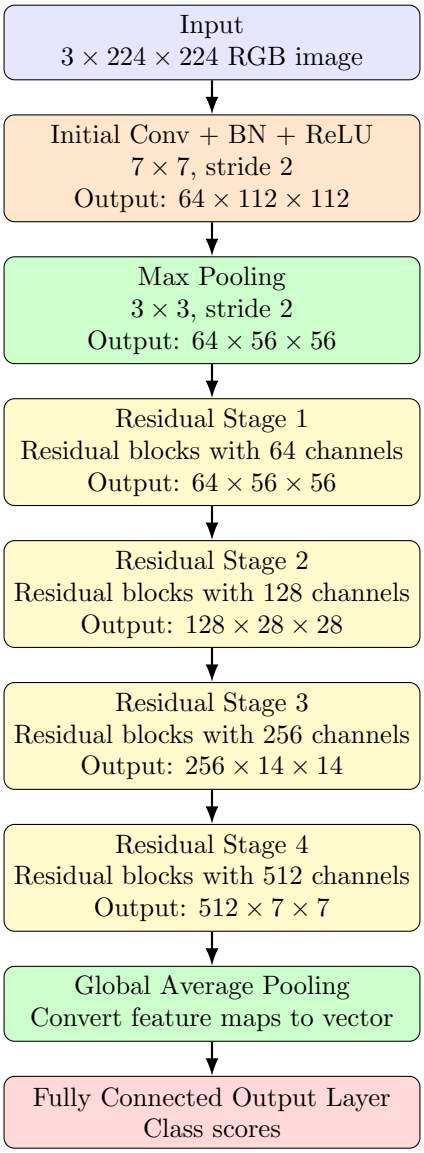


Figure 9.18: A simple flow diagram of a standard ResNet architecture.

9.28.14 Why ResNet is Important

ResNet is important because it made very deep CNNs practical. Its residual connections allow information and gradients to flow more easily through the network. This reduces the optimization difficulty that appears when many layers are stacked directly.

Compared with VGG16 and GoogLeNet, ResNet introduced a simpler and more general solution for training deep networks: add shortcut connections. This idea became widely used not only in CNNs, but also in many later deep learning models, including segmentation networks, detection networks, and Transformer-based architectures.

9.29 Gabor Filters, Saliency Maps, and Grad-CAM

Gabor filters, saliency maps, and Grad-CAM are useful tools for understanding image features and CNN behavior. They are not the same type of method. A Gabor filter is a handcrafted

image filter, while saliency maps and Grad-CAM are model interpretation methods. Together, they help explain how visual patterns are detected and how CNN predictions can be interpreted.

9.29.1 Gabor Filters

A Gabor filter is a handcrafted filter used to detect oriented edges, lines, ridges, and texture patterns in an image. It combines a Gaussian envelope with a sinusoidal wave. The Gaussian part localizes the filter in space, while the sinusoidal part detects a specific frequency and orientation.

A simplified two-dimensional Gabor filter can be written as

$$G(x, y) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos\left(2\pi \frac{x'}{\lambda} + \psi\right),$$

where

$$x' = x\cos\theta + y\sin\theta,$$

$$y' = -x\sin\theta + y\cos\theta.$$

Here, θ controls the orientation of the filter, λ controls the wavelength, σ controls the size of the Gaussian window, γ controls the shape of the filter, and ψ controls the phase.

The intuition is simple. A Gabor filter asks whether a certain local pattern exists in an image region. For example, one Gabor filter may detect vertical edges, while another may detect horizontal or diagonal textures.

Table 9.13: Simple interpretation of Gabor filter parameters.

Parameter	**Meaning**	**Effect**
θ	Orientation	Controls the direction of the detected edge or texture.
λ	Wavelength	Controls the frequency of the sinusoidal pattern.
σ	Gaussian width	Controls the spatial size of the filter.
γ	Aspect ratio	Controls the shape or elongation of the filter.
ψ	Phase offset	Shifts the sinusoidal wave.

In CNNs, Gabor filters are useful conceptually because early convolution layers often learn filters that look similar to edge and texture detectors. The important difference is that Gabor filters are fixed handcrafted filters, while CNN filters are learned automatically from data.

9.29.2 Saliency Maps

A saliency map is an interpretation method that shows which pixels in an input image are most important for a model's prediction. In image classification, a saliency map tries to answer the question:

Which input pixels most influence the predicted class?

Suppose a CNN produces a class score y_c for class c, and the input image is X. A simple saliency map can be computed using the gradient of the class score with respect to the input image:

$$S = \left| \frac{\partial y_c}{\partial X} \right|.$$

This gradient tells us how sensitive the class score is to changes in each input pixel. If a small change in a pixel causes a large change in the class score, that pixel receives a high saliency value.

For example, if a CNN predicts that an image contains a dog, a saliency map may highlight the dog's face, body outline, ears, or fur texture. If the saliency map highlights mostly background regions, this may indicate that the model is relying on irrelevant visual cues.

Basic saliency maps are easy to compute, but they can be noisy because they operate directly at the pixel level. Therefore, they may highlight scattered pixels instead of smooth object regions.

9.29.3 Grad-CAM

Grad-CAM stands for Gradient-weighted Class Activation Mapping. It is a CNN interpretation method that produces a class-specific heatmap. Unlike basic saliency maps, which work directly on input pixels, Grad-CAM uses the feature maps of a convolutional layer, usually the last convolutional layer.

The main idea of Grad-CAM is to identify which spatial regions in the convolutional feature maps are important for a target class.

Let the feature maps of a convolutional layer be

$$A^1, A^2, \ldots, A^K,$$

where A^k is the k-th feature map. For a target class c, Grad-CAM first computes the importance weight for each feature map:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y_c}{\partial A_{ij}^k},$$

where A_{ij}^k is the activation at spatial location (i, j) in feature map k, and Z is the number of spatial locations.

Then the Grad-CAM heatmap is computed as

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k \right).$$

The ReLU keeps only the regions that positively support the target class. The resulting heatmap is usually resized to the input image size and overlaid on the original image.

The intuition is that Grad-CAM asks:

Which high-level CNN regions support this class prediction?

For example, if a CNN predicts "cat," Grad-CAM may highlight the cat's face, ears, and body instead of highlighting unrelated background regions.

9.29.4 Comparison

Table 9.14: Comparison of Gabor filters, saliency maps, and Grad-CAM.

Method	**Main Purpose**	**Works On**	**Output Interpretation**
Gabor filter	Detects edges, ridges, and textures at specific orientations and frequencies.	Input image.	Shows where a selected handcrafted visual pattern appears.
Saliency map	Explains which input pixels affect a prediction.	Input pixels and model gradients.	Shows pixel-level importance for a target class.
Grad-CAM	Explains which image regions support a CNN class prediction.	Convolutional feature maps and gradients.	Shows region-level class activation as a heatmap.

9.29.5 Short Memory Summary

A simple way to remember these three methods is:

Gabor Filter → handcrafted edge and texture detector.

Saliency Map → pixel-level importance map.

Grad-CAM → region-level CNN explanation heatmap.

In short, Gabor filters detect visual patterns, saliency maps explain which pixels matter, and Grad-CAM explains which image regions matter for a CNN decision.

9.30 Upsampling in Convolutional Neural Networks

Convolutional neural networks used for dense prediction tasks such as semantic segmentation often contain an encoder that progressively reduces the spatial resolution of the feature maps. While this reduction allows the network to learn increasingly abstract features, dense prediction requires the spatial resolution to be restored in the decoder.

Several operations can be used for this purpose, including interpolation-based upsampling, unpooling, and transposed convolution. Although these methods all increase spatial resolution, they operate differently and should not be treated as equivalent.

Conceptually, the decoder performs the transformation

$$C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}} \quad \longrightarrow \quad C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}},$$

where typically

$$H_{\text{out}} > H_{\text{in}}, \qquad W_{\text{out}} > W_{\text{in}}.$$

9.30.1 Upsampling

Upsampling is the general process of increasing the spatial resolution of a feature map. If an input feature map has spatial dimensions

$$H \times W,$$

upsampling by a scale factor s produces an output with approximate spatial dimensions

$$sH \times sW.$$

Upsampling methods can broadly be divided into two categories:

1. **non-learnable upsampling**, where the enlargement operation itself contains no trainable parameters;
2. **learnable upsampling**, where trainable parameters determine how the higher-resolution representation is constructed.

Common non-learnable methods include nearest-neighbor and bilinear interpolation, whereas transposed convolution is a common learnable upsampling operation.

9.30.2 Non-Learnable Upsampling

Interpolation-based upsampling enlarges a feature map according to a predefined mathematical rule rather than learned parameters.

For example, nearest-neighbor interpolation assigns each new output location the value of the nearest input location. Bilinear interpolation instead calculates each new value from a weighted combination of neighboring input values.

These operations change the spatial dimensions but do not themselves learn how spatial detail should be reconstructed:

$$\text{Low-Resolution Feature Map} \rightarrow \text{Interpolation} \rightarrow \text{Higher-Resolution Feature Map}.$$

A convolutional layer is often applied after interpolation so that the network can learn how to refine the enlarged feature representation:

$$\text{Upsampling} \rightarrow \text{Convolution} \rightarrow \text{Refined Features}.$$

9.30.3 Unpooling

Unpooling is another method for increasing spatial resolution. It is closely related to pooling but does not mathematically invert all information removed by the pooling operation.

Consider max pooling applied to a small feature region. During max pooling, only the maximum value is retained:

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \xrightarrow{\text{Max Pool}} 4.$$

If the position of the maximum value is stored during pooling, max unpooling can later place the retained value back into the corresponding spatial location:

$$4 \xrightarrow{\text{Unpool}} \begin{bmatrix} 0 & 0 \\ 0 & 4 \end{bmatrix}.$$

The remaining positions are commonly filled with zeros. Therefore, unpooling restores the spatial dimensions but does not recover the values that were discarded during pooling.

The process can be represented as

$$\text{Pooling} \rightarrow \text{Store Maximum Indices} \rightarrow \text{Unpooling Using Stored Indices}.$$

Networks such as encoder–decoder segmentation architectures can use these pooling indices to preserve information about the locations of strong activations during spatial reconstruction.

9.30.4 Learnable Upsampling

Learnable upsampling refers to operations in which trainable parameters participate directly in constructing the higher-resolution feature map.

Unlike fixed interpolation, where the enlargement rule is predetermined, learnable upsampling allows the network to optimize the reconstruction operation together with the rest of the model.

A common implementation is transposed convolution:

$$\text{Low-Resolution Features} \rightarrow \text{Transposed Convolution} \rightarrow \text{Learned Higher-Resolution Features}.$$

The kernel weights of the transposed convolution are updated during backpropagation. Consequently, the network can learn feature transformations that are useful for the target task.

9.30.5 Transposed Convolution

Transposed convolution is a learnable operation commonly used to increase the spatial dimensions of feature maps. It uses trainable convolution kernels, but the spatial mapping differs from that of a conventional convolution.

A conventional convolution typically maps a larger spatial representation to a smaller one:

$$H_{\text{in}} \times W_{\text{in}} \rightarrow H_{\text{out}} \times W_{\text{out}},$$

whereas a transposed convolution can be configured to perform

$$H_{\text{in}} \times W_{\text{in}} \rightarrow H_{\text{out}} \times W_{\text{out}}, \qquad H_{\text{out}} > H_{\text{in}}, \quad W_{\text{out}} > W_{\text{in}}.$$

One intuitive way to understand a transposed convolution with stride greater than one is that the input activations are spatially expanded and the learned kernel contributions are distributed over the output feature map. Overlapping kernel contributions are accumulated.

The important point is that the convolution kernel remains trainable:

$$K = \begin{bmatrix} k_{11} & k_{12} & \cdots \\ k_{21} & k_{22} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix},$$

and its values are learned during model training.

For a transposed convolution with input size H_{in}, stride S, padding P, kernel size K, dilation D, and output padding P_{out}, the general output-size relation is

$$H_{\text{out}} = (H_{\text{in}} - 1)S - 2P + D(K - 1) + P_{\text{out}} + 1. \tag{9.28}$$

For the common case

$$D = 1, \qquad P_{\text{out}} = 0,$$

this simplifies to

$$H_{\text{out}} = (H_{\text{in}} - 1)S - 2P + K. \tag{9.29}$$

The same calculation applies independently to the width.

9.30.6 Why It Is Sometimes Called Deconvolution

Transposed convolution is sometimes referred to as *deconvolution* in deep-learning literature. However, this terminology can be misleading.

A true deconvolution operation generally refers to recovering an unknown signal from a signal that has previously undergone convolution. Transposed convolution does not, in general, invert a previous convolution and does not reconstruct the original input exactly.

The term *transposed convolution* is more precise because the operation is related to the transpose of the linear transformation associated with a conventional convolution.

Therefore,

$$\boxed{\text{Transposed Convolution} \neq \text{Mathematical Inverse of Convolution}}$$

even though the term *deconvolution* is sometimes informally used to describe it.

9.30.7 Unpooling versus Transposed Convolution

Unpooling and transposed convolution can both increase spatial resolution, but their mechanisms are fundamentally different.

Table 9.15: Comparison of common spatial upsampling operations in CNN decoders.

Method	**Learnable Parameters**	**Main Mechanism**	**Interpretation**
Nearest-neighbor upsampling	No	Copies nearby values	Simple fixed spatial enlargement.
Bilinear upsampling	No	Interpolates neighboring values	Produces a smoother fixed-resolution enlargement.
Unpooling	No	Places pooled values back into larger spatial locations	Can use stored pooling indices to restore activation locations.
Transposed convolution	Yes	Applies trainable kernels while increasing spatial resolution	Allows the network to learn the upsampling transformation.

9.30.8 Conceptual Comparison

The major difference among these operations can be summarized as

$$\begin{aligned} \text{Interpolation}: &\quad \text{fixed enlargement rule,} \\ \text{Unpooling}: &\quad \text{restore spatial locations,} \\ \text{Transposed Convolution}: &\quad \text{learn spatial upsampling.} \end{aligned}$$

Thus, *upsampling* is the general objective, whereas interpolation, unpooling, and transposed convolution are different mechanisms that can be used to achieve increased spatial resolution.

Chapter 10

Image Segmentation

10.1 Motivation

Image segmentation is one of the most important tasks in computer vision. Unlike image classification, where the model predicts one label for the whole image, segmentation predicts labels at the pixel level. This makes segmentation useful when we need to know not only what objects are present, but also where they are located.

For example, in an autonomous driving image, classification may tell us that the image contains cars, roads, sidewalks, and pedestrians. Segmentation goes further and assigns each pixel to a class such as road, building, vehicle, person, sky, or vegetation.

Image segmentation is widely used in medical imaging, autonomous driving, robotics, remote sensing, industrial inspection, additive manufacturing, and defect detection.

10.2 What is Image Segmentation?

Image segmentation is the process of dividing an image into meaningful regions. In deep learning, this is usually formulated as a pixel-wise prediction problem.

Given an input image

$$X \in \mathbb{R}^{C \times H \times W},$$

a segmentation model predicts an output mask

$$Y \in \{0, 1, \ldots, K-1\}^{H \times W},$$

where C is the number of image channels, H is the height, W is the width, and K is the number of semantic classes.

Each pixel (i, j) receives a class label:

$$Y(i, j) = k,$$

where k is one of the possible class indices.

For example, if the classes are

$$\{\text{background}, \text{road}, \text{car}, \text{person}\},$$

then each pixel is assigned one of these class labels.

10.3 Classification, Detection, and Segmentation

Image classification, object detection, and image segmentation are related tasks, but they provide different levels of information.

Table 10.1: Comparison of image classification, object detection, and image segmentation.

Task	**Model Output**	**Example Question**	**Main Use**
Image classification	One or more labels for the whole image.	Is there a car in the image?	Image-level recognition.
Object detection	Bounding boxes and class labels.	Where are the cars?	Object localization.
Image segmentation	Pixel-wise class labels or object masks.	Which pixels belong to the car?	Precise region understanding.

In short, classification gives an image-level answer, detection gives object-level boxes, and segmentation gives pixel-level regions.

10.4 Types of Image Segmentation

There are three common types of image segmentation:

- **Semantic segmentation**
- **Instance segmentation**
- **Panoptic segmentation**

Each type answers a slightly different question.

10.4.1 Semantic Segmentation

Semantic segmentation assigns a class label to every pixel. Pixels belonging to the same class receive the same label, even if they belong to different object instances.

For example, if an image contains three cars, semantic segmentation labels all car pixels as "car." It does not separate car 1, car 2, and car 3.

Mathematically, semantic segmentation predicts

$$Y(i, j) \in \{0, 1, \ldots, K-1\},$$

where each pixel receives one semantic class label.

Semantic segmentation is useful for scene understanding, road detection, land-cover mapping, medical tissue classification, and defect region segmentation.

10.4.2 Instance Segmentation

Instance segmentation separates individual object instances. It assigns both a class label and an instance identity to each object.

For example, if an image contains three cars, instance segmentation produces three separate masks:

$$\text{car}_1, \quad \text{car}_2, \quad \text{car}_3.$$

Unlike semantic segmentation, instance segmentation distinguishes between different objects of the same class.

Instance segmentation is useful when we need object-level counting, separation, and precise localization. Mask R-CNN is a common deep learning model for instance segmentation.

10.4.3 Panoptic Segmentation

Panoptic segmentation combines semantic segmentation and instance segmentation. It assigns every pixel a semantic class label and, for countable objects, an instance ID.

Panoptic segmentation separates classes into two groups:

- **Things:** countable objects, such as cars, people, bicycles, and animals.
- **Stuff:** amorphous background regions, such as sky, road, grass, water, and wall.

For example, panoptic segmentation can label road pixels as "road" and also separate three different cars as "car 1," "car 2," and "car 3."

10.5 Comparison of Segmentation Types

Table 10.2: Comparison of semantic, instance, and panoptic segmentation.

Segmentation Type	**Pixel Label**	**Separates Instances?**	**Example**
Semantic segmentation	Class label only.	No.	All car pixels are labeled as car.
Instance segmentation	Object mask and class label.	Yes.	Each car has its own mask.
Panoptic segmentation	Class label and instance ID when needed.	Yes, for countable objects.	Road is labeled as road, and each car is separated.

10.6 Segmentation as Pixel-Wise Classification

Semantic segmentation can be viewed as classification at every pixel. Instead of predicting one class for the whole image, the network predicts a class distribution for each pixel.

For an image of size $H \times W$ with K classes, the model usually produces logits

$$Z \in \mathbb{R}^{K \times H \times W}.$$

For each pixel location (i, j), the model outputs K class scores:

$$Z(:, i, j) = [z_1(i, j), z_2(i, j), \ldots, z_K(i, j)].$$

The predicted class is usually obtained using the argmax operation:

$$\hat{Y}(i, j) = \arg\max_k Z_k(i, j).$$

Therefore, segmentation can be understood as many small classification problems, one for each pixel.

10.7 Segmentation Masks

A segmentation mask is the ground-truth label image used to train or evaluate a segmentation model. It has the same spatial size as the input image, but instead of RGB values, it contains class IDs.

For example, a mask may use the following encoding:

Table 10.3: Example class encoding for a segmentation mask.

Class ID	**Class Name**	**Example Pixels**
0	Background	Unknown or ignored regions.
1	Road	Road surface pixels.
2	Vehicle	Car, truck, or bus pixels.
3	Person	Pedestrian pixels.

The mask is not usually treated as a normal RGB image during training. It is treated as a class-index image.

10.8 Important Note About Mask Resizing

Input images and segmentation masks are often resized before training. However, they must be resized differently.

RGB images can be resized using bilinear interpolation because image values are continuous:

$$\text{image resize} \rightarrow \text{bilinear interpolation.}$$

Segmentation masks must be resized using nearest-neighbor interpolation because mask values are discrete class IDs:

$$\text{mask resize} \rightarrow \text{nearest-neighbor interpolation.}$$

Using bilinear interpolation on masks can create invalid class values. For example, if class 1 and class 2 are averaged, the result may be 1.5, which is not a valid class label.

Table 10.4: Correct resizing methods for images and segmentation masks.

Data Type	**Recommended Resize Method**	**Reason**
RGB image	Bilinear interpolation.	Pixel intensities are continuous.
Segmentation mask	Nearest-neighbor interpolation.	Class labels are discrete integers.

10.9 Encoder-Decoder Structure

Many segmentation networks use an encoder-decoder structure.

The **encoder** gradually reduces spatial resolution while increasing feature depth. It extracts high-level features from the input image.

The **decoder** gradually increases spatial resolution and produces a dense pixel-wise prediction.

A simple segmentation pipeline is:

$$\text{Input Image} \to \text{Encoder} \to \text{Bottleneck} \to \text{Decoder} \to \text{Segmentation Mask}.$$

The encoder learns what is present in the image, and the decoder learns where it is located.

10.10 TikZ Flow Diagram of Image Segmentation

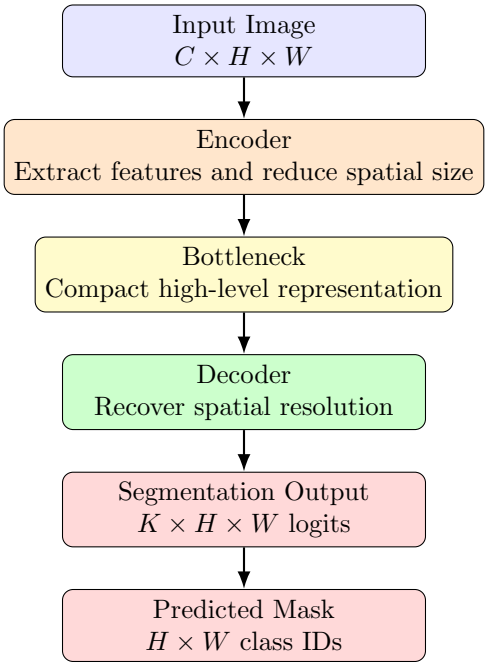


Figure 10.1: A simple encoder-decoder flow for image segmentation.

10.11 Fully Convolutional Networks

A Fully Convolutional Network, or FCN, is one of the earliest deep learning architectures for semantic segmentation. The main idea is to replace fully connected layers with convolutional layers so that the network can produce spatial output maps.

In image classification, CNNs often end with fully connected layers that produce one vector of class scores. In segmentation, we need one prediction for every pixel. Therefore, FCN keeps the spatial structure and produces dense predictions.

A simplified FCN structure is:

$$\text{CNN Feature Extractor} \to \text{Convolutional Classifier} \to \text{Upsampling} \to \text{Pixel-wise Prediction}.$$

FCNs are important because they showed that classification CNNs can be adapted for dense prediction tasks.

10.12 U-Net

U-Net is a popular encoder-decoder architecture for image segmentation. It was originally designed for biomedical image segmentation, but it is now widely used in many segmentation tasks.

The main feature of U-Net is the use of **skip connections** between the encoder and decoder. These skip connections transfer high-resolution spatial details from the encoder to the decoder.

The encoder captures semantic information, while the decoder reconstructs the segmentation mask. The skip connections help preserve fine boundary details.

A simplified U-Net flow is:

$$\text{Input} \rightarrow \text{Encoder} \rightarrow \text{Bottleneck} \rightarrow \text{Decoder with Skip Connections} \rightarrow \text{Mask}.$$

10.13 TikZ Flow Diagram of U-Net

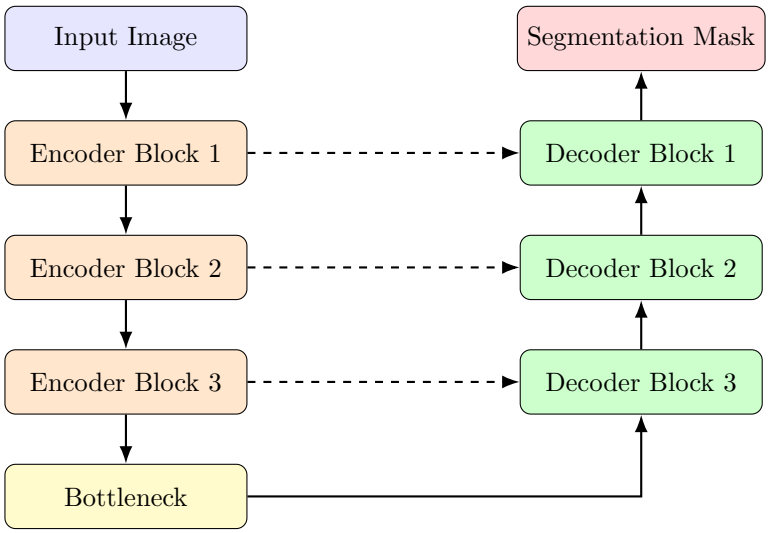


Figure 10.2: A simplified U-Net architecture. Dashed arrows represent skip connections from encoder blocks to decoder blocks.

10.14 Segmentation Loss Functions

A segmentation model is trained by comparing the predicted mask with the ground-truth mask. Since segmentation is pixel-wise prediction, the loss is usually computed over all pixels.

10.14.1 Pixel-Wise Cross-Entropy Loss

For multi-class semantic segmentation, pixel-wise cross-entropy is commonly used. For each pixel, the model predicts a probability distribution over K classes.

The loss can be written as

$$\mathcal{L}_{CE} = -\frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} \sum_{k=1}^{K} Y_{ijk} \log(\hat{P}_{ijk}),$$

where Y_{ijk} is the one-hot encoded ground-truth label and $\hat{P}_{ijk}$ is the predicted probability for class k at pixel (i, j).

10.14.2 Dice Loss

Dice loss is based on the Dice coefficient, which measures overlap between the predicted mask and the ground-truth mask.

For binary segmentation, the Dice coefficient is

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|},$$

where A is the predicted region and B is the ground-truth region.

Dice loss is usually written as

$$\mathcal{L}_{Dice} = 1 - \frac{2\sum_i p_i g_i + \epsilon}{\sum_i p_i + \sum_i g_i + \epsilon},$$

where p_i is the predicted probability, g_i is the ground-truth label, and ϵ is a small constant used to avoid division by zero.

Dice loss is useful when the target region is small compared with the background.

10.14.3 Focal Loss

Focal Loss is a modification of cross-entropy designed to reduce the contribution of easy, well-classified examples and place greater emphasis on difficult or misclassified examples. It is particularly useful when the training data are highly imbalanced, such as when background pixels or instances greatly outnumber foreground examples.

For binary classification, define

$$p_t = \begin{cases} p_i, & y_i = 1, \\ 1 - p_i, & y_i = 0, \end{cases} \tag{10.1}$$

where $p_i \in [0, 1]$ is the predicted foreground probability and $y_i \in \{0, 1\}$ is the corresponding ground-truth label.

Focal Loss is defined as

$$\mathcal{L}_{\text{Focal}} = -\frac{1}{N} \sum_{i=1}^{N} \alpha_t (1 - p_t)^\gamma \log(p_t), \tag{10.2}$$

where $\gamma \geq 0$ is the focusing parameter and α_t is an optional weighting factor used to address class imbalance.

The factor

$$(1 - p_t)^\gamma \tag{10.3}$$

reduces the contribution of examples that are already classified correctly with high confidence. When p_t approaches one, this factor approaches zero, causing easy examples to contribute less to the total loss. In contrast, difficult examples with smaller p_t retain a larger contribution.

When

$$\gamma = 0,$$

the modulating factor becomes one, and Focal Loss reduces to weighted cross-entropy. Increasing γ places progressively greater emphasis on hard examples.

10.14.4 IoU Loss

Intersection over Union, or IoU, measures the overlap between prediction and ground truth:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}.$$

IoU loss is commonly written as

$$\mathcal{L}_{IoU} = 1 - \text{IoU}.$$

IoU is strict because it penalizes both false positives and false negatives.

10.15 Segmentation Evaluation Metrics

Segmentation performance is usually evaluated using pixel-level and region-level metrics.

10.15.1 Pixel Accuracy

Pixel accuracy measures the fraction of correctly classified pixels:

$$\text{Pixel Accuracy} = \frac{\text{Number of correctly classified pixels}}{\text{Total number of pixels}}.$$

Pixel accuracy is easy to understand, but it can be misleading when the background class dominates the image.

10.15.2 Intersection over Union

Intersection over Union compares the overlap between the predicted region and the ground-truth region:

$$\text{IoU} = \frac{TP}{TP + FP + FN},$$

where TP is true positive pixels, FP is false positive pixels, and FN is false negative pixels. For multi-class segmentation, IoU is computed for each class and then averaged:

$$\text{mIoU} = \frac{1}{K} \sum_{k=1}^{K} \text{IoU}_k.$$

Mean IoU is one of the most common metrics for semantic segmentation.

10.15.3 Dice Score

The Dice score measures overlap between prediction and ground truth:

$$\text{Dice} = \frac{2TP}{2TP + FP + FN}.$$

Dice is closely related to IoU, but it gives slightly different weighting to the overlap.

10.15.4 Precision, Recall, and F1 Score

Precision measures how many predicted positive pixels are correct:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Recall measures how many ground-truth positive pixels are detected:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

The F1 score is the harmonic mean of precision and recall:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

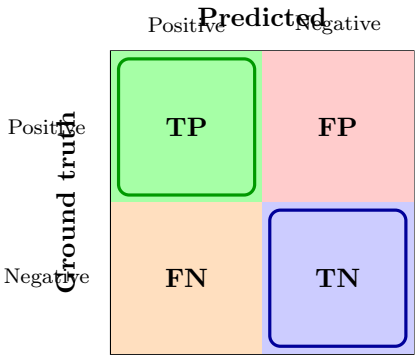
These metrics are useful when studying class imbalance and error type.

10.16 Common Segmentation Metrics Summary

Table 10.5: Common evaluation metrics for image segmentation.

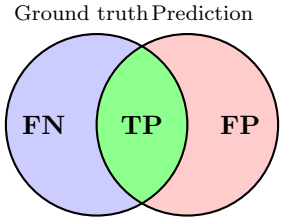
Metric	**Formula**	**Interpretation**
Pixel accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Measures overall pixel correctness. Can be misleading with class imbalance.
IoU	$\frac{TP}{TP+FP+FN}$	Measures overlap between predicted and ground-truth regions.
Mean IoU	$\frac{1}{K}\sum_{k=1}^{K} \text{IoU}_k$	Averages IoU across classes. Common for semantic segmentation.
Dice score	$\frac{2TP}{2TP+FP+FN}$	Measures region overlap. Useful for small objects or imbalanced masks.
Precision	$\frac{TP}{TP+FP}$	Measures how many predicted positive pixels are correct.
Recall	$\frac{TP}{TP+FN}$	Measures how many ground-truth positive pixels are found.
F1 score	$\frac{2PR}{P+R}$	Balances precision and recall.

10.17 Visual Interpretation of Common Segmentation Metrics



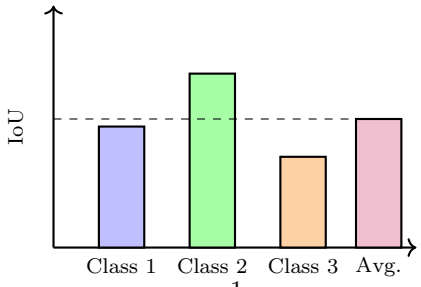
$$\text{Pixel Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

(a) Pixel accuracy counts all correctly classified pixels, including both foreground and background.



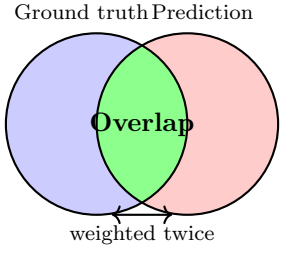
$$\text{IoU} = \frac{TP}{TP + FP + FN}$$

(b) IoU measures the overlap between prediction and ground truth divided by their union.



$$\text{Mean IoU} = \frac{1}{K} \sum_{k=1}^{K} \text{IoU}_k$$

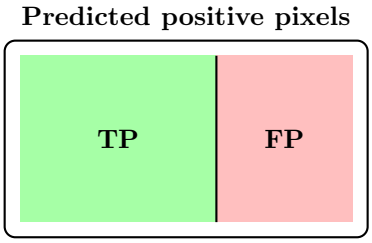
(c) Mean IoU averages the IoU values over all classes, giving equal importance to each class.



$$\text{Dice} = \frac{2TP}{2TP + FP + FN}$$

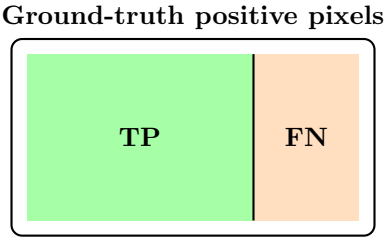
(d) Dice score emphasizes the overlap region more strongly and is often useful for small objects.

Figure 10.3: Conceptual visualizations of pixel accuracy, IoU, mean IoU, and Dice score used in image segmentation.



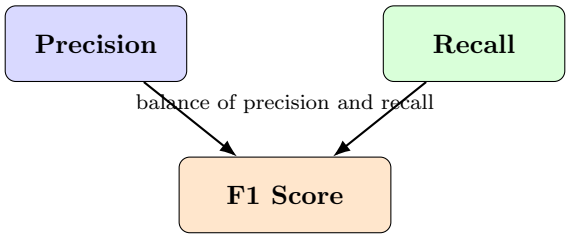
$$\text{Precision} = \frac{TP}{TP + FP}$$

(a) Precision asks: among the pixels predicted as positive, how many are actually correct?



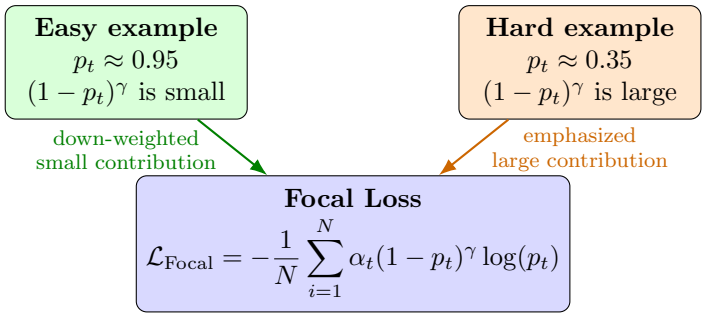
$$\text{Recall} = \frac{TP}{TP + FN}$$

(b) Recall asks: among the true positive pixels, how many were successfully found by the model?



$$\text{F1} = \frac{2PR}{P + R}$$

(c) F1 score combines precision and recall into a single balanced metric.



Focal loss reduces the influence of easy examples and focuses training on hard examples.

(d) Focal loss emphasizes difficult examples and down-weights easy, well-classified examples.

Figure 10.4: Conceptual visualizations of precision, recall, F1 score, and Focal Loss.

10.18 Precision–Recall Curve, Average Precision, and Mean Average Precision

Precision and recall evaluated at a single operating threshold provide only a partial description of segmentation performance. In many segmentation and object-detection models, the predicted output is associated with a confidence score or probability. Changing the confidence threshold changes the number of predictions classified as positive and consequently changes both precision and recall. The precision–recall (PR) curve characterizes this trade-off over a range of confidence thresholds.

10.18.1 Precision–Recall Curve

For a given confidence threshold, precision and recall are defined as

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{10.4}$$

and

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{10.5}$$

where TP, FP, and FN denote the numbers of true positives, false positives, and false negatives, respectively.

As the confidence threshold is varied, a sequence of precision–recall pairs is obtained. These points form the PR curve, with recall typically shown on the horizontal axis and precision on the vertical axis. A model with strong performance maintains high precision while achieving high recall, causing the curve to remain close to the upper-right region of the plot.

A high confidence threshold generally produces fewer positive predictions. This can reduce false positives and increase precision, but it may also increase false negatives and reduce recall. Conversely, lowering the confidence threshold usually increases recall by accepting more predictions, although additional false positives may reduce precision. Therefore, the PR curve provides a more complete assessment than precision or recall measured at only one threshold.

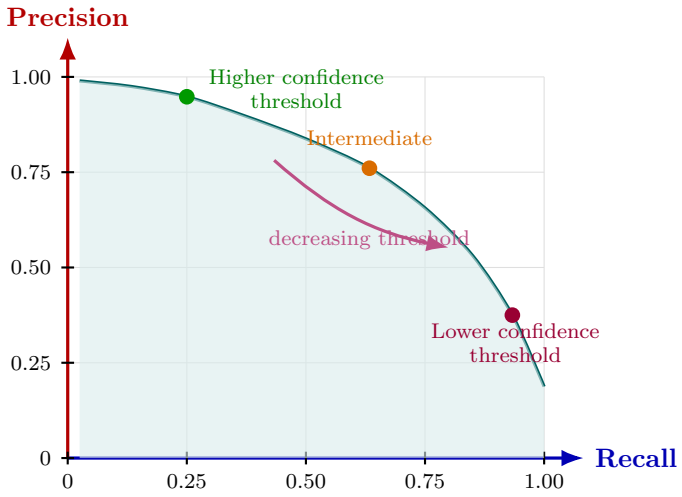


Figure 10.5: Conceptual precision–recall curve obtained by varying the prediction confidence threshold. Stronger models maintain high precision over a broad range of recall values.

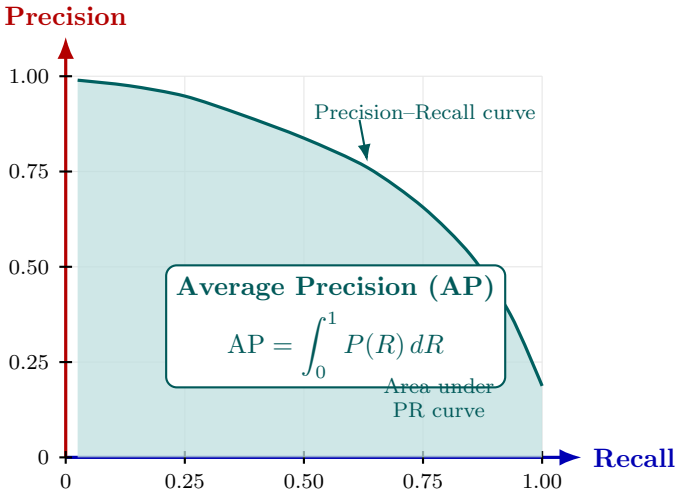


Figure 10.6: Conceptual visualization of Average Precision (AP). AP summarizes the precision–recall relationship as the area under the precision–recall curve. A larger area indicates that high precision is maintained over a broader range of recall values.

10.18.2 Average Precision

Although the PR curve provides detailed information about model behavior, a single numerical value is often desirable for quantitative comparison. Average Precision (AP) summarizes the PR curve by measuring the area under the precision–recall relationship. Conceptually, AP can be expressed as

$$\mathrm{AP} = \int_0^1 P(R)\, dR, \tag{10.6}$$

where $P(R)$ represents precision as a function of recall.

In practice, the PR curve consists of discrete points rather than a continuous function. AP is therefore calculated using numerical or interpolated approximations of the area under the PR curve. A larger AP value indicates that the model maintains relatively high precision across a wide range of recall values.

For instance segmentation and object detection, determining whether a predicted instance is a true positive requires an additional spatial matching criterion. Intersection over Union (IoU) is commonly used for this purpose. For a predicted region B_p and its corresponding ground-truth region B_g,

$$\mathrm{IoU} = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \tag{10.7}$$

A prediction is considered a true positive when its IoU with an appropriate ground-truth instance is greater than or equal to a specified IoU threshold. Otherwise, it may be classified as a false positive. Consequently, AP depends not only on confidence scores but also on the selected IoU criterion.

For example,

$$\mathrm{AP}_{50} \tag{10.8}$$

denotes AP evaluated using an IoU threshold of 0.50, whereas

$$\mathrm{AP}_{75} \tag{10.9}$$

uses the stricter IoU threshold of 0.75. Because a larger IoU threshold requires closer spatial agreement between predicted and ground-truth regions, AP_{75} is generally more demanding than AP_{50}.

10.18.3 Mean Average Precision

When a model predicts multiple object or segmentation classes, AP can be calculated independently for each class. Mean Average Precision (mAP) is then obtained by averaging the class-specific AP values:

$$\mathrm{mAP} = \frac{1}{K} \sum_{k=1}^{K} \mathrm{AP}_k, \tag{10.10}$$

where K is the number of evaluated classes and AP_k is the Average Precision for class k.

For example, if a segmentation problem contains three classes with corresponding AP values AP_1, AP_2, and AP_3, then

$$\mathrm{mAP} = \frac{\mathrm{AP}_1 + \mathrm{AP}_2 + \mathrm{AP}_3}{3}. \tag{10.11}$$

This averaging prevents a class with a large number of instances from automatically dominating the overall evaluation because each evaluated class contributes one AP value to the final mean.

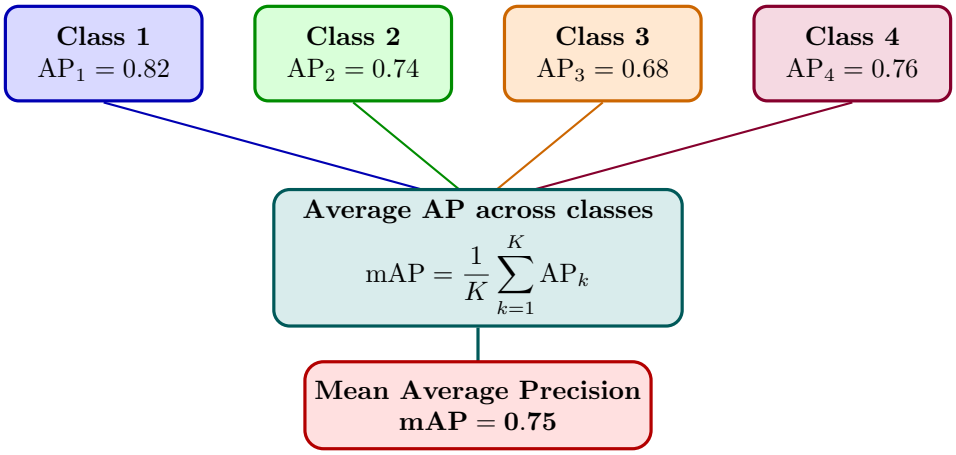


Figure 10.7: Conceptual visualization of mean Average Precision (mAP). Average Precision is first determined independently for each evaluated class, and the class-specific AP values are subsequently averaged to obtain mAP. The numerical values shown are illustrative.

10.18.4 mAP Across Multiple IoU Thresholds

In instance segmentation benchmarks, performance can also be averaged across multiple IoU thresholds. A commonly used evaluation considers IoU thresholds from 0.50 to 0.95 in increments of 0.05:

$$T = \{0.50,\ 0.55,\ 0.60,\ \ldots,\ 0.90,\ 0.95\}. \tag{10.12}$$

For each threshold $t \in T$, AP is evaluated for the individual classes. The resulting AP values are then averaged across the evaluated classes and IoU thresholds. This can be represented conceptually as

$$\text{mAP}_{50:95} = \frac{1}{|T|K} \sum_{t \in T} \sum_{k=1}^{K} \text{AP}_{k,t}, \tag{10.13}$$

where $\text{AP}_{k,t}$ denotes Average Precision for class k evaluated using IoU threshold t.

Unlike mAP_{50}, which accepts predictions having at least 50% IoU with their matched ground-truth instances, $\text{mAP}_{50:95}$ evaluates the model across increasingly strict overlap requirements. It therefore provides a more comprehensive measure of both detection or segmentation accuracy and spatial localization quality.

10.18.5 mAP at Different IoU Thresholds

The subscript associated with mAP specifies the Intersection over Union (IoU) threshold used to determine whether a predicted instance sufficiently overlaps with a corresponding ground-truth instance. For example, mAP_{50} uses an IoU threshold of 0.50, whereas mAP_{90} requires an IoU of at least 0.90. Increasing the IoU threshold therefore imposes a progressively stricter spatial matching requirement.

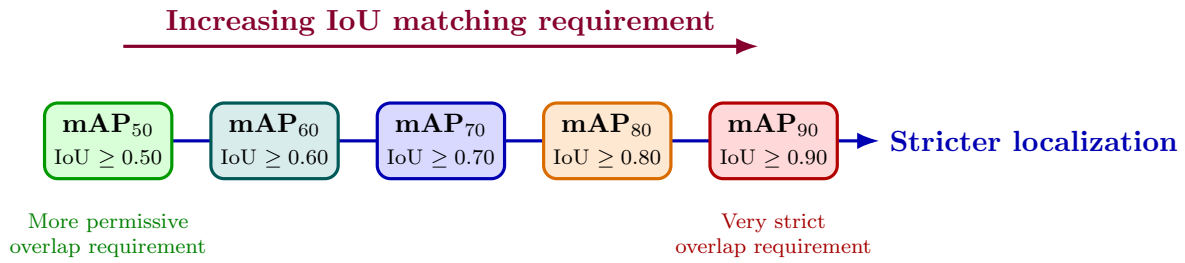


Figure 10.8: Conceptual interpretation of mAP evaluated at different IoU thresholds. Lower IoU thresholds permit greater differences between predicted and ground-truth regions, whereas higher IoU thresholds require increasingly accurate spatial localization.

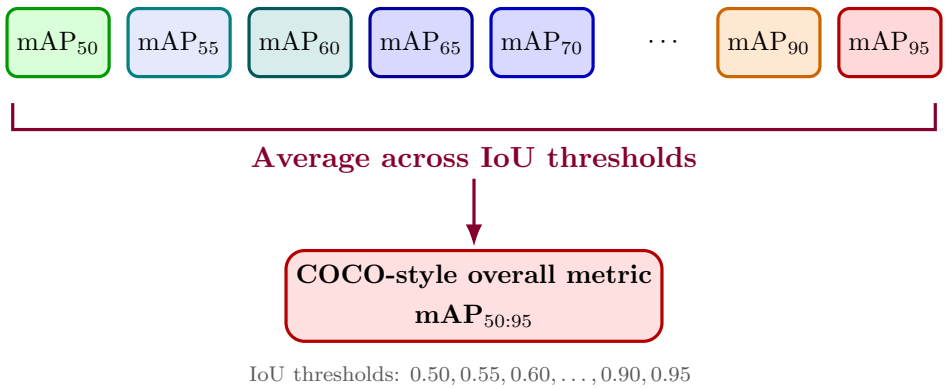


Figure 10.9: Conceptual calculation of $\text{mAP}_{50:95}$. Performance is evaluated at multiple IoU thresholds from 0.50 to 0.95 in increments of 0.05, and the resulting values are averaged to obtain an overall measure that incorporates both permissive and stringent localization requirements.

10.18.6 Relationship Among PR Curve, AP, and mAP

The relationship among these metrics can be summarized as follows:

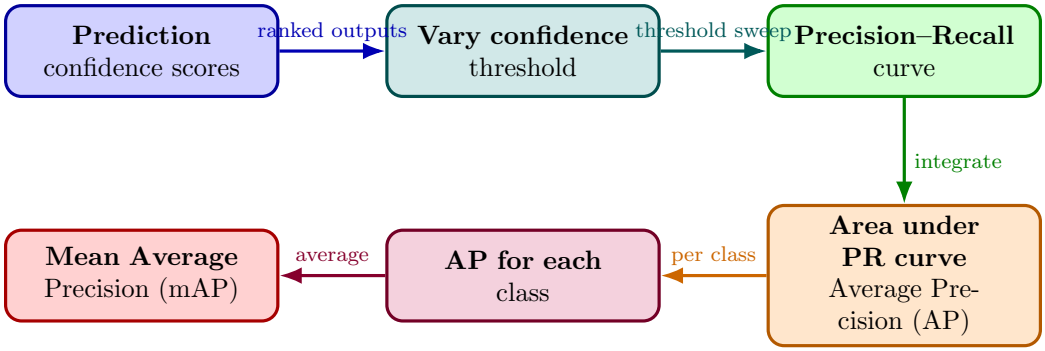


Figure 10.10: Conceptual relationship among confidence thresholds, the precision–recall curve, Average Precision, and mean Average Precision.

10.18.7 Non-Maximum Suppression

Object detection and instance segmentation models often generate multiple predictions for the same underlying object. These predictions may have similar bounding boxes or masks but different confidence scores. If all overlapping predictions are retained, a single object can be counted multiple times, which increases the number of false-positive detections. Non-Maximum Suppression (NMS) is a post-processing procedure used to remove such redundant predictions while retaining the most confident detection.

NMS operates by ranking all predicted detections according to their confidence scores. The prediction with the highest confidence score is retained first. The overlap between this prediction and the remaining predictions is then evaluated, commonly using Intersection over Union (IoU):

$$\mathrm{IoU}(B_i, B_j) = \frac{|B_i \cap B_j|}{|B_i \cup B_j|}, \tag{10.14}$$

where B_i and B_j represent two predicted bounding boxes. A remaining prediction is suppressed if its IoU with the retained prediction exceeds a predefined NMS threshold τ_{NMS}. The procedure is repeated with the highest-confidence prediction among those that remain until all predictions have either been retained or suppressed.

The basic NMS procedure can therefore be summarized as follows:

1. Sort all candidate predictions in descending order according to their confidence scores.
2. Select the prediction with the highest confidence score and retain it.
3. Compute the IoU between the retained prediction and each remaining prediction.
4. Suppress predictions satisfying
$$\mathrm{IoU}(B_{\mathrm{selected}}, B_i) > \tau_{\mathrm{NMS}}.$$
5. Repeat the procedure using the highest-confidence prediction among the remaining candidates.

The NMS threshold controls how aggressively overlapping predictions are removed. A relatively low threshold suppresses predictions even when their overlap is moderate, whereas a high threshold allows substantially overlapping predictions to remain. Consequently, an excessively low threshold may remove valid detections corresponding to nearby objects, while an excessively high threshold may retain multiple predictions for the same object.

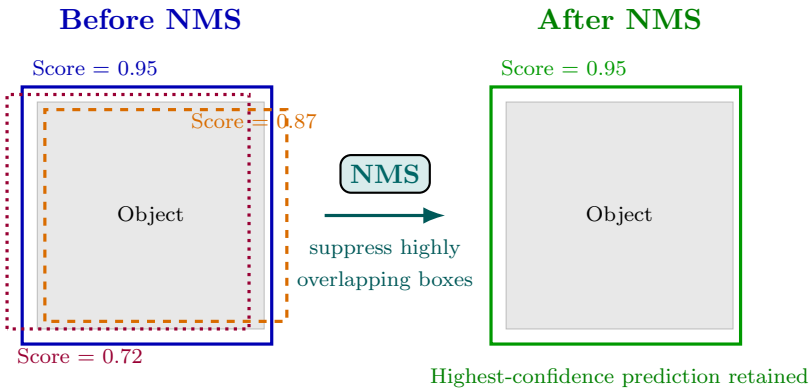


Figure 10.11: Conceptual illustration of Non-Maximum Suppression. Multiple highly overlapping predictions may initially correspond to the same object. NMS retains the highest-confidence prediction and suppresses redundant predictions whose overlap exceeds the specified NMS threshold.

The distinction between the confidence threshold and the NMS threshold is important. The confidence threshold determines whether an individual prediction has sufficient confidence to be considered for further processing, whereas the NMS threshold determines whether two surviving predictions overlap sufficiently for one of them to be removed. Therefore, the two thresholds serve different purposes:

$$s_i \geq \tau_{\text{conf}} \tag{10.15}$$

determines whether prediction i passes the confidence filtering stage, whereas

$$\text{IoU}(B_i, B_j) > \tau_{\text{NMS}} \tag{10.16}$$

determines whether sufficiently overlapping predictions should be suppressed.
A typical prediction pipeline can consequently be represented as

$$\text{Model Predictions} \rightarrow \text{Confidence Filtering} \rightarrow \text{NMS} \rightarrow \text{Final Predictions}. \tag{10.17}$$

For multi-class detection and instance segmentation, NMS is generally applied on a class-wise basis so that overlapping detections belonging to different classes are not automatically suppressed solely because of their spatial overlap. This is particularly important when objects from different classes occupy similar image regions.

NMS also affects downstream evaluation metrics such as precision, recall, Average Precision, and mean Average Precision. Retaining redundant detections can increase the number of false positives and therefore reduce precision and AP. Conversely, overly aggressive suppression can remove valid detections, increase false negatives, and reduce recall. The selected NMS threshold therefore influences the balance between eliminating duplicate predictions and preserving distinct nearby objects.

10.18.8 Conditional Random Field for Segmentation Refinement

A Conditional Random Field (CRF) is a probabilistic graphical model that can be used to refine the output of a semantic segmentation network. A CNN typically predicts the class probability of each pixel primarily from learned image features. However, neighboring pixels are not independent. Pixels that are spatially close and visually similar often belong to the same semantic region, whereas pixels separated by a strong image boundary may belong to different classes.

A CRF incorporates these relationships into the final labeling process. Therefore, a common segmentation pipeline can be represented as

$$\text{Input Image} \rightarrow \text{CNN} \rightarrow \text{Initial Segmentation} \rightarrow \text{CRF} \rightarrow \text{Refined Segmentation}.$$

The CNN provides the initial semantic evidence, while the CRF encourages spatial consistency and better alignment of the predicted labels with image boundaries.

CRF Energy Function

Let

$$\mathbf{x} = (x_1, x_2, \ldots, x_N)$$

represent the class labels assigned to N pixels. A common CRF formulation defines an energy function

$$E(\mathbf{x}) = \sum_{i=1}^{N} \psi_u(x_i) + \sum_{i<j} \psi_p(x_i, x_j), \tag{10.18}$$

where

- $\psi_u(x_i)$ is the **unary potential**, which measures how strongly the segmentation network supports assigning label x_i to pixel i;
- $\psi_p(x_i, x_j)$ is the **pairwise potential**, which models the relationship between the labels assigned to pixels i and j.

The final segmentation corresponds to a labeling with low CRF energy:

$$\mathbf{x}^* = \arg\min_{\mathbf{x}} E(\mathbf{x}). \tag{10.19}$$

Unary Potential

The unary potential is usually derived from the class probabilities predicted by the CNN. If the network predicts probability

$$P_i(c)$$

that pixel i belongs to class c, the unary potential can be expressed conceptually as

$$\psi_u(x_i = c) = -\log P_i(c). \tag{10.20}$$

Therefore, a class assigned a high probability by the CNN produces a small unary penalty, whereas a class having low predicted probability produces a larger penalty.

Conceptually,

$$\boxed{\text{Unary Potential} \rightarrow \text{What class does the CNN think this pixel belongs to?}}$$

Pairwise Potential

The pairwise potential introduces relationships between different pixels. It can encourage nearby pixels with similar appearance to receive the same class label while allowing different labels across strong image boundaries.

Conceptually,

Pairwise Potential → Should these pixels receive similar labels?

The pairwise term can depend on information such as

- spatial distance between pixels,
- color or intensity similarity,
- image edges or boundaries, and
- compatibility between semantic labels.

Thus, two nearby pixels with similar color may be strongly encouraged to have the same label. In contrast, two pixels separated by a strong intensity or color boundary may be allowed to receive different labels.

Dense CRF

A Dense CRF extends this idea by allowing interactions between many, or potentially all, pairs of pixels rather than only immediately neighboring pixels. A commonly used pairwise formulation can be written as

$$\psi_p(x_i, x_j) = \mu(x_i, x_j) \sum_m w_m k^{(m)} \left(\mathbf{f}_i, \mathbf{f}_j\right), \tag{10.21}$$

where

- $\mu(x_i, x_j)$ is a label compatibility function;
- w_m controls the contribution of the m-th pairwise kernel;
- $k^{(m)}(\mathbf{f}_i, \mathbf{f}_j)$ measures similarity between pixels using features such as spatial coordinates and image intensity or color.

The combination of spatial and appearance information allows a Dense CRF to smooth predictions within homogeneous regions while preserving boundaries between visually different regions.

Visual Interpretation of CRF Refinement

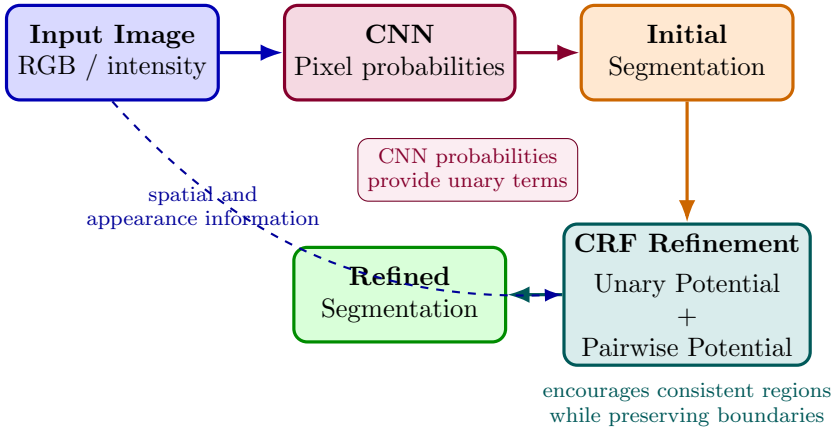


Figure 10.12: Conceptual use of a Conditional Random Field for semantic segmentation. The CNN produces an initial pixel-wise prediction that supplies unary evidence. The CRF combines this evidence with spatial and appearance relationships between pixels through pairwise potentials to produce a spatially refined segmentation.

Conceptual Pixel-Level Example

Consider an initial prediction in which one pixel inside a homogeneous road region is incorrectly classified:

$$\begin{array}{cccc} \text{Road} & \text{Road} & \text{Road} & \text{Road} \\ \text{Road} & \text{Road} & \text{Car} & \text{Road} \\ \text{Road} & \text{Road} & \text{Road} & \text{Road} \end{array}$$

If the incorrectly classified pixel is spatially surrounded by road pixels and has similar appearance, the pairwise CRF terms can favor a consistent road label. The refined prediction may therefore become

$$\begin{array}{cccc} \text{Road} & \text{Road} & \text{Road} & \text{Road} \\ \text{Road} & \text{Road} & \text{Road} & \text{Road} \\ \text{Road} & \text{Road} & \text{Road} & \text{Road}. \end{array}$$

However, the objective of a CRF is not simply to smooth every prediction. Appearance-sensitive pairwise terms can preserve genuine class boundaries when neighboring pixels differ substantially in color, intensity, or other image features.

Role of CRF in Semantic Segmentation

The roles of the CNN and CRF can be summarized as

CNN :	learn semantic evidence for individual pixels,
CRF :	model relationships among pixel labels,
CNN + CRF :	semantic prediction + spatial refinement.

CRFs have historically been used as post-processing components in semantic segmentation systems to improve spatial consistency and boundary localization. Modern segmentation architectures can often learn strong spatial refinement directly through decoder networks, skip

connections, multi-scale features, attention mechanisms, or boundary-aware objectives. Nevertheless, CRFs remain an important example of structured prediction and provide a useful framework for understanding how relationships among neighboring pixel labels can be incorporated into segmentation.

10.18.9 Summary

Table 10.6: Summary of precision–recall-based evaluation metrics.

Metric	**Definition**	**Interpretation**
Precision–Recall curve	Precision plotted against recall while varying the confidence threshold	Shows the trade-off between false-positive control and the ability to recover ground-truth positives across different operating thresholds.
Average Precision (AP)	AP $\approx$ area under the PR curve	Summarizes precision–recall performance for a particular class and evaluation criterion using a single numerical value.
AP_{50}	AP evaluated at IoU ≥ 0.50	Uses a relatively permissive spatial-overlap criterion for matching predictions to ground-truth instances.
AP_{75}	AP evaluated at IoU ≥ 0.75	Uses a stricter spatial-overlap requirement and therefore provides greater emphasis on localization or mask accuracy.
Mean Average Precision (mAP)	$\frac{1}{K}\sum_{k=1}^{K} AP_k$	Averages AP across the evaluated classes.
$mAP_{50:95}$	Average AP over IoU thresholds $0.50 : 0.05 : 0.95$	Evaluates performance across both relatively permissive and highly stringent spatial-overlap requirements.

It is important to distinguish mAP from mean Intersection over Union (mIoU). mIoU directly evaluates class-wise spatial overlap and is widely used for semantic segmentation, where every pixel receives a class label. In contrast, AP and mAP incorporate ranked prediction confidence and are particularly important for object detection and instance segmentation, where individual predicted instances must first be matched with corresponding ground-truth instances. Thus, although both metric families use IoU in segmentation evaluation, they quantify different aspects of model performance.

10.19 Typical Segmentation Training Pipeline

A typical deep learning pipeline for semantic segmentation includes the following steps:

1. Load image and corresponding segmentation mask.
2. Resize the image and mask using appropriate interpolation methods.
3. Apply data augmentation, such as flipping, cropping, rotation, or color changes.
4. Convert the image into a tensor.

5. Keep the mask as integer class labels.
6. Pass the image through the segmentation model.
7. Compute the segmentation loss.
8. Backpropagate the loss and update model parameters.
9. Evaluate the predicted masks using metrics such as mIoU, Dice, and pixel accuracy.

10.20 Common Challenges in Image Segmentation

Image segmentation is more difficult than image classification because it requires pixel-level accuracy. Common challenges include:

- **Class imbalance:** Some classes may occupy many pixels, while others appear rarely.
- **Boundary ambiguity:** Object boundaries may be unclear or fuzzy.
- **Small objects:** Small regions may disappear after downsampling.
- **Occlusion:** Objects may be partially hidden.
- **Annotation noise:** Pixel-level labels may contain errors.
- **High memory cost:** Large images require significant GPU memory.

10.21 Applications of Image Segmentation

Image segmentation is used in many real-world applications.

Table 10.7: Example applications of image segmentation.

Application Area	**Example Use**
Medical imaging	Tumor segmentation, organ segmentation, tissue classification.
Autonomous driving	Road, vehicle, pedestrian, sidewalk, lane, and traffic object segmentation.
Robotics	Scene understanding, object manipulation, navigation.
Remote sensing	Land-cover mapping, building detection, vegetation analysis.
Industrial inspection	Crack detection, defect segmentation, surface anomaly localization.
Additive manufacturing	Melt-pool region extraction, defect localization, thermal region segmentation.

10.22 Short Memory Summary

Image segmentation can be remembered as pixel-level image understanding.

$$\text{Classification} \rightarrow \text{What is in the image?}$$

$$\text{Detection} \rightarrow \text{Where are the objects?}$$

Segmentation → Which pixels belong to each object or class?

Semantic segmentation gives class labels to pixels. Instance segmentation separates individual objects. Panoptic segmentation combines both ideas.

A segmentation model usually follows this structure:

Image → Feature extraction → Upsampling → Pixel-wise class prediction.

In short, image segmentation turns a computer vision model from an image-level classifier into a pixel-level scene understanding system.

Chapter 11

Vision Transformer (ViT)

11.1 Overview

The Vision Transformer (ViT) is a neural architecture that applies the Transformer model (originally developed for natural language processing) to image understanding tasks. Instead of relying on convolutional operations, ViT represents an image as a sequence of embedded patches and processes this sequence using stacked self-attention layers.

Unlike convolutional neural networks (which impose locality and translation equivariance through kernel design), ViT relies on global self-attention to model long-range spatial relationships. This design choice introduces different scaling behaviors in memory usage and computational cost, which must be carefully considered when ViT is used as a backbone in detection or segmentation frameworks such as Mask R-CNN.

11.2 Architectural View of ViT

The Vision Transformer (ViT) follows a flat image-token architecture. The input image is first divided into non-overlapping patches. Each patch is flattened and linearly projected into a token embedding. After adding positional information, the full sequence of image tokens is processed by a stack of Transformer encoder layers.

The defining architectural feature of ViT is that self-attention is applied globally. In each Transformer layer, every patch token can attend to every other patch token. Therefore, long-range spatial relationships can be modeled directly, even between distant image regions. However, this global attention also leads to quadratic growth in computational cost with respect to the number of image tokens.

For an image divided into N patch tokens, the attention matrix has size:

$$N \times N. \tag{11.1}$$

Thus, increasing the input resolution or decreasing the patch size can substantially increase the memory and computational cost of ViT.

The flat structure of ViT can be summarized as:

$$\text{Image} \to \text{Patches} \to \text{Patch Tokens} \to \text{Global Transformer Encoder} \to \text{Output}.$$

In this design, the token resolution remains mostly unchanged across the encoder stack:

$$N \to N \to N \to \cdots \to N.$$

This flat representation is simple and powerful, but it does not naturally produce a multi-scale feature hierarchy. For dense prediction tasks such as object detection, semantic segmentation, and instance segmentation, additional architectural components are often required to obtain multi-scale features.

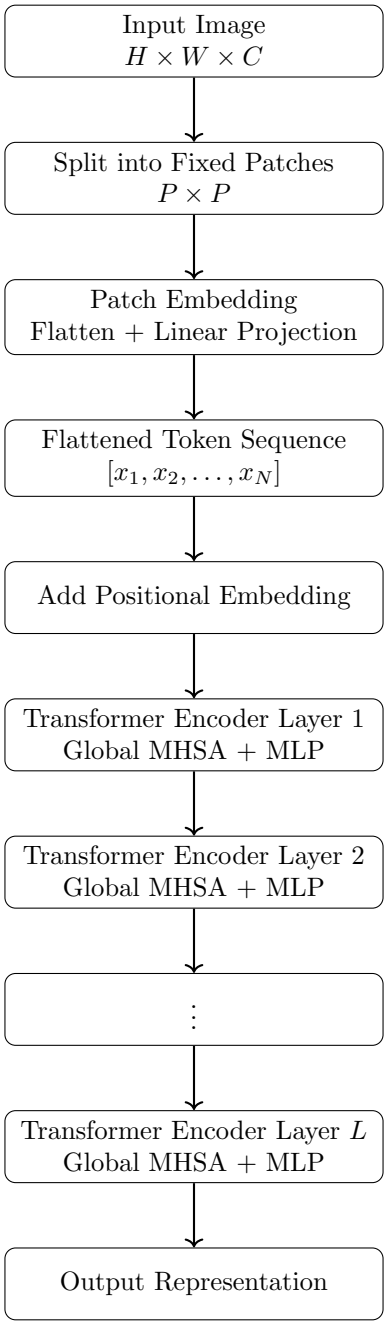


Figure 11.1: Architectural flow of the Vision Transformer. ViT converts the image into a flat sequence of patch tokens and applies global self-attention across all tokens in every encoder layer.

11.3 Patch Embedding and Tokenization

Given an input image of size $H \times W \times C$ (height $\times$ width $\times$ channels), ViT first partitions the image into non-overlapping square patches of size $P \times P$. Each patch is flattened and projected into a fixed-dimensional embedding space.

The number of patch tokens is given by:

$$N_{\text{patch}} = \frac{H}{P} \cdot \frac{W}{P} \tag{11.2}$$

A special learnable classification token (`[CLS]`) is prepended to the patch sequence, yielding the total number of tokens:

$$N = N_{\text{patch}} + 1 \tag{11.3}$$

Each token is represented as a vector in $\mathbb{R}^D$, where D is the embedding dimension.

11.4 Transformer Depth and Attention Modules

A Vision Transformer consists of a stack of identical Transformer layers. Each layer contains exactly one multi-head self-attention (MHSA) module followed by a feed-forward multilayer perceptron (MLP).

- The number of **attention modules** is equal to the Transformer depth L.
- Each attention module processes the full token sequence.
- Depth is a user-defined architectural hyperparameter and is independent of token count, embedding dimension, and number of attention heads.

Thus:

$$\text{Number of attention modules} = L \tag{11.4}$$

11.5 Multi-Head Self-Attention

Within a single attention module, multi-head self-attention projects the token embeddings into queries, keys, and values, which are then split across multiple attention heads.

Let:

- D be the embedding dimension
- h be the number of attention heads

The per-head dimension is:

$$d_h = \frac{D}{h}, \quad \text{with } D \bmod h = 0 \tag{11.5}$$

For each head, attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) V \tag{11.6}$$

The attention matrix for each head has size:

$$N \times N \tag{11.7}$$

This quadratic dependence on the number of tokens is the primary source of memory and computational cost in ViT.

11.6 Computational Scaling

The dominant computational and memory terms in ViT arise from self-attention and MLP layers.

11.6.1 Attention Cost

Self-attention scales as:

$$\mathcal{O}(N^2 \cdot D) \tag{11.8}$$

This quadratic scaling with respect to the number of tokens makes patch size and input resolution critical design parameters.

11.6.2 MLP Cost

Each Transformer layer contains an MLP with hidden dimension:

$$D_{\text{MLP}} = r \cdot D \tag{11.9}$$

where r is the MLP expansion ratio, typically set to 4.

The MLP cost scales as:

$$\mathcal{O}(N \cdot D \cdot D_{\text{MLP}}) \tag{11.10}$$

11.7 Parameter Impact Summary

Table 11.1 summarizes the role and impact of key Vision Transformer parameters.

Parameter	**Controls**	**Primary Impact**
Patch size P	Number of tokens N	Quadratic attention memory and compute
Embedding dim D	Token feature width	Linear memory, quadratic MLP cost
Depth L	Number of attention modules	Linear scaling in memory and compute
Heads h	Attention parallelism	Minor overhead, $D \bmod h = 0$
MLP ratio r	Feed-forward width	Significant compute increase
Image resolution $H \times W$	Token count	Quadratic attention scaling

Table 11.1: Impact of Vision Transformer architectural parameters.

11.8 Numerical Example

11.8.1 Small Example: 16×16 Image

Consider a small input image of size 16×16 with patch size $P = 2$. The number of patches along the height and width is:

$$\frac{16}{2} \times \frac{16}{2} = 8 \times 8. \tag{11.11}$$

Therefore, the number of image patches is:

$$N_{\text{patch}} = 8 \times 8 = 64. \tag{11.12}$$

Since each patch is represented as one token, the number of image tokens is:

$$N = 64. \tag{11.13}$$

In ViT, self-attention is global. Therefore, every token attends to every other token. The attention matrix per head has size:

$$N \times N = 64 \times 64. \tag{11.14}$$

Equivalently, using the patch grid notation:

$$(8 \times 8)(8 \times 8) = 4096. \tag{11.15}$$

Thus, for a 16×16 image with 2×2 patches, ViT produces 64 image tokens and computes 4096 pairwise attention scores per head.

If a classification token is added, the total number of tokens becomes:

$$N = 64 + 1 = 65, \tag{11.16}$$

and the attention matrix size becomes:

$$65 \times 65 = 4225. \tag{11.17}$$

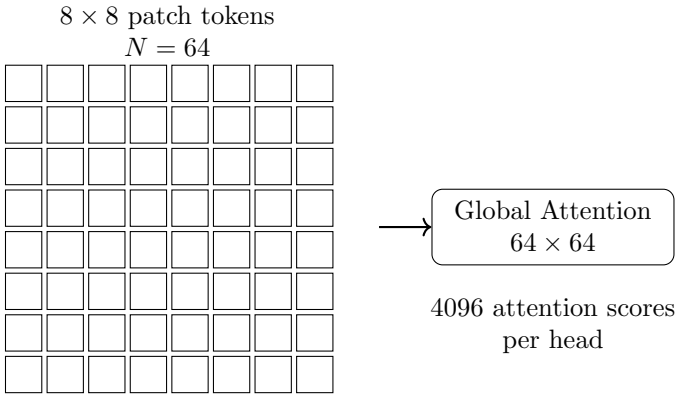


Figure 11.2: Small ViT attention example. A 16×16 image with 2×2 patches produces an 8×8 token grid. Since ViT uses global self-attention, the attention matrix has size 64×64.

For a standard image classification example, consider an input image of size 224×224 with patch size $P = 16$.

$$N_{\text{patch}} = (224/16)^2 = 14^2 = 196 \tag{11.18}$$

$$N = 196 + 1 = 197 \tag{11.19}$$

For an embedding dimension $D = 192$ and $h = 3$ heads:

$$d_h = 192/3 = 64 \tag{11.20}$$

The attention matrix size per layer has size:

$$h \cdot N^2 = 3 \times 197^2 = 116{,}427 \tag{11.21}$$

If the patch size is reduced to $P = 8$, the number of tokens increases to $N = 785$, and the attention matrix size increases to approximately 1.85 million entries per layer, illustrating the quadratic scaling effect.

11.9 Implications for Detection and Segmentation

When used as a backbone in detection or segmentation frameworks, ViT must often process images at higher resolutions than standard classification benchmarks. As a result, token count and attention memory become the dominant constraints.

Practical deployments typically rely on:

- Larger patch sizes
- Reduced depth
- Windowed or local attention mechanisms
- Hierarchical token merging

These modifications preserve the representational benefits of self-attention while keeping memory usage within feasible limits.

11.10 Summary

Vision Transformers replace convolutional inductive biases with global self-attention over patch tokens. The number of attention modules is determined solely by the Transformer depth, while computational cost is dominated by the quadratic dependence on the number of tokens. Understanding these scaling relationships is essential when integrating ViT into large-scale vision systems such as Mask R-CNN.

Parameter	**Controls**	**Compute / Memory Scaling**	**Effect on Representation**
MLP ratio r	Hidden width of MLP (rD)	$\mathcal{O}(N \cdot rD^2)$ compute, $\mathcal{O}(N \cdot rD)$ activations	Channel-wise capacity (no token mixing)

Table 11.2: Impact of the MLP ratio in Vision Transformers.

The MLP ratio determines the width of the channel-wise feed-forward network applied independently to each token, increasing parameter count, compute, and activation memory linearly while leaving token–token interactions unchanged, which are governed exclusively by self-attention. In contrast to self-attention, which mixes information across tokens, the MLP operates only along the feature dimension and therefore controls expressive capacity rather than spatial interaction.

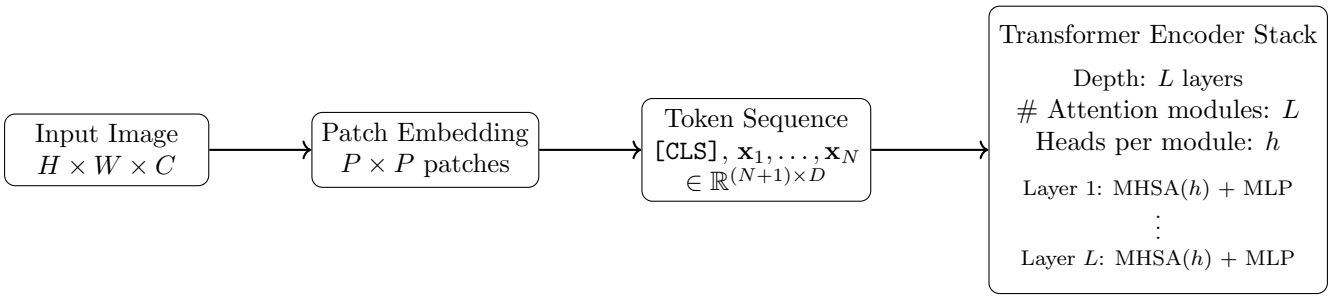


Figure 11.3: High-level architecture of the Vision Transformer (ViT). An input image is split into non-overlapping patches, embedded into a token sequence, and processed by a Transformer encoder stack of depth L. Each encoder layer contains exactly one multi-head self-attention (MHSA) module (so the number of attention modules is L), and each MHSA uses h attention heads.

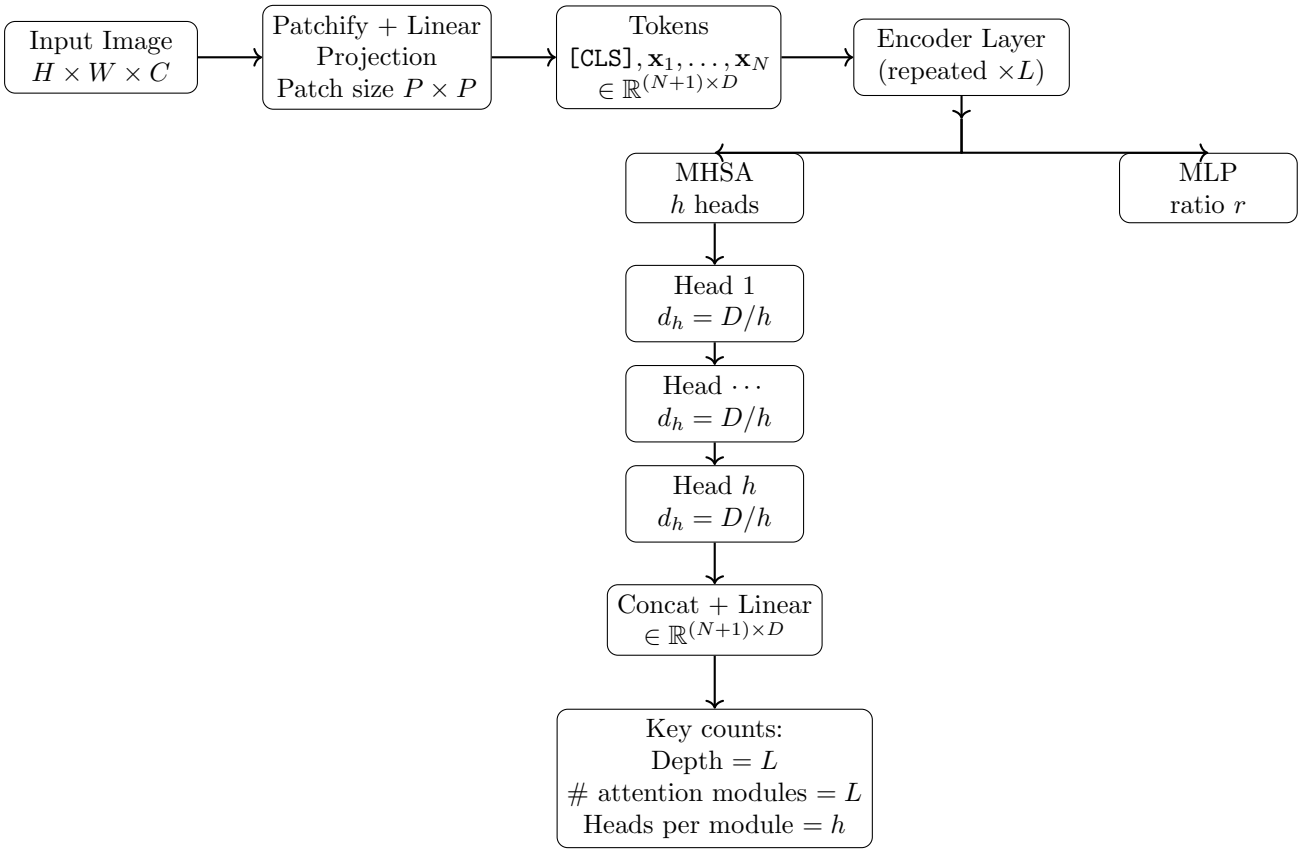


Figure 11.4: Hydra/tree-style view of ViT without overflow. Tokens pass through an encoder layer repeated $\times L$. Each layer contains exactly one MHSA (so the number of attention modules is L) with h heads and one MLP block. The MHSA splits into h heads and merges back via concatenation to dimension D.

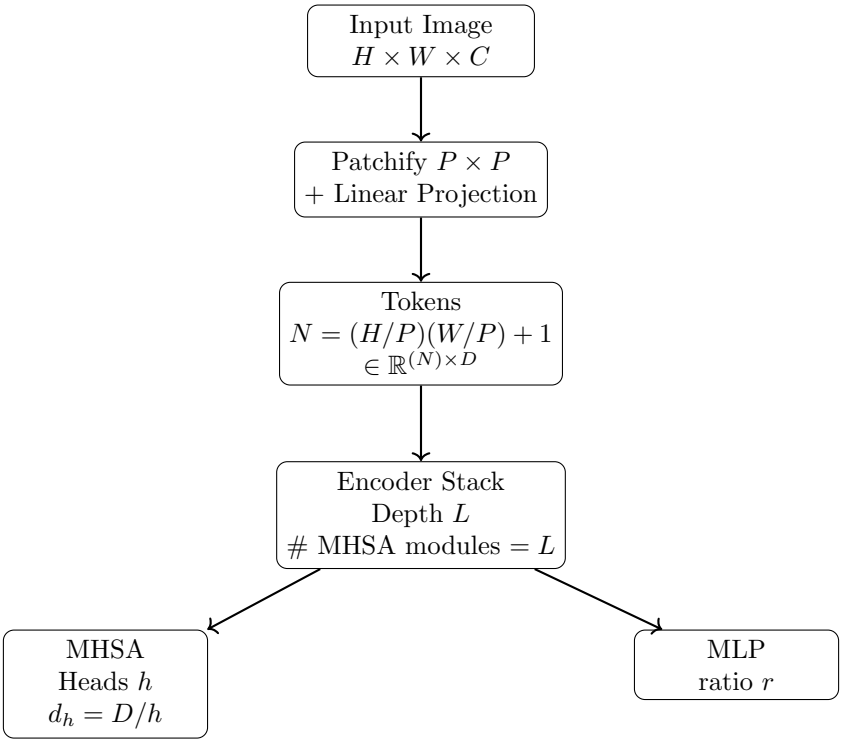


Figure 11.5: Tree-style view of ViT. The encoder stack has depth L; each layer contains one MHSA (therefore total attention modules $= L$) with h heads and an MLP block.

Chapter 12

Swin Transformer

12.1 Overview

The Swin Transformer (Shifted Window Transformer) is a hierarchical Vision Transformer architecture designed for image understanding tasks. Unlike the original Vision Transformer, which applies global self-attention over the full token sequence, Swin Transformer restricts self-attention to local non-overlapping windows. This design substantially reduces computational complexity while preserving the representational power of attention-based models.

A key feature of Swin Transformer is the use of **shifted windows**. In alternating Transformer blocks, the window partition is shifted so that tokens in neighboring windows can interact across block boundaries. This enables cross-window information flow without incurring the full quadratic cost of global attention.

In addition, Swin Transformer is **hierarchical**: feature maps are progressively downsampled and channel dimensions are increased through patch merging. This makes Swin especially suitable as a backbone for dense prediction tasks, including object detection, semantic segmentation, and instance segmentation.

12.2 Architectural View of Swin Transformer

The Swin Transformer modifies the standard Vision Transformer architecture by replacing global self-attention with local window-based self-attention. Instead of flattening all image patches into a single sequence and applying attention over the entire image, Swin keeps the patch tokens arranged as a two-dimensional feature map. Self-attention is then computed inside local windows.

This design has two main architectural consequences. First, the attention computation becomes more efficient because each token attends only to tokens within its local window. Second, Swin can build a hierarchical representation through patch merging, where spatial resolution is reduced while the channel dimension is increased.

A typical Swin Transformer backbone has four stages:

$$\frac{H}{4} \times \frac{W}{4} \to \frac{H}{8} \times \frac{W}{8} \to \frac{H}{16} \times \frac{W}{16} \to \frac{H}{32} \times \frac{W}{32}.$$

At the same time, the channel dimension usually increases:

$$C \to 2C \to 4C \to 8C.$$

Therefore, Swin behaves more like a convolutional backbone than a flat Transformer. This makes it suitable for detection, segmentation, and other dense prediction tasks.

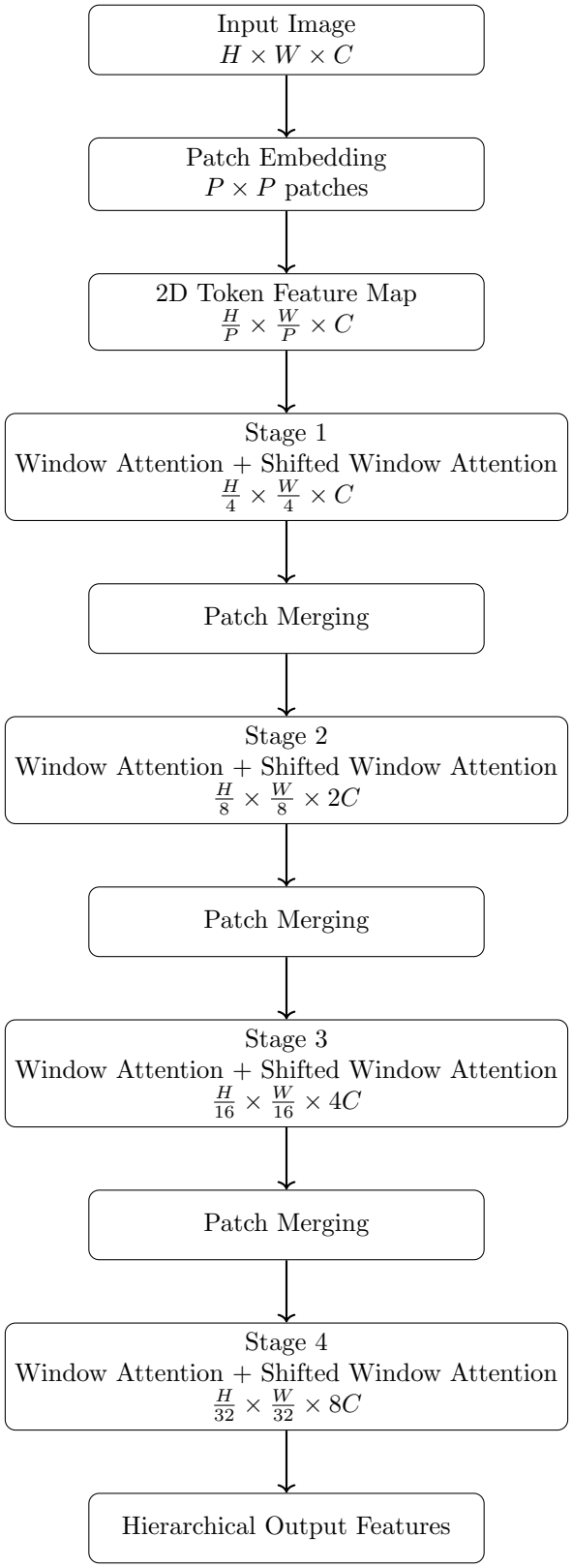


Figure 12.1: Architectural flow of the Swin Transformer. Swin keeps image tokens as a two-dimensional feature map, applies attention inside local windows, uses shifted windows for cross-window interaction, and builds a hierarchical feature representation through patch merging.

The Swin architecture can be summarized as:

Image → Patch Embedding → Window Attention → Shifted Window Attention → Patch Merging → Hierarch

The main difference from ViT is that Swin does not apply global attention over all patch tokens at every layer. Instead, it alternates between regular window attention and shifted window attention. Regular window attention models local relationships inside each window, while shifted window attention allows information to pass across neighboring windows.

Thus:

$$\text{ViT} = \text{global attention on a flat token sequence},$$

whereas

$$\text{Swin} = \text{local window attention on a hierarchical token feature map.}$$

12.3 Patch Embedding and Initial Tokenization

Given an input image of size $H \times W \times C$, Swin Transformer first partitions the image into non-overlapping patches, typically of size $P \times P$. Each patch is flattened and projected into an embedding space of dimension D.

The number of initial patch tokens is:

$$N_0 = \frac{H}{P} \cdot \frac{W}{P} \tag{12.1}$$

Unlike standard ViT, Swin Transformer does **not** prepend a classification token. Instead, all tokens remain arranged on a 2D spatial grid, which is preserved throughout the hierarchy.

The embedded token tensor can be viewed as:

$$\mathbf{X}_0 \in \mathbb{R}^{\frac{H}{P} \times \frac{W}{P} \times D} \tag{12.2}$$

12.4 Hierarchical Stage Structure

A Swin Transformer consists of multiple stages, typically four. Each stage contains a stack of Swin Transformer blocks, and between consecutive stages, a patch merging layer reduces spatial resolution while increasing channel dimension.

If stage s has spatial dimensions $H_s \times W_s$ and channel dimension D_s, then patch merging reduces spatial size by a factor of 2 in each spatial direction:

$$H_{s+1} = \frac{H_s}{2}, \qquad W_{s+1} = \frac{W_s}{2} \tag{12.3}$$

and typically doubles the channel dimension:

$$D_{s+1} = 2D_s \tag{12.4}$$

Thus, token count decreases by a factor of 4 after each merging step:

$$N_{s+1} = \frac{N_s}{4} \tag{12.5}$$

This pyramid-like hierarchy makes Swin Transformer behave more like a convolutional backbone than a flat Transformer.

12.5 Swin Transformer Blocks

Each Swin Transformer block contains:

- Layer normalization
- Window-based multi-head self-attention
- A feed-forward MLP
- Residual connections

Within a stage, blocks are arranged in alternating pairs:

- **W-MSA**: Window-based multi-head self-attention over regular non-shifted windows
- **SW-MSA**: Shifted-window multi-head self-attention over shifted windows

This alternating structure enables local processing and cross-window communication.

12.6 Window-Based Multi-Head Self-Attention

Suppose a stage has feature map size $H_s \times W_s$ with channel dimension D_s. The feature map is partitioned into non-overlapping windows of size $M \times M$, where M is the window size hyperparameter.

The number of windows in that stage is:

$$N_{\text{win}} = \frac{H_s}{M} \cdot \frac{W_s}{M} \tag{12.6}$$

Each window contains:

$$N_w = M^2 \tag{12.7}$$

tokens.

Self-attention is computed only within each window, not across the full feature map. For one attention head, the attention operation within a window is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + B\right)V \tag{12.8}$$

where:

- $d_h = D_s/h$ is the per-head dimension
- h is the number of attention heads
- B is the relative position bias

The attention matrix for each window has size:

$$N_w \times N_w = M^2 \times M^2 \tag{12.9}$$

Because M is fixed and small, the attention cost depends linearly on the number of windows rather than quadratically on the total number of tokens.

12.7 Shifted Window Mechanism

If attention were always computed within the same fixed windows, tokens near window boundaries would never interact with tokens in adjacent windows. Swin Transformer addresses this by shifting the window partition by

$$\left(\frac{M}{2}, \frac{M}{2}\right) \tag{12.10}$$

in alternating blocks.

For a window size of $M \times M$, the shift size is commonly chosen as

$$\left\lfloor \frac{M}{2} \right\rfloor \times \left\lfloor \frac{M}{2} \right\rfloor.$$

For example, if $M = 2$, the shift is 1×1; if $M = 7$, the shift is commonly 3×3. The shifted window block keeps the same number of attention scores as the regular window block, but it changes which neighboring tokens are placed in the same local attention window.

This shifted-window strategy allows neighboring regions to exchange information across layers while keeping the attention computation localized.

To maintain efficient batched computation, shifted windows are implemented together with an attention mask that prevents invalid interactions across wrapped boundaries.

12.8 Patch Merging

Between stages, Swin Transformer uses patch merging to reduce spatial resolution and increase feature richness.

In patch merging, each group of neighboring 2×2 tokens is concatenated:

$$4D_s \rightarrow \text{Linear Projection} \rightarrow 2D_s \tag{12.11}$$

Thus, the spatial dimensions are halved:

$$H_{s+1} = \frac{H_s}{2}, \qquad W_{s+1} = \frac{W_s}{2} \tag{12.12}$$

while the channel dimension increases:

$$D_{s+1} = 2D_s \tag{12.13}$$

Patch merging serves a role analogous to pooling or strided convolution in convolutional networks.

12.9 Computational Scaling

The dominant computational terms in Swin Transformer arise from windowed self-attention and MLP layers.

12.9.1 Windowed Attention Cost

If the stage has $N_s = H_s W_s$ spatial positions and window size $M \times M$, then self-attention is computed independently inside windows of fixed size M^2. The attention cost scales as:

$$\mathcal{O}(N_s \cdot M^2 \cdot D_s) \tag{12.14}$$

Because M is fixed, this is effectively linear in the number of spatial positions. This is a major advantage over global ViT attention, which scales as:

$$\mathcal{O}(N_s^2 \cdot D_s) \tag{12.15}$$

12.9.2 MLP Cost

As in other Transformer architectures, each block contains an MLP with hidden dimension:

$$D_{\text{MLP}} = r \cdot D_s \tag{12.16}$$

where r is the MLP expansion ratio.

The MLP cost scales as:

$$\mathcal{O}(N_s \cdot D_s \cdot D_{\text{MLP}}) \tag{12.17}$$

12.10 Parameter Impact Summary

Table 12.1 summarizes the primary role of key Swin Transformer parameters.

Parameter	Controls	Primary Impact
Patch size P	Initial token count	Initial spatial resolution and compute
Embedding dim D	Feature width	Linear memory, MLP cost
Stage depths $\{L_s\}$	Number of blocks per stage	Linear scaling in compute and memory
Heads h	Attention parallelism	Per-head partitioning, $D_s \bmod h = 0$
Window size M	Tokens per local attention region	Quadratic cost inside each window
MLP ratio r	Feed-forward width	Significant compute increase
Patch merging	Downsampling	Reduces token count, increases channels

Table 12.1: Impact of Swin Transformer architectural parameters.

12.11 Numerical Example

12.11.1 Small Example: 16×16 Image

Consider a small input image of size 16×16 with patch size $P = 2$. The initial token grid is:

$$\frac{16}{2} \times \frac{16}{2} = 8 \times 8. \tag{12.18}$$

Thus, the number of image tokens is:

$$N = 8 \times 8 = 64. \tag{12.19}$$

Now suppose the Swin window size is $M = 2$, so each local window has size:

$$2 \times 2. \tag{12.20}$$

Each window therefore contains:

$$M^2 = 2 \times 2 = 4 \tag{12.21}$$

tokens.

Since the token grid is 8×8, the number of non-overlapping windows is:

$$\frac{8}{2} \times \frac{8}{2} = 4 \times 4 = 16. \tag{12.22}$$

Inside each window, self-attention is computed only among the 4 tokens in that window. Therefore, the attention matrix inside one window has size:

$$(2 \times 2)(2 \times 2) = 4 \times 4 = 16. \tag{12.23}$$

Across all windows, the total number of attention scores per head is:

$$(4 \times 4) \times (2 \times 2)(2 \times 2) = 16 \times 16 = 256. \tag{12.24}$$

Therefore, for this example, Swin computes 256 local attention scores per head in one window-attention block. In contrast, global ViT attention over the same 64 tokens would require:

$$64 \times 64 = 4096 \tag{12.25}$$

attention scores per head.

The shifted-window block uses the same window size. For $M = 2$, the shift size is:

$$\left(\frac{M}{2}, \frac{M}{2}\right) = (1, 1). \tag{12.26}$$

Thus, regular window attention and shifted-window attention have the same number of attention scores:

$$\text{W-MSA} = 256, \qquad \text{SW-MSA} = 256. \tag{12.27}$$

The shift does not increase the size of the attention matrix. Instead, it changes which neighboring tokens are grouped into the same window. Consequently, one pair of consecutive Swin blocks has:

$$\text{W-MSA} + \text{SW-MSA} = 256 + 256 = 512 \tag{12.28}$$

attention scores per head.

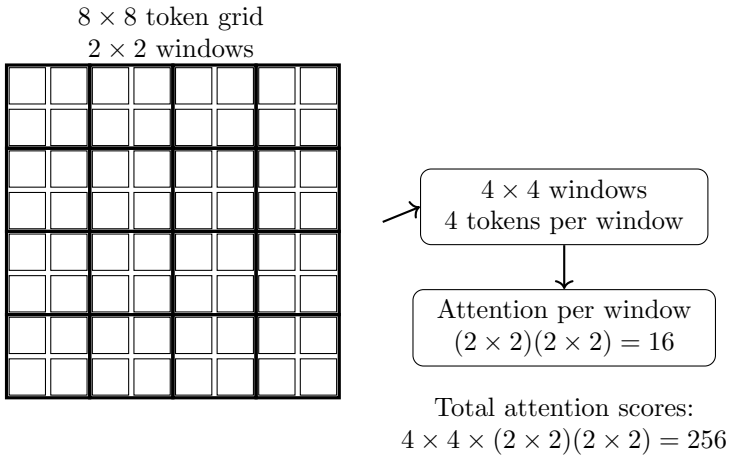


Figure 12.2: Small Swin Transformer attention example. A 16×16 image with 2×2 patches produces an 8×8 token grid. With a 2×2 window size, the grid contains 4×4 windows, and attention is computed locally inside each window.

For a standard Swin Transformer example, consider an input image of size 224×224 with patch size $P = 4$. The initial token grid is:

$$\frac{224}{4} \times \frac{224}{4} = 56 \times 56 \tag{12.29}$$

so the initial number of tokens is:

$$N_0 = 56^2 = 3136 \tag{12.30}$$

Suppose the window size is $M = 7$. Then each window contains:

$$N_w = 7^2 = 49 \tag{12.31}$$

tokens, and the number of windows is:

$$N_{\text{win}} = \frac{56}{7} \cdot \frac{56}{7} = 8 \cdot 8 = 64 \tag{12.32}$$

Therefore, each attention matrix per head has size:

$$49 \times 49 = 2401 \tag{12.33}$$

and across all windows:

$$64 \times 49^2 = 153{,}664 \tag{12.34}$$

entries per head.

This is far smaller than global attention over all 3136 tokens, which would require:

$$3136^2 = 9{,}834{,}496 \tag{12.35}$$

entries per head.

After one patch merging step, the grid becomes:

$$28 \times 28 \tag{12.36}$$

with channel dimension doubled. This illustrates how Swin Transformer progressively reduces spatial cost across stages.

12.12 Implications for Detection and Segmentation

Swin Transformer is well suited for dense vision tasks because its hierarchical design naturally produces multi-scale feature maps. This is essential in frameworks such as:

- Mask R-CNN
- UPerNet
- semantic segmentation pipelines
- feature pyramid networks

Compared with flat ViT backbones, Swin Transformer provides:

- lower memory usage
- better scaling to high-resolution inputs
- native multi-scale representations
- local and cross-window contextual modeling

These properties make Swin Transformer a practical Transformer backbone for large-scale vision systems.

12.13 Architectural Comparison with ViT

Although both ViT and Swin Transformer are attention-based image architectures, their internal organization is different. ViT treats the image as a flat sequence of patch tokens and applies global self-attention across all tokens. Swin Transformer keeps the image tokens in a two-dimensional structure, applies attention locally inside windows, shifts the windows to enable cross-window interaction, and progressively builds a hierarchical feature representation.

The main architectural distinction is therefore:

ViT uses global attention on a flat token sequence, while Swin uses local shifted-window attention in a hiera

Table 12.2: Architectural comparison between ViT and Swin Transformer.

Aspect	**Vision Transformer (ViT)**	**Swin Transformer**
Token structure	Flat sequence of patch tokens	Two-dimensional token feature map
Attention region	Global attention over all tokens	Local attention inside windows
Cross-region interaction	Direct, because every token attends to every other token	Indirect, through shifted windows across consecutive blocks
Spatial hierarchy	Mostly flat; token resolution is usually preserved	Hierarchical; spatial resolution decreases through patch merging
Computational scaling	Expensive for high-resolution inputs because attention grows quadratically with token count	More efficient because attention is restricted to fixed-size windows
Feature representation	Single-scale representation unless additional modules are added	Multi-scale representation produced naturally by the architecture
Similarity to CNN backbones	Less similar to CNNs	More similar to CNN backbones because of staged downsampling
Typical suitability	Image classification and global representation learning	Detection, semantic segmentation, instance segmentation, and dense prediction

12.14 Summary

The Swin Transformer replaces global self-attention with window-based local self-attention and introduces shifted windows to enable cross-window interaction. Through patch merging, it constructs a hierarchical representation with decreasing spatial resolution and increasing feature dimension. This combination of local attention, shifted connectivity, and pyramid structure makes Swin Transformer computationally efficient and highly effective for classification, detection, and segmentation tasks.

Parameter	**Controls**	**Compute / Memory Scaling**	**Effect on Representation**
MLP ratio r	Hidden width of MLP (rD_s)	$\mathcal{O}(N_s \cdot rD_s^2)$ compute, $\mathcal{O}(N_s \cdot rD_s)$ activations	Channel-wise capacity inside each stage; does not alter window partitioning or cross-window connectivity

Table 12.3: Impact of the MLP ratio in Swin Transformer.

The MLP ratio determines the width of the channel-wise feed-forward network applied after window-based attention within each Swin block. Increasing r enlarges parameter count, compute, and activation memory, while token interaction remains governed by the windowed attention mechanism and shifted-window connectivity.

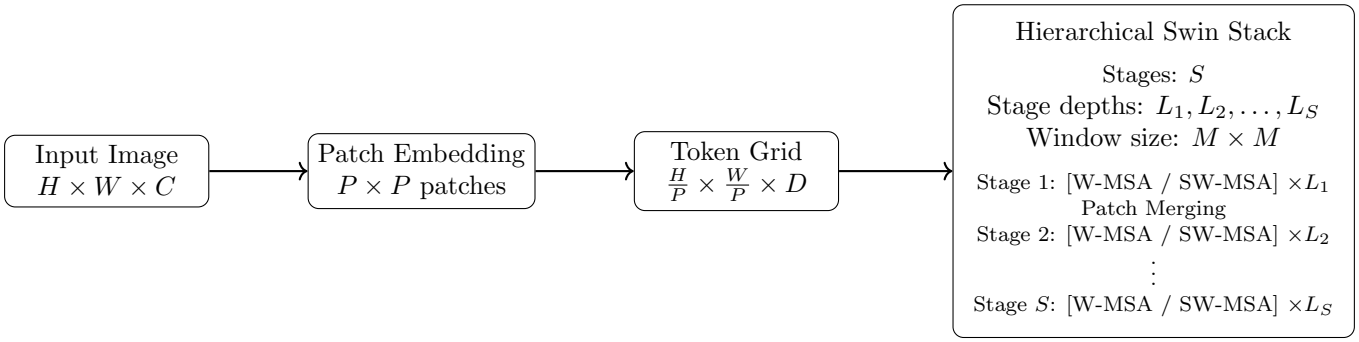


Figure 12.3: High-level architecture of the Swin Transformer. An input image is partitioned into patches and embedded into a 2D token grid. The token grid is processed by a hierarchical sequence of stages. Within each stage, Swin blocks alternate between regular window attention (W-MSA) and shifted-window attention (SW-MSA). Patch merging layers between stages reduce spatial resolution and increase channel dimension.

Input → PatchEmbed → Stage 1 → PatchMerging → Stage 2 → PatchMerging → Stage 3 → PatchMerging →

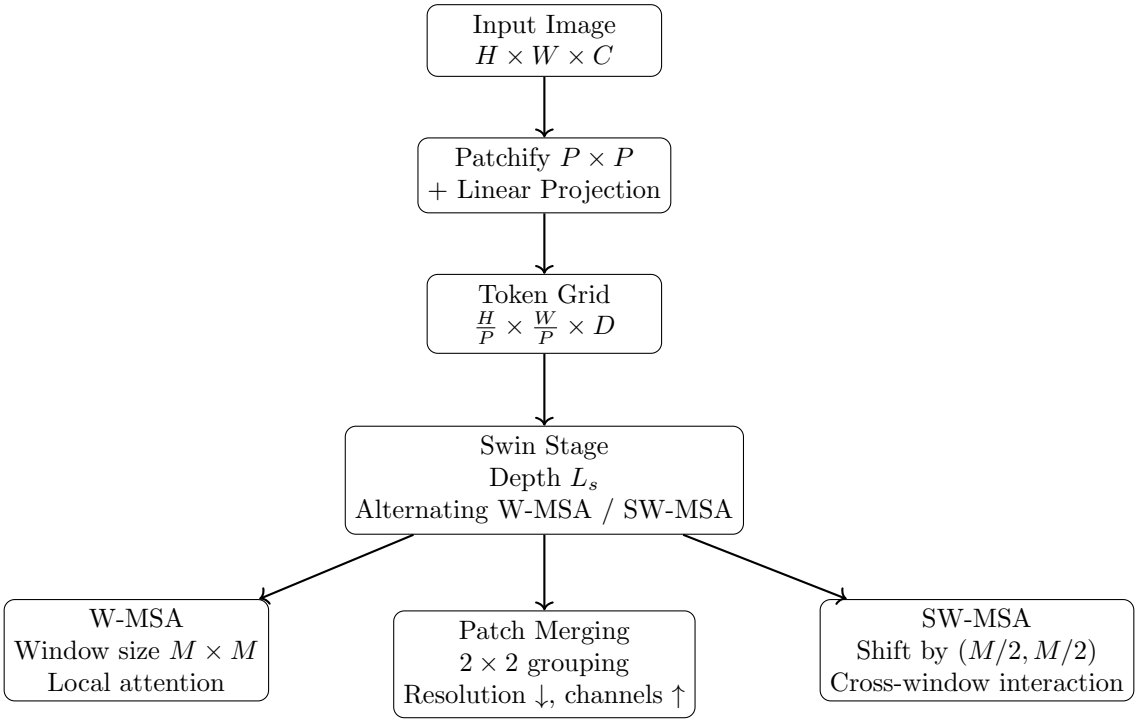


Figure 12.4: Tree-style view of one Swin Transformer stage. Each stage contains L_s Swin blocks with alternating regular window attention (W-MSA) and shifted-window attention (SW-MSA). Patch merging connects one stage to the next by reducing spatial resolution and increasing feature dimension.

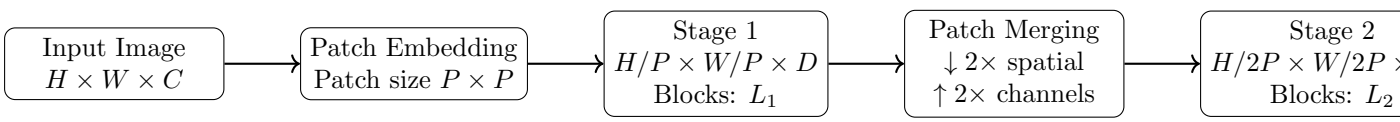


Figure 12.5: Flow of Swin Transformer across stages. The model begins with patch embedding, then processes the token grid through a hierarchy of stages. Each stage applies alternating windowed and shifted-window attention blocks, while patch merging progressively reduces spatial resolution and increases channel dimension.

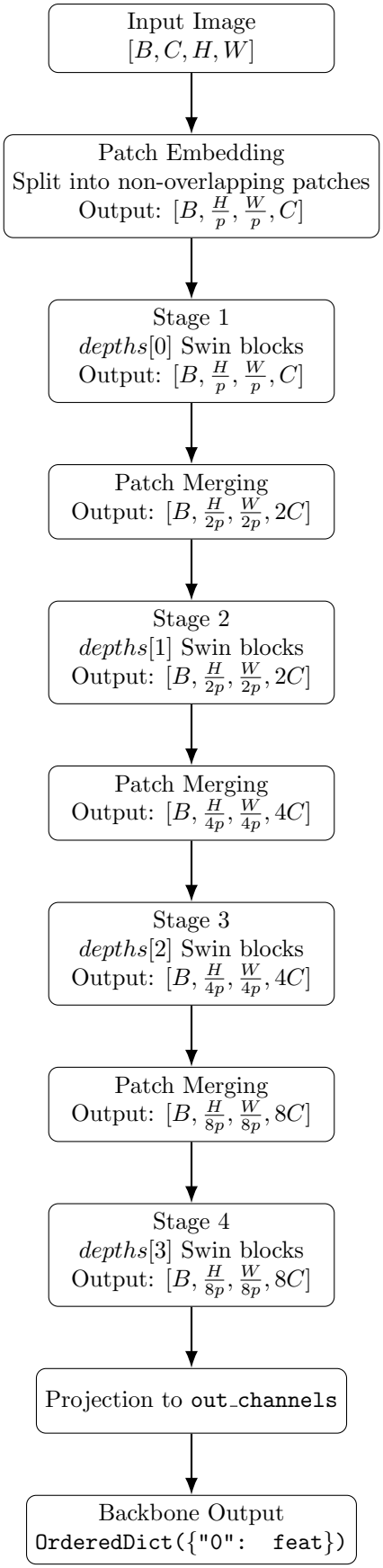


Figure 12.6: Overall flow of a Swin Transformer backbone. Here, p denotes patch size and C denotes the base embedding dimension.

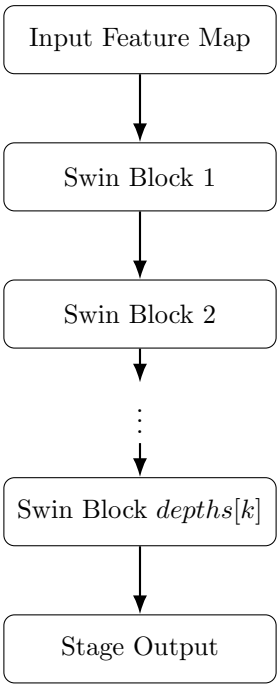


Figure 12.7: Internal structure of Swin stage k, consisting of a stack of Swin blocks.

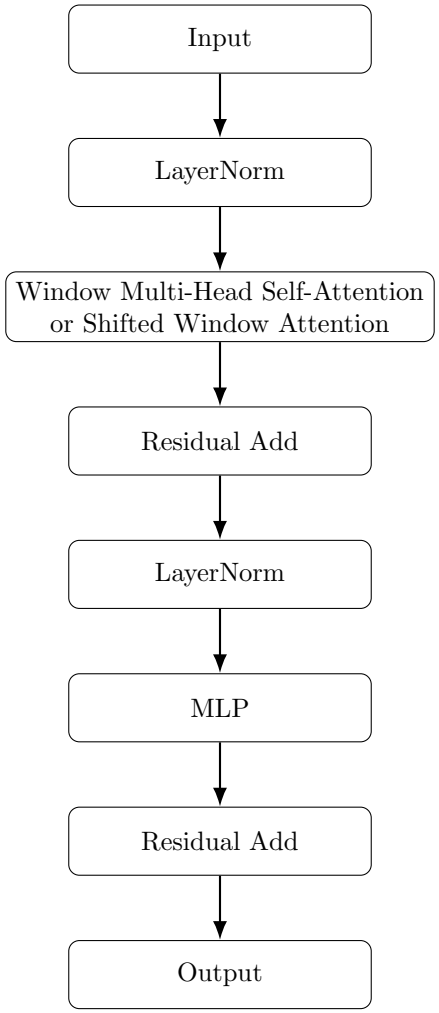


Figure 12.8: Internal flow of a Swin Transformer block.

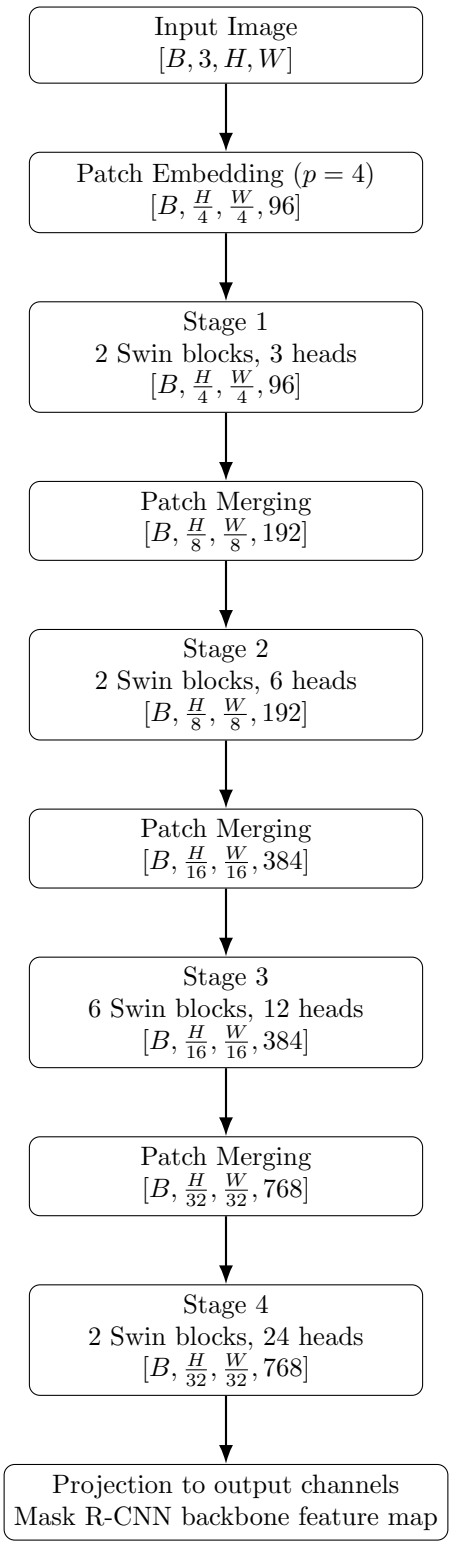


Figure 12.9: Example Swin Transformer backbone configuration with $embed_dim = 96$, $depths = (2, 2, 6, 2)$, and $num_heads = (3, 6, 12, 24)$.

Chapter 13

Model Structure Explanation

This chapter explains the structure of three commonly used deep learning models: ResNet-50, FCN-ResNet50, and DeepLabV3-ResNet50. ResNet-50 is mainly used as a feature extraction backbone, while FCN-ResNet50 and DeepLabV3-ResNet50 extend this backbone for semantic segmentation. The goal of this chapter is to describe each model in a clear and implementation-oriented way.

13.1 ResNet-50

ResNet-50 is mainly used as a feature extractor or backbone. It takes an input image and gradually converts it into deeper feature maps. These feature maps contain information at different levels, starting from simple visual patterns such as edges and textures and moving toward more abstract semantic information.

The ResNet-50 stages are sequential. This means that the original input image is not sent separately to every layer. Instead, the output of one stage becomes the input to the next stage. However, inside each stage, ResNet uses repeated residual bottleneck blocks. Each bottleneck block contains a skip connection, also called a residual connection.

```
ResNet50_Key_Idea = {
    "stage flow": "sequential",
    "stage order": "stem -> layer1 -> layer2 -> layer3 -> layer4",
    "inside each stage": "bottleneck residual block is repeated",
    "inside each block": "main conv path + skip connection",
    "why skip connection": "passes the input forward and adds it to the processed output",
    "benefit": "helps train deeper networks by improving gradient flow"
}
```

A ResNet-50 bottleneck block has two paths. The first path is the main convolution path. The second path is the shortcut or skip path. The output of the block is obtained by adding these two paths together.

```
ResNet50_Bottleneck_Block = {
    "input": "x",

    "main path": {
        "1x1 conv": "reduces or adjusts channels",
        "3x3 conv": "extracts spatial features",
        "1x1 conv": "expands channels"
    },

    "skip path": {
        "identity shortcut": "used when input and output shapes match",
        "projection shortcut": "1x1 conv used when channel size or spatial size changes"
    },

    "block output": {
        "operation": "main path output + skip path output",
        "formula": "y = F(x) + x, or y = F(x) + projection(x)",
        "meaning": "the block learns a residual correction instead of learning the full
    mapping directly"
    }
}
```

The repeated-block structure of ResNet-50 is:

```
ResNet50_Repetitions = {
    "layer1 / conv2_x": {
        "repeated block": "bottleneck residual block",
        "number of blocks": 3,
        "notation": "Bottleneck -> Bottleneck -> Bottleneck"
    },

    "layer2 / conv3_x": {
        "repeated block": "bottleneck residual block",
        "number of blocks": 4,
        "notation": "Bottleneck -> ... -> Bottleneck"
    },

    "layer3 / conv4_x": {
        "repeated block": "bottleneck residual block",
        "number of blocks": 6,
        "notation": "Bottleneck -> ... -> Bottleneck"
    },

    "layer4 / conv5_x": {
        "repeated block": "bottleneck residual block",
        "number of blocks": 3,
        "notation": "Bottleneck -> Bottleneck -> Bottleneck"
    }
}
```

Therefore, the layers themselves are not repeated as a full group. Instead, each layer is a ResNet stage, and each stage repeats the bottleneck residual block a specific number of times.

```
ResNet50_Stage_Clarification = {
    "not correct": "layer1 -> layer2 -> layer3 -> layer4 repeated many times",

    "correct": {
        "layer1": "bottleneck block repeated 3 times, then output goes to layer2",
        "layer2": "bottleneck block repeated 4 times, then output goes to layer3",
        "layer3": "bottleneck block repeated 6 times, then output goes to layer4",
        "layer4": "bottleneck block repeated 3 times, then output goes to classifier or
    segmentation head"
    }
}
```

```
ResNet50_Pseudocode = {
    "x": "input image [B, 3, H, W]",

    "x0": "Stem(x)",
    "Stem": "7x7 conv -> BN -> ReLU -> MaxPool",

    "C2": "Layer1(x0)",
    "Layer1": "repeat Bottleneck block 3 times",

    "C3": "Layer2(C2)",
    "Layer2": "repeat Bottleneck block 4 times",

    "C4": "Layer3(C3)",
    "Layer3": "repeat Bottleneck block 6 times",

    "C5": "Layer4(C4)",
    "Layer4": "repeat Bottleneck block 3 times",

    "z": "GlobalAveragePooling(C5)",
    "logits": "FullyConnected(z)",

    "important": "Layer1, Layer2, Layer3, and Layer4 are sequential",
    "skip connections": "inside each Bottleneck block, not between all stages"
}
```

```
Bottleneck_Block_Code_Diagram = {
    "input": "x",

    "main path": {
        "step 1": "a = Conv1x1(x)",
        "step 2": "a = BN(a) -> ReLU(a)",
        "step 3": "a = Conv3x3(a)",
        "step 4": "a = BN(a) -> ReLU(a)",
        "step 5": "a = Conv1x1(a)",
        "step 6": "a = BN(a)"
    },

    "skip path": {
        "case 1": "shortcut = x, if shape does not change",
        "case 2": "shortcut = Conv1x1(x), if channel or spatial size changes"
    },

    "output": {
        "code": "y = ReLU(a + shortcut)",
        "meaning": "main path output is added to skip path output"
    }
}
```

The input and output flow of ResNet-50 is:

```
ResNet50_Sequential_Flow = {
    "input image": "RGB image, shape [B, 3, H, W]",

    "stem": {
        "input": "RGB image",
        "operations": "7x7 conv -> batch norm -> ReLU -> max pooling",
        "output": "early feature map",
        "output shape": "[B, 64, H/4, W/4]",
        "feature type": "very low-level visual features",
        "examples": "edges, color changes, simple textures"
    },

    "layer1 / conv2_x": {
        "input": "stem output",
        "internal structure": "bottleneck residual block repeated 3 times",
        "skip connections": "one residual shortcut inside each bottleneck block",
        "output": "C2",
        "output channels": 256,
        "output shape": "[B, 256, H/4, W/4]",
        "feature type": "low-level features",
        "examples": "edges, corners, simple textures"
    },

    "layer2 / conv3_x": {
        "input": "C2 from layer1",
        "internal structure": "bottleneck residual block repeated 4 times",
        "skip connections": "one residual shortcut inside each bottleneck block",
        "output": "C3",
        "output channels": 512,
        "output shape": "[B, 512, H/8, W/8]",
        "feature type": "mid-level features",
        "examples": "local shapes, object parts"
    },

    "layer3 / conv4_x": {
        "input": "C3 from layer2",
        "internal structure": "bottleneck residual block repeated 6 times",
        "skip connections": "one residual shortcut inside each bottleneck block",
        "output": "C4",
        "output channels": 1024,
        "output shape": "[B, 1024, H/16, W/16]",
        "feature type": "high-level features",
        "examples": "larger structures, semantic regions"
    },

    "layer4 / conv5_x": {
        "input": "C4 from layer3",
        "internal structure": "bottleneck residual block repeated 3 times",
        "skip connections": "one residual shortcut inside each bottleneck block",
        "output": "C5",
        "output channels": 2048,
        "output shape": "[B, 2048, H/32, W/32]",
        "feature type": "deep semantic features",
        "examples": "class-level object meaning"
    },

    "global average pooling": {
        "input": "C5",
        "output": "one feature vector per image",
        "output shape": "[B, 2048]"
    },

    "fully connected layer": {
        "input": "pooled feature vector",
        "output": "classification logits",
        "output shape": "[B, num_classes]"
    }
}
```

The important point is that each stage receives the previous stage output. Within each stage, the bottleneck block is repeated several times before the result is passed to the next stage.

```
ResNet50_Main_Flow = {
    "step 1": "input image -> stem",

    "step 2": {
        "operation": "stem output -> layer1",
        "inside layer1": "bottleneck residual block x3",
        "output": "C2"
    },

    "step 3": {
        "operation": "C2 -> layer2",
        "inside layer2": "bottleneck residual block x4",
        "output": "C3"
    },

    "step 4": {
        "operation": "C3 -> layer3",
        "inside layer3": "bottleneck residual block x6",
        "output": "C4"
    },

    "step 5": {
        "operation": "C4 -> layer4",
        "inside layer4": "bottleneck residual block x3",
        "output": "C5"
    },

    "step 6": "C5 -> global average pooling",
    "step 7": "pooled vector -> fully connected layer",
    "final output": "image-level class prediction"
}
```

For image classification, ResNet-50 ends with a global average pooling layer and a fully connected layer. The global average pooling layer compresses the spatial feature map into a single feature vector, and the fully connected layer predicts the image-level class.

```
ResNet50_Output = {
    "input to classifier": "C5 feature map",
    "global average pooling": "converts [B, 2048, H/32, W/32] to [B, 2048]",
    "fully connected layer": "converts [B, 2048] to [B, num_classes]",
    "final output": "one class prediction for the whole image"
}
```

For semantic segmentation, the final classification part is usually removed. In that case, ResNet-50 is used only as a backbone. The network keeps the convolutional feature extraction layers, but removes the global average pooling and fully connected classification layers.

```
ResNet50_Backbone = {
    "input": "RGB image, shape [B, 3, H, W]",

    "kept": {
        "stem": "kept",
        "layer1 / conv2_x": "kept, produces C2",
        "layer2 / conv3_x": "kept, produces C3",
        "layer3 / conv4_x": "kept, produces C4",
        "layer4 / conv5_x": "kept, produces C5"
    },

    "inside each layer": {
        "layer1": "3 repeated bottleneck residual blocks",
        "layer2": "4 repeated bottleneck residual blocks",
        "layer3": "6 repeated bottleneck residual blocks",
        "layer4": "3 repeated bottleneck residual blocks"
    },

    "skip connections": {
        "location": "inside every bottleneck residual block",
        "operation": "main path output is added to shortcut path output",
        "purpose": "preserve information and improve training stability"
    },

    "removed": {
        "global average pooling": "removed",
        "fully connected layer": "removed"
    },

    "backbone outputs": {
        "C2": "[B, 256, H/4, W/4]",
        "C3": "[B, 512, H/8, W/8]",
        "C4": "[B, 1024, H/16, W/16]",
        "C5": "[B, 2048, H/32, W/32]"
    },

    "use in segmentation": "C2, C3, C4, and C5 can be passed to a segmentation head, FPN,
    or decoder"
}
```

In simple terms, ResNet-50 is a sequential feature extraction network made of repeated residual bottleneck blocks. The stages are connected one after another, but each bottleneck block contains its own skip connection. This is why ResNet can be deep while still being trainable. By itself, ResNet-50 is normally designed for image classification. When its classification layers are removed, it becomes a strong backbone for segmentation models.

13.2 FCN-ResNet50

FCN-ResNet50 means that a ResNet-50 backbone is combined with a Fully Convolutional Network segmentation head. Instead of predicting one class for the whole image, FCN-ResNet50 predicts one class score for every pixel.

The ResNet-50 backbone is sequential. The input image passes through the stem, then layer1, layer2, layer3, and layer4. The output of each stage becomes the input to the next stage. For segmentation, the classification layers of ResNet-50 are removed, and the deep feature map is passed to the FCN segmentation head.

```
FCN_ResNet50 = {
    "task": "semantic segmentation",
    "input": "RGB image, shape [B, 3, H, W]",
    "backbone": "ResNet-50 without global average pooling and FC layer",
    "segmentation head": "FCN classifier head",
    "output": "pixel-wise class prediction",
    "output shape": "[B, num_classes, H, W]"
}
```

The ResNet-50 backbone extracts hierarchical visual features from the input image. The early stages preserve more spatial detail, while the deeper stages contain stronger semantic information. In the standard FCN-ResNet50 structure, the main classifier head mainly uses the deep feature map from layer4.

```
FCN_ResNet50_Backbone = {
    "input": "RGB image, shape [B, 3, H, W]",

    "stem": {
        "input": "RGB image",
        "operations": "7x7 conv -> batch norm -> ReLU -> max pooling",
        "output": "early feature map",
        "output shape": "[B, 64, H/4, W/4]",
        "feature type": "very low-level visual features"
    },

    "layer1 / conv2_x": {
        "input": "stem output",
        "blocks": 3,
        "output": "C2",
        "output channels": 256,
        "output shape": "[B, 256, H/4, W/4]",
        "feature type": "low-level features",
        "examples": "edges, corners, simple textures"
    },

    "layer2 / conv3_x": {
        "input": "C2 from layer1",
        "blocks": 4,
        "output": "C3",
        "output channels": 512,
        "output shape": "[B, 512, H/8, W/8]",
        "feature type": "mid-level features",
        "examples": "local shapes, object parts"
    },

    "layer3 / conv4_x": {
        "input": "C3 from layer2",
        "blocks": 6,
        "output": "C4",
        "output channels": 1024,
        "output shape": "[B, 1024, H/16, W/16]",
        "feature type": "high-level features",
        "examples": "larger structures, semantic regions"
    },

    "layer4 / conv5_x": {
        "input": "C4 from layer3",
        "blocks": 3,
        "output": "C5",
        "output channels": 2048,
        "output shape": "[B, 2048, H/32, W/32]",
        "feature type": "deep semantic features",
        "examples": "class-level object meaning"
    }
}
```

The FCN head converts the deep feature map into pixel-level class scores. It does not use global average pooling or a fully connected classification layer. Instead, it keeps the spatial feature map and applies convolutional layers so that each spatial location receives class scores.

```
FCN_Head = {
    "input": {
        "source": "C5 from ResNet-50 layer4",
        "shape": "[B, 2048, H/32, W/32]",
        "meaning": "deep semantic feature map"
    },

    "3x3 conv": {
        "input": "C5",
        "output": "processed deep feature map",
        "purpose": "refines semantic features while keeping spatial structure"
    },

    "batch norm": "stabilizes feature values during training",

    "ReLU": "adds nonlinearity",

    "dropout": "regularization",

    "1x1 conv": {
        "input": "processed feature map",
        "output": "class-score map",
        "output shape": "[B, num_classes, H/32, W/32]",
        "purpose": "maps feature channels to class channels"
    },

    "upsampling": {
        "input": "low-resolution class-score map",
        "operation": "bilinear interpolation",
        "output shape": "[B, num_classes, H, W]",
        "purpose": "restores prediction to original image size"
    }
}
```

The main flow of FCN-ResNet50 can be summarized as follows.

```
FCN_ResNet50_Flow = {
    "step 1": "input image -> ResNet stem",
    "step 2": "stem output -> layer1 -> C2",
    "step 3": "C2 -> layer2 -> C3",
    "step 4": "C3 -> layer3 -> C4",
    "step 5": "C4 -> layer4 -> C5",
    "step 6": "C5 -> FCN head -> low-resolution class-score map",
    "step 7": "class-score map -> upsample to original image size",
    "final output": "pixel-wise segmentation logits, shape [B, num_classes, H, W]"
}
```

In segmentation, the final output is not a single class label. The model outputs a class-score vector for every pixel. During training, these scores are compared with the ground-truth segmentation mask. During inference, the predicted class at each pixel is usually obtained by taking the class with the highest score.

```
FCN_ResNet50_Output = {
    "logits": {
        "shape": "[B, num_classes, H, W]",
        "meaning": "raw class scores for each pixel"
    },

    "prediction": {
        "operation": "argmax over class dimension",
        "shape": "[B, H, W]",
        "meaning": "predicted class ID for each pixel"
    }
}
```

In short:

```
FCN_ResNet50 = {
    "strength": "simple and stable segmentation baseline",
    "uses ResNet-50": "yes, as a sequential feature extraction backbone",
    "uses global average pooling": "no, removed for segmentation",
    "uses fully connected layer": "no, removed for segmentation",
    "uses FCN head": "yes",
    "uses ASPP": "no",
    "multi-scale context": "limited compared with DeepLabV3",
    "best use": "baseline semantic segmentation"
}
```

FCN-ResNet50 is a good baseline model for semantic segmentation because it is simple, stable, and easy to understand. However, its context modeling is limited compared with DeepLabV3 because it does not use an ASPP module. In simple terms, FCN-ResNet50 is ResNet-50 used as a sequential feature extractor, followed by a convolutional pixel classifier and an upsampling step.

13.3 DeepLabV3-ResNet50

DeepLabV3-ResNet50 also uses ResNet-50 as the backbone. However, instead of using a simple FCN head, it uses an Atrous Spatial Pyramid Pooling module, usually called ASPP. This makes DeepLabV3 better at capturing context at multiple spatial scales.

The ResNet-50 backbone is still sequential. The input image passes through the stem, then layer1, layer2, layer3, and layer4. The output of one stage becomes the input to the next stage. After the backbone extracts the deep feature map, the ASPP module processes that feature map using several parallel dilation branches.

```
DeepLabV3_ResNet50 = {
    "task": "semantic segmentation",
    "input": "RGB image, shape [B, 3, H, W]",
    "backbone": "ResNet-50 without global average pooling and FC layer",
    "segmentation head": "ASPP + classifier",
    "output": "pixel-wise class prediction",
    "output shape": "[B, num_classes, H, W]"
}
```

The ResNet-50 backbone extracts hierarchical visual features from the input image. The early stages preserve more spatial structure, while the deeper stages contain stronger semantic information. In DeepLabV3-ResNet50, the classification layers of ResNet-50 are removed, and

the deep feature map is passed to the ASPP module.

```
DeepLabV3_ResNet50_Backbone = {
    "input": "RGB image, shape [B, 3, H, W]",

    "stem": {
        "input": "RGB image",
        "operations": "7x7 conv -> batch norm -> ReLU -> max pooling",
        "output": "early feature map",
        "output shape": "[B, 64, H/4, W/4]",
        "feature type": "very low-level visual features"
    },

    "layer1 / conv2_x": {
        "input": "stem output",
        "blocks": 3,
        "output": "C2",
        "output channels": 256,
        "output shape": "[B, 256, H/4, W/4]",
        "feature type": "low-level features",
        "examples": "edges, corners, simple textures"
    },

    "layer2 / conv3_x": {
        "input": "C2 from layer1",
        "blocks": 4,
        "output": "C3",
        "output channels": 512,
        "output shape": "[B, 512, H/8, W/8]",
        "feature type": "mid-level features",
        "examples": "local shapes, object parts"
    },

    "layer3 / conv4_x": {
        "input": "C3 from layer2",
        "blocks": 6,
        "output": "C4",
        "output channels": 1024,
        "output shape": "often [B, 1024, H/16, W/16]",
        "feature type": "high-level features",
        "examples": "larger structures, semantic regions"
    },

    "layer4 / conv5_x": {
        "input": "C4 from layer3",
        "blocks": 3,
        "output": "C5",
        "output channels": 2048",
        "output shape": "often [B, 2048, H/16, W/16] in segmentation settings",
        "feature type": "deep semantic features",
        "examples": "class-level object meaning"
    }
}
```

In many DeepLabV3 implementations, the last ResNet stages are modified using dilation so that the final feature map keeps more spatial resolution. This is different from the original classification ResNet-50, where the final feature map is usually around $H/32 \times W/32$. In segmentation, keeping a larger feature map helps produce better pixel-level predictions.

```
DeepLabV3_Backbone_Modification = {
    "classification ResNet-50": {
        "usual final stride": "32",
        "C5 shape": "[B, 2048, H/32, W/32]",
        "purpose": "image-level classification"
    },

    "DeepLabV3 ResNet-50": {
        "usual output stride": "16 or 8 depending on implementation",
        "C5 shape": "often [B, 2048, H/16, W/16]",
        "method": "replace some stride with dilation",
        "purpose": "keep more spatial detail for segmentation"
    }
}
```

The key part of DeepLabV3 is the ASPP module. ASPP does not apply dilation branches sequentially. Instead, it applies several branches in parallel to the same deep feature map. Each branch uses a different receptive field. The outputs from all branches are then concatenated and projected using a 1×1 convolution.

```
ASPP = {
    "input": {
        "source": "deep feature map from ResNet-50 layer4",
        "typical shape": "[B, 2048, H/16, W/16]",
        "meaning": "deep semantic feature map"
    },

    "parallel branch 1": {
        "operation": "1x1 conv",
        "purpose": "local feature projection"
    },

    "parallel branch 2": {
        "operation": "3x3 conv",
        "dilation": 12,
        "purpose": "medium-range context"
    },

    "parallel branch 3": {
        "operation": "3x3 conv",
        "dilation": 24,
        "purpose": "large-range context"
    },

    "parallel branch 4": {
        "operation": "3x3 conv",
        "dilation": 36,
        "purpose": "very-large-range context"
    },

    "parallel branch 5": {
        "operation": "global average pooling",
        "purpose": "image-level context"
    },

    "concatenation": {
        "input": "outputs from all ASPP branches",
        "purpose": "combine local, multi-scale, and image-level context"
    },

    "1x1 projection": {
        "input": "concatenated ASPP features",
        "output": "fused ASPP feature map",
        "purpose": "compress and fuse combined features"
    }
}
```

The full DeepLabV3-ResNet50 structure can be described as follows.

```
DeepLabV3_ResNet50 = {
    "input": "RGB image, shape [B, 3, H, W]",

    "ResNet-50 backbone": {
        "stem": {
            "input": "RGB image",
            "output": "early feature map"
        },

        "layer1": {
            "input": "stem output",
            "output": "C2",
            "feature type": "low-level features"
        },

        "layer2": {
            "input": "C2",
            "output": "C3",
            "feature type": "mid-level features"
        },

        "layer3": {
            "input": "C3",
            "output": "C4",
            "feature type": "high-level features"
        },

        "layer4": {
            "input": "C4",
            "output": "C5",
            "feature type": "deep semantic features, often with dilation"
        }
    },

    "ASPP module": {
        "input": "C5 from layer4",
        "1x1 conv": "local context",
        "3x3 conv dilation 12": "medium-range context",
        "3x3 conv dilation 24": "large-range context",
        "3x3 conv dilation 36": "very-large-range context",
        "global average pooling": "whole-image context",
        "concatenate": "merge all parallel branches",
        "project": "compress merged features"
    },

    "classifier": {
        "input": "fused ASPP feature map",
        "conv layers": "refine ASPP output",
        "1x1 conv": "maps features to num_classes",
        "output": "low-resolution class-score map"
    },

    "upsampling": {
        "input": "low-resolution class-score map",
        "operation": "bilinear interpolation",
        "output shape": "[B, num_classes, H, W]",
        "purpose": "resizes output to original image size"
    },

    "output": {
        "shape": "[B, num_classes, H, W]",
        "meaning": "class score for each pixel"
    }
}
```

The main flow of DeepLabV3-ResNet50 can be summarized as follows.

```
DeepLabV3_ResNet50_Flow = {
    "step 1": "input image -> ResNet stem",
    "step 2": "stem output -> layer1 -> C2",
    "step 3": "C2 -> layer2 -> C3",
    "step 4": "C3 -> layer3 -> C4",
    "step 5": "C4 -> layer4 -> C5",
    "step 6": "C5 -> ASPP parallel dilation branches",
    "step 7": "ASPP branch outputs -> concatenate",
    "step 8": "concatenated features -> 1x1 projection",
    "step 9": "projected ASPP feature -> classifier",
    "step 10": "class-score map -> upsample to original image size",
    "final output": "pixel-wise segmentation logits, shape [B, num_classes, H, W]"
}
```

In segmentation, the final output is a class-score vector for every pixel. During training, these scores are compared with the ground-truth segmentation mask. During inference, the predicted class at each pixel is usually obtained by taking the class with the highest score.

```
DeepLabV3_ResNet50_Output = {
    "logits": {
        "shape": "[B, num_classes, H, W]",
        "meaning": "raw class scores for each pixel"
    },

    "prediction": {
        "operation": "argmax over class dimension",
        "shape": "[B, H, W]",
        "meaning": "predicted class ID for each pixel"
    }
}
```

In short:

```
DeepLabV3_ResNet50 = {
    "strength": "better context understanding than FCN",
    "uses ResNet-50": "yes, as a sequential feature extraction backbone",
    "uses global average pooling": "not as classifier, but ASPP uses image-level pooling
    branch",
    "uses fully connected layer": "no, removed for segmentation",
    "uses dilation": "yes, in the backbone and ASPP",
    "uses ASPP": "yes",
    "ASPP branches": "parallel, not sequential",
    "multi-scale context": "strong",
    "best use": "semantic segmentation where object size and context matter"
}
```

DeepLabV3-ResNet50 is usually stronger than FCN-ResNet50 when objects or regions appear at different sizes. The ASPP module helps the model look at the image using multiple receptive fields, which improves contextual understanding. In simple terms, DeepLabV3-ResNet50 is ResNet-50 used as a sequential feature extractor, followed by a parallel multi-scale ASPP module and a pixel-wise classifier.understanding.

13.4 Main Difference Between the Models

The main difference between ResNet-50, FCN-ResNet50, and DeepLabV3-ResNet50 is summarized below.

```
Model_Comparison = {
    "ResNet-50": {
        "task": "classification backbone",
        "output": "one class for whole image",
        "segmentation ready": "not by itself"
    },

    "FCN-ResNet50": {
        "task": "semantic segmentation",
        "backbone": "ResNet-50",
        "head": "simple FCN classifier",
        "context handling": "basic"
    },

    "DeepLabV3-ResNet50": {
        "task": "semantic segmentation",
        "backbone": "ResNet-50",
        "head": "ASPP classifier",
        "context handling": "multi-scale dilated context"
    }
}
```

The simplest way to remember the three models is:

```
Summary = {
    "ResNet-50": "extracts image features",

    "FCN-ResNet50": "ResNet-50 + simple pixel classifier",

    "DeepLabV3-ResNet50": "ResNet-50 + ASPP multi-scale pixel classifier"
}
```

For semantic segmentation code, the two segmentation models can be viewed as follows.

```
Segmentation_Models = {
    "FCN-ResNet50": {
        "encoder": "ResNet-50",
        "decoder/head": "simple convolutional classifier",
        "context": "mainly from deep ResNet features"
    },

    "DeepLabV3-ResNet50": {
        "encoder": "ResNet-50",
        "decoder/head": "ASPP classifier",
        "context": "multi-scale dilated convolutions + image-level pooling"
    }
}
```

13.5 Chapter Summary

ResNet-50 is a convolutional neural network originally designed for image classification. In segmentation models, it is commonly used as a backbone after removing its classification layers.

FCN-ResNet50 extends ResNet-50 by adding a simple fully convolutional segmentation head. It converts the deep ResNet feature map into pixel-wise class predictions and upsamples the result to the original image size.

DeepLabV3-ResNet50 also uses ResNet-50 as the backbone, but it adds an ASPP module before classification. The ASPP module uses multiple dilation rates and image-level pooling to capture multi-scale context. Therefore, DeepLabV3-ResNet50 usually provides stronger contextual understanding than FCN-ResNet50.

```
Final_Summary = {
    "ResNet-50": "feature extractor / classification backbone",
    "FCN-ResNet50": "ResNet-50 + simple segmentation head",
    "DeepLabV3-ResNet50": "ResNet-50 + ASPP multi-scale segmentation head"
}
```

Chapter 14

Fundamental Concepts in Reinforcement Learning

14.1 Introduction

Reinforcement Learning (RL) is a framework in which an agent learns to make sequential decisions through interaction with an environment. At each time step, the agent observes the current state, selects an action, receives a reward, and moves to a new state. The objective is to learn a policy that maximizes long-term cumulative reward rather than only the immediate reward.

Throughout this chapter, we use a simple robot navigation example.

- **State:** The robot observes a wall ahead and a charging station located to its left.
- **Actions:** {Left, Right, Forward}.
- **Goal:** Reach the charging station.
- **Rewards:**
  - +100 for reaching the charging station,
  - −10 for hitting a wall,
  - −1 for each movement step.

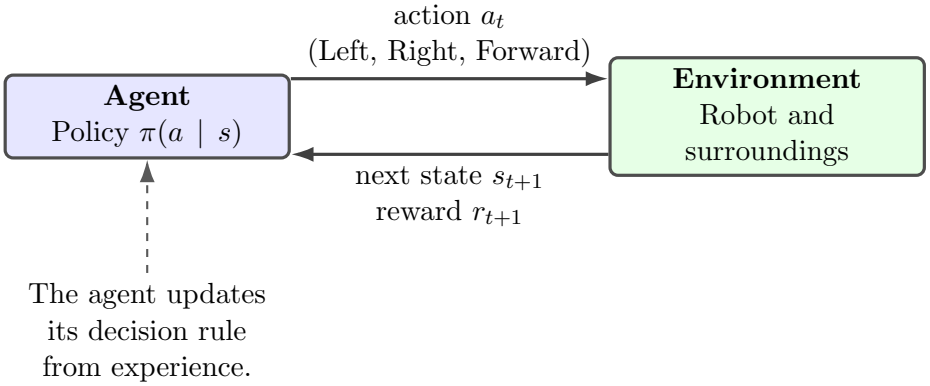


Figure 14.1: The reinforcement learning interaction loop. The agent selects an action from the current state, and the environment returns a reward and the next state.

14.2 Markov Decision Processes

Most reinforcement learning problems are modeled as a Markov Decision Process (MDP) defined by

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma),$$

where

- $\mathcal{S}$ is the state space,
- $\mathcal{A}$ is the action space,
- $P(s' \mid s, a)$ is the transition probability,
- $R(s, a)$ is the reward function,
- $\gamma \in [0, 1]$ is the discount factor.

The Markov property states that the future depends only on the current state and action, not on the full history:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots) = P(s_{t+1} \mid s_t, a_t).$$

In the robot example, the state may describe the robot position, wall distance, and charging-station direction. The action may be Left, Right, or Forward. The transition function describes how the robot moves after selecting an action, and the reward function assigns positive or negative feedback.

14.2.1 State

A state s_t is the information available to the agent at time t. It should contain enough information for the agent to make a useful decision.

14.2.2 Action

An action a_t is the decision selected by the agent. In discrete-control problems, actions may come from a finite set such as {Left, Right, Forward}. In continuous-control problems, actions may be real-valued, such as steering angle, torque, or velocity.

14.2.3 Reward

The reward r_t is the feedback signal received from the environment. It defines what behavior is desirable. A poorly designed reward function can lead to poor or unintended behavior even when the learning algorithm works correctly.

14.2.4 Discount Factor

The discount factor γ controls the importance of future rewards. If γ is close to zero, the agent focuses mainly on immediate reward. If γ is close to one, the agent considers long-term reward more strongly.

14.3 Return and Long-Term Reward

The return is the cumulative discounted reward collected from a given time step:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots .$$

The goal of reinforcement learning is usually to find a policy that maximizes the expected return:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_t].$$

For example, the robot should not move Forward into a wall simply because it reduces the step count. It should learn that turning Left may have a slightly longer immediate path but a higher long-term return because it reaches the charging station safely.

14.4 Policy

A policy defines the behavior of the agent. It specifies which action should be taken in a given state. A stochastic policy is written as

$$\pi(a \mid s) = P(a \mid s),$$

where s represents the state and a represents the action.

14.4.1 Example

Suppose the robot observes a wall ahead and the charging station slightly to the left. The policy may produce the probabilities shown in Table 14.1.

Table 14.1: Example policy output.

Action	Probability
Left	0.80
Right	0.10
Forward	0.10

The policy indicates that moving left is the most favorable action.

14.5 Stochastic Policies

A stochastic policy outputs a probability distribution over possible actions. Instead of always selecting the same action, the agent samples from the distribution:

$$a_t \sim \pi(\cdot \mid s_t).$$

14.5.1 Example

Table 14.2: Example stochastic policy.

Action	Probability
Left	70%
Right	20%
Forward	10%

The robot usually turns left but occasionally explores alternative actions. PPO and SAC use stochastic policies.

14.6 Deterministic Policies

A deterministic policy directly outputs a single action:

$$a = \pi(s).$$

For each state, there is exactly one selected action.

14.6.1 Example

If the robot observes a wall ahead and the charging station to the left, a deterministic policy may output

$$\pi(s) = \text{Left}.$$

TD3 uses a deterministic policy for the actor.

14.7 On-Policy Learning

On-policy algorithms learn from data generated by the current policy. After the policy changes, previously collected data may no longer represent the updated policy well and is often discarded.

14.7.1 Example

Suppose the current policy selects Left with probability 0.80 and Right with probability 0.20. After training, the policy may select Left with probability 0.95 and Right with probability 0.05. Since the older data were collected under the previous behavior, an on-policy method such as PPO generally collects fresh experience before the next update.

14.8 Off-Policy Learning

Off-policy algorithms can learn from data generated by older policies or different behavior policies. Experiences are commonly stored in a replay buffer and reused during training.

14.8.1 Example

Table 14.3: Replay buffer samples.

State	Action	Reward
s_1	Left	5
s_2	Right	-2
s_3	Forward	1

SAC and TD3 can train on these stored samples many times. This often makes them more sample-efficient than strictly on-policy methods.

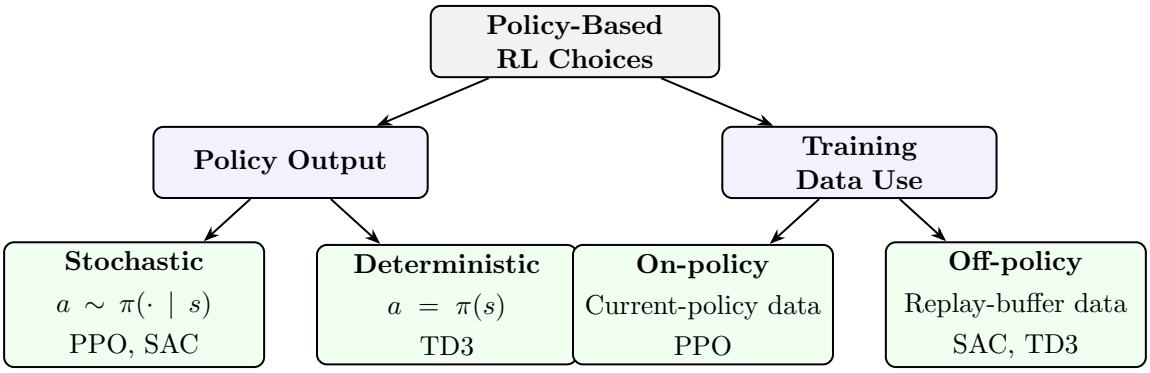


Figure 14.2: Two independent ways to classify reinforcement learning algorithms: how the policy represents actions and how training data are reused.

14.9 Replay Buffers

A replay buffer is a memory storage mechanism used in off-policy RL. It stores transitions of the form

$$(s_t, a_t, r_{t+1}, s_{t+1}, d_t),$$

where d_t indicates whether the episode ended. During training, the algorithm samples mini-batches from this buffer and updates the actor and critic networks.

Replay buffers are important because they improve data reuse and reduce the correlation between consecutive training samples. This makes neural-network training more stable.

14.10 Objective Functions in Reinforcement Learning

The objective function defines what the agent seeks to optimize. In supervised learning, one common loss is

$$L = (y - \hat{y})^2.$$

In reinforcement learning, the objective is usually based on expected return:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_t].$$

The central difference among many RL algorithms is how they approximate, regularize, or stabilize this objective.

14.10.1 Example

Table 14.4: Policy comparison based on average reward.

Policy	Average Reward
A	50
B	120

Policy B is preferred because it produces a larger expected reward.

14.11 Value Functions

Value functions estimate the long-term quality of states or state–action pairs. They are central to critic-based reinforcement learning.

14.11.1 State-Value Function

The state-value function estimates how good it is to be in state s and then follow policy π:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s].$$

14.11.2 Example

Table 14.5: Example state values.

State	Value
Near charging station	90
Lost in maze	15

14.11.3 Action-Value Function

The action-value function, or Q-function, estimates how good it is to take action a in state s and then follow policy π:

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a].$$

14.11.4 Example

Table 14.6: Example action values.

Action	$Q(s,a)$
Left	90
Right	15
Forward	-20

The optimal action corresponds to the highest Q-value.

14.12 Actor-Critic Methods

Actor-critic methods use two components: an actor and a critic. The actor selects actions, while the critic evaluates states or actions and provides a learning signal.

14.12.1 Actor

The actor is the policy network. It selects actions according to

$$a_t \sim \pi_\theta(\cdot \mid s_t)$$

for a stochastic actor, or

$$a_t = \pi_\theta(s_t)$$

for a deterministic actor. In the robot example, the actor is the part of the model that decides whether to move Left, Right, or Forward.

14.12.2 Critic

The critic estimates $V(s)$ or $Q(s, a)$. It does not directly choose the action. Instead, it evaluates how good the current state or selected action is expected to be.

Table 14.7: Example critic evaluation.

Action	Predicted Return
Left	80
Right	10

14.12.3 Interaction Between Actor and Critic

The actor proposes an action, and the critic estimates whether that action is beneficial. The actor is then updated in a direction that increases the critic's predicted value.

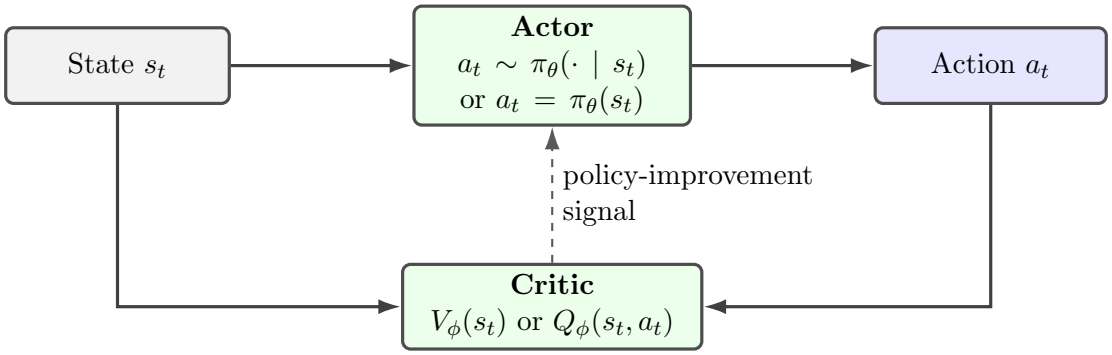


Figure 14.3: Actor–critic architecture. The actor proposes an action, while the critic evaluates the state or state–action pair and supplies a learning signal to improve the actor.

14.13 Critic Targets

Critics are trained using target values. A common one-step target is

$$y_t = r_{t+1} + \gamma V(s_{t+1}).$$

For Q-learning-style critics, a common target is

$$y_t = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}).$$

The exact form of the target depends on the algorithm. SAC uses a soft target that includes entropy, while TD3 uses the minimum of two target critics.

14.13.1 Example

Suppose

$$r_{t+1} = 5, \qquad V(s_{t+1}) = 80, \qquad \gamma = 0.99.$$

Then

$$y_t = 5 + 0.99(80) = 84.2.$$

The critic attempts to predict this target value.

14.14 Advantage Function

The advantage function measures whether an action is better or worse than the average action in a state:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

14.14.1 Example

Suppose

$$V(s) = 50, \qquad Q(s, \text{Left}) = 80.$$

Then

$$A(s, \text{Left}) = 80 - 50 = 30.$$

The action is better than average. If

$$Q(s, \text{Right}) = 20,$$

then

$$A(s, \text{Right}) = 20 - 50 = -30.$$

The action is worse than average. PPO uses advantage estimates heavily when updating the policy.

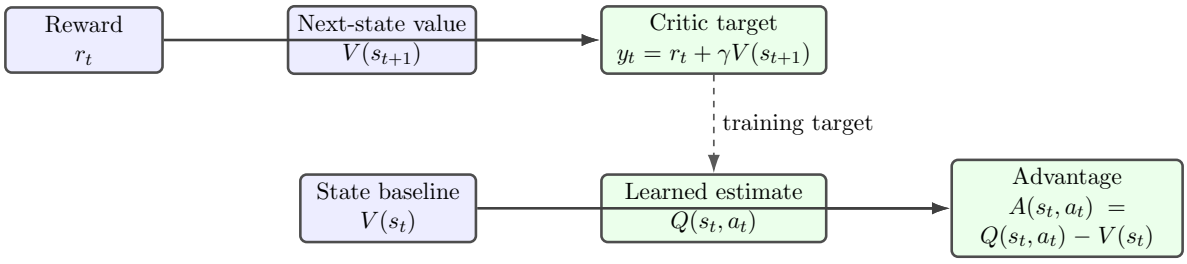


Figure 14.4: Relationship among the temporal-difference target, the learned action value, the state-value baseline, and the advantage estimate.

14.15 Entropy and Exploration

Entropy measures randomness in a policy:

$$H(\pi(\cdot \mid s)) = -\sum_{a} \pi(a \mid s) \log \pi(a \mid s).$$

Higher entropy means the policy explores more. Lower entropy means the policy behaves more deterministically.

14.15.1 Example

A low-entropy policy may assign probability 1.0 to Left and 0.0 to all other actions. A high-entropy policy may assign similar probabilities to Left, Right, and Forward. SAC explicitly rewards entropy so that the policy remains exploratory during learning.

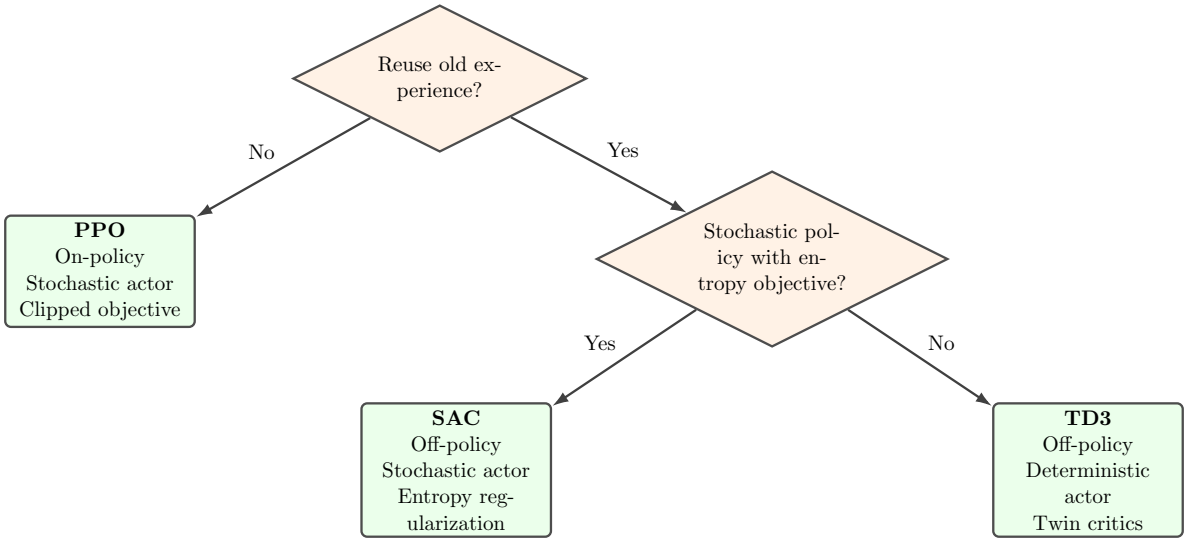


Figure 14.5: Conceptual decision map distinguishing PPO, SAC, and TD3. The diagram is a learning aid rather than a universal algorithm-selection rule.

14.16 Proximal Policy Optimization

Proximal Policy Optimization (PPO) is an on-policy actor-critic algorithm. It uses a stochastic policy and limits how much the updated policy can deviate from the previous policy.

14.16.1 Motivation

In policy-gradient learning, a large update can make the policy worse even if the estimated gradient appears beneficial. PPO addresses this issue by discouraging excessive policy changes.

14.16.2 Clipped Objective

Let

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

be the probability ratio between the new policy and the old policy. PPO optimizes the clipped surrogate objective

$$L^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t \right) \right].$$

The clipping term prevents the policy from changing too aggressively.

14.16.3 Example

Suppose the old policy selected Left with probability 0.50, but the new policy tries to assign probability 0.99 after one update. PPO limits the effect of this large jump. Conceptually, PPO says: improve the policy, but do not overreact to one batch of data.

14.16.4 Advantages and Limitations

PPO is simple, stable, and widely used. However, because it is on-policy, it typically requires more environment interactions than off-policy methods such as SAC and TD3.

14.17 Soft Actor-Critic

Soft Actor-Critic (SAC) is an off-policy actor-critic algorithm that uses a stochastic policy and a maximum-entropy objective.

14.17.1 Motivation

Standard RL maximizes expected reward. SAC maximizes reward while also encouraging exploration. This is useful in continuous-control problems where premature convergence to a poor deterministic behavior can be harmful.

14.17.2 Maximum-Entropy Objective

SAC optimizes

$$J(\pi) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} \left[r(s_t, a_t) + \alpha H(\pi(\cdot \mid s_t)) \right],$$

where α controls the importance of entropy.

14.17.3 Soft Q Target

A simplified SAC target is

$$y_t = r_{t+1} + \gamma \left(Q_{\text{target}}(s_{t+1}, a_{t+1}) - \alpha \log \pi(a_{t+1} \mid s_{t+1}) \right).$$

The entropy-related term encourages the policy to keep exploring high-value alternatives.

14.17.4 Example

Suppose one policy has reward 100 and entropy 0, while another policy has reward 98 and entropy 20. If $\alpha = 0.2$, the second policy receives

$$98 + 0.2(20) = 102.$$

SAC may prefer the second policy because it balances reward and exploration.

14.17.5 Advantages and Limitations

SAC is usually sample-efficient and robust because it reuses replay-buffer data and maintains exploration through entropy regularization. Its main limitation is algorithmic complexity, including soft Q targets and temperature tuning.

14.18 Twin Delayed Deep Deterministic Policy Gradient

Twin Delayed Deep Deterministic Policy Gradient (TD3) is an off-policy deterministic actor-critic algorithm. It improves Deep Deterministic Policy Gradient (DDPG) by reducing Q-value overestimation and stabilizing actor updates.

14.18.1 Motivation

In deterministic actor-critic methods, the actor may exploit errors in the critic. If the critic overestimates some actions, the actor may learn to choose those actions even when they are not truly good. TD3 addresses this problem using three main mechanisms: twin critics, delayed actor updates, and target policy smoothing.

14.18.2 Twin Critics

TD3 trains two critics, Q_1 and Q_2, and uses the smaller predicted value in the target:

$$y_t = r_{t+1} + \gamma \min_{i=1,2} Q_{i,\text{target}}(s_{t+1}, a_{t+1}).$$

14.18.3 Example

If

$$Q_1 = 120, \qquad Q_2 = 90,$$

then TD3 uses

$$\min(Q_1, Q_2) = 90.$$

This reduces overestimation bias.

14.18.4 Delayed Actor Updates

TD3 updates the critic more frequently than the actor. This gives the critic more time to become reliable before the actor is updated using the critic's estimate.

14.18.5 Target Policy Smoothing

TD3 adds small clipped noise to the target action:

$$a' = \pi_{\text{target}}(s') + \epsilon.$$

This prevents the critic from relying on sharp, narrow peaks in the learned value function.

14.18.6 Advantages and Limitations

TD3 is effective for continuous-control problems with deterministic policies. However, it is not naturally designed for discrete action spaces, and it requires careful tuning of exploration noise and target smoothing.

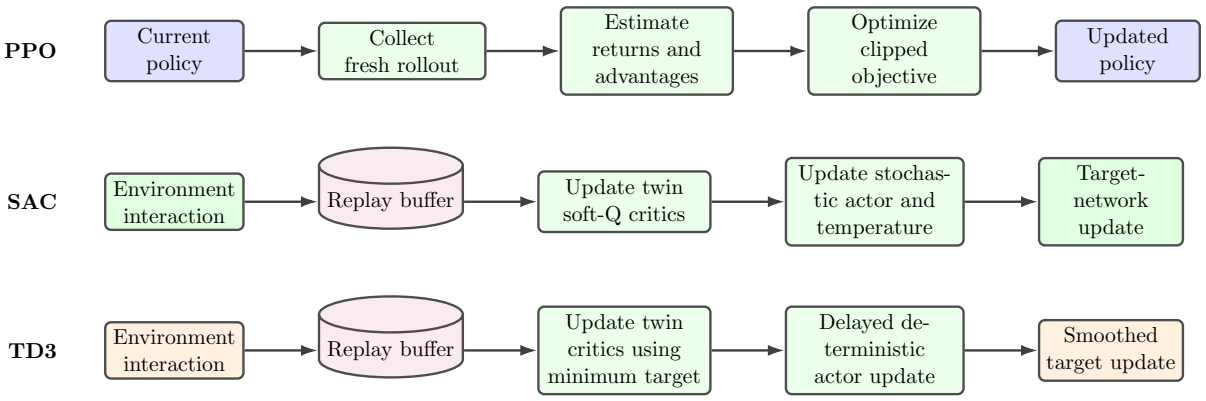


Figure 14.6: High-level training pipelines for PPO, SAC, and TD3. PPO uses newly collected on-policy rollouts, whereas SAC and TD3 reuse samples from a replay buffer.

14.19 Optimizers in Reinforcement Learning

An optimizer updates neural-network parameters using gradients. Common optimizers include Stochastic Gradient Descent (SGD), RMSProp, and Adam.

Suppose

$$w = 2.0, \qquad \frac{\partial L}{\partial w} = 0.5.$$

An optimizer updates the parameter, for example,

$$w \leftarrow 1.995.$$

The optimizer is not the main difference among PPO, SAC, and TD3. All three can use Adam. Their main differences are the policy type, objective function, critic target, data usage, and stabilization mechanisms.

14.20 Comparison of PPO, SAC, and TD3

Table 14.8: Comparison of PPO, SAC, and TD3.

Feature	PPO	SAC	TD3
Policy type	Stochastic	Stochastic	Deterministic
Training data	On-policy	Off-policy	Off-policy
Replay buffer	No	Yes	Yes
Main objective	Clipped policy-gradient objective	Reward plus entropy	Deterministic actor objective
Critic target	Value or advantage target	Soft Q target	Minimum of twin critics
Entropy bonus	Usually no explicit maximum-entropy objective	Yes	No
Main stabilization	Policy-ratio clipping	Entropy regularization and target networks	Twin critics, delayed actor update, target smoothing
Sample efficiency	Moderate	High	High
Common action space	Discrete or continuous	Mostly continuous; discrete variants exist	Continuous
Typical optimizer	Adam	Adam	Adam

The main idea is that PPO, SAC, and TD3 are not merely different optimizers. They are different learning recipes built from different objectives, target definitions, policy classes, sampling strategies, and stabilization mechanisms.

14.21 End-to-End Learning Workflow

A complete RL workflow can be summarized as follows:

1. Observe the current state s_t.
2. Select an action a_t using the policy.
3. Execute the action in the environment.
4. Receive reward r_{t+1} and next state s_{t+1}.
5. Estimate value, Q-value, or advantage using the critic.
6. Compute the relevant loss or objective.
7. Update the actor, critic, or both using gradient-based optimization.
8. Repeat until the policy achieves satisfactory performance.

For PPO, this workflow uses fresh on-policy rollouts and a clipped policy objective. For SAC, it uses replay-buffer samples and a reward-plus-entropy objective. For TD3, it uses replay-buffer samples, twin critics, delayed actor updates, and target policy smoothing.

14.22 Summary

Modern reinforcement learning algorithms can be interpreted as different combinations of

Policy type+Objective function+Critic target+Data collection strategy+Stabilization mechanism.

PPO, SAC, and TD3 all train neural networks using gradient-based optimization, often with Adam. However, their mathematical and algorithmic designs are different. PPO emphasizes stable on-policy improvement through clipping. SAC augments reward maximization with entropy maximization to encourage exploration. TD3 improves deterministic actor-critic learning through twin critics, delayed actor updates, and target policy smoothing.

Therefore, the difference among PPO, SAC, and TD3 is not simply the optimizer. The main differences are the objective function, the policy representation, the critic target, the data-use strategy, and the stability mechanisms used during training.

Index

1x1 convolution, 142

action, 261
Activation function, 31, 32, 37
  choice, 32
  derivative, 50, 53
  ReLU, 33, 57
  sigmoid, 33, 57
  tanh, 33, 57
activation function, 153
Activation Functions, 15
  GELU, 16
  Leaky ReLU, 16
  ReLU, 16
  Sigmoid, 15
  Softmax, 16
  Tanh, 15
actor, 267
actor-critic methods, 267
Adagrad, 32
Adam, 32
Adam optimizer, 272
advantage function, 268
agent, 261
AlexNet, 169
  architecture, 170
Architecture parameters, 224, 234
atrous convolution, *see* dilated convolution
Attention
  cost, 224, 233
Attention head, 223, 224, 232, 234, 236
  count, 223, 227, 228, 232
Attention mask, 233
Attention matrix, 223, 232
  size, 225
Attention model, 229
Attention module, 223
  count, 223, 224, 226–228
Autograd, 48
Automatic differentiation, 31, 36, 48, 49
average pooling, 153

Backbone network, 221, 226, 229, 236
Backpropagation, 15, 28, 31
  chronological table, 61
  definition, 31
  gradient accumulation, 39
  initial upstream gradient, 61
  Jacobian view, 49
  matrix form, 37
  numerical example, 68
  operations, 64
  parameter gradients, 64
  reference implementation, 65
  single neuron, 33
  steps, 32
  summary, 32, 56
  universality, 32
  worked example, 57
Backward pass, 31
Bagging, 127
  comparison with boosting, 130
basic residual block, 182
Batch computation, 233
Bias, 31, 33, 57
Bias–variance trade-off, 119
Boosting, 128
  comparison with bagging, 130
Bootstrap, 124
Bootstrap aggregation, 127
Bootstrap sampling, 124
bottleneck residual block, 183
Boundary masking, 233

Chain rule, 31–33, 49
  accumulation, 39
  computational graph, 61
  explicit calculation, 53
channel, 137, 143
Channel dimension, 231, 234
channel mixing, 139
channel transformation, 140
Channels, 223, 231
Classification token, 225
clipped surrogate objective, 269
`[CLS]` token, 223, 231
CNN, *see* convolutional neural network (CNN)
  image classification, 154

image data, 167
object detection, 156
semantic segmentation, 155
versus fully connected neural network, 167
CNN architecture, 154
CNN architectures
architecture-level comparison, 180
comparison, 179
CNN hierarchy, 152
Committee methods, 117, 118
Commutativity, 65
Computation graph, 33, 36, 48
definition, 48
Computational complexity, 49, 221, 223, 224, 226, 229, 233, 234, 238
Computational efficiency, 238
Compute, 224, 233, 234
Computer vision, 221, 229
systems, 226, 236
Transformers, 221, 229
Conditional Random Field, 214
Context modeling, 236
Convolution, 221
convolution, 133
1x1, 142
mathematical definition, 134
multiple channels, 137
numerical example, 135
output size, 145
output size example, 146
parameter count, 153
versus cross-correlation, 134
Convolutional neural network (CNN), 33, 221, 233
backbone, 229, 231
convolutional neural network (CNN), 133
CRF, 214
critic, 267
critic target, 267
cross-correlation, 134
numerical example, 135
Cross-window interaction, 229, 232, 233, 238

deconvolution, 192, 195
Deep Deterministic Policy Gradient, 271
deep learning libraries
convolution implementation, 136
Deep neural network (DNN), 33
degradation problem, 182
delayed actor update, 271
Delta (backprop), 37
dense layer, *see* fully connected layer
Dense prediction, 229, 236
Deployment, 226
Depth, 32
Depth L, 223, 224, 226
Derivative
activation function, 37
ReLU, 34, 57
sigmoid, 34, 57
tanh, 34, 57
dilated convolution, 147
2D example, 148
dilation
motivation, 149
output size, 148
Direction vector, 46
discount factor, 262
Divisibility constraint, 224, 234
Dot product, 65
Double descent, 123
Downsampling, 229, 233, 234

effective kernel size, 147
Embedding, 223, 231
Embedding dimension, 223–225, 234
per-head, 223
Embedding dimension D, 223
Ensemble learning, 117
entropy, 269
environment, 261
Example
attention matrix, 224, 234
full forward backprop update, 57
numerical values, 34, 57
single neuron, 33
Swin attention, 234
three-layer backpropagation, 33
token count, 224
ViT attention, 224
window count, 234
Expected error
committee, 120
individual model, 119
exploration, 269

Fast R-CNN, 157
Faster R-CNN, 160
Feature dimension, 234
Feature hierarchy, 221
Feature map, 229
feature map, 143
example, 144
multiple output channels, 139

Feature pyramid, 231
Feature pyramid network (FPN), 236
Feature representation, 233
Feed-forward network, 223, 226, 232, 238
    cost, 224, 234
Feedforward neural network, 31
filter, *see* kernel
Flattening, 223, 231
Focal Loss, 203
Forward Pass, 15, 28
Forward pass, 31
    chronological table, 61
    data flow, 31
    definition, 31
    matrix form, 37
    memory, 37, 38
    single neuron, 33
    summary, 32
    worked example, 57
fully connected layer, 154
fully connected neural network
    comparison with CNN, 167

GoogLeNet, 176
    architecture, 177
Gradient, 31
    biases, 38
    shape, 64
    summary, 59, 64
    weights, 38
Gradient descent, 32
    manual implementation, 65
    update, 59
    update rule, 57
gradient flow, 182

Hadamard product, 37
Head dimension, 223, 232
Heads h, 223–225, 234
Hessian-free optimization, 40, 46
Hessian-vector product, 39, 40, 55
    example, 40
hierarchical feature learning, 152
Hierarchical transformer, 226, 229, 231, 238
High-resolution image, 236
Hyperparameter, 223, 224, 234
    typical values, 224
    window size, 232

identity shortcut, 183
Image, 223, 231
Image classification, 226, 238
image classification, 154
Image resolution, 223–226, 231, 235
Image segmentation, 221, 226, 229, 236, 238
image tensor, 133
Inception module, 176
Inception network, 176
Inductive bias, 226
    locality, 221
    translation equivariance, 221
Inference mode, 48
input channel, 138
input tensor, 133
Instance segmentation, 221, 229
instance segmentation, 156
Interpolation threshold, 123

Jacobian matrix, 49
    chronological, 61
    definition, 49
    numerical example, 50
Jacobian–vector product, 49

Kernel, 221
kernel, 133, 143
    learned weights, 145
kernel flipping, 134
Key (K), 223

Layer
    fully connected, 64
    hidden, 31
    input, 31
    output, 31
Layer normalization, 232
LeNet, 168
    architecture, 168
LeNet-5, 168
Linear algebra
    backpropagation, 65
Linear projection, 223, 231
Linear scaling, 232, 233
Local attention, 229, 232, 233
local connectivity, 167
Long-range dependency, 221
Loss function, 31
    choice, 32
    derivative, 50, 53, 61
    effect on dynamics, 68
Loss Functions, 15, 19
    Cross Entropy, 19
    cross-entropy, 31, 32
    Dice Loss, 19

Focal Loss, 19
Huber Loss, 19
IoU Loss, 19
KL Divergence, 19
MAE, 19
MSE, 19, 31, 32, 57

Markov Decision Process, 262
Markov property, 262
Mask R-CNN, 161, 221, 226, 236
Matrix calculus, 37
Matrix multiplication, 65
Matrix–vector product, 37
max pooling, 153
maximum-entropy objective, 270
Memory complexity, 49
Memory usage, 48, 221, 223, 224, 226, 236, 238
attention, 226
MLP, 223, 224, 232–234
cost, 224, 234
width, 224, 234
MLP ratio, 226, 238
MLP ratio r, 224, 234
Model averaging, 117, 118
multi-channel convolution, 138
Multi-head self-attention (MHSA), 223, 227, 232
multi-scale feature extraction, 176
Multi-scale feature map, 236
Multi-scale representation, 236
Multilayer perceptron (MLP), 31, 50

Nabla operator, 39
Natural language processing (NLP), 221
Neighborhood interaction, 233
Network architecture
layers, 31
Neural network
basic, 31
training, 31
Neuron
single, 64
Newton–CG, 40, 46
Notation
dimensions, 37

Object detection, 221, 226, 229, 236, 238
object detection, 156
objective function, 265
off-policy learning, 264
on-policy learning, 264
Optimizer, 32
choice, 32
optimizer, 272
Optimizers, 15, 23
Adagrad, 23
Adam, 23
AdamW, 23
LAMB, 23
Momentum, 23
RMSProp, 23
SGD, 23
Out-of-bag observations, 124
Outer product, 37, 38, 65
output channel, 139

padding, 145, 146
Parallelism, 224, 234
parameter count
convolution layer, 153
Parameter sharing, 121
hard, 122
soft, 122
Patch, 223, 231
non-overlapping, 221, 223, 231
Patch embedding, 221, 223, 227, 231, 238, 239
Patch merging, 229–231, 233, 234, 236, 238, 239
Patch size, 223–226, 231, 234, 235
policy, 261
deterministic, 264
stochastic, 263
Pooling, 233
pooling, 153
Positional embedding, 221
Product types, 38
projection shortcut, 183
Proximal Policy Optimization, 269
PyTorch, 48
image tensor format, 133

Q-function, *see* value function, action-value
Quadratic complexity, 223, 226, 229
local, 234
Query (Q), 223

R-CNN, 156
receptive field, 150
stride, 151
Recurrent neural network (RNN), 33
reinforcement learning, 261
Relative position bias, 232
ReLU, 31, 32, 153
derivative, 37

replay buffer, 265
Resampling, 124
residual branch, 182
Residual connection, 232
residual connection, 181
residual learning, 181
residual network, *see* ResNet
ResNet, 181
  architecture, 185
  basic block, 182
  basic residual block, 182
  bottleneck block, 183
  bottleneck repetition, 183
  variants, 186
ResNet-101, 186
ResNet-152, 186
ResNet-18, 186
ResNet-34, 186
ResNet-50, 183, 186
return, 263
reward, 261
RMSProp, 32, 272

Sampling with replacement, 124
Saved tensors, 37
Scaling laws, 221, 224, 226, 233, 234
Second backward pass, 40, 46
Second-order derivatives, 39
Second-order optimization, 49
segmentation
  CRF, 214
Self-attention, 221, 223, 224, 226, 229
  cost, 224
  global, 221, 222, 225, 226, 229, 233, 238
  local, 226
  quadratic scaling, 223–225
semantic segmentation, 155
Sequence
  prepend token, 223
Sequence model, 221
Sequential model combination, 128
Shifted window attention, 229, 230, 233, 235, 238, 239
Shifted Window Transformer, *see* Swin Transformer, 229
shortcut branch, 182
Sigmoid, 31, 32
  derivative, 37
skip connection, 181
Soft Actor-Critic, 270
soft Q target, 270
Spatial cost, 236
Spatial grid, 231
Spatial position, 233
Spatial resolution, 231, 233
Stacked layers, 223
Stage, 231, 239
Stage depth, 234, 239
Stage structure, 231
state, 261
Stochastic Gradient Descent, 272
Stochastic gradient descent (SGD), 32
stride, 145, 146
Strided convolution, 233
`SW-MSA`, 232
Swin Transformer, 229, 238
  architecture, 229, 238, 239
  attention example, 235
  comparison with ViT, 236, 237
  flow, 239
  four-stage hierarchy, 231
  hierarchical architecture, 230
  parameters, 234
  small example, 234
  summary, 238
Swin Transformer block, 231, 232

tanh, 31, 32
  derivative, 37
Target, 31
target policy smoothing, 271
Token, 223, 224, 227
  classification token, 223
  count, 223, 224, 226, 232, 234
  embedding, 224
  merging, 226
  patch token, 223, 226, 231
  sequence, 223
Token grid, 238
Token grouping, 233
Tokenization, 223, 231
Training mode, 48
Transformer, 221, 234
  backbone, 236
  block, 229, 232
  depth, 223, 224, 226–228, 234
  encoder, 221, 227
  flat architecture, 221, 231
  layer, 223, 224
  overview, 221, 229
translation equivariance, 167
transposed convolution, 192, 194
twin critics, 271

Twin Delayed Deep Deterministic Policy Gradient, 271

unpooling, 192, 193
UPerNet, 236
upsampling, 192
  CNN, 193
  learnable, 194

Value (V), 223
value function, 266
  action-value, 266
  state-value, 266
Vector–Jacobian product, 49
VGG network, 172
VGG16, 172
  architecture, 173
Vision Transformer (ViT), 221, 223, 226, 229, 231
  architecture, 221, 227, 228
  attention example, 225
  backbone, 236
  comparison with Swin Transformer, 236, 237
  flat architecture, 222
  parameters, 224
  small example, 224
  summary, 226
ViT, *see* Vision Transformer (ViT)

`W-MSA`, 232
Weight and Bias Updates, 15, 28
weight sharing, 167
Weighted sum, 31
Weights, 33, 57
  vector vs matrix, 64
Width, 32
Window boundary, 233
Window partition, 229, 232
Window shift, 233
Window size, 232, 234, 235
Window-based self-attention, 229, 230, 232, 233, 235
  cost, 233
Windowed attention, 226, 229, 234, 238, 239